

DevOps Handbook — Complete Edition

1 DevOps Handbook — Master Table of Contents

- [1.1 How to read this handbook](#)
- [1.2 Part I — Introduction & Mindset \(Pages 1–60\)](#)
- [1.3 Part II — Linux & Shell Mastery \(Pages 61–140\)](#)
- [1.4 Part III — Git & Collaboration \(Pages 141–200\)](#)
- [1.5 Part IV — Networking for DevOps \(Pages 201–280\)](#)
- [1.6 Part V — Scripting & Programming \(Pages 281–360\)](#)
- [1.7 Part VI — Infrastructure as Code \(Pages 361–480\)](#)
- [1.8 Part VII — Containers \(Pages 481–580\)](#)
- [1.9 Part VIII — Kubernetes \(Pages 581–720\)](#)
- [1.10 Part IX — CI/CD \(Pages 721–840\)](#)
- [1.11 Part X — Observability & SRE \(Pages 841–960\)](#)
- [1.12 Part XI — Security \(DevSecOps\) \(Pages 961–1080\)](#)
- [1.13 Part XII — Advanced Topics \(Pages 1081–1200\)](#)
- [1.14 Expansion status](#)

2 Chapter 1: What Is DevOps? History and Culture

- [2.1 1.1 The problem DevOps solves](#)
- [2.2 1.2 A working definition](#)
- [2.3 1.3 Brief history](#)
- [2.4 1.4 Culture before tools](#)
- [2.5 1.5 DevOps practices \(overview\)](#)
- [2.6 1.6 Organizational models](#)
- [2.7 1.7 What DevOps is NOT](#)
- [2.8 1.8 Your learning path in this handbook](#)
- [2.9 1.9 Chapter summary](#)
- [2.10 Lab 1.1 — Reflect on your current workflow](#)
- [2.11 Review questions](#)

3 Chapter 2: CALMS, Three Ways, and Team Topologies

- [3.1 2.1 The CALMS framework](#)
- [3.2 2.2 The Three Ways \(Gene Kim\)](#)
- [3.3 2.3 Team Topologies \(Skelton & Pais\)](#)
- [3.4 2.4 Conway's Law and reverse Conway maneuver](#)
- [3.5 2.5 Maturity model \(informal\)](#)
- [3.6 2.6 Chapter summary](#)
- [3.7 Lab 2.1 — CALMS assessment](#)
- [3.8 Review questions](#)

4 Chapter 3: DevOps vs SRE vs Platform Engineering

- [4.1 3.1 Three related disciplines](#)
- [4.2 3.2 Site Reliability Engineering \(SRE\)](#)
- [4.3 3.3 Platform Engineering](#)
- [4.4 3.4 How they fit together](#)
- [4.5 3.5 Job titles in the market](#)
- [4.6 3.6 Chapter summary](#)
- [4.7 Lab 3.1](#)

5 Chapter 4: Career Paths, Certifications, and Learning Strategy

- [5.1 4.1 DevOps is a skill set, not a single job title](#)
- [5.2 4.2 Common career paths](#)
- [5.3 4.3 Certifications: value and limits](#)
- [5.4 4.4 Building a learning strategy](#)
- [5.5 4.5 Portfolio and interview preparation](#)
- [5.6 4.6 Soft skills that separate good from great](#)
- [5.7 4.7 Staying current without burnout](#)
- [5.8 4.8 Chapter summary](#)
- [5.9 Lab 4.1 — Personal learning plan](#)

[5.10 Lab 4.2 — Job description analysis](#)

[5.11 Review questions](#)

6 Chapter 5: Linux Fundamentals — Filesystem, Users, Permissions

[6.1 5.1 Why Linux dominates DevOps](#)

[6.2 5.2 Filesystem hierarchy \(FHS\)](#)

[6.3 5.3 Essential navigation commands](#)

[6.4 5.4 File operations](#)

[6.5 5.5 Users, groups, and permissions](#)

[6.6 5.6 Package management \(Debian/Ubuntu\)](#)

[6.7 5.7 Text processing \(daily DevOps tools\)](#)

[6.8 5.8 Environment and shell](#)

[6.9 5.9 SSH — remote access](#)

[6.10 5.10 Chapter summary](#)

[6.11 Lab 5.1 — Linux sandbox](#)

[6.12 Review questions](#)

7 Chapter 6: Processes, systemd, and Service Management

[7.1 6.1 Understanding Linux processes](#)

[7.2 6.2 systemd — modern service manager](#)

[7.3 6.3 Cron and timers](#)

[7.4 6.4 Resource limits and troubleshooting](#)

[7.5 6.5 Chapter summary](#)

[7.6 Lab 6.1](#)

8 Chapter 7: Networking on Linux — ip, ss, curl, dig

[8.1 7.1 Why networking skills matter for DevOps](#)

[8.2 7.2 The `ip` command suite](#)

[8.3 7.3 Sockets and connections with `ss`](#)

[8.4 7.4 HTTP debugging with `curl`](#)

[8.5 7.5 DNS with `dig`](#)

[8.6 7.6 Putting it together: diagnostic workflow](#)

[8.7 7.7 `/etc/hosts`, `/etc/resolv.conf`, and `systemd-resolved`](#)

[8.8 7.8 Firewalls at a glance](#)

[8.9 7.9 Chapter summary](#)

[8.10 Lab 7.1 — Local service bind address](#)

[8.11 Lab 7.2 — DNS and latency baseline](#)

[8.12 Review questions](#)

9 Chapter 8: Bash Scripting for Automation

[9.1 8.1 When bash is the right tool](#)

[9.2 8.2 Script anatomy and safety](#)

[9.3 8.3 Variables, quoting, and expansion](#)

[9.4 8.4 Conditionals and tests](#)

[9.5 8.5 Loops and functions](#)

[9.6 8.6 Exit codes and error handling](#)

[9.7 8.7 External commands and JSON](#)

[9.8 8.8 Logging and observability](#)

[9.9 8.9 Argument parsing](#)

[9.10 8.10 Script layout in repos](#)

[9.11 8.11 Security considerations](#)

[9.12 8.12 Chapter summary](#)

[9.13 Lab 8.1 — Health check script](#)

[9.14 Lab 8.2 — Batch service restart](#)

[9.15 Review questions](#)

10 Chapter 9: Git Internals — Objects, Refs, and the DAG

[10.1 9.1 Git is a content-addressable filesystem](#)

[10.2 9.2 How a commit is stored](#)

[10.3 9.3 Essential Git commands \(beyond basics\)](#)

[10.4 9.4 Branches and merging](#)

[10.5 9.5 Remote repositories](#)

[10.6 9.6 `.gitignore` and hooks](#)

[10.7 9.7 Chapter summary](#)

[10.8 Lab 9.1](#)

[11 Chapter 10: Branching Strategies and Code Review](#)

[11.1 10.1 Branches as a collaboration contract](#)

[11.2 10.2 Trunk-based development in depth](#)

[11.3 10.3 Git Flow and when it still fits](#)

[11.4 10.4 Pull requests and merge mechanics](#)

[11.5 10.5 Code review: purpose and culture](#)

[11.6 10.6 Size and speed metrics](#)

[11.7 10.7 Release tags and semantic versioning](#)

[11.8 10.8 Handling hotfixes](#)

[11.9 10.9 Chapter summary](#)

[11.10 Lab 10.1 — Strategy comparison](#)

[11.11 Lab 10.2 — Mock code review](#)

[11.12 Review questions](#)

[12 Chapter 11: GitHub/GitLab Workflows — PRs, Issues, Actions Intro](#)

[12.1 11.1 Platform-native Git workflows](#)

[12.2 11.2 Issues and project hygiene](#)

[12.3 11.3 Pull request lifecycle \(GitHub\)](#)

[12.4 11.4 CODEOWNERS and review routing](#)

[12.5 11.5 GitHub Actions introduction](#)

[12.6 11.6 GitLab CI introduction](#)

[12.7 11.7 Branch protection and environments](#)

[12.8 11.8 Fork and open-source workflows](#)

[12.9 11.9 Integrations DevOps teams use](#)

[12.10 11.10 Comparing Actions vs GitLab CI \(practical\)](#)

[12.11 11.11 CLI automation: gh and glab](#)

[12.12 11.12 Chapter summary](#)

[12.13 Lab 11.1 — First GitHub Actions workflow](#)

[12.14 Lab 11.2 — GitLab mirror \(optional\)](#)

[12.15 Review questions](#)

[13 Chapter 12: OSI Model, TCP/IP, DNS Deep Dive](#)

[13.1 12.1 Why models matter during incidents](#)

[13.2 12.2 OSI seven layers \(reference\)](#)

[13.3 12.3 TCP/IP four-layer model \(operational\)](#)

[13.4 12.4 IP addressing and CIDR](#)

[13.5 12.5 TCP vs UDP](#)

[13.6 12.6 DNS: the distributed phone book](#)

[13.7 12.7 DNS operations DevOps engineers perform](#)

[13.8 12.8 ICMP and path discovery](#)

[13.9 12.9 ARP and L2 in VPC context](#)

[13.10 12.10 Common failure patterns](#)

[13.11 12.11 Chapter summary](#)

[13.12 Lab 12.1 — DNS TTL experiment](#)

[13.13 Lab 12.2 — Layer mapping drill](#)

[13.14 Review questions](#)

[14 Chapter 13: HTTP/HTTPS, TLS, and Certificates](#)

[14.1 13.1 HTTP: the application protocol of the web](#)

[14.2 13.2 HTTP/1.1 vs HTTP/2 vs HTTP/3](#)

[14.3 13.3 HTTPS and TLS](#)

[14.4 13.4 Certificates and PKI](#)

[14.5 13.5 mTLS \(mutual TLS\)](#)

[14.6 13.6 Cipher suites and hardening](#)

[14.7 13.7 HTTP headers DevOps should know](#)

[14.8 13.8 Timeouts, keep-alive, and load balancers](#)

[14.9 13.9 Debugging TLS in production](#)

[14.10 13.10 Chapter summary](#)

[14.11 Lab 13.1 — Inspect a live certificate](#)

[14.12 Lab 13.2 — Broken chain simulation](#)

[14.13 Review questions](#)

[15 Chapter 14: Load Balancing, Proxies, and CDN](#)

[15.1 14.1 Why load balancing exists](#)

[15.2 14.2 Layer 4 vs Layer 7 load balancing](#)

[15.3 14.3 Load balancing algorithms](#)

[15.4 14.4 Reverse proxies vs forward proxies](#)

[15.5 14.5 Common proxy software](#)

[15.6 14.6 Session persistence \(stickiness\)](#)

[15.7 14.7 Content Delivery Networks \(CDN\)](#)

[15.8 14.8 Global load balancing and failover](#)

[15.9 14.9 Kubernetes ingress pattern](#)

[15.10 14.10 Observability for load balancers](#)

[15.11 14.11 Chapter summary](#)

[15.12 Lab 14.1 — nginx upstream failover](#)

[15.13 Lab 14.2 — CDN cache headers](#)

[15.14 Review questions](#)

[16 Chapter 15: Cloud Networking Patterns — VPC Overview](#)

[16.1 15.1 VPC: your private network in the cloud](#)

[16.2 15.2 CIDR design fundamentals](#)

[16.3 15.3 Public vs private subnets](#)

[16.4 15.4 Security groups vs NACLs](#)

[16.5 15.5 NAT, egress, and VPC endpoints](#)

[16.6 15.6 Hybrid and multi-VPC connectivity](#)

[16.7 15.7 Kubernetes networking in VPC](#)

[16.8 15.8 IaC example \(Terraform sketch\)](#)

[16.9 15.9 Operational checklist](#)

[16.10 15.10 Chapter summary](#)

[16.11 Lab 15.1 — Paper VPC design](#)

[16.12 Lab 15.2 — Terraform mini-VPC \(optional cloud\)](#)

[16.13 Review questions](#)

[17 Chapter 16: Python for DevOps — boto3, requests, pathlib](#)

[17.1 16.1 Why Python dominates DevOps automation](#)

[17.2 16.2 Project layout and virtual environments](#)

[17.3 16.3 pathlib: filesystem without footguns](#)

[17.4 16.4 requests: HTTP for APIs and health checks](#)

[17.5 16.5 boto3: AWS automation](#)

[17.6 16.6 Idempotency and dry-run patterns](#)

[17.7 16.7 CLI entry points with argparse](#)

[17.8 16.8 Logging instead of print](#)

[17.9 16.9 Testing Python automation](#)

[17.10 16.10 Security notes](#)

[17.11 16.11 Chapter summary](#)

[17.12 Lab 16.1 — EC2 inventory script](#)

[17.13 Lab 16.2 — Health gate for CI](#)

[17.14 Review questions](#)

[18 Chapter 17: Go for CLI Tools and Operators](#)

[18.1 17.1 Why Go in the DevOps toolchain](#)

[18.2 17.2 Module setup](#)

[18.3 17.3 A minimal CLI with cobra \(pattern\)](#)

[18.4 17.4 Error handling and exit codes](#)

[18.5 17.5 Concurrency for parallel probes](#)

[18.6 17.6 Kubernetes operators \(conceptual\)](#)

[18.7 17.7 Testing Go CLIs](#)

[18.8 17.8 Release and distribution](#)

[18.9 17.9 When not to use Go](#)

[18.10 17.10 Chapter summary](#)

[18.11 Lab 17.1 — Multi-target HTTP checker](#)

[18.12 Lab 17.2 — Cross-compile release](#)

[18.13 Review questions](#)

[19 Chapter 18: JSON/YAML, Jinja2, and Templating](#)

[19.1 18.1 Structured data in DevOps](#)

[19.2 18.2 JSON essentials](#)

[19.3 18.3 YAML essentials and pitfalls](#)

[19.4 18.4 JSON vs YAML: when to use which](#)

[19.5 18.5 Templating concepts](#)

[19.6 18.6 Jinja2 in Ansible](#)

[19.7 18.7 Helm and Go templates \(brief\)](#)

[19.8 18.8 Schema validation](#)

[19.9 18.9 Secrets and templating](#)

[19.10 18.10 Chapter summary](#)

[19.11 Lab 18.1 — Render nginx from Jinja2](#)

[19.12 Lab 18.2 — YAML lint pipeline](#)

[19.13 Review questions](#)

[20 Chapter 19: Testing Automation Scripts](#)

[20.1 19.1 Why test automation code?](#)

[20.2 19.2 Test pyramid for DevOps scripts](#)

[20.3 19.3 Testing bash scripts](#)

[20.4 19.4 Testing Python automation](#)

[20.5 19.5 Testing Go CLIs](#)

[20.6 19.6 Integration testing with containers](#)

[20.7 19.7 Testing Terraform and Ansible](#)

[20.8 19.8 CI pipeline layout for script repos](#)

[20.9 19.9 Test data and fixtures](#)

[20.10 19.10 When tests are enough](#)

[20.11 19.11 Chapter summary](#)

[20.12 Lab 19.1 — pytest suite for inventory script](#)

[20.13 Lab 19.2 — bats for bash healthcheck](#)

[20.14 Review questions](#)

[21 Chapter 20: IaC Principles — State, Idempotency, and Drift](#)

[21.1 20.1 What is Infrastructure as Code?](#)

[21.2 20.2 Declarative vs imperative](#)

[21.3 20.3 Idempotency](#)

[21.4 20.4 State management](#)

[21.5 20.5 Configuration drift](#)

[21.6 20.6 IaC workflow in teams](#)

[21.7 20.7 Testing IaC](#)

[21.8 20.8 Chapter summary](#)

[21.9 Lab 20.1](#)

[22 Chapter 21: Terraform — HCL, Providers, and Modules](#)

[22.1 21.1 Terraform overview](#)

[22.2 21.2 Basic configuration](#)

[22.3 21.3 Resources, data sources, and dependencies](#)

[22.4 21.4 Modules — reusable infrastructure](#)

[22.5 21.5 Lifecycle meta-arguments](#)

[22.6 21.6 Workspaces and environments](#)

[22.7 21.7 CI/CD integration](#)

[22.8 21.8 Chapter summary](#)

[22.9 Lab 21.1](#)

[23 Chapter 22: Terraform — Workspaces, Backends, and CI Integration](#)

[23.1 22.1 Why remote state and backends matter](#)

[23.2 22.2 Configuring S3 backend with locking](#)

[23.3 22.3 Terraform workspaces](#)

[23.4 22.4 State operations and safety](#)

[23.5 22.5 CI/CD integration patterns](#)

[23.6 22.6 Atlantis and Terraform Cloud](#)

[23.7 22.7 Testing and policy in the pipeline](#)

[23.8 22.8 Chapter summary](#)

[23.9 Lab 22.1](#)

[23.10 Lab 22.2](#)

[23.11 Review questions](#)

[24 Chapter 23: Ansible — Playbooks, Roles, and Inventories](#)

[24.1 23.1 Ansible in the IaC landscape](#)

[24.2 23.2 Inventory — defining targets](#)

[24.3 23.3 Playbooks — ordered automation](#)

[24.4 23.4 Modules, idempotency, and handlers](#)

[24.5 23.5 Roles — reusable bundles](#)

[24.6 23.6 Variables and facts precedence](#)

[24.7 23.7 Vault, collections, and project structure](#)

[24.8 23.8 Ansible in CI/CD](#)

[24.9 23.9 Chapter summary](#)

[24.10 Lab 23.1](#)

[24.11 Lab 23.2](#)

[24.12 Review questions](#)

[25 Chapter 24: Packer & Golden Images](#)

[25.1 24.1 Golden images and the immutable infrastructure pattern](#)

[25.2 24.2 Packer workflow](#)

[25.3 24.3 AWS EBS builder example](#)

[25.4 24.4 Provisioners](#)

[25.5 24.5 Multi-region and shared images](#)

[25.6 24.6 Versioning, promotion, and CI](#)

[25.7 24.7 Security and compliance](#)

[25.8 24.8 Chapter summary](#)

[25.9 Lab 24.1](#)

[25.10 Lab 24.2](#)

[25.11 Review questions](#)

[26 Chapter 25: Policy as Code — OPA, Checkov, and Sentinel](#)

[26.1 25.1 Why policy as code?](#)

[26.2 25.2 Policy evaluation points](#)

[26.3 25.3 Checkov — static IaC scanning](#)

[26.4 25.4 Open Policy Agent \(OPA\) and Rego](#)

[26.5 25.5 HashiCorp Sentinel \(Terraform Cloud/Enterprise\)](#)

[26.6 25.6 Comparison matrix](#)

[26.7 25.7 Building a policy program](#)

[26.8 25.8 CI pipeline integration](#)

[26.9 25.9 Chapter summary](#)

[26.10 Lab 25.1](#)

[26.11 Lab 25.2](#)

[26.12 Review questions](#)

[27 Chapter 26: Docker — Images, Dockerfile, and Multi-stage Builds](#)

[27.1 26.1 Containers in one paragraph](#)

[27.2 26.2 Core concepts](#)

[27.3 26.3 Essential commands](#)

[27.4 26.4 Dockerfile best practices](#)

[27.5 26.5 Docker Compose for local development](#)

[27.6 26.6 Security basics](#)

[27.7 26.7 Chapter summary](#)

[27.8 Lab 26.1](#)

[28 Chapter 27: Container Networking, Volumes, and Security](#)

[28.1 27.1 Container networking model](#)

[28.2 27.2 Docker network drivers](#)

[28.3 27.3 Container-to-container and host communication](#)

[28.4 27.4 Volumes and persistence](#)

[28.5 27.5 Linux capabilities and seccomp](#)

[28.6 27.6 Image and runtime security](#)
[28.7 27.7 Resource limits and health](#)
[28.8 27.8 Debugging networking issues](#)
[28.9 27.9 Chapter summary](#)
[28.10 Lab 27.1](#)
[28.11 Lab 27.2](#)
[28.12 Review questions](#)

[29 Chapter 28: Docker Compose and Local Dev Environments](#)

[29.1 28.1 What Docker Compose solves](#)
[29.2 28.2 Compose file structure](#)
[29.3 28.3 Essential commands](#)
[29.4 28.4 Environment-specific overrides](#)
[29.5 28.5 Networking in Compose](#)
[29.6 28.6 Development workflows](#)
[29.7 28.7 Testing with Compose](#)
[29.8 28.8 Compose vs production](#)
[29.9 28.9 Chapter summary](#)
[29.10 Lab 28.1](#)
[29.11 Lab 28.2](#)
[29.12 Review questions](#)

[30 Chapter 29: Container Registries and Image Signing](#)

[30.1 29.1 Container registries overview](#)
[30.2 29.2 Image naming and tagging strategy](#)
[30.3 29.3 Authenticating to registries](#)
[30.4 29.4 Pull policies in Kubernetes and beyond](#)
[30.5 29.5 Vulnerability scanning](#)
[30.6 29.6 Image signing and verification](#)
[30.7 29.7 Registry operations](#)
[30.8 29.8 Supply chain best practices](#)
[30.9 29.9 Chapter summary](#)
[30.10 Lab 29.1](#)
[30.11 Lab 29.2](#)
[30.12 Review questions](#)

[31 Chapter 30: Rootless Containers and Podman](#)

[31.1 30.1 Why rootless containers matter](#)
[31.2 30.2 Podman overview](#)
[31.3 30.3 Rootless setup](#)
[31.4 30.4 Pods and systemd integration](#)
[31.5 30.5 Podman Compose and Kubernetes YAML](#)
[31.6 30.6 Rootless Docker \(Moby\)](#)
[31.7 30.7 Security comparison](#)
[31.8 30.8 Buildah and Skopeo](#)
[31.9 30.9 When to use Podman vs Docker vs Kubernetes](#)
[31.10 30.10 Chapter summary](#)
[31.11 Lab 30.1](#)
[31.12 Lab 30.2](#)
[31.13 Review questions](#)

[32 Chapter 31: Kubernetes Architecture — Control Plane and Node Components](#)

[32.1 31.1 What Kubernetes provides](#)
[32.2 31.2 Cluster architecture overview](#)
[32.3 31.3 Control plane components](#)
[32.4 31.4 Node components](#)
[32.5 31.5 Kubernetes objects and namespaces](#)
[32.6 31.6 kubectl and kubeconfig](#)
[32.7 31.7 Managed vs self-managed clusters](#)
[32.8 31.8 Addons and the CNCF landscape](#)
[32.9 31.9 High availability and etcd sizing](#)
[32.10 31.10 Chapter summary](#)

[32.11 Lab 31.1](#)

[32.12 Lab 31.2](#)

[32.13 Review questions](#)

[33 Chapter 32: Kubernetes Workloads — Pod, Deployment, StatefulSet, DaemonSet](#)

[33.1 32.1 Pods — the atomic unit](#)

[33.2 32.2 Deployments — stateless apps](#)

[33.3 32.3 ReplicaSets and labels](#)

[33.4 32.4 StatefulSets — stable identity and storage](#)

[33.5 32.5 DaemonSets — one pod per node](#)

[33.6 32.6 Jobs and CronJobs](#)

[33.7 32.7 Choosing a workload type](#)

[33.8 32.8 Pod scheduling controls](#)

[33.9 32.9 Chapter summary](#)

[33.10 Lab 32.1](#)

[33.11 Lab 32.2](#)

[33.12 Review questions](#)

[34 Chapter 33: Kubernetes Services, Ingress, and Network Policies](#)

[34.1 33.1 Kubernetes networking model](#)

[34.2 33.2 Service types](#)

[34.3 33.3 Endpoints and EndpointSlices](#)

[34.4 33.4 Ingress — HTTP routing](#)

[34.5 33.5 LoadBalancer and NodePort in cloud](#)

[34.6 33.6 Gateway API \(modern alternative\)](#)

[34.7 33.7 Network Policies — zero trust inside the cluster](#)

[34.8 33.8 Debugging connectivity](#)

[34.9 33.9 Chapter summary](#)

[34.10 Lab 33.1](#)

[34.11 Lab 33.2](#)

[34.12 Review questions](#)

[35 Chapter 34: Kubernetes ConfigMaps, Secrets, and Storage \(PV/PVC\)](#)

[35.1 34.1 Separating config from images](#)

[35.2 34.2 ConfigMaps](#)

[35.3 34.3 Secrets](#)

[35.4 34.4 Storage basics — PV, PVC, StorageClass](#)

[35.5 34.5 StatefulSet storage pattern](#)

[35.6 34.6 Config reload and updates](#)

[35.7 34.7 Backup and snapshots](#)

[35.8 34.8 Security checklist](#)

[35.9 34.9 Chapter summary](#)

[35.10 Lab 34.1](#)

[35.11 Lab 34.2](#)

[35.12 Review questions](#)

[36 Chapter 35: Helm, Kustomize, and GitOps \(Argo CD\)](#)

[36.1 35.1 The packaging problem](#)

[36.2 35.2 Helm charts](#)

[36.3 35.3 Helm hooks and releases](#)

[36.4 35.4 Kustomize — patch without templates](#)

[36.5 35.5 GitOps principles](#)

[36.6 35.6 Argo CD](#)

[36.7 35.7 Promotion pipeline](#)

[36.8 35.8 Flux \(brief comparison\)](#)

[36.9 35.9 Secrets in GitOps](#)

[36.10 35.10 Chapter summary](#)

[36.11 Lab 35.1](#)

[36.12 Lab 35.2](#)

[36.13 Review questions](#)

[37 Chapter 36: Kubernetes RBAC, Pod Security, and Hardening](#)

[37.1 36.1 Defense in depth for Kubernetes](#)

[37.2 36.2 Authentication overview](#)
[37.3 36.3 RBAC — Roles and Bindings](#)
[37.4 36.4 Aggregated roles and default bindings](#)
[37.5 36.5 Pod Security Admission \(PSA\)](#)
[37.6 36.6 OPA Gatekeeper and Kyverno](#)
[37.7 36.7 Workload hardening checklist](#)
[37.8 36.8 Secrets, audit logs, and API security](#)
[37.9 36.9 Node and supply chain security](#)
[37.10 36.10 Incident response primitives](#)
[37.11 36.11 Chapter summary](#)
[37.12 Lab 36.1](#)
[37.13 Lab 36.2](#)
[37.14 Review questions](#)

38 Chapter 37: CI/CD Concepts and Pipeline Design

[38.1 37.1 Continuous Integration \(CI\)](#)
[38.2 37.2 Continuous Delivery vs Continuous Deployment](#)
[38.3 37.3 Pipeline stages](#)
[38.4 37.4 GitHub Actions example](#)
[38.5 37.5 Branching and trunk-based development](#)
[38.6 37.6 Deployment strategies](#)
[38.7 37.7 Pipeline secrets and artifacts](#)
[38.8 37.8 DORA metrics in your pipeline](#)
[38.9 37.9 Chapter summary](#)
[38.10 Lab 37.1](#)

39 Chapter 38: GitHub Actions — Workflows, Secrets, Runners

[39.1 38.1 GitHub Actions architecture](#)
[39.2 38.2 Workflow syntax fundamentals](#)
[39.3 38.3 Reusable workflows and composite actions](#)
[39.4 38.4 Secrets, variables, and security](#)
[39.5 38.5 Runners: hosted vs self-hosted](#)
[39.6 38.6 Matrix builds, caching, and artifacts](#)
[39.7 38.7 Environments, deployments, and approvals](#)
[39.8 38.8 Debugging and troubleshooting](#)
[39.9 38.9 Chapter summary](#)
[39.10 Lab 38.1 — Multi-environment deploy pipeline](#)
[39.11 Lab 38.2 — Self-hosted runner in Docker](#)
[39.12 Review questions](#)

40 Chapter 39: Jenkins Pipelines as Code

[40.1 39.1 Jenkins in the modern CI/CD landscape](#)
[40.2 39.2 Architecture: controller, agents, and plugins](#)
[40.3 39.3 Declarative Pipeline structure](#)
[40.4 39.4 Agents and parallelism](#)
[40.5 39.5 Credentials, parameters, and shared libraries](#)
[40.6 39.6 Multibranch and organization folders](#)
[40.7 39.7 Pipeline durability and best practices](#)
[40.8 39.8 Jenkins vs cloud-native CI \(when to stay\)](#)
[40.9 39.9 Chapter summary](#)
[40.10 Lab 39.1 — Declarative pipeline from scratch](#)
[40.11 Lab 39.2 — Kubernetes ephemeral agents](#)
[40.12 Review questions](#)

41 Chapter 40: GitLab CI and Multi-platform Comparison

[41.1 40.1 GitLab CI/CD overview](#)
[41.2 40.2 .gitlab-ci.yml fundamentals](#)
[41.3 40.3 Runners, executors, and isolation](#)
[41.4 40.4 Variables, environments, and child pipelines](#)
[41.5 40.5 GitLab-specific features](#)
[41.6 40.6 Multi-platform CI/CD comparison](#)
[41.7 40.7 Portable pipeline patterns](#)

[41.8 40.8 Migration considerations](#)

[41.9 40.9 Chapter summary](#)

[41.10 Lab 40.1 — GitLab pipeline with services](#)

[41.11 Lab 40.2 — Platform comparison exercise](#)

[41.12 Review questions](#)

[42 Chapter 41: Deployment Strategies — Blue/Green, Canary, Feature Flags](#)

[42.1 41.1 Why deployment strategy matters](#)

[42.2 41.2 Rolling deployment](#)

[42.3 41.3 Recreate deployment](#)

[42.4 41.4 Blue/green deployment](#)

[42.5 41.5 Canary deployment](#)

[42.6 41.6 Feature flags \(dark launches\)](#)

[42.7 41.7 Database migrations and deployment coupling](#)

[42.8 41.8 Choosing a strategy](#)

[42.9 41.9 Chapter summary](#)

[42.10 Lab 41.1 — Kubernetes rolling vs recreate](#)

[42.11 Lab 41.2 — Feature flag rollout](#)

[42.12 Review questions](#)

[43 Chapter 42: DORA Metrics and Continuous Improvement](#)

[43.1 42.1 What DORA measures](#)

[43.2 42.2 Why these four metrics matter](#)

[43.3 42.3 Measuring deployment frequency](#)

[43.4 42.4 Measuring lead time for changes](#)

[43.5 42.5 Change failure rate](#)

[43.6 42.6 Time to restore service \(MTTR\)](#)

[43.7 42.7 Performance clusters and benchmarking](#)

[43.8 42.8 Building a metrics program](#)

[43.9 42.9 Continuous improvement loops](#)

[43.10 42.10 Chapter summary](#)

[43.11 Lab 42.1 — Deployment event pipeline](#)

[43.12 Lab 42.2 — Lead time value stream map](#)

[43.13 Review questions](#)

[44 Chapter 43: Monitoring Fundamentals — Metrics, Logs, Traces](#)

[44.1 43.1 From monitoring to observability](#)

[44.2 43.2 Metrics: types and best practices](#)

[44.3 43.3 Logs: structure and aggregation](#)

[44.4 43.4 Traces: distributed request flow](#)

[44.5 43.5 The observability data pipeline](#)

[44.6 43.6 Alerting philosophy](#)

[44.7 43.7 Golden signals and SLIs](#)

[44.8 43.8 Instrumentation ownership](#)

[44.9 43.9 Chapter summary](#)

[44.10 Lab 43.1 — RED metrics for a sample API](#)

[44.11 Lab 43.2 — Correlate logs and traces](#)

[44.12 Review questions](#)

[45 Chapter 44: Prometheus & Grafana Stack](#)

[45.1 44.1 The Prometheus ecosystem](#)

[45.2 44.2 Prometheus architecture](#)

[45.3 44.3 Prometheus configuration](#)

[45.4 44.4 PromQL essentials](#)

[45.5 44.5 Alerting rules and Alertmanager](#)

[45.6 44.6 Grafana dashboards](#)

[45.7 44.7 Kubernetes deployment patterns](#)

[45.8 44.8 Exporters and instrumentation](#)

[45.9 44.9 Operations and troubleshooting](#)

[45.10 44.10 Chapter summary](#)

[45.11 Lab 44.1 — Local Prometheus + Grafana](#)

[45.12 Lab 44.2 — ServiceMonitor on Kubernetes](#)

[45.13 Review questions](#)

[46 Chapter 45: ELK/EFK Log Aggregation](#)

[46.1 45.1 Centralized logging rationale](#)

[46.2 45.2 Architecture overview](#)

[46.3 45.3 Elasticsearch fundamentals](#)

[46.4 45.4 Fluent Bit vs Logstash vs Filebeat](#)

[46.5 45.5 Logstash pipelines](#)

[46.6 45.6 Kibana for search and dashboards](#)

[46.7 45.7 EFK on Kubernetes \(ECK\)](#)

[46.8 45.8 Security and compliance](#)

[46.9 45.9 Loki as an alternative](#)

[46.10 45.10 Operational best practices](#)

[46.11 45.11 Chapter summary](#)

[46.12 Lab 45.1 — Docker Compose ELK](#)

[46.13 Lab 45.2 — Fluent Bit on Kubernetes](#)

[46.14 Review questions](#)

[47 Chapter 46: OpenTelemetry Distributed Tracing](#)

[47.1 46.1 The tracing problem](#)

[47.2 46.2 Traces, spans, and context propagation](#)

[47.3 46.3 OpenTelemetry architecture](#)

[47.4 46.4 Instrumentation: auto vs manual](#)

[47.5 46.5 OTel Collector configuration](#)

[47.6 46.6 Sampling strategies](#)

[47.7 46.7 Backends: Jaeger and Grafana Tempo](#)

[47.8 46.8 Correlating traces, logs, and metrics](#)

[47.9 46.9 Service mesh tracing](#)

[47.10 46.10 Adoption roadmap](#)

[47.11 46.11 Chapter summary](#)

[47.12 Lab 46.1 — OTel + Jaeger locally](#)

[47.13 Lab 46.2 — Tail sampling](#)

[47.14 Review questions](#)

[48 Chapter 47: SLOs, Error Budgets, and Incident Response](#)

[48.1 47.1 Reliability as a product decision](#)

[48.2 47.2 Choosing SLIs and SLOs](#)

[48.3 47.3 Error budgets in practice](#)

[48.4 47.4 Incident severity and lifecycle](#)

[48.5 47.5 On-call and escalation](#)

[48.6 47.6 Incident response procedures](#)

[48.7 47.7 Blameless postmortems](#)

[48.8 47.8 Status communication](#)

[48.9 47.9 Tooling integration](#)

[48.10 47.10 Chapter summary](#)

[48.11 Lab 47.1 — Define SLOs for a sample service](#)

[48.12 Lab 47.2 — Tabletop incident exercise](#)

[48.13 Review questions](#)

[49 Chapter 48: Chaos Engineering and Game Days](#)

[49.1 48.1 Proactive resilience testing](#)

[49.2 48.2 Principles of chaos engineering](#)

[49.3 48.3 Failure modes to inject](#)

[49.4 48.4 Chaos tools landscape](#)

[49.5 48.5 Safe experimentation guardrails](#)

[49.6 48.6 Game days](#)

[49.7 48.7 From experiment to improvement](#)

[49.8 48.8 Organizational maturity](#)

[49.9 48.9 Chapter summary](#)

[49.10 Lab 48.1 — Pod delete experiment](#)

[49.11 Lab 48.2 — Mini game day](#)

[49.12 Review questions](#)

50 Chapter 49: Shift-Left Security in the Pipeline

[50.1 49.1 DevSecOps and shift-left](#)

[50.2 49.2 Security activities across the SDLC](#)

[50.3 49.3 Threat modeling fundamentals](#)

[50.4 49.4 Pipeline security gates](#)

[50.5 49.5 Secret scanning](#)

[50.6 49.6 Secure defaults and platform guardrails](#)

[50.7 49.7 Security champions program](#)

[50.8 49.8 Compliance integration](#)

[50.9 49.9 Culture and psychological safety](#)

[50.10 49.10 Chapter summary](#)

[50.11 Lab 49.1 — Pre-commit security stack](#)

[50.12 Lab 49.2 — Threat model exercise](#)

[50.13 Review questions](#)

51 Chapter 50: SAST, DAST, SCA, and Container Scanning

[51.1 50.1 Application security testing taxonomy](#)

[51.2 50.2 SAST — static analysis](#)

[51.3 50.3 DAST — dynamic testing](#)

[51.4 50.4 SCA — dependency analysis](#)

[51.5 50.5 Container and image scanning](#)

[51.6 50.6 IaC and Kubernetes scanning](#)

[51.7 50.7 Unified vulnerability management](#)

[51.8 50.8 Reducing noise](#)

[51.9 50.9 Chapter summary](#)

[51.10 Lab 50.1 — Full scan pipeline](#)

[51.11 Lab 50.2 — DAST against staging](#)

[51.12 Review questions](#)

52 Chapter 51: Secrets Management — Vault, SOPS, External Secrets

[52.1 51.1 The secrets problem](#)

[52.2 51.2 HashiCorp Vault architecture](#)

[52.3 51.3 Vault on Kubernetes](#)

[52.4 51.4 Mozilla SOPS — Git-friendly encryption](#)

[52.5 51.5 External Secrets Operator \(ESO\)](#)

[52.6 51.6 Cloud-native secret managers](#)

[52.7 51.7 CI/CD secrets patterns](#)

[52.8 51.8 Rotation and emergency response](#)

[52.9 51.9 Chapter summary](#)

[52.10 Lab 51.1 — SOPS with age](#)

[52.11 Lab 51.2 — External Secrets on Kubernetes](#)

[52.12 Review questions](#)

53 Chapter 52: Supply Chain Security — SBOM, Sigstore, SLSA

[53.1 52.1 Software supply chain threats](#)

[53.2 52.2 SBOM — Software Bill of Materials](#)

[53.3 52.3 SLSA — supply chain levels](#)

[53.4 52.4 Sigstore and cosign](#)

[53.5 52.5 Dependency and build hardening](#)

[53.6 52.6 in-toto and attestations](#)

[53.7 52.7 Organizational supply chain program](#)

[53.8 52.8 Chapter summary](#)

[53.9 Lab 52.1 — SBOM generation and CVE lookup](#)

[53.10 Lab 52.2 — Sign and verify with cosign](#)

[53.11 Review questions](#)

54 Chapter 53: Runtime Security — Falco and OPA Gatekeeper

[54.1 53.1 Defense beyond the pipeline](#)

[54.2 53.2 Falco — cloud-native runtime detection](#)

[54.3 53.3 Falco response actions](#)

[54.4 53.4 OPA and Gatekeeper](#)

[54.5 53.5 Policy libraries and Pod Security](#)

[54.6 53.6 Runtime security program](#)

[54.7 53.7 Chapter summary](#)

[54.8 Lab 53.1 — Falco shell detection](#)

[54.9 Lab 53.2 — Gatekeeper privileged block](#)

[54.10 Review questions](#)

[55 Chapter 54: Compliance Automation and Audit Trails](#)

[55.1 54.1 Compliance in continuous delivery](#)

[55.2 54.2 Control frameworks mapping](#)

[55.3 54.3 Audit trails — what to log](#)

[55.4 54.4 Policy as Code for compliance](#)

[55.5 54.5 Continuous control monitoring](#)

[55.6 54.6 Change management automation](#)

[55.7 54.7 Data privacy and retention](#)

[55.8 54.8 Audit preparation workflow](#)

[55.9 54.9 Chapter summary](#)

[55.10 Lab 54.1 — Kubernetes audit logging](#)

[55.11 Lab 54.2 — Compliance evidence pipeline](#)

[55.12 Review questions](#)

[56 Chapter 55: Platform Engineering and Internal Developer Platforms](#)

[56.1 55.1 From DevOps to platform engineering](#)

[56.2 55.2 Internal Developer Platform components](#)

[56.3 55.3 Backstage — reference IDP portal](#)

[56.4 55.4 Golden paths and paved roads](#)

[56.5 55.5 Platform as a product](#)

[56.6 55.6 Infrastructure abstractions](#)

[56.7 55.7 Developer experience metrics](#)

[56.8 55.8 Anti-patterns](#)

[56.9 55.9 Chapter summary](#)

[56.10 Lab 55.1 — Backstage software catalog](#)

[56.11 Lab 55.2 — Golden path template](#)

[56.12 Review questions](#)

[57 Chapter 56: Multi-Cloud and Hybrid Patterns](#)

[57.1 56.1 Why multi-cloud and hybrid exist](#)

[57.2 56.2 Architecture patterns](#)

[57.3 56.3 Portability strategies](#)

[57.4 56.4 Networking across clouds and on-prem](#)

[57.5 56.5 GitOps and multi-cluster management](#)

[57.6 56.6 Data and state in multi-cloud](#)

[57.7 56.7 Operational model](#)

[57.8 56.8 When NOT to multi-cloud](#)

[57.9 56.9 Chapter summary](#)

[57.10 Lab 56.1 — Terraform multi-provider module](#)

[57.11 Lab 56.2 — Argo CD ApplicationSet](#)

[57.12 Review questions](#)

[58 Chapter 57: FinOps and Cost-Aware Engineering](#)

[58.1 57.1 Cloud cost as engineering concern](#)

[58.2 57.2 Cost visibility and allocation](#)

[58.3 57.3 Right-sizing and waste elimination](#)

[58.4 57.4 Pricing models and savings](#)

[58.5 57.5 Cost in CI/CD and IaC](#)

[58.6 57.6 Unit economics](#)

[58.7 57.7 FinOps maturity stages](#)

[58.8 57.8 Chapter summary](#)

[58.9 Lab 57.1 — Cost allocation tags](#)

[58.10 Lab 57.2 — Infracost in CI](#)

[58.11 Review questions](#)

[59 Chapter 58: MLOps Overview for DevOps Engineers](#)

[59.1 58.1 Why DevOps engineers need MLOps](#)

[59.2 58.2 ML lifecycle vs traditional SDLC](#)
[59.3 58.3 Key MLOps concepts](#)
[59.4 58.4 Reproducible training pipeline](#)
[59.5 58.5 Model deployment patterns](#)
[59.6 58.6 Feature stores and training-serving skew](#)
[59.7 58.7 Monitoring ML in production](#)
[59.8 58.8 Governance and responsible ML](#)
[59.9 58.9 DevOps + MLOps collaboration](#)
[59.10 58.10 Chapter summary](#)
[59.11 Lab 58.1 — MLflow end-to-end](#)
[59.12 Lab 58.2 — Model monitoring dashboard](#)
[59.13 Review questions](#)

60 Chapter 59: Capstone — End-to-End Production System Design

[60.1 59.1 Capstone objectives](#)
[60.2 59.2 Architecture overview](#)
[60.3 59.3 Repository and team structure](#)
[60.4 59.4 CI/CD pipeline design](#)
[60.5 59.5 Infrastructure as Code](#)
[60.6 59.6 Security architecture](#)
[60.7 59.7 Observability and SRE](#)
[60.8 59.8 Data, DR, and scaling](#)
[60.9 59.9 FinOps and platform metrics](#)
[60.10 59.10 Capstone delivery roadmap](#)
[60.11 59.11 Design review checklist](#)
[60.12 59.12 Chapter summary](#)
[60.13 Capstone Lab — Mini ShopStream](#)
[60.14 Review questions](#)

61 Chapter 60: Appendix — Cheatsheets, Glossary, and Index

[61.1 60.1 kubectrl quick reference](#)
[61.2 60.2 Docker quick reference](#)
[61.3 60.3 Terraform quick reference](#)
[61.4 60.4 Git essentials](#)
[61.5 60.5 Linux troubleshooting](#)
[61.6 60.6 PromQL cheatsheet](#)
[61.7 60.7 CI/CD YAML snippets](#)
[61.8 60.8 Security tool commands](#)
[61.9 60.9 Observability checklist](#)
[61.10 60.10 SLO error budget reference](#)
[61.11 60.11 Glossary](#)
[61.12 60.12 Handbook chapter index](#)
[61.13 60.13 Recommended reading](#)
[61.14 60.14 Certification paths \(optional\)](#)
[61.15 60.15 Final review questions \(handbook-wide\)](#)
[61.16 60.16 Chapter summary](#)

1 DevOps Handbook — Master Table of Contents

Target length: ~1,200 pages (~480,000 words)
Audience: Beginner → Advanced
Format: Each numbered section ≈ one “page” of ~400 words in the final edition

1.1 How to read this handbook

Work through **Parts I-XII** in order. Labs (marked) add hands-on time beyond page counts. Cross-reference the [AWS Handbook](#) for cloud-specific depth.

1.2 Part I — Introduction & Mindset (Pages 1-60)

Ch	Title	Pages
1	What Is DevOps? History and Culture	1-15
2	CALMS, Three Ways, and Team Topologies	16-30
3	DevOps vs SRE vs Platform Engineering	31-42
4	Career Paths, Certifications, and Learning Strategy	43-60

Written: [Chapters 1-4](#) (Introduction complete)

1.3 Part II — Linux & Shell Mastery (Pages 61-140)

Ch	Title	Pages
5	Linux Fundamentals: Filesystem, Users, Permissions	61-85
6	Processes, systemd, and Service Management	86-105
7	Networking on Linux: ip, ss, curl, dig	106-120
8	Bash Scripting for Automation	121-140

Written: [Chapter 5](#), [Chapter 6](#), [Chapter 7](#), [Chapter 8](#)

1.4 Part III — Git & Collaboration (Pages 141-200)

Ch	Title	Pages
9	Git Internals: Objects, Refs, and the DAG	141-158
10	Branching Strategies and Code Review	159-175
11	GitHub/GitLab Workflows: PRs, Issues, Actions intro	176-200

Written: [Chapter 9](#), [Chapter 10](#), [Chapter 11](#)

1.5 Part IV — Networking for DevOps (Pages 201-280)

Ch	Title	Pages
12	OSI Model, TCP/IP, DNS Deep Dive	201-225
13	HTTP/HTTPS, TLS, and Certificates	226-245
14	Load Balancing, Proxies, and CDN Concepts	246-265
15	Cloud Networking Patterns (VPC overview)	266-280

Written: [Chapter 12](#) through [Chapter 15](#)

1.6 Part V — Scripting & Programming (Pages 281-360)

Ch	Title	Pages
16	Python for DevOps: boto3, requests, pathlib	281-305
17	Go for CLI Tools and Operators	306-325
18	JSON/YAML, Jinja2, and Templating	326-345
19	Testing Automation Scripts	346-360

Written: [Chapter 16](#) through [Chapter 19](#)

1.7 Part VI — Infrastructure as Code (Pages 361-480)

Ch	Title	Pages
20	IaC Principles: State, Idempotency, Drift	361-380
21	Terraform: HCL, Providers, Modules	381-410
22	Terraform: Workspaces, Backends, CI Integration	411-430
23	Ansible: Playbooks, Roles, Inventories	431-450
24	Packer & Golden Images	451-465
25	Policy as Code: OPA, Checkov, Sentinel	466-480

1.8 Part VII — Containers (Pages 481-580)

Ch	Title	Pages
26	Docker: Images, Dockerfile, Multi-stage Builds	481-505
27	Container Networking, Volumes, and Security	506-525
28	Docker Compose and Local Dev Environments	526-540
29	Container Registries and Image Signing	541-560
30	Rootless Containers and Alternatives (Podman)	561-580

1.9 Part VIII — Kubernetes (Pages 581-720)

Ch	Title	Pages
31	K8s Architecture: Control Plane & Node Components	581-605
32	Workloads: Pod, Deployment, StatefulSet, DaemonSet	606-630
33	Services, Ingress, and Network Policies	631-655
34	ConfigMaps, Secrets, and Storage (PV/PVC)	656-675
35	Helm, Kustomize, and GitOps (Argo CD)	676-700
36	RBAC, Pod Security, and Hardening	701-720

1.10 Part IX — CI/CD (Pages 721-840)

Ch	Title	Pages
37	CI/CD Concepts and Pipeline Design	721-745
38	GitHub Actions: Workflows, Secrets, Runners	746-770
39	Jenkins: Pipelines as Code (Declarative)	771-790
40	GitLab CI and Multi-platform Comparison	791-810
41	Deployment Strategies: Blue/Green, Canary, Feature Flags	811-825
42	DORA Metrics and Continuous Improvement	826-840

1.11 Part X — Observability & SRE (Pages 841-960)

Ch	Title	Pages
43	Monitoring Fundamentals: Metrics, Logs, Traces	841-865
44	Prometheus & Grafana Stack	866-890
45	ELK/EFK and Log Aggregation	891-910
46	OpenTelemetry and Distributed Tracing	911-930
47	SLOs, Error Budgets, and Incident Response	931-945
48	Chaos Engineering and Game Days	946-960

1.12 Part XI — Security (DevSecOps) (Pages 961-1080)

Ch	Title	Pages
49	Shift-Left Security in the Pipeline	961-980
50	SAST, DAST, SCA, and Container Scanning	981-1000
51	Secrets Management: Vault, SOPS, External Secrets	1001-1020
52	Supply Chain: SBOM, Sigstore, SLSA	1021-1040

53	Runtime Security: Falco, OPA Gatekeeper	1041-1060
54	Compliance Automation and Audit Trails	1061-1080

1.13 Part XII — Advanced Topics (Pages 1081-1200)

Ch	Title	Pages
55	Platform Engineering & Internal Developer Platforms	1081-1100
56	Multi-Cloud and Hybrid Patterns	1101-1115
57	FinOps and Cost-Aware Engineering	1116-1130
58	MLOps Overview for DevOps Engineers	1131-1145
59	Capstone: End-to-End Production System Design	1146-1180
60	Appendix: Command Cheatsheets, Glossary, Index	1181-1200

1.14 Expansion status

Part	Status
I	Ch 1-4 complete
II	Ch 5-8 complete
III	Ch 9-11 complete
IV	Ch 12-15 complete
V	Ch 16-19 complete
VI-XII	Ch 20-21, 26, 37 partial (~remaining chapters)

To request a chapter: *“Expand DevOps Handbook Chapter 21 (Terraform modules)”*

2 Chapter 1: What Is DevOps? History and Culture

DevOps Handbook — Part I, Pages 1–15

2.1 1.1 The problem DevOps solves

Before DevOps, software organizations often split into two camps:

- **Development** — wrote features quickly, measured by release velocity.
- **Operations** — kept systems stable, measured by uptime and incident count.

These goals frequently conflicted. Developers threw code “over the wall”; operators resisted change because every deployment risked an outage. Releases were rare, painful, and manual. When something broke in production, blame flowed in both directions.

DevOps is not a single tool or job title. It is a **cultural and technical movement** that unifies development and operations around shared goals: deliver value to users **fast** and **reliably**.

2.2 1.2 A working definition

***DevOps** is the practice of aligning people, processes, and technology so that software can be built, tested, released, and operated continuously—with feedback loops that shorten the time from idea to production.*

Key outcomes:

Outcome	What it means
Shorter lead time	Hours or days from commit to production, not months
Higher deployment frequency	Many small releases instead of rare big-bang releases
Lower change failure rate	Most changes succeed without rollback
Faster recovery	When failures happen, restore service quickly

These four metrics are known as **DORA metrics** (DevOps Research and Assessment) and are the standard way to measure DevOps maturity.

2.3 1.3 Brief history

Year	Milestone
2007–2009	Flickr, Etsy, and others share “10+ deploys per day” stories
2009	First DevOpsDays conference in Ghent, Belgium (Patrick Debois)
2010	“The Phoenix Project” novel popularizes DevOps narrative
2013	“The DevOps Handbook” (Gene Kim et al.) codifies practices
2016+	Cloud-native, containers, and Kubernetes accelerate adoption
2020s	Platform engineering, GitOps, and AI-assisted ops emerge

The term **DevOps** blends **Development** + **Operations**. It intentionally breaks down silos rather than creating a

third silo called “DevOps team” that sits between dev and ops.

2.4 1.4 Culture before tools

A common mistake is buying Jenkins, Kubernetes, or Terraform and calling it “DevOps.” Tools enable DevOps; they do not create it.

Cultural pillars:

1. **Shared ownership** — The team that builds a service also runs it (“you build it, you run it”).
 2. **Blameless postmortems** — Focus on systems and processes, not individuals, after incidents.
 3. **Automation of toil** — Repetitive manual work is eliminated or codified.
 4. **Continuous learning** — Experimentation, feedback, and improvement are expected.
 5. **Customer focus** — Every decision ties back to user value.
-

2.5 1.5 DevOps practices (overview)

You will study each in depth later in this handbook:

Practice	Summary
Version control	Everything in Git: code, config, IaC, docs
CI/CD	Automated build, test, deploy pipelines
Infrastructure as Code	Reproducible environments (Terraform, Ansible)
Monitoring & observability	Know what production is doing in real time
Microservices & containers	Smaller deployable units (optional but common)
Security integration (DevSecOps)	Security checks in the pipeline, not at the end

2.6 1.6 Organizational models

2.6.1 Functional silos (anti-pattern for DevOps)

Separate dev, QA, ops, and security teams with handoffs.

2.6.2 Cross-functional product teams (recommended)

One team owns a product or service end-to-end: code, infra, on-call, roadmap.

2.6.3 DevOps enablement / platform team

A **platform team** builds internal tools (CI templates, K8s clusters, golden paths) so product teams move faster. This is **not** a ticket-taking ops team—it is a **product for developers**.

2.7 1.7 What DevOps is NOT

- **Not “NoOps”** — Operations expertise is more important at scale, not less.
- **Not only for startups** — Enterprises adopt DevOps for compliance and speed.
- **Not replacing developers with operators** — It merges responsibilities skillfully.
- **Not a certification checkbox** — Culture and practice matter more than badges.

2.8 1.8 Your learning path in this handbook

```

Foundations → Automation → Containers → CI/CD → Observability → Security → Advanced
    ↑                                     ↓
Linux, Git, Networking ————— Platform & SRE

```

Invest 70% of study time in **hands-on labs**. Reading without doing builds false confidence.

2.9 1.9 Chapter summary

- DevOps unifies dev and ops around **speed** and **reliability**.
- **Culture** (shared ownership, blameless learning) precedes tools.
- Measure progress with **DORA metrics**.
- Cross-functional teams and platform engineering are modern expressions of DevOps.

2.10 Lab 1.1 — Reflect on your current workflow

1. Map your (or a sample) team's path from code commit to production.
2. Count manual steps and handoffs.
3. Identify one bottleneck and one automation opportunity.
4. Write a one-paragraph "DevOps vision" for that team.

2.11 Review questions

1. Name the four DORA metrics.
2. Why is “DevOps team” as a silo problematic?
3. What year and event is credited with coining “DevOps”?
4. Give one cultural and one technical practice of DevOps.

Next: Chapter 2 — CALMS, Three Ways, and Team Topologies

3 Chapter 2: CALMS, Three Ways, and Team Topologies

DevOps Handbook — Part I, Pages 16–30

3.1 2.1 The CALMS framework

CALMS is a mnemonic for assessing DevOps maturity across five dimensions:

Letter	Dimension	Description
C	Culture	Collaboration, trust, shared goals
A	Automation	Eliminate manual, error-prone work
L	Lean	Reduce waste, small batches, flow
M	Measurement	Data-driven decisions, DORA metrics
S	Sharing	Knowledge, tools, failures, successes

3.1.1 Culture (C)

Healthy DevOps culture means developers understand production constraints and operators understand business priorities. Psychological safety allows people to report problems early.

3.1.2 Automation (A)

Automate builds, tests, deployments, infrastructure provisioning, and compliance checks. Manual runbooks become code. **Toil**—repetitive operational work that scales linearly with service growth—should be targeted for elimination.

3.1.3 Lean (L)

Inspired by Toyota Production System: optimize **flow**, limit work in progress (WIP), and deliver **small batches**. Large releases hide defects; small releases surface them quickly.

3.1.4 Measurement (M)

You cannot improve what you do not measure. Track deployment frequency, lead time, MTTR, change failure rate, plus business KPIs (revenue, user satisfaction).

3.1.5 Sharing (S)

Internal conferences, shared Slack channels, communal dashboards, and open postmortems spread expertise. Tools and patterns are reused, not reinvented per team.

3.2 2.2 The Three Ways (Gene Kim)

From *The Phoenix Project* and *The DevOps Handbook*, the **Three Ways** describe the principles of DevOps flow:

3.2.1 First Way: Flow (left to right)

Accelerate delivery from **development** → **operations** → **customer**.

- Make work visible (Kanban boards, pipeline status).

- Reduce batch sizes.
- Build quality in (automated tests at every stage).
- Never pass defects downstream.

Example: A commit triggers CI; only green builds reach staging; only approved artifacts reach production.

3.2.2 Second Way: Feedback (right to left)

Amplify feedback from **production** → **development**.

- Monitor user experience and system health.
- Swarm on problems; shorten feedback loops.
- Developers get production access (with guardrails).

Example: Error rate spike in Grafana pages the on-call engineer; linked traces show the bad deploy; rollback is automated.

3.2.3 Third Way: Continuous learning and experimentation

Create a culture of **improvement** and **risk-taking**.

- Blameless postmortems.
- Reserve time for improvement (20% ops time, hack days).
- Rehearse failures (game days, chaos engineering).

3.3 2.3 Team Topologies (Skelton & Pais)

Modern DevOps organizations structure teams intentionally:

Team type	Purpose
Stream-aligned	Delivers product features; primary value stream
Platform	Provides self-service infra (K8s, CI, databases)
Enabling	Coaches stream teams on new tech (temporary)
Complicated-subsystem	Owns deep specialty (e.g., video encoding, ML)

3.3.1 Interaction modes

- **Collaboration** — Two teams work closely for discovery.
- **X-as-a-Service** — Platform provides API/portal; stream teams consume.
- **Facilitating** — Enabling team mentors others.

Anti-pattern: One ops team serves 50 dev teams via tickets.

Better: Platform team offers paved roads; stream teams deploy autonomously.

3.4 2.4 Conway's Law and reverse Conway maneuver

| *"Organizations design systems that mirror their communication structure."* — Melvin Conway

If dev and ops are separate departments, you will get separate deployment artifacts and fragile handoffs.

Reverse Conway maneuver: Structure teams the way you want the architecture to look. Want autonomous microservices? Give teams end-to-end ownership of each service.

3.5 2.5 Maturity model (informal)

Level	Characteristics
1 — Initial	Manual deploys, silos, hero culture
2 — Managed	Some CI, version control, basic monitoring
3 — Defined	IaC, automated testing, standardized pipelines
4 — Measured	SLOs, DORA tracking, blameless postmortems
5 — Optimizing	Continuous experimentation, platform self-service

Use this for self-assessment, not as a rigid certification ladder.

3.6 2.6 Chapter summary

- **CALMS** evaluates Culture, Automation, Lean, Measurement, Sharing.
 - The **Three Ways** optimize flow, feedback, and learning.
 - **Team Topologies** align org structure with delivery goals.
 - Apply **Conway's Law** consciously when designing teams.
-

3.7 Lab 2.1 — CALMS assessment

Score your team 1–5 on each CALMS letter. Pick the lowest score and draft one actionable improvement per dimension.

3.8 Review questions

1. What does the “L” in CALMS stand for, and what Lean concept supports it?
 2. Describe the First Way in one sentence.
 3. Name the four team types in Team Topologies.
 4. What is a reverse Conway maneuver?
-

Next: Part II — [Chapter 5: Linux Fundamentals](#)

4 Chapter 3: DevOps vs SRE vs Platform Engineering

DevOps Handbook — Part I, Pages 31–42

4.1 3.1 Three related disciplines

DevOps, **Site Reliability Engineering (SRE)**, and **Platform Engineering** are often confused because they share goals: reliable systems, fast delivery, and reduced toil. They are complementary, not competing.

Discipline	Origin	Primary focus
DevOps	Movement (2009+)	Culture + practices bridging dev and ops
SRE	Google (2003+, public ~2016)	Engineering approach to operations at scale
Platform Engineering	Industry evolution (2020s)	Internal platforms that accelerate product teams

4.2 3.2 Site Reliability Engineering (SRE)

Google introduced **SRE** as “what happens when you ask a software engineer to design an operations function.”

4.2.1 Core SRE principles

1. **Embrace risk** — 100% uptime is impossible and expensive; use **error budgets** to balance reliability and velocity.
2. **Service level objectives (SLOs)** — Define measurable targets (e.g., 99.9% availability).
3. **Eliminate toil** — Manual, repetitive work that scales with traffic must be automated.
4. **Monitoring** — Systems must be observable; alerts fire on symptoms users feel.
5. **Release engineering** — Safe, gradual rollouts with fast rollback.
6. **Simplicity** — Prefer boring, well-understood technology.

4.2.2 SRE team responsibilities

- Define and track SLOs/SLIs
- On-call rotation and incident response
- Capacity planning
- Performance and reliability testing
- Building automation to replace toil

4.2.3 Error budget example

If SLO = 99.9% monthly availability, you have ~43 minutes of downtime budget. When budget is exhausted, **freeze feature releases** and focus on reliability until budget recovers.

4.3 3.3 Platform Engineering

Platform engineering treats the internal developer platform as a **product** whose customers are other engineers.

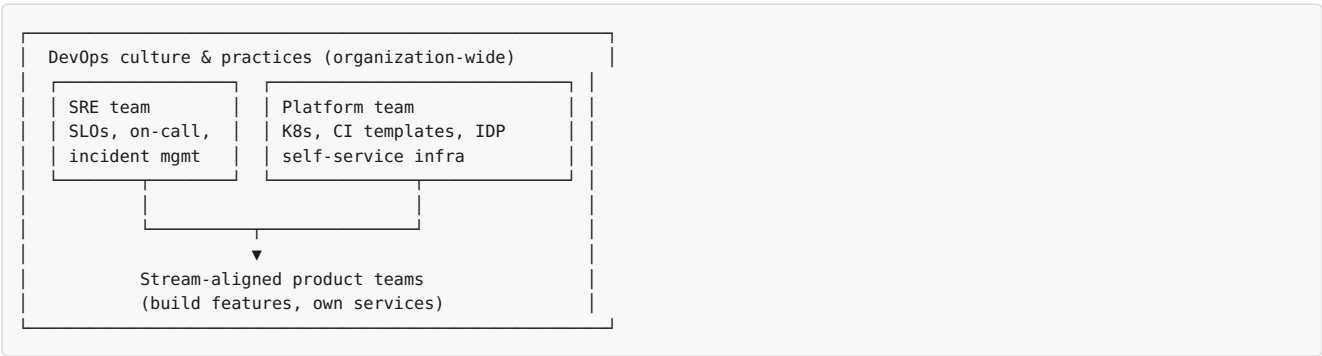
4.3.1 What platform teams build

- Kubernetes clusters with guardrails
- CI/CD templates and golden pipelines
- Self-service databases, caches, and queues
- Developer portals (Backstage) with docs and service catalogs
- Standardized observability (dashboards, tracing, logging)

4.3.2 Platform vs traditional ops

Traditional ops	Platform engineering
Ticket queue for infra requests	Self-service APIs and portals
Snowflake servers	Standardized golden paths
“Call us to deploy”	Teams deploy autonomously with paved roads

4.4 3.4 How they fit together



- **DevOps** sets the cultural north star.
- **SRE** ensures reliability is measurable and engineered.
- **Platform** removes friction so product teams ship faster.

4.5 3.5 Job titles in the market

Titles vary widely. Focus on **responsibilities**, not labels:

Title	Often means
DevOps Engineer	CI/CD, IaC, cloud infra, automation
SRE	SLOs, on-call, reliability tooling, toil reduction
Platform Engineer	Internal platform, K8s, developer experience
Cloud Engineer	AWS/Azure/GCP architecture and implementation
Infrastructure Engineer	Servers, networking, IaC

Many roles blend all three disciplines.

4.6 3.6 Chapter summary

- **SRE** engineers reliability with SLOs, error budgets, and toil automation.
 - **Platform engineering** provides self-service paved roads for developers.
 - **DevOps** is the umbrella culture; SRE and platform are specialized implementations.
-

4.7 Lab 3.1

Draft SLOs for a sample API: pick one SLI (latency or availability), set a target, and calculate the monthly error budget in minutes.

Next: [*Chapter 6 — Processes and systemd*](#)

5 Chapter 4: Career Paths, Certifications, and Learning Strategy

DevOps Handbook — Part I, Pages 43–60

5.1 4.1 DevOps is a skill set, not a single job title

Organizations use many titles for similar work: **DevOps Engineer**, **Site Reliability Engineer (SRE)**, **Platform Engineer**, **Cloud Engineer**, **Release Engineer**, **Infrastructure Engineer**, and **Build/Release Manager**. The labels differ; the underlying competencies overlap heavily.

What employers actually hire for:

Competency area	What you must demonstrate
Linux & shell	Debug servers, write automation, read logs
Networking	DNS, HTTP, TLS, load balancers, firewalls
Version control & CI/CD	Git workflows, pipelines, deployment strategies
Infrastructure as Code	Terraform, Ansible, reproducible environments
Containers & orchestration	Docker, Kubernetes, service mesh basics
Observability	Metrics, logs, traces, incident response
Security	Secrets, scanning, least privilege, compliance
Communication	Docs, postmortems, cross-team collaboration

A junior role might emphasize scripting and pipeline maintenance; a senior role adds architecture, reliability design, and mentoring. **Titles follow scope and seniority**, not a separate “DevOps career ladder” disconnected from software engineering.

5.2 4.2 Common career paths

5.2.1 Path A: Developer → DevOps / Platform

Developers who enjoy deployment, performance, and production debugging often move toward platform or DevOps roles. Strengths: application mindset, code quality, API design. Gaps to fill: Linux depth, networking, IaC, on-call culture.

5.2.2 Path B: Sysadmin / NOC → DevOps / SRE

Traditional operations professionals bring production instincts, incident handling, and infrastructure knowledge. Gaps to fill: Git-centric workflows, programming (Python/Go), cloud-native patterns, automation-first thinking.

5.2.3 Path C: QA / Release → CI/CD specialist

Test and release engineers often own pipelines and quality gates. Natural progression: expand into IaC, container platforms, and observability.

5.2.4 Path D: Security → DevSecOps

Security engineers who embed controls in pipelines and infrastructure are increasingly critical. Requires comfort with CI/CD, containers, and cloud IAM.

5.2.5 Senior and leadership trajectories

Level	Typical focus
Junior	Execute runbooks, fix pipeline failures, learn the stack
Mid	Own services end-to-end, design automation, participate in on-call
Senior	Architecture, SLO design, cross-team standards, incident command
Staff / Principal	Org-wide platforms, technical strategy, golden paths
Engineering Manager	People, hiring, delivery—still technical enough to review designs

DevOps careers rarely stay in one lane. The best engineers combine **breadth** (full stack of delivery) with **depth** in one or two areas (e.g., Kubernetes, Terraform, observability).

5.3 4.3 Certifications: value and limits

Certifications **validate baseline knowledge** and help pass HR filters. They do **not** replace hands-on experience or cultural fit.

Certification	Provider	Best for
AWS Certified DevOps Engineer - Professional	AWS	AWS-heavy shops, CI/CD + IaC on AWS
AWS Solutions Architect / SysOps	AWS	Foundational cloud before DevOps Pro
CKA / CKAD / CKS	CNCF	Kubernetes administration and security
Terraform Associate	HashiCorp	IaC fundamentals
LPIC-1 / LPIC-2	Linux Professional Institute	Linux fundamentals (less common in job reqs)
GitHub Actions / GitLab certifications	Vendors	Pipeline-specific credentials
Professional Cloud DevOps Engineer	Google	GCP-centric organizations

How to use certs effectively:

1. **Align with your target stack** — CKA helps for K8s roles; AWS DevOps Pro helps for AWS platform teams.
2. **Study by doing** — Spin up labs; exams that include performance tasks reward practice, not memorization.
3. **Pair with a portfolio** — GitHub repos with Terraform modules, CI workflows, and READMEs beat cert walls alone.
4. **Avoid cert collecting** — Two relevant certs plus projects outweigh five unrelated badges.

Remember Chapter 1: DevOps is **not a certification checkbox**. Hiring managers ask *“What have you shipped and operated?”*

5.4 4.4 Building a learning strategy

5.4.1 The 70/20/10 model (adapted for DevOps)

Allocation	Activity
70%	Hands-on labs, homelab, work projects, break/fix exercises

20%	Peers, communities, code review, conference talks
10%	Courses, books, documentation reading

Reading this handbook without a terminal open builds **false confidence**. Every chapter includes labs; treat them as required.

5.4.2 Spaced repetition and depth-first learning

Avoid “tutorial hell”—watching endless videos without retention. Instead:

- 1. **Pick one vertical** (e.g., Linux networking) for two weeks.
- 2. **Complete labs** until commands are muscle memory.
- 3. **Teach it** — write a short internal doc or blog post.
- 4. **Move to the next topic** in the handbook sequence.

5.4.3 Homelab options (low to high cost)

Option	Cost	Good for
Local VM (VirtualBox, UTM, KVM)	Free	Linux, Ansible, K8s (kind/minikube)
Cloud free tier	Low	Real AWS/GCP/Azure patterns (watch billing!)
Raspberry Pi cluster	Moderate	Physical networking, k3s
Second-hand server	Moderate	Bare-metal, storage, RAID

Document your homelab in Git: `README.md` , architecture diagram, and teardown scripts prove discipline to interviewers.

5.5 4.5 Portfolio and interview preparation

5.5.1 What belongs in a DevOps portfolio

- **IaC repository** — Terraform or OpenTofu provisioning VPC + EKS/GKE/AKS or a simpler stack.
- **CI/CD pipeline** — Build, test, scan, deploy a small app (GitHub Actions or GitLab CI).
- **Observability demo** — Prometheus + Grafana or OpenTelemetry traces on a sample service.
- **Runbook or postmortem sample** — Shows operational maturity (sanitize any employer data).

5.5.2 STAR stories for behavioral interviews

Prepare stories for:

- **Production incident** you helped resolve (what broke, how you mitigated, what changed after).
- **Automation** that saved time (before/after metrics).
- **Disagreement** with dev or security resolved constructively.
- **Learning something hard** under time pressure.

5.5.3 Technical interview topics

Expect live exercises or take-homes involving: read a broken pipeline YAML, explain Git merge vs rebase, sketch a CI/CD diagram, debug `curl / kubectl` output, or write a small Python script against an API.

5.6 4.6 Soft skills that separate good from great

DevOps engineers spend as much time with **people and processes** as with servers.

--	--

Skill	Why it matters
Written communication	Runbooks, RFCs, and Slack updates must be clear under stress
Blameless mindset	Incidents improve systems when fear is removed
Prioritization	Not every alert deserves a 2 a.m. page
Empathy for developers	Platform work succeeds when users adopt it willingly
Security partnership	Say “no” with alternatives, not blockers

5.7 4.7 Staying current without burnout

The toolchain changes constantly (Kubernetes releases, new CI features, AI-assisted ops). Sustainable habits:

1. **Follow a few high-quality sources** — official docs, CNCF blog, vendor release notes for *your* stack only.
2. **Quarterly learning goals** — one cert attempt OR one major project, not both every month.
3. **On-call boundaries** — chronic burnout kills careers faster than missing a new tool launch.
4. **Contribute upstream** — small doc fixes to open source count as learning and networking.

5.8 4.8 Chapter summary

- DevOps careers span multiple titles; **competencies** matter more than labels.
- Paths from dev, ops, QA, or security are all valid with targeted gap-filling.
- Certifications **help hiring filters** but must pair with **hands-on projects**.
- Use **70% practice / 20% collaboration / 10% theory** and depth-first study.
- Portfolio, STAR stories, and communication skills differentiate senior candidates.

5.9 Lab 4.1 — Personal learning plan

1. Write your current role (or target role) and list five skill gaps from Section 4.1.
2. Choose **one** certification OR **one** portfolio project aligned with your target stack.
3. Block **5 hours per week** on your calendar for hands-on labs (not video watching).
4. Create a private Git repo `devops-learning-journal` with a `WEEKLY.md` log; add your first entry.

5.10 Lab 4.2 — Job description analysis

1. Collect three real DevOps/SRE job postings from companies you admire.
2. Highlight repeated requirements (tools, years, responsibilities).
3. Build a spreadsheet: requirement → handbook chapter → lab to complete.
4. Identify the **top three** gaps and schedule them for the next 30 days.

5.11 Review questions

1. Name four competency areas common to DevOps job descriptions.
2. Why is “cert collecting” a weak strategy on its own?
3. What is the 70/20/10 learning split recommended in this chapter?
4. Describe two valid career entry paths into DevOps.
5. What three items would you include in a minimal DevOps portfolio?

Next: [*Chapter 5 — Linux Fundamentals: Filesystem, Users, Permissions*](#)

6 Chapter 5: Linux Fundamentals — Filesystem, Users, Permissions

DevOps Handbook — Part II, Pages 61–85

6.1 5.1 Why Linux dominates DevOps

Most servers, containers, and cloud instances run **Linux**. As a DevOps engineer you will SSH into machines, debug production issues, write shell scripts, and configure services daily. Windows matters too (Active Directory, .NET shops), but Linux fluency is non-negotiable.

Common distributions:

Distro	Typical use
Ubuntu	Cloud default, tutorials, desktops
Debian	Stable base for Ubuntu and containers
RHEL / Rocky / Alma	Enterprise, regulated industries
Amazon Linux	AWS-optimized

6.2 5.2 Filesystem hierarchy (FHS)

Path	Purpose
/	Root of entire tree
/home	User home directories (/home/alice)
/root	root user’s home
/etc	Configuration files
/var	Variable data: logs (/var/log), caches
/tmp	Temporary files (often cleared on reboot)
/usr	User programs and libraries
/bin , /sbin	Essential binaries
/opt	Optional third-party software
/proc , /sys	Virtual filesystems for kernel info

Everything is a **file** in Unix—including devices and sockets.

6.3 5.3 Essential navigation commands

```
pwd           # Print working directory
ls -lah       # List files (human-readable, including hidden)
cd /etc/nginx # Change directory
cd ..         # Parent directory
cd ~          # Home directory
tree -L 2 /etc # Tree view (install: apt install tree)
```

Tab completion saves time: type partial names and press `Tab`.

6.4 5.4 File operations

```
cp source dest           # Copy
cp -r dir1 dir2         # Copy directory recursively
mv old new              # Move or rename
rm file                 # Delete (careful!)
rm -rf directory        # Delete directory recursively (very careful!)
mkdir -p path/to/dir    # Create nested directories
touch file.txt          # Create empty file or update timestamp
cat file                # Print file contents
less /var/log/syslog    # Paginated view (q to quit)
head -n 20 file         # First 20 lines
tail -f /var/log/app.log # Follow log in real time
```

6.4.1 Finding files

```
find /var/log -name "*.log" -mtime -7 # Logs modified in 7 days
find / -type f -size +100M 2>/dev/null # Files larger than 100MB
locate nginx.conf # Fast index search (updatedb)
```

6.5 5.5 Users, groups, and permissions

6.5.1 Users and groups

```
id # Current user UID, GID, groups
whoami
cat /etc/passwd # User accounts
cat /etc/group # Groups
sudo command # Run as root (if permitted)
su - username # Switch user
```

6.5.2 Permission model

Each file has three permission sets for **user (u)**, **group (g)**, **others (o)**:

```
-rwxr-xr-- 1 alice developers 4096 Jan 15 10:00 deploy.sh
||| |||||
||| ||||| others: read
||| _____ group: read, execute
|_____ user: read, write, execute
```

Symbol	Meaning
r (4)	Read
w (2)	Write
x (1)	Execute (directories: enter)

```
chmod 755 script.sh      # rwXr-Xr-X
chmod u+x script.sh      # Add execute for user
chown alice:developers file
chgrp developers file
```

6.5.3 Special permissions

- **setuid** — Run as file owner (e.g., `passwd`)

- **setgid** — Run as group; dirs inherit group
- **sticky bit** — On `/tmp`, only owner can delete own files

6.6 5.6 Package management (Debian/Ubuntu)

```
sudo apt update           # Refresh package index
sudo apt upgrade -y       # Upgrade installed packages
sudo apt install nginx -y # Install package
apt search nginx          # Search
apt show nginx            # Package info
sudo apt remove nginx     # Remove
dpkg -l | grep nginx     # List installed
```

RHEL/Rocky equivalent: `dnf install`, `dnf update`.

6.7 5.7 Text processing (daily DevOps tools)

```
grep -r "ERROR" /var/log/app/ # Search recursively
grep -E "timeout|refused" log.txt # Extended regex
awk '{print $1}' access.log    # Column extraction
sed 's/old/new/g' file.txt     # Stream editor
sort file | uniq -c | sort -rn | head # Count unique lines
wc -l file                    # Line count
```

Pipes (`|`) chain commands: stdout of one → stdin of next.

6.8 5.8 Environment and shell

```
echo $PATH                # Executable search path
export MY_VAR=hello       # Set environment variable
env                       # All environment variables
which python3             # Path to executable
alias ll='ls -lah'        # Command shortcut
```

Shell config files: `~/.bashrc`, `~/.profile` (sourced on login).

6.9 5.9 SSH — remote access

```
ssh user@hostname
ssh -i ~/.ssh/mykey.pem ec2-user@1.2.3.4
scp file user@host:/remote/path
rsync -avz ./local/ user@host:/remote/ # Efficient sync
```

`~/.ssh/config` simplifies connections:

```
Host prod-web
  HostName 10.0.1.50
  User ubuntu
  IdentityFile ~/.ssh/prod.pem
```

Then: `ssh prod-web`

6.10 5.10 Chapter summary

- Linux FHS organizes `/etc`, `/var`, `/home`, and more.
 - Master `ls`, `cd`, `cp`, `mv`, `rm`, `find`, `grep`, `tail -f`.
 - Permissions: user/group/other + `chmod` / `chown`.
 - Use `apt` / `dnf` for packages; SSH for remote ops.
-

6.11 Lab 5.1 — Linux sandbox

1. Create user `devops-lab` with home directory.
 2. Create `~/scripts/deploy.sh` with `#!/bin/bash` and `echo "Deploying..."`.
 3. Set permissions to `755`; run it.
 4. Write a one-liner to count ERROR lines in a sample log file.
 5. Configure SSH key auth to a local VM or cloud instance.
-

6.12 Review questions

1. What directory holds system configuration files?
 2. What does permission `644` mean in symbolic notation?
 3. How do you follow a log file in real time?
 4. What is the difference between `cp` and `mv`?
-

Continue: Chapter 6 — Processes, systemd, and Service Management (outline in TOC)

7 Chapter 6: Processes, systemd, and Service Management

DevOps Handbook — Part II, Pages 86–105

7.1 6.1 Understanding Linux processes

A **process** is a running instance of a program. Each has a **PID** (Process ID), owner, state, and resource usage.

```
ps aux          # All processes (BSD style)
ps -ef         # All processes (POSIX style)
top            # Interactive process viewer
htop          # Enhanced top (install separately)
pgrep nginx    # Find PIDs by name
kill -HUP nginx # Send signal to processes by name
```

7.1.1 Process states

State	Code	Meaning
Running	R	Executing or runnable
Sleeping	S	Waiting for event
Zombie	Z	Terminated but not reaped by parent
Stopped	T	Paused (e.g., Ctrl+Z)

7.1.2 Signals

```
kill -15 <pid> # SIGTERM – graceful shutdown (default)
kill -9 <pid>  # SIGKILL – force kill (last resort)
kill -HUP <pid> # SIGHUP – reload config (many daemons)
```

7.2 6.2 systemd — modern service manager

Most Linux distros use **systemd** to boot the system and manage services (units).

```
systemctl status nginx
systemctl start nginx
systemctl stop nginx
systemctl restart nginx
systemctl reload nginx      # Reload config without full restart
systemctl enable nginx     # Start on boot
systemctl disable nginx
systemctl list-units --type=service --state=running
journalctl -u nginx -f     # Follow service logs
journalctl -u nginx --since "1 hour ago"
```

7.2.1 Unit files

Service definitions live in `/etc/systemd/system/` or `/lib/systemd/system/`.

Example `/etc/systemd/system/myapp.service`:

```
[Unit]
Description=My Application
After=network.target
```

```
[Service]
Type=simple
User=appuser
WorkingDirectory=/opt/myapp
ExecStart=/opt/myapp/bin/server
Restart=on-failure
RestartSec=5
Environment=PORT=8080

[Install]
WantedBy=multi-user.target
```

After creating or editing a unit:

```
sudo systemctl daemon-reload
sudo systemctl enable --now myapp
```

7.3 6.3 Cron and timers

7.3.1 cron — scheduled tasks

```
crontab -e      # Edit user crontab
crontab -l      # List crontab
```

Cron format: `minute hour day month weekday command`

```
# Every day at 2:30 AM – backup database
30 2 * * * /opt/scripts/backup.sh >> /var/log/backup.log 2>&1
```

System-wide jobs: `/etc/cron.d/`, `/etc/cron.daily/`, etc.

7.3.2 systemd timers (preferred on modern systems)

Timers are more flexible and integrate with journald logging.

```
# /etc/systemd/system/backup.timer
[Unit]
Description=Daily backup

[Timer]
OnCalendar=*-*-* 02:30:00
Persistent=true

[Install]
WantedBy=timers.target
```

```
systemctl enable --now backup.timer
systemctl list-timers
```

7.4 6.4 Resource limits and troubleshooting

```
# CPU and memory per process
ps -o pid,user,%cpu,%mem,cmd -p <pid>

# Open files by process
lsof -p <pid>
ss -tlnp          # Listening TCP ports with process
df -h             # Disk usage
free -h           # Memory usage
dmesg | tail      # Kernel messages
```

7.4.1 Common production issues

Symptom	Likely cause	Check
Service won't start	Bad config, missing binary	<code>journalctl -u service -n 50</code>
Port in use	Another process bound	<code>ss -tlnp \ grep :80</code>
Disk full	Logs, temp files	<code>df -h</code> , <code>du -sh /var/log/*</code>
OOM killed	Memory exhaustion	<code>dmesg \ grep -i oom</code>

7.5 6.5 Chapter summary

- Manage processes with `ps`, `kill`, and signals.
- Use **systemctl** and **journalctl** for services on systemd systems.
- Schedule jobs with cron or systemd timers.
- Check logs first when debugging service failures.

7.6 Lab 6.1

1. Create a simple bash script that writes a timestamp to a log every minute.
2. Run it as a systemd service + timer.
3. Practice `systemctl status`, `journalctl -f`, and graceful stop/start.

Next: [Chapter 9 — Git Internals](#)

8 Chapter 7: Networking on Linux — ip, ss, curl, dig

DevOps Handbook — Part II, Pages 106–120

8.1 7.1 Why networking skills matter for DevOps

Every deployment, health check, and incident touches the network: DNS resolution, TCP connections, TLS handshakes, load balancer health probes, and firewall rules. When an application is “down,” the failure is often **not** the app—it is DNS, routing, a security group, or a saturated socket backlog.

Linux provides modern tools that replaced much of the legacy `ifconfig` / `netstat` era:

Tool	Purpose
<code>ip</code>	Interfaces, addresses, routes, rules
<code>ss</code>	Socket statistics (connections, listening ports)
<code>curl</code>	HTTP/HTTPS client for APIs and debugging
<code>dig</code>	DNS queries and troubleshooting
<code>traceroute</code> / <code>tracert</code>	Path discovery
<code>tcpdump</code> / <code>tshark</code>	Packet capture (advanced)

This chapter focuses on daily diagnostic workflows.

8.2 7.2 The ip command suite

The `ip` utility from **iproute2** is the standard for network configuration and inspection on modern Linux.

8.2.1 View interfaces and addresses

```
ip link show           # Layer 2: interfaces, MAC, state
ip addr show           # Layer 3: IPv4/IPv6 addresses
ip -br addr            # Brief one-line-per-interface summary
ip addr show dev eth0  # Single interface
```

Example brief output interpretation:

lo	UNKNOWN	127.0.0.1/8 ::1/128
eth0	UP	10.0.1.25/24 fe80::5054:ff:fe12:3456/64

- **UP** — interface administratively enabled and carrier detected (for physical links).
- **127.0.0.1/8** — loopback; always present.
- **/24** — subnet prefix; 256 addresses in this example.

8.2.2 Routing table

```
ip route show          # Main routing table
ip route get 8.8.8.8    # Which path would a packet take?
ip rule show           # Policy routing rules (multi-table)
```

If `ip route get` shows an unexpected interface, compare with your cloud subnet route tables and default gateway.

8.2.3 Temporary address assignment (lab only)

```
sudo ip addr add 192.168.99.10/24 dev eth0
sudo ip link set eth0 up
sudo ip route add default via 192.168.99.1
```

Changes made with `ip` are **not persistent** across reboot unless saved via netplan, NetworkManager, or distro-specific config under `/etc`.

8.3 7.3 Sockets and connections with `ss`

`ss` replaces `netstat` for inspecting listening ports and established connections. It reads from `/proc` and is faster on busy servers.

8.3.1 Common patterns

```
ss -tln           # TCP/UDP listening ports, numeric
ss -tunap        # Include process names (needs root for all PIDs)
ss -tn state established # Established TCP connections
ss -tn sport = :443  # Connections from local port 443
ss -tn dst 10.0.1.50 # Connections to specific destination
```

Flags:

Flag	Meaning
-t	TCP
-u	UDP
-l	Listening sockets
-n	Numeric (no DNS reverse lookup)
-a	All (listening + non-listening)
-p	Show process

8.3.2 DevOps debugging scenario

A service fails health checks but the process is running:

```
sudo ss -tlnp | grep 8080
curl -v http://127.0.0.1:8080/health
curl -v http://10.0.1.25:8080/health # bind address matters!
```

If `ss` shows `127.0.0.1:8080` but not `0.0.0.0:8080`, the app binds **localhost only**—external load balancers cannot reach it.

8.4 7.4 HTTP debugging with `curl`

`curl` is the universal CLI for HTTP/S, file transfers, and API testing.

8.4.1 Essential options

```
curl -I https://example.com # HEAD request (headers only)
curl -v https://api.example.com/v1/users # Verbose: TLS, headers, redirects
curl -sS -o /dev/null -w '%{http_code}\n' https://example.com # Status only
curl -H 'Authorization: Bearer TOKEN' https://api.example.com/me
curl -X POST -H 'Content-Type: application/json' \
```

```
-d '{"name":"svc"}' https://api.example.com/items
curl --connect-timeout 5 --max-time 30 https://slow.example.com
curl -k https://self-signed.local # Skip TLS verify (debug only!)
curl -o out.bin https://example.com/file # Save body to file
curl -L https://short.url/abc # Follow redirects
```

8.4.2 Timing breakdown

```
curl -w '\nnlookup:%{time_namelookup} connect:%{time_connect} tls:%{time_appconnect} ttfb:%{time_starttransfer} total:%{time_total}\n' \
-o /dev/null -sS https://example.com
```

Phase	Indicates
time_namelookup	DNS delay
time_connect	TCP handshake to server
time_appconnect	TLS handshake complete
time_starttransfer	Time to first byte (server processing)

High time_namelookup → DNS issue. High time_connect → routing/firewall. High time_starttransfer → backend overload.

8.5 7.5 DNS with dig

DNS failures cause mysterious outages: apps cache stale records, TTLs expire during migrations, and split-horizon DNS confuses internal vs external views.

8.5.1 Basic queries

```
dig example.com # Default A record query
dig example.com AAAA # IPv6
dig example.com MX # Mail exchangers
dig example.com NS # Name servers
dig @8.8.8.8 example.com # Query specific resolver
dig +short example.com # Answer only
dig +trace example.com # Full delegation trace (education/debug)
```

8.5.2 Useful flags for SRE work

```
dig example.com +noall +answer +stats
dig -x 93.184.216.34 # Reverse DNS (PTR)
dig @1.1.1.1 app.internal.company.com # Test internal resolver
```

Compare answers from **corporate resolver**, **public resolver (8.8.8.8)**, and **authoritative NS** when debugging propagation.

8.6 7.6 Putting it together: diagnostic workflow

When `curl https://api.myapp.com/health` fails from a server:

1. `dig api.myapp.com` → Does DNS resolve? Correct IP?
2. `ip route get <that-ip>` → Expected egress path?
3. `curl -v` (with IP / Host) → TLS? HTTP status? Timeout?
4. `ss -tn dst <ip>:443` → Connection state stuck?
5. Check firewall (nftables/iptables, cloud SG)
6. `tcpdump` (if permitted) → Packets leaving/returning?

Document findings in the incident channel as you go—future you will thank present you.

8.7 7.7 /etc/hosts , resolv.conf , and systemd-resolved

File / service	Role
/etc/hosts	Static local overrides (break-glass, testing)
/etc/resolv.conf	Resolver configuration (may be managed)
systemd-resolved	Caching stub resolver on many distros

```
resolvectl status           # Current DNS servers and domains
resolvectl query example.com # Query through systemd-resolved
```

On containers, DNS is often injected by Docker/Kubernetes—always check `/etc/resolv.conf` inside the pod, not only on the node.

8.8 7.8 Firewalls at a glance

```
sudo nft list ruleset      # nftables (modern default on many distros)
sudo iptables -L -n -v     # Legacy iptables
sudo ufw status verbose    # Ubuntu uncomplicated firewall
```

DevOps engineers rarely invent firewall rules from scratch—they align **host firewall** with **cloud security groups** and **Kubernetes NetworkPolicies**. Still, verifying `DROP` rules locally saves hours of cloud-console hunting.

8.9 7.9 Chapter summary

- Use `ip` for interfaces, addresses, and routes; prefer it over deprecated `ifconfig`.
- Use `ss` to see listening ports, bind addresses, and connection states.
- Use `curl -v` and timing flags to separate DNS, TCP, TLS, and application latency.
- Use `dig` to validate DNS from multiple resolvers during cutovers.
- Follow a **structured workflow** instead of random command guessing during incidents.

8.10 Lab 7.1 — Local service bind address

1. Run `python3 -m http.server 9090` on a VM.
2. Use `ss -tlnp` to see which address it binds.
3. From another host (or curl to the VM’s private IP), confirm reachability.
4. Restart bound to `127.0.0.1` only (if supported) and observe external failure.
5. Write two sentences explaining why bind address matters for load balancers.

8.11 Lab 7.2 — DNS and latency baseline

1. Pick three production-like hostnames (or use `example.com`, `cloudflare.com`, `github.com`).
2. For each, run `dig +short` and `curl -w` timing from your lab machine.
3. Record results in a table: hostname, IP, nlookup, connect, ttfb, total.

4. Change resolver to `8.8.8.8` temporarily and repeat—note differences.
-

8.12 Review questions

1. What does `ss -tuln` show, and what does each letter in the flags mean?
 2. How do you test whether an app listens on localhost only vs all interfaces?
 3. Which `curl -w` timing field isolates DNS delay?
 4. Why run `dig @8.8.8.8` during an internal DNS incident?
 5. Name two files/services that affect name resolution on Linux.
-

Next: [*Chapter 8 — Bash Scripting for Automation*](#)

9 Chapter 8: Bash Scripting for Automation

DevOps Handbook — Part II, Pages 121–140

9.1 8.1 When bash is the right tool

Bash excels at **gluing** existing CLI tools: deploy hooks, cron jobs, CI steps, and quick operational scripts. Reach for Python or Go when you need complex logic, libraries, or testability—but bash remains the **lingua franca** of Linux automation.

Rules of thumb:

Use bash	Use Python/Go instead
Wrapping <code>kubectl</code> , <code>aws</code> , <code>curl</code> , <code>tar</code>	Parsing complex JSON at scale
CI job steps under 50 lines	Long-running services
One-off migration with tight deadline	Shared libraries across teams
Systemd <code>ExecStart</code> wrappers	Heavy unit testing requirements

9.2 8.2 Script anatomy and safety

Every production script should start with:

```
#!/usr/bin/env bash
set -euo pipefail
IFS=$'\n\t'
```

Option	Effect
<code>set -e</code>	Exit on first command failure
<code>set -u</code>	Error on unset variables
<code>set -o pipefail</code>	Pipeline fails if any stage fails
<code>IFS</code>	Safer word splitting for loops

Optional: `set -x` for trace mode during debugging (remove before committing secrets to logs).

9.2.1 Shebang and portability

```
#!/usr/bin/env bash    # Preferred: finds bash in PATH
#!/bin/bash            # OK on most Linux; macOS may be old bash
```

Avoid bashisms in scripts that must run on **Alpine** `/bin/sh` (ash)—use `#!/bin/sh` and POSIX syntax, or explicitly require bash.

9.3 8.3 Variables, quoting, and expansion

```
name="api-server"
```

```
count=3
echo "Deploying ${name} x${count}"
echo 'Literal ${name} not expanded'

# Command substitution
today=$(date -u +%Y-%m-%d)
files=$(find /var/log -name '*.log' | wc -l)

# Default values
region="${AWS_REGION:-us-east-1}"
```

Always quote variable expansions unless you intentionally want word splitting:

```
rm -rf "$tmpdir"      # Good
rm -rf $tmpdir        # Dangerous if empty or contains spaces
```

Arrays (bash-specific):

```
services=(nginx api worker)
for svc in "${services[@]}; do
    systemctl is-active --quiet "$svc" || echo "$svc down"
done
```

9.4 8.4 Conditionals and tests

```
if [[ -f /etc/nginx/nginx.conf ]]; then
    echo "nginx config present"
elif [[ -d /opt/myapp ]]; then
    echo "app directory exists"
else
    echo "neither found" >&2
    exit 1
fi
```

Common tests:

Test	Meaning
[[-f path]]	Regular file exists
[[-d path]]	Directory exists
[[-z "\$var"]]	String empty
[["\$a" == "\$b"]]	String equality
[["\$n" -gt 0]]	Numeric greater than

Use `[[ ]]` in bash (not single-bracket `[ ]` unless POSIX required).

9.5 8.5 Loops and functions

```
for env in staging production; do
    echo "Deploying to $env"
    ./deploy.sh "$env"
done

while read -r line; do
    echo "Log: $line"
done < /var/log/app.log

deploy() {
    local env="$1"
    local version="$2"
```

```

    echo "deploy $version -> $env"
}

deploy staging "v1.2.3"

```

Use `local` inside functions to avoid polluting global scope.

9.6 8.6 Exit codes and error handling

Exit code `0` = success; non-zero = failure. CI systems and `set -e` depend on this.

```

if ! kubectl apply -f manifest.yaml; then
    echo "apply failed" >&2
    exit 1
fi

# Or with trap for cleanup
cleanup() {
    rm -f "$lockfile"
}
trap cleanup EXIT

lockfile=$(mktemp)
# ... critical section ...

```

`trap` ensures cleanup runs on exit, interrupt, or error—essential for temp files and lock release.

9.7 8.7 External commands and JSON

Prefer structured tools over fragile `grep` of command output when possible:

```

# jq for JSON (install in CI images)
instance_id=$(curl -s http://169.254.169.254/latest/meta-data/instance-id)
aws ec2 describe-instances --instance-ids "$instance_id" \
    | jq -r '.Reservations[0].Instances[0].PrivateIpAddress'

# Avoid parsing human tables with awk unless no JSON API exists

```

For simple key=value files:

```

# shellcheck disable=SC1091
source /etc/default/myapp
echo "$APP_PORT"

```

Never `source` untrusted files.

9.8 8.8 Logging and observability

```

log() {
    printf "[%s] %s\n" "$(date -u +%Y-%m-%dT%H:%M:%SZ)" "$*" >&2
}

log "Starting backup"

```

- Send logs to **stderr**; reserve stdout for data piped to other commands.
 - Include **timestamps in UTC** for correlating with centralized logging.
 - Avoid echoing secrets; redact tokens in debug output.
-

9.9 8.9 Argument parsing

```
usage() {
    echo "Usage: $0 -e ENV -v VERSION" >&2
    exit 1
}

env=""
version=""

while getopts ":e:v:h" opt; do
    case "$opt" in
        e) env="$OPTARG" ;;
        v) version="$OPTARG" ;;
        h) usage ;;
        *) usage ;;
    esac
done

[[ -n "$env" && -n "$version" ]] || usage
```

For long options (`--env`), consider `getopt` or a thin Python wrapper; keep bash CLIs simple.

9.10 8.10 Script layout in repos

```
scripts/
  deploy.sh
lib/
  common.sh      # shared functions, sourced by other scripts
README.md       # usage, env vars, examples
```

```
# deploy.sh
SCRIPT_DIR=$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)
# shellcheck source=lib/common.sh
source "$SCRIPT_DIR/lib/common.sh"
```

Run `shellcheck` in CI on all shell scripts—it catches quoting bugs and portability issues early.

9.11 8.11 Security considerations

Risk	Mitigation
Command injection	Never pass untrusted input to <code>eval</code> ; quote variables
Secrets in args	Use env vars or secret managers; avoid <code>ps</code> exposure
World-writable paths	Check <code>TMPDIR</code> ; use <code>mktemp</code>
<code>curl \</code> <code>bash</code>	Pin checksums; avoid in production install flows
SUID / excessive sudo	Run with least privilege

9.12 8.12 Chapter summary

- Start scripts with `set -euo pipefail` and quote expansions.
- Use functions, `local` , and `trap` for maintainable automation.
- Prefer **JSON APIs + jq** over parsing human-formatted CLI tables.
- Log to **stderr** with UTC timestamps; never leak secrets.
- Run **shellcheck** and document usage in README files.

9.13 Lab 8.1 — Health check script

1. Write `healthcheck.sh` that accepts `-u URL` and exits 0 only if HTTP status is 200.
 2. Use `curl -sS -o /dev/null -w '%{http_code}'`.
 3. Add `--timeout SECONDS` optional flag.
 4. Run shellcheck and fix all warnings.
-

9.14 Lab 8.2 — Batch service restart

1. Create `restart-services.sh` reading service names from a file (one per line).
 2. For each service: check active with `systemctl`, restart if failed, log result.
 3. Use `trap` to write a summary count on exit.
 4. Test with dummy unit names and intentional failures.
-

9.15 Review questions

1. What do `set -e`, `set -u`, and `pipefail` each prevent?
 2. Why quote `"$variable"` when passing paths to commands?
 3. When should you choose Python over bash for automation?
 4. What is the purpose of `trap cleanup EXIT`?
 5. How do you safely share functions across multiple bash scripts in a repo?
-

Next: [*Chapter 9 — Git Internals: Objects, Refs, and the DAG*](#)

10 Chapter 9: Git Internals — Objects, Refs, and the DAG

DevOps Handbook — Part III, Pages 141–158

10.1 9.1 Git is a content-addressable filesystem

At its core, Git stores **snapshots** of your project, not just diffs. Every piece of content is identified by a **SHA-1 hash** (Git is migrating to SHA-256).

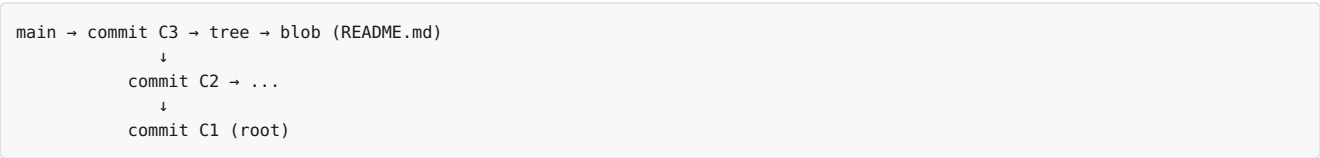
Four object types:

Type	Contains
blob	File content
tree	Directory listing (names → blobs/trees)
commit	Snapshot + metadata (author, message, parent)
tag	Named pointer to a commit (annotated tags)

10.2 9.2 How a commit is stored

When you `git add` and `git commit`:

1. Git compresses file content into **blobs**.
2. Directory structure becomes **trees**.
3. A **commit** object points to the root tree and parent commit(s).
4. Branch name (e.g., `main`) is a **ref** — a pointer to the latest commit.



This forms a **DAG** (directed acyclic graph). Merge commits have two parents.

10.3 9.3 Essential Git commands (beyond basics)

```
git log --oneline --graph --all
git show <commit>           # Commit details + diff
git diff HEAD~1             # Compare with previous commit
git stash push -m "wip"
git stash pop
git cherry-pick <commit>
git rebase main              # Replay commits on top of main
git reflog                  # History of HEAD movements (recovery!)
```

10.3.1 Undoing mistakes

```
git restore file.txt         # Discard working tree changes
git restore --staged file.txt # Unstage
git reset --soft HEAD~1      # Undo commit, keep changes staged
```

```
git reset --hard HEAD~1      # Undo commit, discard changes (dangerous)
git revert <commit>          # New commit that undoes a commit (safe for shared branches)
```

10.4 9.4 Branches and merging

```
git branch feature/login
git switch feature/login      # or: git checkout feature/login
git merge feature/login       # Merge into current branch
git merge --no-ff feature/login # Always create merge commit
```

10.4.1 Merge vs rebase

Merge	Rebase
Preserves history	Linear history
Creates merge commits	Rewrites commits
Safe for shared branches	Use only on local/feature branches

Golden rule: Never rebase commits that others have pulled.

10.5 9.5 Remote repositories

```
git remote -v
git fetch origin      # Download objects, don't merge
git pull origin main  # fetch + merge
git push origin main
git push -u origin feature/x # Set upstream
```

10.5.1 Tracking branches

`origin/main` is a **remote-tracking branch** — your local copy of what's on the server.

10.6 9.6 .gitignore and hooks

10.6.1 .gitignore

```
node_modules/
*.log
.env
dist/
.terraform/
```

10.6.2 Git hooks (`.git/hooks/`)

Scripts that run on events: `pre-commit`, `commit-msg`, `pre-push`. Used for linting, tests, and commit message format.

Example with **pre-commit** framework:

```
pip install pre-commit
# .pre-commit-config.yaml defines hooks
pre-commit install
```

10.7 9.7 Chapter summary

- Git stores blobs, trees, commits, and tags; branches are movable pointers.
 - Use `reflog` to recover “lost” commits.
 - Prefer **merge** on shared branches; **rebase** to clean up local work.
 - Automate quality with hooks and `.gitignore`.
-

10.8 Lab 9.1

1. Create a repo with 5 commits on `main`.
 2. Create a feature branch, make 2 commits, merge with `--no-ff`.
 3. Use `git log --graph --oneline --all` to visualize the DAG.
 4. Reset hard to an earlier commit, then recover using `git reflog`.
-

Next: [Chapter 20 — IaC Principles](#)

11 Chapter 10: Branching Strategies and Code Review

DevOps Handbook — Part III, Pages 159–175

11.1 10.1 Branches as a collaboration contract

Git branches are movable pointers to commits. How your team uses branches defines **release cadence**, **review culture**, and **incident risk**. There is no universal “best” strategy—only trade-offs aligned with deployment frequency, team size, and compliance needs.

Common strategies:

Strategy	Summary	Typical deployment
Trunk-based development (TBD)	Short-lived branches; merge to <code>main</code> daily	Continuous (many times/day)
GitHub Flow	<code>main</code> + feature branches + PR	On merge to <code>main</code>
Git Flow	<code>main</code> , <code>develop</code> , release/hotfix branches	Scheduled releases
Release branches	Stabilize on <code>release/x.y</code> before prod	Versioned releases

DevOps maturity correlates with **shorter branch lifetime** and **smaller changesets**.

11.2 10.2 Trunk-based development in depth

Trunk-based development keeps `main` **always releasable**. Developers integrate frequently—ideally at least once per day.

Practices:

1. **Feature flags** hide incomplete work instead of long-lived branches.
2. **Small PRs** (< 400 lines changed is a common guideline) review faster and fail less.
3. **Branch lifetime** measured in hours or days, not weeks.
4. **Automated gates** — CI must pass before merge; no manual “it works on my machine.”

```
developer → short feature branch → PR → CI → review → merge to main → deploy
```

Anti-patterns for TBD:

- “Integration branch” that lives for a sprint.
- Merging without CI green because “we’ll fix later.”
- Shared long-lived branches per environment (`dev` , `qa` , `prod` in Git)—prefer **promote artifacts**, not cherry-pick commits.

11.3 10.3 Git Flow and when it still fits

Git Flow (Vincent Driessen) defines:

- **main** — production releases only (tags).
- **develop** — integration branch for next release.
- **feature/*** — from develop, merge back to develop.
- **release/*** — stabilize version, bugfix only.
- **hotfix/*** — emergency fix from **main**, back-merge to develop.

It suits **shrink-wrapped software**, mobile apps with store review, or regulated release windows. Costs:

- Merge complexity between **develop**, **release**, and **main**.
- Slower feedback loops vs trunk-based.
- Temptation to batch large releases.

Many cloud-native teams **simplified Git Flow to GitHub Flow + tags** for semver.

11.4 10.4 Pull requests and merge mechanics

11.4.1 Merge commit vs squash vs rebase

Method	History	Pros	Cons
Merge commit	Preserves branch topology	True audit trail	Noisy graph
Squash merge	One commit on main	Clean main history	Loses granular commits
Rebase merge	Linear history	Readable log	Rewrites SHAs on branch

Platform teams often standardize on **squash to main** for product repos and **merge commits** for open-source where contributor attribution matters.

11.4.2 Protected branches

Configure on GitHub/GitLab:

- Require PR before merge to **main**.
- Require **status checks** (CI jobs) to pass.
- Require **approving reviews** (1–2 depending on risk).
- Block force-push and deletion.
- Optional: require signed commits.

11.5 10.5 Code review: purpose and culture

Code review is **not** a gate for senior engineers to flex syntax preferences. It is a **risk reduction and knowledge sharing** mechanism.

Goals:

1. Catch defects, security issues, and operational gaps before production.
2. Spread context so bus factor > 1.
3. Enforce team standards (tests, observability, docs) consistently.

11.5.1 What reviewers should examine

Area	Questions
Correctness	Does it do what the ticket says? Edge cases?

Tests	Adequate coverage for changed behavior?
Observability	Logs, metrics, traces for new paths?
Security	Injection, authz, secrets, dependencies?
Operability	Rollback, config, migrations, feature flags?
Performance	N+1 queries, unbounded loops, cache behavior?

11.5.2 Author responsibilities

- Keep PRs **focused**—one logical change.
- Write a **clear description**: problem, approach, test plan, rollout notes.
- Self-review the diff before requesting others.
- Respond to feedback with commits or comments; don't take review personally.

11.5.3 Reviewer etiquette

- Distinguish **blockers** from **nits** (`nit:` prefix).
- Approve when you'd be comfortable **on-call** for the change.
- Time-box reviews—slow reviews stall delivery and hurt morale.

11.6 10.6 Size and speed metrics

Google's research and DORA reports align: **small batches** and **fast review** improve stability.

Track informally:

Metric	Healthy signal
PR size (lines)	Median under ~200–400
Time to first review	< 4 business hours
Time to merge	< 1 day for most PRs
Revert rate	Low; postmortems when reverts happen

If review latency exceeds a day, fix process (rotation, alerts) before adding more tools.

11.7 10.7 Release tags and semantic versioning

Even with continuous deployment, **tags** mark releases for support and changelogs:

```
git tag -a v1.4.0 -m "Release 1.4.0: payment retry logic"
git push origin v1.4.0
```

Semantic versioning (semver):

- **MAJOR** — incompatible API change.
- **MINOR** — backward-compatible feature.
- **PATCH** — backward-compatible bug fix.

CI can build container images as `myapp:1.4.0` and `myapp:1.4.0-<git-sha>` for traceability.

11.8 10.8 Handling hotfixes

With trunk-based flow:

1. Branch from `main` (or fix forward on main directly if policy allows).
2. Minimal fix + test; fast-track review.
3. Merge; deploy; monitor.
4. Postmortem if customer-impacting.

With Git Flow:

1. `hotfix/*` from tagged `main`.
2. Merge to `main` and `develop`.
3. Tag patch release.

Either way: **deploy the same artifact** built from the merged commit—never hand-edit production.

11.9 10.9 Chapter summary

- Match branching strategy to **release frequency** and compliance—not hype.
 - Trunk-based + feature flags supports **continuous delivery**; Git Flow suits **scheduled releases**.
 - Protect `main` with **CI gates and reviews**; prefer small PRs.
 - Code review reduces risk and spreads knowledge when done with **respect and clarity**.
 - Tag releases with **semver** for supportability even if deploys are continuous.
-

11.10 Lab 10.1 — Strategy comparison

1. For a hypothetical SaaS (daily deploys) and a mobile app (monthly store release), pick a branching strategy for each.
 2. Write one paragraph justifying each choice using trade-offs from this chapter.
 3. List three protected-branch rules you would enable on `main`.
-

11.11 Lab 10.2 — Mock code review

1. Open a small PR in a practice repo (or use a provided diff).
 2. Leave three comments: one blocker, one suggestion, one nit.
 3. As author, respond and push a fix for the blocker.
 4. Record time from PR open to merge; aim under 30 minutes for this exercise.
-

11.12 Review questions

1. What is the main goal of trunk-based development?
 2. Name the five branch types in classic Git Flow.
 3. Compare squash merge vs merge commit for mainline history.
 4. List four areas a DevOps-minded code reviewer should check beyond syntax.
 5. When might Git Flow be preferable to trunk-based development?
-

Next: [Chapter 11 — GitHub/GitLab Workflows](#)

12 Chapter 11: GitHub/GitLab Workflows — PRs, Issues, Actions Intro

DevOps Handbook — Part III, Pages 176–200

12.1 11.1 Platform-native Git workflows

Git is the protocol; **GitHub** and **GitLab** are collaboration platforms. They add pull/merge requests, issue tracking, CI/CD, permissions, and audit trails. DevOps engineers live in these UIs and APIs daily.

Concept mapping:

Concept	GitHub	GitLab
Pull request	Pull Request (PR)	Merge Request (MR)
CI system	GitHub Actions	GitLab CI/CD
Issue	Issue	Issue
Code owners	CODEOWNERS	CODEOWNERS
Environments	Environments + rules	Environments + protected branches
Package registry	GHCR	GitLab Container Registry

Master one platform deeply; skills transfer with vocabulary swaps.

12.2 11.2 Issues and project hygiene

Issues are the **system of record** for work: bugs, features, tech debt, incidents.

Effective issue template fields:

Field	Purpose
Summary	One-line title
Context	User impact, links to alerts
Steps to reproduce	For bugs
Acceptance criteria	For features
Rollout / rollback	For infra changes

Link PRs to issues (`Fixes #123`) so merges auto-close tickets and audit trails stay connected.

Labels (`bug` , `infra` , `security` , `P1`) and milestones (sprints, releases) help prioritization—avoid label sprawl beyond ~15 types.

12.3 11.3 Pull request lifecycle (GitHub)

Typical flow:

```
1. Create branch from main    git checkout -b feat/retry-webhook
```

2. Commit with clear messages `git commit -m "feat: add webhook retry"`
3. Push and open PR `gh pr create`
4. CI runs on PR event
5. Reviewers approve
6. Merge (squash/merge/rebase per policy)
7. Deployment pipeline triggers on main

12.3.1 PR description template

```
## Summary
Brief description of change.

## Test plan
- [ ] Unit tests pass locally
- [ ] Staging deploy verified

## Rollout
Feature flag `webhook_retry` default off; enable 10% canary.

## Screenshots / logs
(if applicable)
```

Store templates in `.github/pull_request_template.md`.

12.4 11.4 CODEOWNERS and review routing

`CODEOWNERS` auto-requests reviewers based on path:

```
# .github/CODEOWNERS
* @platform-team
/terraform/ @infra-team @security
/apps/payments/ @payments-squad
```

GitLab equivalent lives at `.gitlab/CODEOWNERS` or project settings with **approval rules**.

This scales review without a central gatekeeper bottleneck—**domain experts** review their turf.

12.5 11.5 GitHub Actions introduction

GitHub Actions runs CI/CD in YAML workflows stored in `.github/workflows/`.

Minimal workflow:

```
# .github/workflows/ci.yml
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.12'

      - name: Install dependencies
        run: pip install -r requirements.txt
```

```
- name: Run tests
  run: pytest -q
```

Key concepts:

Term	Meaning
Workflow	YAML file; one or more jobs
Job	Runs on a runner VM; parallelizable
Step	Shell command or <code>uses:</code> action
Action	Reusable unit (checkout, setup-node)
Runner	GitHub-hosted or self-hosted machine

Secrets: Settings → Secrets and variables → Actions . Reference as `${{ secrets.API_TOKEN }}` —never commit secrets.

12.6 11.6 GitLab CI introduction

GitLab CI uses `.gitlab-ci.yml` at repo root:

```
# .gitlab-ci.yml
stages:
  - test
  - build

variables:
  PIP_CACHE_DIR: "${CI_PROJECT_DIR}/.cache/pip"

test:
  stage: test
  image: python:3.12-slim
  script:
    - pip install -r requirements.txt
    - pytest -q
  cache:
    paths:
      - .cache/pip

build:
  stage: build
  image: docker:24
  services:
    - docker:24-dind
  script:
    - docker build -t "${CI_REGISTRY_IMAGE}:${CI_COMMIT_SHA}" .
    - docker push "${CI_REGISTRY_IMAGE}:${CI_COMMIT_SHA}"
  only:
    - main
```

GitLab runners execute jobs; register shared or project-specific runners for private clusters.

12.7 11.7 Branch protection and environments

12.7.1 GitHub environments

Define **staging** and **production** with:

- Required reviewers before deploy job runs.
- Wait timers for change windows.
- Secrets scoped per environment.

```

jobs:
  deploy-prod:
    runs-on: ubuntu-latest
    environment: production
    needs: [test]
    steps:
      - run: ./deploy.sh production

```

12.7.2 GitLab protected environments

Similar via **protected branches**, **deployment tiers**, and **manual jobs** (`when: manual`).

12.8 11.8 Fork and open-source workflows

External contributors fork repos, push branches on their fork, and open PRs/MRs upstream.

Maintainers:

- Run CI on PRs from forks with **limited secrets** (GitHub uses `pull_request_target` carefully—security sensitive).
- Use **CLA bots** or DCO sign-off for legal compliance.
- Label `good first issue` to grow contributors.

12.9 11.9 Integrations DevOps teams use

Integration	Use case
Slack / Teams	PR and deploy notifications
Jira / Linear	Bi-directional issue sync
Dependabot / Renovate	Automated dependency PRs
CODE scanning	SAST on push
OIDC to cloud	Keyless AWS/GCP/Azure deploy from CI

Example OIDC pattern (GitHub → AWS): configure IAM role trust for `token.actions.githubusercontent.com`; workflow assumes role without long-lived AWS keys.

12.10 11.10 Comparing Actions vs GitLab CI (practical)

Dimension	GitHub Actions	GitLab CI
Marketplace actions	Huge ecosystem	Smaller; more shell/scripts
Self-hosted runners	Per repo/org	First-class runner manager
Monorepo	Path filters, reusable workflows	<code>rules: changes</code> , child pipelines
Container registry	GHCR integrated	Built into GitLab
Enterprise	GitHub Enterprise	GitLab dedicated/self-managed

Many organizations use **both**—product on GitHub, internal mirrors on GitLab—so learn YAML patterns for each.

12.11 11.11 CLI automation: `gh` and `glab`

```
# GitHub CLI
gh pr create --title "feat: retry" --body "See template"
gh pr checks
gh run list --workflow ci.yml

# GitLab CLI (glab)
glab mr create --title "feat: retry"
glab pipeline status
```

Scripting PR creation and CI status in onboarding docs reduces friction for new engineers.

12.12 11.12 Chapter summary

- GitHub **PRs** and GitLab **MRs** are the hub for review, CI, and audit.
 - Issues linked to PRs create traceability from idea to production.
 - **CODEOWNERS** routes review to the right experts automatically.
 - **GitHub Actions** and **GitLab CI** define pipelines as code in YAML.
 - Use **environments**, branch protection, and OIDC for safe deployments.
-

12.13 Lab 11.1 — First GitHub Actions workflow

1. Add `.github/workflows/ci.yml` to a practice repo running `shellcheck` on `scripts/`.
 2. Open a PR that breaks a script intentionally; confirm CI fails.
 3. Fix and merge; confirm CI passes on `main`.
 4. Add a job summary with `$GITHUB_STEP_SUMMARY`.
-

12.14 Lab 11.2 — GitLab mirror (optional)

1. If using GitLab, create `.gitlab-ci.yml` with `lint` + `test` stages.
 2. Configure a manual `deploy_staging` job only on `main`.
 3. Document required CI variables in `README`.
-

12.15 Review questions

1. What is the GitLab equivalent of a GitHub Pull Request?
 2. Where should CI workflow files live in GitHub vs GitLab repos?
 3. How does CODEOWNERS improve review routing?
 4. Why should production secrets be scoped to environments?
 5. Name two integrations that reduce toil for DevOps teams on these platforms.
-

Next: [Chapter 12 — OSI Model, TCP/IP, DNS Deep Dive](#)

13 Chapter 12: OSI Model, TCP/IP, DNS Deep Dive

DevOps Handbook — Part IV, Pages 201–225

13.1 12.1 Why models matter during incidents

When a user reports “the site is down,” you need a **mental stack** to isolate the layer at fault: physical link, IP routing, DNS, TLS, HTTP, or application logic. The **OSI model** (7 layers) and the practical **TCP/IP model** (4 layers) provide shared vocabulary for engineers and vendors.

You will not configure “Layer 5” in AWS—but you will say “this looks like L4” or “DNS is broken” and save hours.

13.2 12.2 OSI seven layers (reference)

Layer	Name	PDU	Examples
7	Application	Data	HTTP, DNS, SMTP
6	Presentation	Data	TLS encryption, compression
5	Session	Data	Session management (often folded into app)
4	Transport	Segment	TCP, UDP
3	Network	Packet	IP, ICMP, routing
2	Data Link	Frame	Ethernet, MAC, switches
1	Physical	Bits	Cables, fiber, Wi-Fi radio

Mnemonic: *All People Seem To Need Data Processing* (top-down) or *Please Do Not Throw Sausage Pizza Away* (bottom-up).

DevOps work concentrates on **Layers 3–7** in cloud environments; hardware L1/L2 is abstracted but still appears in hybrid/on-prem links.

13.3 12.3 TCP/IP four-layer model (operational)

TCP/IP layer	Maps to OSI	DevOps touchpoints
Application	5–7	HTTP APIs, DNS queries, gRPC
Transport	4	TCP ports, UDP, load balancer targets
Internet	3	IP addresses, routes, security groups
Network access	1–2	NIC, VPC, subnets, ARP

Most troubleshooting flows **top-down** (curl fails → dig → ping/trace → SG rules) or **bottom-up** when link issues suspected.

13.4 12.4 IP addressing and CIDR

IPv4 addresses are 32-bit dotted quads; subnets use **CIDR** notation:

10.0.1.25/24

prefix length: 24 network bits, 8 host bits → 256 addresses

private RFC1918 range (common in VPCs)

Private ranges (RFC 1918):

Range	CIDR
10.0.0.0 – 10.255.255.255	10.0.0.0/8
172.16.0.0 – 172.31.255.255	172.16.0.0/12
192.168.0.0 – 192.168.255.255	192.168.0.0/16

NAT maps private IPs to public IPs at egress—understand this before debugging “works inside VPC, fails from internet.”

IPv6 adoption grows (128-bit addresses, /64 typical on subnets). Tools: `ip -6 addr`, `AAAA` records, dual-stack load balancers.

13.5 12.5 TCP vs UDP

Aspect	TCP	UDP
Connection	Connection-oriented (3-way handshake)	Connectionless
Reliability	Retransmits, ordering	Best-effort
Use cases	HTTP, SSH, DB	DNS, QUIC, video, metrics
Flow control	Yes	No
DevOps example	HTTPS API	CoreDNS, StatsD

TCP connection establishment:

Client → SYN → Server

Client ← SYN-ACK ← Server

Client → ACK → Server

`ss` states like `SYN-SENT`, `ESTAB`, `TIME-WAIT` map to this lifecycle—**TIME-WAIT** accumulation can exhaust ports on high-churn load generators.

13.6 12.6 DNS: the distributed phone book

DNS resolves names to records. Critical for **every** deploy that changes IPs or uses CDNs.

13.6.1 Record types

Type	Purpose	Example
A	IPv4 address	<code>api.example.com</code> → <code>203.0.113.10</code>
AAAA	IPv6 address	<code>api.example.com</code> → <code>2001:db8::1</code>
CNAME	Alias to another name	<code>www</code> → <code>example.com</code>
MX	Mail server	Priority + hostname

TXT	Arbitrary text	SPF, DKIM, domain verification
NS	Delegates zone	Points to authoritative servers
SRV	Service location	Port + host for protocols
CAA	Certificate authority restriction	Security

13.6.2 Resolution chain

```

Browser/OS stub resolver
  → Recursive resolver (ISP, 8.8.8.8, corporate DNS)
    → Root (.)
      → TLD (.com)
        → Authoritative NS (example.com)
          → Answer (A record)

```

Results are **cached** at each hop per **TTL** (seconds). Low TTL (60–300) speeds cutovers; high TTL reduces query load but slows rollback.

13.7 12.7 DNS operations DevOps engineers perform

13.7.1 Cutover checklist

1. Lower TTL **24–48 hours** before change.
2. Apply new records on authoritative provider.
3. Verify with `dig @authoritative-ns` and public resolvers.
4. Monitor error rates and `curl` from multiple regions.
5. Restore TTL after stable.

13.7.2 Split-horizon and private zones

Internal services use **private DNS** (Route 53 private zones, Cloud DNS private, Azure Private DNS):

- `db.internal` resolves to `10.0.5.12` inside VPC.
- Same name may not resolve publicly—or resolves differently (**split brain** by design).

Document which resolvers apps use (`/etc/resolv.conf` , CoreDNS `forward` plugin).

13.8 12.8 ICMP and path discovery

ICMP supports diagnostics (ping, unreachable messages). Many firewalls block ICMP—“ping fails” does not always mean host down.

```

ping -c 4 203.0.113.10
traceroute 203.0.113.10
tracepath 203.0.113.10    # No root required on Linux
mtr 203.0.113.10          # Interactive (install mtr)

```

Use traceroute when latency spikes or routing loops suspected after VPC peering changes.

13.9 12.9 ARP and L2 in VPC context

ARP maps IPv4 to MAC on local subnet. In cloud VPCs, you rarely touch ARP directly—but **security groups** and **NACLs** behave like L3/L4 filters on virtual interfaces.

On-prem hybrid: VPN/ Direct Connect extends L3; understand whether broadcasts traverse (usually not).

13.10 12.10 Common failure patterns

Symptom	Likely layer	First checks
Unknown host	DNS	<code>dig</code> , resolver config
Connection timed out	L3/L4 firewall/route	SG, NACL, <code>ip route get</code>
Connection refused	L4 app not listening	<code>ss -tlnp</code> , bind address
SSL certificate error	L6/TLS	cert SANs, expiry, chain
HTTP 502/504	L7/L4 LB	target health, upstream timeout

13.11 12.11 Chapter summary

- OSI provides vocabulary; **TCP/IP** maps to daily cloud networking.
 - **CIDR** and private IP planning prevent overlapping VPC peering disasters.
 - **TCP** reliability vs **UDP** speed trade-offs drive protocol choice.
 - **DNS TTL**, record types, and resolver chains explain most “works after 5 minutes” incidents.
 - Troubleshoot **top-down** with `curl` , `dig` , `ss` , and cloud network ACLs.
-

13.12 Lab 12.1 — DNS TTL experiment

1. Create a test subdomain with TTL 60 on your DNS provider.
 2. Point A record to IP A; verify with `dig +trace` or `dig @ns` .
 3. Change to IP B; measure propagation time from three resolvers.
 4. Repeat with TTL 3600 in a lab zone (document expected delay).
-

13.13 Lab 12.2 — Layer mapping drill

1. For five past incidents (or hypothetical scenarios), identify the failing layer.
 2. Write the first diagnostic command you would run for each.
 3. Peer-review with a colleague or checklist.
-

13.14 Review questions

1. Map HTTP, TCP, and IP to OSI layers.
 2. What is the difference between A and CNAME records?
 3. Why lower DNS TTL before a migration?
 4. What does TCP `TIME-WAIT` indicate, and when is it a problem?
 5. Explain split-horizon DNS in one paragraph.
-

Next: [Chapter 13 — HTTP/HTTPS, TLS, and Certificates](#)

14 Chapter 13: HTTP/HTTPS, TLS, and Certificates

DevOps Handbook — Part IV, Pages 226–245

14.1 13.1 HTTP: the application protocol of the web

HTTP (Hypertext Transfer Protocol) is a stateless request/response protocol at OSI Layer 7. DevOps engineers encounter HTTP in health checks, API gateways, ingress controllers, and CI cache headers daily.

14.1.1 Request structure

```
GET /api/v1/users HTTP/1.1
Host: api.example.com
Accept: application/json
Authorization: Bearer eyJhbGc...
User-Agent: curl/8.5.0
```

14.1.2 Response structure

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 42
Cache-Control: no-store

{"users":[]}
```

14.1.3 Common methods

Method	Idempotent	Typical use
GET	Yes	Read resource
POST	No	Create, actions
PUT	Yes	Replace resource
PATCH	No*	Partial update
DELETE	Yes	Remove resource
HEAD	Yes	Headers only (health)

*PATCH idempotency depends on implementation.

14.1.4 Status codes (DevOps essentials)

Code	Meaning	Action
200	OK	Success
301/302	Redirect	Check Location header
400	Bad request	Client payload issue
401/403	Authn/authz	Credentials or policy
404	Not found	Route or resource missing
429	Rate limited	Backoff, quotas
500	Server error	App bug or dependency
502/503/504	Gateway/upstream	LB, overload, timeout

14.2 13.2 HTTP/1.1 vs HTTP/2 vs HTTP/3

Version	Transport	Notes
HTTP/1.1	TCP	Head-of-line blocking on one connection
HTTP/2	TCP + TLS (common)	Multiplexed streams, header compression
HTTP/3	QUIC (UDP)	Faster handshakes, mobile lossy networks

Ingress controllers (nginx, Envoy, ALB) terminate HTTP/2 from clients and may speak HTTP/1.1 to legacy backends—know your **end-to-end protocol** when debugging streaming and timeouts.

14.3 13.3 HTTPS and TLS

HTTPS is HTTP over **TLS** (Transport Layer Security). TLS provides:

- 1. **Confidentiality** — encrypted payload.
- 2. **Integrity** — tamper detection.
- 3. **Authentication** — server (and optionally client) identity via certificates.

14.3.1 TLS handshake (simplified)

ClientHello (supported ciphers, SNI)
 ← ServerHello, certificate, key exchange
Client verifies cert chain → generates session keys
Encrypted application data (HTTP)

SNI (Server Name Indication) lets one IP host many HTTPS sites—virtual hosting at TLS layer. Missing SNI breaks multi-tenant ingress.

Inspect with:

```
openssl s_client -connect api.example.com:443 -servername api.example.com </dev/null 2>/dev/null \  
| openssl x509 -noout -subject -issuer -dates  
curl -vI https://api.example.com
```

14.4 13.4 Certificates and PKI

A **certificate** binds a public key to identities (DNS names in **SAN**—Subject Alternative Name). Browsers trust **Certificate Authorities (CAs)** rooted in the system trust store.

Component	Role
Private key	Stays secret on server or HSM
CSR	Certificate Signing Request → sent to CA
Leaf cert	Presented to clients
Intermediate + root	Chain of trust to a trusted root

14.4.1 Let’s Encrypt and ACME

ACME automates issuance (cert-manager on Kubernetes, Caddy, Traefik). Certs are short-lived (90 days)—automation is mandatory.

```
# cert-manager Certificate (illustrative)
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: api-tls
spec:
  secretName: api-tls-secret
  dnsNames:
    - api.example.com
  issuerRef:
    name: letsencrypt-prod
    kind: ClusterIssuer
```

14.5 13.5 mTLS (mutual TLS)

Standard TLS authenticates the **server** to the client. **mTLS** also requires clients to present certificates—common in service meshes (Istio, Linkerd) and zero-trust internal APIs.

DevOps tasks:

- Rotate client certs before expiry.
- Store CA bundles in trust stores for sidecars.
- Monitor handshake failure metrics.

14.6 13.6 Cipher suites and hardening

Weak ciphers and old TLS versions (1.0, 1.1) are disabled in modern compliance baselines. Prefer **TLS 1.2+** and **TLS 1.3** where supported.

Test external posture:

```
# Using testssl.sh or SSL Labs API for audits
nmap --script ssl-enum-ciphers -p 443 api.example.com
```

Align ingress `ssl_protocols` / `ssl_ciphers` with security team policy—document exceptions.

14.7 13.7 HTTP headers DevOps should know

Header	Purpose
Host	Virtual host routing
X-Forwarded-For / Forwarded	Client IP through proxies
X-Request-ID	Correlation across services
Strict-Transport-Security	Force HTTPS (HSTS)
Content-Security-Policy	Mitigate XSS
Cache-Control	CDN and browser caching

Misconfigured **X-Forwarded-For** trust allows IP spoofing in rate limits—only trust headers from known L7

proxies.

14.8 13.8 Timeouts, keep-alive, and load balancers

Setting	Symptom if wrong
Client idle timeout	Premature connection close
LB idle timeout	502 during long requests
Upstream read timeout	504 gateway timeout
Keep-alive	Extra TCP/TLS handshakes if disabled

AWS ALB default idle timeout is 60s—long-running exports may need tuning on both app and LB.

14.9 13.9 Debugging TLS in production

Checklist:

1. **Expiry** — `openssl x509 -enddate` ; alert 30 days ahead.
 2. **Chain** — missing intermediate → Android/old clients fail.
 3. **SAN** — cert must cover exact hostname (`api` vs `api.internal`).
 4. **Clock skew** — NTP drift causes “cert not yet valid.”
 5. **SNI** — curl without `-H 'Host: '` or wrong `-servername` .
- `echo | openssl s_client -connect wrong-host.example.com:443 -servername api.example.com 2>&1 | grep subject`

14.10 13.10 Chapter summary

- HTTP methods and status codes drive **health checks and SLOs**.
- **TLS** encrypts and authenticates; understand handshake, SNI, and chains.
- Automate cert lifecycle with **ACME/cert-manager**—manual renewals fail at scale.
- **mTLS** secures east-west traffic in meshes and zero-trust designs.
- Align **timeouts and forwarded headers** between apps, proxies, and LBs.

14.11 Lab 13.1 — Inspect a live certificate

1. Run `openssl s_client` against three HTTPS sites you operate or public APIs.
2. Record issuer, SANs, and expiry dates in a table.
3. Simulate SNI: connect by IP with `-servername` and observe cert change on shared IP hosts.

14.12 Lab 13.2 — Broken chain simulation

1. In a lab nginx/ingress, serve a leaf cert without intermediate bundle.
2. Verify failure with `curl` on a minimal trust store container.
3. Fix by appending intermediate; document the full chain file order.

14.13 Review questions

1. What is the difference between HTTP 502 and 504?
2. What role does SNI play on shared load balancer IPs?
3. Name the three security properties TLS provides.
4. Why are short-lived Let's Encrypt certs acceptable?
5. What header carries client IP through proxies, and what is the spoofing risk?

Next: [*Chapter 14 — Load Balancing, Proxies, and CDN*](#)

15 Chapter 14: Load Balancing, Proxies, and CDN

DevOps Handbook — Part IV, Pages 246–265

15.1 14.1 Why load balancing exists

Single servers fail and saturate. **Load balancers** distribute traffic across healthy backends, terminate TLS, enforce routing rules, and provide a stable **virtual IP/DNS name** while instances churn behind them.

Goals:

Goal	Mechanism
Availability	Health checks remove bad targets
Scalability	Add backends horizontally
Maintainability	Rolling deploys without client reconfiguration
Security	WAF, TLS offload, DDoS absorption (with CDN)

15.2 14.2 Layer 4 vs Layer 7 load balancing

Type	OSI layer	Sees	Routing basis	Examples
L4 (NLB)	Transport	IP, port, TCP/UDP	IP:port, flow hash	AWS NLB, HAProxy TCP mode
L7 (ALB/HTTP)	Application	HTTP headers, path, cookies	URL path, Host, headers	AWS ALB, nginx, Envoy

L4 is faster and protocol-agnostic—great for databases, gRPC-TLS passthrough, or extreme throughput.

L7 enables path-based routing (/api → service A), host-based routing (api. vs www.), sticky sessions, and request manipulation.

Many architectures stack both: **CDN/L7 edge → L7 ingress → L4 service mesh**.

15.3 14.3 Load balancing algorithms

Algorithm	Behavior	Use when
Round robin	Cycle targets evenly	Homogeneous backends
Least connections	Send to least busy	Long-lived connections
Weighted	Favor stronger instances	Mixed instance sizes
IP hash	Same client → same backend	Simple session stickiness
Random / Maglev	Hash tables for stability	Large clusters (Google LB)

Health checks must reflect **application** health—not just TCP open port. Return 200 on /health with dependency checks (DB ping) when appropriate.

Unhealthy threshold and interval tune flapping vs slow detection:

```
interval: 10s
timeout: 5s
healthy_threshold: 2
unhealthy_threshold: 3
```

15.4 14.4 Reverse proxies vs forward proxies

	Reverse proxy	Forward proxy
Sits	In front of servers	In front of clients
Clients know	Backend exists (often hidden)	Proxy exists
Examples	nginx, Traefik, ALB	Corporate HTTP proxy, Squid
DevOps use	Ingress, API gateway	Egress control from CI/VPC

A **reverse proxy** terminates client connections and opens new ones to upstreams—enabling TLS offload, buffering, and rate limits.

15.5 14.5 Common proxy software

15.5.1 nginx (ingress workhorse)

```
upstream api_backend {
    least_conn;
    server 10.0.1.10:8080 max_fails=3 fail_timeout=30s;
    server 10.0.1.11:8080 max_fails=3 fail_timeout=30s;
}

server {
    listen 443 ssl http2;
    server_name api.example.com;

    location / {
        proxy_pass http://api_backend;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

15.5.2 HAProxy

Popular for TCP and HTTP with advanced health checks and stick tables.

15.5.3 Envoy

Data plane for Istio and modern API gateways; dynamic config via xDS.

Choose based on team expertise and platform (managed cloud LB vs self-run on VMs/K8s).

15.6 14.6 Session persistence (stickiness)

Stateful apps without shared session store may need **sticky sessions**:

- **Cookie-based** (ALB insert cookie).

- **Source IP affinity** (breaks behind carrier NAT—fragile).

Prefer **external session store** (Redis) over stickiness when possible—stickiness complicates deploys and failover.

15.7 14.7 Content Delivery Networks (CDN)

A **CDN** caches static and cacheable dynamic content at **edge PoPs** close to users—reducing latency and origin load.

Benefit	Explanation
Latency	RTT to edge << RTT to origin region
Scale	Absorb traffic spikes at edge
DDoS	Distributed scrubbing (vendor-dependent)
TLS at edge	Terminate close to user

Providers: CloudFront, Fastly, Cloudflare, Akamai, Google Cloud CDN.

15.7.1 Cache control

Origin sets policy via headers:

```
Cache-Control: public, max-age=3600
Cache-Control: no-store
ETag: "abc123"
```

Cache key often includes path, query string rules, and `Vary` headers—misconfiguration serves wrong content to wrong users (classic bug: caching authenticated pages without `private`).

15.7.2 Purge and invalidation

Deploy new `app.js` → purge `/static/*` or use **filename hashing** (`app.a1b2c3.js`) so cache busting is automatic.

15.8 14.8 Global load balancing and failover

Geo DNS or **anycast** routes users to nearest healthy region:

```
User in EU → GSLB → eu-west-1 (healthy)
Failover → us-east-1 if region unhealthy
```

Requires **state replication** or accept read-only failover; DNS TTL and health probe design determine RTO.

15.9 14.9 Kubernetes ingress pattern

```
Internet → Cloud LB → Ingress Controller (nginx/Envoy)
→ Service (ClusterIP) → Pod endpoints
```

`Ingress` resources define host/path rules; **Gateway API** is the evolving standard with clearer L4/L7 separation.

Service mesh adds **sidecar proxies** for east-west L7 policy without changing Service types.

15.10 14.10 Observability for load balancers

Monitor:

Metric	Indicates
Request count / latency p99	User experience
5xx rate	Upstream or LB misconfig
Healthy host count	Deploy or health check failures
TLS handshake time	Cert or cipher issues
Cache hit ratio	CDN efficiency

Log **X-Request-ID** from edge through origin for distributed traces.

15.11 14.11 Chapter summary

- **L4** balances connections; **L7** routes HTTP semantics—choose per use case.
 - Health checks and algorithms must match **workload shape** (short vs long connections).
 - **Reverse proxies** enable TLS, routing, and security at the edge.
 - **CDNs** cache at PoPs—control with headers, hashed assets, and purge strategy.
 - Instrument LB and CDN metrics as part of **SLO dashboards**.
-

15.12 Lab 14.1 — nginx upstream failover

1. Run two backend containers on ports 8081/8082 returning different bodies.
 2. Configure nginx upstream with one server marked down.
 3. curl through nginx; confirm traffic to healthy only.
 4. Stop healthy container; observe 502 until nginx marks unhealthy.
-

15.13 Lab 14.2 — CDN cache headers

1. Serve a static file from a simple web server with `Cache-Control: max-age=60`.
 2. If CDN unavailable, use two curl requests and `Age` header via local cache proxy—or document expected CDN behavior.
 3. Change file content without purge; explain stale risk.
 4. Switch to content-hashed filename; show safe long `max-age`.
-

15.14 Review questions

1. When would you choose L4 over L7 load balancing?
 2. Why is TCP-open health check insufficient for many apps?
 3. Difference between reverse and forward proxy?
 4. How do cache keys cause authenticated content leaks?
 5. What Kubernetes component typically replaces raw nginx on bare VMs?
-

Next: [Chapter 15 — Cloud Networking Patterns \(VPC overview\)](#)

16 Chapter 15: Cloud Networking Patterns — VPC Overview

DevOps Handbook — Part IV, Pages 266–280

16.1 15.1 VPC: your private network in the cloud

A **Virtual Private Cloud (VPC)** is an isolated virtual network in AWS, GCP (**VPC**), or Azure (**VNet**). You define IP ranges, subnets, routing, firewalls, and attachments to on-prem or other clouds.

DevOps engineers provision VPCs via Terraform, deploy workloads into **private subnets**, and expose services through **controlled ingress**—not public IPs on every VM.

Conceptual map (AWS-centric; others analogous):

AWS	GCP	Azure
VPC	VPC	Virtual Network
Subnet	Subnet	Subnet
Security Group	Firewall rule (VPC)	NSG
NACL	— (less common pattern)	NACL (optional)
Internet Gateway	—	Internet Gateway
NAT Gateway	Cloud NAT	NAT Gateway
Route 53 private zone	Cloud DNS private	Private DNS zone

16.2 15.2 CIDR design fundamentals

Plan **non-overlapping** CIDR blocks before peering or VPN:

```
VPC: 10.0.0.0/16
├─ public-a: 10.0.0.0/24 (AZ a) – LB, NAT
├─ public-b: 10.0.1.0/24 (AZ b)
├─ private-a: 10.0.10.0/24 (AZ a) – apps, workers
├─ private-b: 10.0.11.0/24 (AZ b)
└─ data-a/b: 10.0.20.0/24, 10.0.21.0/24 – databases
```

Rules of thumb:

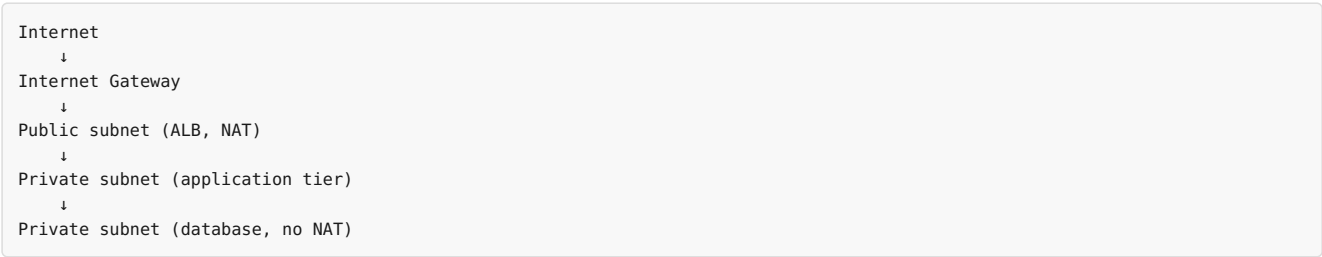
- Leave **spare capacity** in VPC CIDR for growth.
- **Multi-AZ** subnets for HA—one failure domain per AZ.
- Put **internet-facing LBs** in public subnets; **instances** in private subnets without public IPs.

16.3 15.3 Public vs private subnets

Subnet type	Default route	Typical resources
Public	0.0.0.0/0 → Internet Gateway	NAT GW, public ALB, bastion (legacy)
Private	0.0.0.0/0 → NAT Gateway	App servers, EKS nodes, RDS

	(optional)	
--	------------	--

Private subnets **without NAT** cannot reach the internet—ideal for data layers with VPC endpoints only.



16.4 15.4 Security groups vs NACLs

16.4.1 Security groups (stateful)

- Instance/network interface level.
- **Allow rules only**; implicit deny.
- **Stateful**—return traffic automatically allowed.
- Example: allow 443 from 0.0.0.0/0 to ALB SG; allow 8080 from ALB SG to app SG.

16.4.2 Network ACLs (stateless, AWS)

- Subnet level; optional deny rules.
- **Stateless**—must allow ephemeral return ports explicitly if used.
- Less common in day-to-day DevOps than SGs.

Defense in depth: SG for fine-grained app rules; NACL for coarse subnet blocks if compliance requires.

16.5 15.5 NAT, egress, and VPC endpoints

NAT Gateway lets private instances initiate outbound internet (package updates, external APIs) without inbound exposure. Costs and bandwidth add up—monitor NAT metrics.

VPC endpoints (AWS **Interface** and **Gateway** endpoints) keep traffic on AWS backbone:

Endpoint type	Example	Avoids
Gateway	S3, DynamoDB	Internet for AWS API
Interface	ECR, Secrets Manager, STS	Public NAT egress

For EKS pulling images from ECR, **interface endpoints** reduce NAT dependency and improve security posture.

16.6 15.6 Hybrid and multi-VPC connectivity

Pattern	Use case
VPC peering	Connect two VPCs (non-transitive)
Transit Gateway	Hub-and-spoke many VPCs
VPN / Direct Connect	On-prem to cloud

PrivateLink	Consumer access to service without full peering
--------------------	---

Peering requires **non-overlapping CIDRs**. Transitive routing pitfalls break naive “peer everything” designs—document a **network topology diagram**.

16.7 15.7 Kubernetes networking in VPC

Managed Kubernetes (EKS, GKE, AKS) integrates CNI plugins assigning pod IPs from VPC or overlay ranges.

DevOps concerns:

- **Pod CIDR vs VPC CIDR**—must not overlap peered networks.
- **Security groups for pods** (where supported) replace blanket node SG rules.
- **LoadBalancer Service** provisions cloud LB into public/private subnets per annotation.
- **Internal-only services** use internal LB annotations and private DNS.

16.8 15.8 IaC example (Terraform sketch)

```
resource "aws_vpc" "main" {
  cidr_block      = "10.0.0.0/16"
  enable_dns_hostnames = true
  enable_dns_support = true
}

resource "aws_subnet" "private_a" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.0.10.0/24"
  availability_zone = "us-east-1a"
}

resource "aws_security_group" "app" {
  name   = "app-sg"
  vpc_id = aws_vpc.main.id

  ingress {
    from_port = 8080
    to_port   = 8080
    protocol  = "tcp"
    security_groups = [aws_security_group.alb.id]
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

Store network diagrams alongside Terraform for reviewers who don’t read HCL daily.

16.9 15.9 Operational checklist

Before production launch:

1. **No 0.0.0.0/0 SSH** on app SGs—use SSM Session Manager or bastion with MFA.
2. **Flow logs** enabled on VPC or key subnets for forensics.
3. **Route tables** verified per subnet—accidental IGW on DB subnet is a data breach.
4. **DNS** private zones linked to VPC; resolver rules for hybrid.
5. **Quota limits** checked (EIPs, NAT GW per AZ, LB limits).

16.10 15.10 Chapter summary

- **VPC** isolates cloud resources with subnets, routes, and firewalls.
 - **Public subnets** expose ingress paths; **private subnets** protect workloads.
 - **Security groups** are stateful primary filters; know when **NACLs** apply.
 - **NAT** vs **VPC endpoints** trade cost, security, and simplicity for egress.
 - Plan **CIDR and peering** early; overlaps are painful to fix later.
-

16.11 Lab 15.1 — Paper VPC design

1. Design a VPC for a three-tier app (web, app, DB) in two AZs.
 2. Assign CIDR blocks and list route table entries for each subnet type.
 3. Document SG rules as a matrix: source → destination → port.
 4. Identify which traffic uses IGW, NAT, and VPC endpoints.
-

16.12 Lab 15.2 — Terraform mini-VPC (optional cloud)

1. Apply a module creating VPC + one public + one private subnet.
 2. Launch a test instance in private subnet without public IP.
 3. Verify outbound via NAT with `curl` and inbound denial from internet.
 4. Tear down to avoid charges.
-

16.13 Review questions

1. What is the difference between public and private subnets at the routing layer?
 2. Why are security groups described as stateful?
 3. When would you use a VPC endpoint instead of NAT for S3 access?
 4. What problem does non-overlapping CIDR solve in peering?
 5. Where should an internet-facing ALB sit in a well-designed VPC?
-

Next: [Chapter 16 — Python for DevOps](#)

17 Chapter 16: Python for DevOps — boto3, requests, pathlib

DevOps Handbook — Part V, Pages 281–305

17.1 16.1 Why Python dominates DevOps automation

Python balances readability, rich libraries, and fast iteration. It powers Ansible modules, AWS Lambda, glue scripts, data migration tools, and CI steps. You will use it alongside bash (Chapter 8) for anything beyond simple command wrapping.

Core libraries for this handbook:

Library	Purpose
boto3	AWS API SDK
requests	HTTP client
pathlib	Modern filesystem paths
json / yaml	Config and API payloads
subprocess	Safe CLI invocation
logging	Structured operational logs

17.2 16.2 Project layout and virtual environments

Never install packages globally on shared CI runners without pinning. Use **venv** per project:

```
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install boto3 requests
pip freeze > requirements.txt
```

Minimal repo layout:

```
tools/
  aws_inventory/
    __init__.py
    main.py
    requirements.txt
    README.md
tests/
  test_inventory.py
```

Pin versions in `requirements.txt` for reproducible CI:

```
boto3==1.34.162
requests==2.32.3
```

17.3 16.3 pathlib: filesystem without footguns

`pathlib.Path` replaces brittle string path concatenation:

```
from pathlib import Path

root = Path(__file__).resolve().parent
config_dir = root / "config"
log_file = Path("/var/log/app") / "deploy.log"

if not config_dir.exists():
    config_dir.mkdir(parents=True, exist_ok=True)

for yaml_file in config_dir.glob("*.yaml"):
    print(yaml_file.read_text(encoding="utf-8")[:200])

# Safe join – no double slashes
staging = root / ".." / "staging" / "inventory.json"
staging = staging.resolve()
```

Use `.read_text()`, `.write_text()`, `.iterdir()`, and `.glob()` instead of `os.path` when starting new scripts.

17.4 16.4 requests: HTTP for APIs and health checks

```
import requests
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

session = requests.Session()
retries = Retry(total=3, backoff_factor=0.5, status_forcelist=[502, 503, 504])
session.mount("https://", HTTPAdapter(max_retries=retries))

def get_health(url: str, timeout: float = 5.0) -> bool:
    try:
        resp = session.get(url, timeout=timeout)
        resp.raise_for_status()
        return resp.json().get("status") == "ok"
    except requests.RequestException as exc:
        print(f"health check failed: {exc}")
        return False

if __name__ == "__main__":
    ok = get_health("https://api.example.com/health")
    raise SystemExit(0 if ok else 1)
```

Best practices:

- Always set **timeouts** (`timeout=(connect, read)`).
- Use `raise_for_status()` or explicit status handling.
- Reuse **Session** for connection pooling in loops.
- Never disable TLS verify in production (`verify=False` is debug-only).

17.5 16.5 boto3: AWS automation

boto3 uses **botocore** for API calls. Credentials resolve via environment variables, `~/.aws/credentials`, or IAM role on EC2/EKS.

```
import boto3
from botocore.exceptions import ClientError

def list_stopped_instances(region: str = "us-east-1"):
    ec2 = boto3.client("ec2", region_name=region)
    paginator = ec2.get_paginator("describe_instances")
    stopped = []

    for page in paginator.paginate(
        Filters=[{"Name": "instance-state-name", "Values": ["stopped"]}])
```



```

):
    for reservation in page["Reservations"]:
        for inst in reservation["Instances"]:
            name = next(
                (t["Value"] for t in inst.get("Tags", []) if t["Key"] == "Name"),
                "unnamed",
            )
            stopped.append({"id": inst["InstanceId"], "name": name})
    return stopped

def stop_instance(instance_id: str, region: str = "us-east-1") -> None:
    ec2 = boto3.client("ec2", region_name=region)
    try:
        ec2.stop_instances(InstanceIds=[instance_id])
    except ClientError as e:
        raise RuntimeError(f"stop failed: {e}") from e

```

17.5.1 Resource vs client

API	Style	Use
client	Low-level dict responses	Full API coverage
resource	Object-oriented	Convenient for EC2/S3 common ops

```

s3 = boto3.resource("s3")
for bucket in s3.buckets.all():
    print(bucket.name)

```

Handle `ClientError` with error codes (`AccessDenied`, `InvalidParameterValue`) for actionable messages.

17.6 16.6 Idempotency and dry-run patterns

DevOps scripts should be safe to re-run:

```

def ensure_tag(ec2_client, instance_id: str, key: str, value: str) -> None:
    ec2_client.create_tags(
        Resources=[instance_id],
        Tags=[{"Key": key, "Value": value}],
    )
    # create_tags is idempotent for same key/value on AWS

```

Add `--dry-run` CLI flags for destructive operations; boto3 supports `DryRun` on many calls to validate permissions without effect.

17.7 16.7 CLI entry points with argparse

```

import argparse
from pathlib import Path

def main() -> int:
    parser = argparse.ArgumentParser(description="Deploy artifact to S3")
    parser.add_argument("--bucket", required=True)
    parser.add_argument("--file", type=Path, required=True)
    parser.add_argument("--region", default="us-east-1")
    args = parser.parse_args()

    if not args.file.is_file():
        parser.error(f"file not found: {args.file}")

    # upload logic here
    return 0

if __name__ == "__main__":

```

```
raise SystemExit(main())
```

Use `raise SystemExit(main())` so CI captures exit codes.

17.8 16.8 Logging instead of print

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(message)s",
)
log = logging.getLogger(__name__)

log.info("starting sync", extra={"bucket": "my-bucket"})
```

Structured JSON logging (for Loki/CloudWatch) can use `python-json-logger` in larger services.

17.9 16.9 Testing Python automation

```
# tests/test_health.py
from unittest.mock import patch, MagicMock
import inventory # your module

def test_get_health_ok():
    mock_resp = MagicMock()
    mock_resp.json.return_value = {"status": "ok"}
    mock_resp.raise_for_status = MagicMock()
    with patch("inventory.session.get", return_value=mock_resp):
        assert inventory.get_health("http://test") is True
```

Run with `pytest`; mock external APIs—never hit production AWS in unit tests.

17.10 16.10 Security notes

- Store secrets in env vars or secret managers—not in code.
 - Use **least-privilege IAM** policies scoped to required actions.
 - Validate input paths to prevent `../..` escapes when operating on filesystems.
 - Scan dependencies with **pip-audit** or SCA in CI.
-

17.11 16.11 Chapter summary

- Use **venv + pinned requirements** for reproducible automation.
 - **pathlib** simplifies safe filesystem operations.
 - **requests** needs timeouts, retries, and explicit error handling.
 - **boto3** clients/resources automate AWS—paginate and handle `ClientError`.
 - Add **argparse**, logging, dry-run, and **pytest** mocks for production-quality scripts.
-

17.12 Lab 16.1 — EC2 inventory script

1. Write `list_ec2_by_tag.py --tag-key Environment --tag-value staging`.

2. Output JSON list of instance ID, type, state, private IP.
 3. Run with read-only IAM; add `--output table` optional format.
 4. Add pytest with mocked boto3 responses.
-

17.13 Lab 16.2 — Health gate for CI

1. Implement `check_health.py --url URL --retries 5 --interval 10`.
 2. Exit non-zero on failure for use in GitHub Actions step.
 3. Document usage in README with example workflow snippet.
-

17.14 Review questions

1. Why use `requests.Session` in a loop?
 2. Difference between boto3 client and resource?
 3. How does pathlib improve on string path joins?
 4. What three elements belong in every HTTP call in automation?
 5. How do you test boto3 code without calling AWS?
-

Next: [Chapter 17 — Go for CLI Tools and Operators](#)

18 Chapter 17: Go for CLI Tools and Operators

DevOps Handbook — Part V, Pages 306–325

18.1 17.1 Why Go in the DevOps toolchain

Go (Golang) compiles to **single static binaries** with no runtime dependency—ideal for CLIs shipped to servers, minimal container images, and Kubernetes **operators**. The cloud-native ecosystem (Docker, Kubernetes, Terraform, Prometheus, Helm) is largely Go.

Strength	DevOps use
Fast compile	Quick iteration in CI
Static binary	Copy to jump host, air-gapped env
Concurrency	Parallel checks across hosts/regions
Strong stdlib	HTTP, TLS, encoding/json
kubebuilder ecosystem	Custom controllers and operators

Python wins for quick scripts; Go wins for **distributable tools** and **control loops**.

18.2 17.2 Module setup

Go 1.21+ with modules (no `$GOPATH` src layout required):

```
mkdir -p ~/go/devops-check && cd ~/go/devops-check
go mod init github.com/you/devops-check
```

```
devops-check/
cmd/
  devops-check/
    main.go
internal/
  health/
    health.go
go.mod
go.sum
```

Build:

```
go build -o bin/devops-check ./cmd/devops-check
GOOS=linux GOARCH=amd64 go build -o bin/devops-check-linux ./cmd/devops-check
```

18.3 17.3 A minimal CLI with cobra (pattern)

Many projects use **spf13/cobra** for subcommands and flags:

```
// cmd/devops-check/main.go
package main

import (
    "fmt"
    "os"
```

```

    "github.com/spf13/cobra"
    "github.com/you/devops-check/internal/health"
)

func main() {
    var url string
    var timeout int

    root := &cobra.Command{
        Use:   "devops-check",
        Short: "Operational health utilities",
    }

    checkCmd := &cobra.Command{
        Use:   "http",
        Short: "HTTP health check",
        RunE: func(cmd *cobra.Command, args []string) error {
            ok, err := health.CheckHTTP(url, timeout)
            if err != nil {
                return err
            }
            if !ok {
                return fmt.Errorf("unhealthy response from %s", url)
            }
            fmt.Println("OK")
            return nil
        },
    }
    checkCmd.Flags().StringVar(&url, "url", "", "URL to check")
    checkCmd.Flags().IntVar(&timeout, "timeout", 5, "timeout seconds")
    _ = checkCmd.MarkFlagRequired("url")

    root.AddCommand(checkCmd)
    if err := root.Execute(); err != nil {
        os.Exit(1)
    }
}

```

```

// internal/health/health.go
package health

import (
    "net/http"
    "time"
)

func CheckHTTP(url string, timeoutSec int) (bool, error) {
    client := &http.Client{Timeout: time.Duration(timeoutSec) * time.Second}
    resp, err := client.Get(url)
    if err != nil {
        return false, err
    }
    defer resp.Body.Close()
    return resp.StatusCode >= 200 && resp.StatusCode < 300, nil
}

```

18.4 17.4 Error handling and exit codes

Go uses explicit error returns—no exceptions:

```

if err := run(); err != nil {
    fmt.Fprintf(os.Stderr, "error: %v\n", err)
    os.Exit(1)
}

```

Wrap errors for context with Go 1.13+ `%w`:

```

return fmt.Errorf("fetch config: %w", err)

```

CI systems depend on **non-zero exit** for failures—never swallow errors silently.

18.5 17.5 Concurrency for parallel probes

Checking hundreds of endpoints sequentially is slow. Use goroutines with bounded concurrency:

```
sem := make(chan struct{}, 10) // max 10 concurrent
var wg sync.WaitGroup

for _, url := range urls {
    wg.Add(1)
    go func(u string) {
        defer wg.Done()
        sem <- struct{}{}
        defer func() { <-sem }()
        // check u
    }(url)
}
wg.Wait()
```

Always cap concurrency—unbounded goroutines against production can cause incidents.

18.6 17.6 Kubernetes operators (conceptual)

An **operator** extends Kubernetes with custom resources (CRDs) and a **control loop**:

```
Observe CR state → Compare desired vs actual → Act (create/update/delete) → Requeue
```

Built with **controller-runtime** or **kubebuilder**:

```
kubebuilder init --domain example.com --repo github.com/you/sidecar-operator
kubebuilder create api --group app --version v1 --kind SidecarConfig
```

DevOps engineers operate operators; platform teams author them for domain-specific automation (database failover, certificate renewal, tenant provisioning).

You don't need to write an operator on day one—**understand the pattern** when installing cert-manager, Prometheus operator, or custom internal controllers.

18.7 17.7 Testing Go CLIs

```
// internal/health/health_test.go
func TestCheckHTTP(t *testing.T) {
    srv := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        w.WriteHeader(http.StatusOK)
    }))
    defer srv.Close()

    ok, err := CheckHTTP(srv.URL, 2)
    if err != nil || !ok {
        t.Fatalf("expected ok, got ok=%v err=%v", ok, err)
    }
}
```

Run `go test ./...` in CI; use `httptest` for HTTP without network flakiness.

18.8 17.8 Release and distribution

Method	Notes
GitHub Releases	Attach cross-compiled binaries
Homebrew tap	macOS/Linux developer installs
Container	FROM scratch + static binary
go install	Private modules need GOPRIVATE

Example multi-arch build with GoReleaser or raw `G00S / G0ARCH` matrix in Actions.

18.9 17.9 When not to use Go

- One-off 10-line file rename on a laptop → bash.
- Heavy data science or ML pipelines → Python.
- Team lacks Go experience and tool won't be maintained → Python with typed hints (mypy).

Choose Go when **shipping a binary** or **Kubernetes reconciliation** is the core requirement.

18.10 17.10 Chapter summary

- Go produces **static binaries** suited to CLIs and cloud-native tooling.
 - Structure projects with `cmd/` and `internal/` packages.
 - Use **cobra** (or flag stdlib) for UX; return proper **exit codes**.
 - Bounded **concurrency** speeds checks without overloading targets.
 - **Operators** implement control loops for Kubernetes custom resources.
-

18.11 Lab 17.1 — Multi-target HTTP checker

1. Implement `devops-check http --url` accepting multiple URLs or a file of URLs.
 2. Run checks with max 5 concurrent workers; print failures to stderr.
 3. Exit 1 if any fail; add `go test` with httptest server.
-

18.12 Lab 17.2 — Cross-compile release

1. Build linux/amd64 and darwin/arm64 binaries in CI (or locally).
 2. Compute SHA256 checksums; upload as release artifacts in a practice repo.
 3. Document install instructions in README.
-

18.13 Review questions

1. Name two reasons Go is popular for Kubernetes ecosystem tools.
2. What is the purpose of the `internal/` directory convention?
3. Why cap goroutines when probing many hosts?
4. Describe the operator control loop in four steps.
5. When is Python a better choice than Go for DevOps tasks?

Next: [*Chapter 18 — JSON/YAML, Jinja2, and Templating*](#)

19 Chapter 18: JSON/YAML, Jinja2, and Templating

DevOps Handbook — Part V, Pages 326–345

19.1 18.1 Structured data in DevOps

Modern infrastructure is **data**, not manual clicking: CI matrices, Kubernetes manifests, Terraform variables, Ansible inventories, and API payloads. **JSON** and **YAML** are the two dominant serialization formats; **Jinja2** (in Ansible and many generators) templates text from variables.

Misformatted YAML has caused production outages—treat config editing as code review with schema validation.

19.2 18.2 JSON essentials

JSON is strict: double quotes, no trailing commas, limited types (object, array, string, number, boolean, null).

```
{
  "service": "payments-api",
  "replicas": 3,
  "env": {
    "LOG_LEVEL": "info",
    "FEATURE_RETRY": true
  },
  "ports": [8080, 9090]
}
```

19.2.1 CLI tooling

```
echo '{"a":1}' | jq '.a'
jq -r '.items[].name' deploy.json
yq -o=json '.' config.yaml      # yq v4: YAML ↔ JSON
python3 -m json.tool < messy.json  # Pretty-print
```

In Python:

```
import json
from pathlib import Path

data = json.loads(Path("config.json").read_text(encoding="utf-8"))
data["replicas"] = 5
Path("config.out.json").write_text(
    json.dumps(data, indent=2, sort_keys=True) + "\n",
    encoding="utf-8",
)
```

19.3 18.3 YAML essentials and pitfalls

YAML is human-friendly but ambiguous if abused.

```
service: payments-api
replicas: 3
env:
  LOG_LEVEL: info
  FEATURE_RETRY: true
ports:
  - 8080
  - 9090
```

19.3.1 Gotchas

Issue	Example	Fix
Norway problem	N0: off → false in some parsers	Quote "N0"
Octal numbers	0123 misread	Quote strings
Tabs	Tab indentation	Spaces only (2 spaces common)
Multi-document	--- separators	Split or use explicit loader
Aliases	&id, *id	Powerful; confusing in reviews

Kubernetes and GitHub Actions use YAML extensively—run `yamllint` in CI.

Python safe load (never `yaml.load` without Loader):

```
import yaml

with open("deploy.yaml", encoding="utf-8") as f:
    doc = yaml.safe_load(f)
```

19.4 18.4 JSON vs YAML: when to use which

Choose JSON	Choose YAML
API request/response	Kubernetes, Ansible, docker-compose
Machine-generated only	Human-edited config with comments
Strict schema validation	Long nested config files
jq everywhere	Inline multiline strings (<code>\ </code> , <code>></code>)

Many pipelines **author YAML, convert to JSON** for tools that only accept JSON.

19.5 18.5 Templating concepts

Templates separate **structure** from **values**:

```
template + variables → rendered config
```

Engines:

Engine	Context
Jinja2	Ansible, Salt, some CI generators
Go text/template	Helm charts, some CLIs
Terraform HCL	Built-in interpolation
envsubst	Shell-level \$VAR replacement

Jinja2 syntax preview:

```
# nginx.conf.j2
upstream {{ service_name }} {
{% for backend in backends %}
    server {{ backend.host }}:{{ backend.port }};
{% endfor %}
}
```

Rendered with variables:

```
from jinja2 import Environment, FileSystemLoader, select_autoescape

env = Environment(
    loader=FileSystemLoader("templates"),
    autoescape=select_autoescape(default=False),
)
template = env.get_template("nginx.conf.j2")
print(template.render(
    service_name="api",
    backends=[{"host": "10.0.1.10", "port": 8080}],
))
```

19.6 18.6 Jinja2 in Ansible

Ansible playbooks are YAML; **templates** live in `templates/` and deploy via `template` module:

```
- name: Deploy nginx config
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: /etc/nginx/conf.d/api.conf
    mode: "0644"
    notify: Reload nginx
```

Variables come from inventory, group_vars, host_vars, and facts:

```
# group_vars/api.yml
service_name: payments-api
backends:
  - host: 10.0.10.5
    port: 8080
  - host: 10.0.10.6
    port: 8080
```

Filters and tests (`| default('')` , `when: env == 'prod'`) keep templates readable—avoid business logic soup in Jinja.

19.7 18.7 Helm and Go templates (brief)

Helm charts template Kubernetes YAML:

```
# templates/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "mychart.fullname" . }}
spec:
  replicas: {{ .Values.replicaCount }}
```

`helm template` renders locally for review before apply—always render in CI and diff against cluster.

19.8 18.8 Schema validation

Prevent bad config from merging:

Tool	Format
jsonschema	JSON Schema for JSON/YAML
kubeconform	Kubernetes OpenAPI
checkov / tfsec	IaC policies
ajv	JSON in Node CI

Example with Python jsonschema:

```
from jsonschema import validate

schema = {
    "type": "object",
    "required": ["service", "replicas"],
    "properties": {
        "service": {"type": "string"},
        "replicas": {"type": "integer", "minimum": 1},
    },
}
validate(instance=doc, schema=schema)
```

19.9 18.9 Secrets and templating

Never template **plaintext secrets** into Git-tracked files. Patterns:

- Reference `{{ vault_db_password }}` from Ansible Vault or External Secrets.
- Render secrets at deploy time from CI secret store.
- Use `envsubst` on runtime-only files in ephemeral CI job.

Redact rendered output in logs (`no_log: true` in Ansible for sensitive tasks).

19.10 18.10 Chapter summary

- **JSON** is strict and universal for APIs; **YAML** powers K8s and Ansible with comment-friendly syntax.
- Watch YAML **gotchas** (types, tabs, unquoted `NO`).
- **Jinja2** separates templates from data—used heavily in Ansible.
- **Validate** structured config with schemas before apply.
- Keep **secrets out** of rendered artifacts in version control.

19.11 Lab 18.1 — Render nginx from Jinja2

1. Create `templates/upstream.conf.j2` with loop over backends.
2. Write `render.py` reading variables from `vars.yaml`.
3. Add jsonschema validation for `vars.yaml` structure.
4. Break validation intentionally; confirm CI/local script fails.

19.12 Lab 18.2 — YAML lint pipeline

1. Add `yamllint` config to a repo with sample K8s manifests.
 2. Fix indentation and document-start issues.
 3. Add GitHub Actions or GitLab job running yamllint on every PR.
-

19.13 Review questions

1. Why is `yaml.safe_load` preferred over bare `yaml.load`?
 2. Name two YAML features that cause review confusion.
 3. What problem does Jinja2 solve in Ansible?
 4. How does Helm differ from plain Jinja2 templating?
 5. Why validate config against a schema in CI?
-

Next: [*Chapter 19 — Testing Automation Scripts*](#)

20 Chapter 19: Testing Automation Scripts

DevOps Handbook — Part V, Pages 346–360

20.1 19.1 Why test automation code?

Pipeline scripts and operational tools **are production software**. Untested deploy scripts cause outages as surely as application bugs. Testing automation yields:

Benefit	Example
Regression safety	Refactor deploy.sh without fear
Documentation	Tests show expected behavior
Faster review	CI green reduces manual guesswork
Confidence in rollbacks	Verified restore procedures

The investment scales with **blast radius**—a one-off migration may need smoke tests only; a shared deploy tool needs full unit and integration coverage.

20.2 19.2 Test pyramid for DevOps scripts



Layer	Scope	Speed	Cost
Unit	Pure logic, mocks	Fast	Low
Integration	Real local services	Medium	Medium
E2E	Staging environment	Slow	High

Run **many unit, some integration, few E2E** in CI budgets.

20.3 19.3 Testing bash scripts

Tools:

Tool	Role
bats	Bash Automated Testing System
shellcheck	Static analysis (not tests, but required)
shfmt	Formatting consistency

Example bats test:

```
#!/usr/bin/env bats
```

```
# tests/healthcheck.bats

setup() {
    load '../scripts/healthcheck.sh'
}

@test "returns 0 for HTTP 200" {
    run curl_mock 200
    [ "$status" -eq 0 ]
}
```

Pragmatic approach: extract logic to `lib/*.sh` functions, source in tests, mock external commands by placing stubs early in `PATH`:

```
# tests/bin/curl - mock
#!/usr/bin/env bash
echo "200"
```

Run in CI:

```
- name: Shellcheck
  run: shellcheck scripts/*.sh
- name: Bats
  run: bats tests/
```

20.4 19.4 Testing Python automation

Use **pytest** with mocks for cloud and HTTP:

```
from unittest.mock import patch, MagicMock
import boto3
import mymodule

@patch("boto3.client")
def test_list_buckets(mock_client):
    mock_client.return_value.list_buckets.return_value = {
        "Buckets": [{"Name": "a"}, {"Name": "b"}]
    }
    names = mymodule.list_bucket_names()
    assert names == ["a", "b"]
```

Fixtures for temp filesystem:

```
import pytest
from pathlib import Path

@pytest.fixture
def tmp_config(tmp_path):
    p = tmp_path / "config.yaml"
    p.write_text("replicas: 2\n", encoding="utf-8")
    return p
```

Run `pytest --cov=mymodule --cov-report=term-missing` for coverage gaps on critical paths.

20.5 19.5 Testing Go CLIs

Go's `testing` package plus `httptest` cover most CLI logic (Chapter 17). Test `RunE` functions by extracting business logic from `main`.

Table-driven tests:

```
tests := []struct {
```

```

    name    string
    code    int
    wantOK  bool
  }{
    {"ok", 200, true},
    {"fail", 500, false},
  }
  for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
      // ...
    })
  }
}

```

20.6 19.6 Integration testing with containers

docker-compose or **testcontainers** spin real dependencies:

```

# docker-compose.test.yml
services:
  localstack:
    image: localstack/localstack:3
    environment:
      SERVICES: s3,ec2
    ports:
      - "4566:4566"

```

Point boto3 to `endpoint_url=http://localhost:4566` in integration tests marked `@pytest.mark.integration` —skip by default in fast CI jobs.

For Kubernetes manifests: **kind** cluster in CI, `kubectl apply --dry-run=server`, or **kubeconform** validation.

20.7 19.7 Testing Terraform and Ansible

Tool	Test approach
Terraform	<code>terraform validate</code> , <code>tflint</code> , <code>checkov</code> , terraform plan in CI
Ansible	<code>ansible-playbook --syntax-check</code> , molecule with docker driver
Helm	<code>helm template</code> + kubeconform, chart unit tests

Terratest (Go) applies modules in ephemeral AWS accounts—powerful but slow; use for module maintainers.

20.8 19.8 CI pipeline layout for script repos

```

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: shellcheck scripts/*.sh || true
      - run: pip install ruff && ruff check .
      - run: yamllint .

  unit:
    needs: lint
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install -r requirements-dev.txt
      - run: pytest -q --ignore=tests/integration

```



```

integration:
  needs: unit
  runs-on: ubuntu-latest
  services:
    localstack:
      image: localstack/localstack:3
      ports:
        - 4566:4566
  steps:
    - run: pytest tests/integration -q

```

Gate merges on **lint + unit**; run integration on main or nightly if slow.

20.9 19.9 Test data and fixtures

- Store **sanitized** sample JSON/YAML in `tests/fixtures/`.
- Never commit real **secrets** or production dumps.
- Use **factories** for repetitive object graphs (Python `dataclasses`, Go structs).

Golden file testing: compare rendered template to committed expected output; update with explicit `--update-golden` flag when intentional.

20.10 19.10 When tests are enough

Minimum bar before merging shared automation:

1. **shellcheck / ruff / go vet** clean.
2. **Unit tests** for parsing, decision branches, error paths.
3. **Dry-run** or plan-only validation for infra changes.
4. **README** with usage and rollback notes.

For incident one-offs, a **checklist and peer review** may suffice—promote to tested tooling if reused twice.

20.11 19.11 Chapter summary

- Automation scripts deserve the same **quality gates** as application code.
 - Favor **unit tests with mocks**; add integration tests for cloud APIs via LocalStack/containers.
 - Use **bats**, **pytest**, and Go `testing` per language.
 - Validate **Terraform/Ansible/Helm** with domain-specific tools in CI.
 - Match test depth to **blast radius** and reuse frequency.
-

20.12 Lab 19.1 — pytest suite for inventory script

1. Take the EC2 inventory script from Lab 16.1 (or a stub module).
 2. Add unit tests with mocked boto3 for empty, single, and paginated responses.
 3. Wire pytest into GitHub Actions; fail PR on coverage below 80% for module.
-

20.13 Lab 19.2 — bats for bash healthcheck

1. Add bats tests for the healthcheck script from Lab 8.1.
2. Mock `curl` via PATH stub returning 200, 500, and connection failure.

3. Run shellcheck + bats in CI.
-

20.14 Review questions

1. Why are untested deploy scripts risky in DevOps?
 2. What belongs in unit vs integration tests for a boto3 script?
 3. How can you test bash that wraps curl without hitting real URLs?
 4. Name two tools for validating Terraform in CI besides `apply`.
 5. When is it acceptable to skip automated tests?
-

Next: [*Chapter 20 — IaC Principles: State, Idempotency, Drift*](#)

21 Chapter 20: IaC Principles — State, Idempotency, and Drift

DevOps Handbook — Part VI, Pages 361–380

21.1 20.1 What is Infrastructure as Code?

Infrastructure as Code (IaC) means defining servers, networks, databases, and cloud resources in **machine-readable files** stored in version control—treated like application code.

Benefits:

- **Reproducibility** — Identical dev/staging/prod environments
- **Auditability** — Git history shows who changed what
- **Speed** — Provision in minutes, not days
- **Documentation** — Code is the source of truth
- **Disaster recovery** — Rebuild from code

21.2 20.2 Declarative vs imperative

Style	You specify	Example tools
Declarative	Desired end state	Terraform, CloudFormation, Kubernetes YAML
Imperative	Steps to execute	Ansible (mostly), shell scripts, AWS CLI scripts

Declarative tools compare desired state to actual state and compute a plan. **Imperative** tools run commands in sequence.

Most modern IaC is declarative with imperative escape hatches.

21.3 20.3 Idempotency

An operation is **idempotent** if running it multiple times produces the same result.

```
# Ansible – idempotent: "ensure nginx is installed and running"
- name: Install nginx
  apt:
    name: nginx
    state: present

- name: Start nginx
  service:
    name: nginx
    state: started
```

Running the playbook twice does not install nginx twice or restart unnecessarily.

Why it matters: CI/CD pipelines re-run constantly; non-idempotent scripts cause drift and outages.

21.4 20.4 State management

Tools like **Terraform** track what they created in a **state file** (`terraform.tfstate`).

State maps resource addresses to real cloud IDs:

```
{
  "resources": [{
    "type": "aws_s3_bucket",
    "name": "logs",
    "instances": [{
      "attributes": { "id": "my-app-logs-bucket", "arn": "arn:aws:s3::..." }
    }]
  }]
}
```

21.4.1 State best practices

1. **Remote state** — Store in S3 + DynamoDB lock (AWS), GCS, Terraform Cloud
2. **Never commit secrets** in state (may contain sensitive values)
3. **Locking** — Prevent concurrent `terraform apply` from corrupting state
4. **State encryption** at rest

21.5 20.5 Configuration drift

Drift occurs when live infrastructure diverges from IaC definitions (manual console changes, emergency fixes).

Detection:

- `terraform plan` shows drift before apply
- AWS Config rules
- Scheduled drift detection jobs

Remediation:

- **Reconcile** — Apply IaC to restore desired state
- **Import** — Bring manual resources under IaC management
- **Prevent** — Restrict console access; require IaC PRs

21.6 20.6 IaC workflow in teams

Developer → PR with .tf changes → CI runs `terraform plan` → Review → Merge
→ CD runs `terraform apply` → Slack notification

21.6.1 Directory structure (example)

```
infrastructure/
├── modules/
│   ├── vpc/
│   ├── eks/
│   └── rds/
├── environments/
│   ├── dev/
│   ├── staging/
│   └── prod/
└── README.md
```

Each environment calls shared modules with different variables.

21.7 20.7 Testing IaC

Level	Tool	What it checks
Lint	<code>terraform fmt</code> , <code>tflint</code> , <code>cfn-lint</code>	Syntax and style
Static security	<code>checkov</code> , <code>tfsec</code> , <code>kics</code>	Misconfigurations
Plan	<code>terraform plan</code> in CI	Expected changes
Integration	Terratest, kitchen- terraform	Real deploy in test account

21.8 20.8 Chapter summary

- IaC makes infrastructure versioned, repeatable, and reviewable.
- Prefer **declarative**, **idempotent** tools.
- Manage **state** remotely with locking; detect and fix **drift**.

21.9 Lab 20.1

1. Manually change a tag on a Terraform-managed resource in the AWS console.
2. Run `terraform plan` and observe drift output.
3. Run `terraform apply` to reconcile.

Next: [Chapter 21 — Terraform](#)

22 Chapter 21: Terraform — HCL, Providers, and Modules

DevOps Handbook — Part VI, Pages 381–410

22.1 21.1 Terraform overview

Terraform (HashiCorp / OpenTofu fork) is the most widely used multi-cloud IaC tool. You write **HCL** (HashiCorp Configuration Language); Terraform calls cloud APIs via **providers**.

```
terraform init      # Download providers, init backend
terraform fmt       # Format code
terraform validate  # Syntax check
terraform plan      # Preview changes
terraform apply     # Create/update/destroy resources
terraform destroy   # Tear down (use carefully)
```

22.2 21.2 Basic configuration

main.tf:

```
terraform {
  required_version = ">= 1.5"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }

  backend "s3" {
    bucket         = "my-terraform-state"
    key            = "prod/vpc/terraform.tfstate"
    region        = "us-east-1"
    dynamodb_table = "terraform-locks"
    encrypt        = true
  }
}

provider "aws" {
  region = var.aws_region
}

resource "aws_s3_bucket" "logs" {
  bucket = "${var.project_name}-logs-${var.environment}"

  tags = {
    Project      = var.project_name
    Environment  = var.environment
    ManagedBy    = "terraform"
  }
}
```

variables.tf:

```
variable "aws_region" {
  type    = string
  default = "us-east-1"
}

variable "project_name" {
```

```

    type = string
  }

  variable "environment" {
    type = string
    validation {
      condition     = contains(["dev", "staging", "prod"], var.environment)
      error_message = "environment must be dev, staging, or prod."
    }
  }
}

```

outputs.tf:

```

output "logs_bucket_name" {
  value      = aws_s3_bucket.logs.id
  description = "Name of the logs S3 bucket"
}

```

Apply with: `terraform apply -var="project_name=myapp" -var="environment=dev"`

22.3 21.3 Resources, data sources, and dependencies

22.3.1 Data sources — read existing infrastructure

```

data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["al2023-ami-*-x86_64"]
  }
}

resource "aws_instance" "web" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = "t3.micro"
}

```

22.3.2 Implicit vs explicit dependencies

Terraform infers dependencies from references. Use `depends_on` when ordering matters but no attribute reference exists.

22.4 21.4 Modules — reusable infrastructure

modules/vpc/main.tf:

```

resource "aws_vpc" "this" {
  cidr_block      = var.cidr_block
  enable_dns_hostnames = true
  tags = { Name = var.name }
}

resource "aws_subnet" "public" {
  count            = length(var.public_subnet_cidrs)
  vpc_id          = aws_vpc.this.id
  cidr_block      = var.public_subnet_cidrs[count.index]
  availability_zone = var.availability_zones[count.index]
  map_public_ip_on_launch = true
}

```

Using the module:

```

module "vpc" {
  source = "../modules/vpc"

  name           = "prod-vpc"
  cidr_block     = "10.0.0.0/16"
  public_subnet_cidrs = ["10.0.1.0/24", "10.0.2.0/24"]
  availability_zones = ["us-east-1a", "us-east-1b"]
}

```

Publish modules to **Terraform Registry** or private registries for org-wide reuse.

22.5 21.5 Lifecycle meta-arguments

```

resource "aws_instance" "web" {
  ami           = var.ami_id
  instance_type = "t3.micro"

  lifecycle {
    create_before_destroy = true
    prevent_destroy       = false # Set true for prod databases
    ignore_changes        = [tags["LastReboot"]] # Ignore specific drift
  }
}

```

22.6 21.6 Workspaces and environments

```

terraform workspace list
terraform workspace new staging
terraform workspace select prod

```

Workspaces share the same backend key by default—often **separate directories per environment** (with separate state keys) is clearer for production.

22.7 21.7 CI/CD integration

GitHub Actions example (plan on PR):

```

- name: Terraform Plan
  run: |
    terraform init -input=false
    terraform plan -input=false -no-color -out=tfplan
  env:
    AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
    AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }

```

Apply only from protected `main` branch after approval.

22.8 21.8 Chapter summary

- Terraform workflow: **init** → **plan** → **apply** with remote state.
 - Use **variables**, **outputs**, and **modules** for reuse.
 - Run `terraform plan` in CI on every PR; apply from trusted pipelines only.
-

22.9 Lab 21.1

1. Write Terraform to create an S3 bucket with versioning and encryption.
2. Extract a VPC module with 2 public subnets.
3. Configure S3 backend for remote state.
4. Run plan/apply in a free-tier AWS account.

Next: [Chapter 26 — Docker](#)

23 Chapter 22: Terraform — Workspaces, Backends, and CI Integration

DevOps Handbook — Part VI, Pages 411–430

23.1 22.1 Why remote state and backends matter

Terraform’s **state file** is the source of truth mapping HCL resource addresses to real cloud IDs. Local state on a laptop does not scale for teams: it cannot be shared, locked, or audited. A **backend** configures where Terraform stores state and how it acquires **locks** during `plan` and `apply`.

Backend type	Locking	Best for
local	None	Solo learning only
s3 + DynamoDB	Yes	AWS teams
azurerm	Yes	Azure
gcs	Yes	GCP
remote (Terraform Cloud/Enterprise)	Yes	Org policy, run UI, Sentinel
kubernetes	Via API	GitOps-native shops

Remote state enables collaboration, CI pipelines, and disaster recovery. Treat state as sensitive: it often contains secrets even when outputs are marked `sensitive`.

23.2 22.2 Configuring S3 backend with locking

backend.tf:

```
terraform {
  backend "s3" {
    bucket      = "acme-terraform-state"
    key         = "networking/vpc/terraform.tfstate"
    region      = "us-east-1"
    dynamodb_table = "terraform-locks"
    encrypt     = true
    kms_key_id   = "arn:aws:kms:us-east-1:123456789012:key/abcd-1234"
  }
}
```

Bootstrap the backend itself with a one-time local state apply, or use a separate `bootstrap/` directory:

```
# bootstrap/main.tf – run once with local backend
resource "aws_s3_bucket" "state" {
  bucket = "acme-terraform-state"
}

resource "aws_s3_bucket_versioning" "state" {
  bucket = aws_s3_bucket.state.id
  versioning_configuration { status = "Enabled" }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "state" {
  bucket = aws_s3_bucket.state.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "aws:kms"
    }
  }
}
```

```

}
}

resource "aws_dynamodb_table" "locks" {
  name         = "terraform-locks"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "LockID"

  attribute {
    name = "LockID"
    type = "S"
  }
}

```

23.2.1 Partial backend configuration

Pass backend settings at `init` time for multi-environment reuse:

```

terraform init \
  -backend-config="bucket=acme-terraform-state" \
  -backend-config="key=prod/app/terraform.tfstate" \
  -backend-config="region=us-east-1" \
  -backend-config="dynamodb_table=terraform-locks"

```

Use `backend.hcl` files per environment and `-backend-config=backend-prod.hcl`.

23.3 22.3 Terraform workspaces

Workspaces partition a single configuration into multiple isolated state instances within the same backend. Each workspace gets its own state object (for S3 backends, the key becomes `env:/<workspace>/...`).

```

terraform workspace list
terraform workspace new staging
terraform workspace select prod
terraform workspace show

```

Reference workspace in HCL:

```

locals {
  environment = terraform.workspace

  instance_type = {
    dev     = "t3.micro"
    staging = "t3.small"
    prod    = "m6i.large"
  }[terraform.workspace]
}

resource "aws_instance" "app" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = local.instance_type
  tags = {
    Environment = local.environment
  }
}

```

23.3.1 Workspaces vs directory-per-environment

Approach	Pros	Cons
Workspaces	One codebase, quick switching	Easy to apply to wrong workspace
Separate directories (<code>env/prod</code> , <code>env/staging</code>)	Explicit, clearer in CI	Duplication unless using modules

Separate repos	Strong blast-radius isolation	Harder to keep modules in sync
----------------	-------------------------------	--------------------------------

Production teams often prefer **separate state keys per environment** (directory or `-backend-config`) over workspaces alone, because `terraform workspace select` mistakes have caused real outages.

23.4 22.4 State operations and safety

```
terraform state list
terraform state show aws_instance.app
terraform state mv aws_instance.old aws_instance.new
terraform state rm aws_instance.orphan # Remove from state, not from cloud
terraform import aws_instance.app i-0abc123def456
```

`terraform state rm` does not destroy the resource—it only forgets it. `terraform import` adopts existing infrastructure into state.

For large refactors, use `moved blocks` (Terraform 1.1+):

```
moved {
  from = aws_instance.web
  to   = module.compute.aws_instance.web
}
```

Enable **state encryption, versioning** on the S3 bucket, and restrict IAM to CI roles and break-glass admin roles only.

23.5 22.5 CI/CD integration patterns

23.5.1 Pull request: plan only

```
# .github/workflows/terraform.yml
name: Terraform
on:
  pull_request:
    paths: ['infra/**']
  push:
    branches: [main]
    paths: ['infra/**']

permissions:
  id-token: write # OIDC
  contents: read
  pull-requests: write

jobs:
  plan:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: infra
    steps:
      - uses: actions/checkout@v4

      - uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: 1.9.0

      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/github-terraform-plan
          aws-region: us-east-1

      - run: terraform init -input=false
```

```

- run: terraform fmt -check -recursive
- run: terraform validate
- run: terraform plan -input=false -no-color -out=tfplan

- uses: actions/upload-artifact@v4
  with:
    name: tfplan
    path: infra/tfplan

```

Post plan output as a PR comment with `terraform show -no-color tfplan` or tools like **atlantis** or **terraform-cloud** run tasks.

23.5.2 Main branch: apply with approval

```

apply:
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'
  needs: plan
  environment: production # Requires GitHub Environment approval
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: hashicorp/setup-terraform@v3
    - uses: aws-actions/configure-aws-credentials@v4
      with:
        role-to-assume: arn:aws:iam::123456789012:role/github-terraform-apply
        aws-region: us-east-1
    - run: terraform init -input=false
    - run: terraform apply -input=false -auto-approve

```

23.5.3 OIDC over long-lived keys

Prefer **OIDC federation** (`aws-actions/configure-aws-credentials` with `role-to-assume`) instead of storing `AWS_ACCESS_KEY_ID` in GitHub Secrets. Scope IAM roles: plan role read-only, apply role write with conditions on branch and environment.

23.6 22.6 Atlantis and Terraform Cloud

Atlantis runs `terraform plan` on PRs inside your VPC:

```

# atlantis.yaml
version: 3
projects:
  - name: prod-vpc
    dir: infra/networking
    workspace: default
    autoplan:
      when_modified: ["**/*.tf", "**/*.tfvars"]
    apply_requirements: [approved, mergeable]

```

Terraform Cloud (TFC) / HCP Terraform centralizes runs, variables, and policy (Sentinel/OPA). The `cloud` block replaces manual backend config:

```

terraform {
  cloud {
    organization = "acme-corp"
    workspaces {
      name = "prod-networking"
    }
  }
}

```

23.7 22.7 Testing and policy in the pipeline

Tool	Purpose
tflint	Lint provider-specific mistakes
checkov / tfsec	Security misconfiguration scan
terratest	Go integration tests against real APIs
terraform test (1.6+)	Native <code>.tftest.hcl</code> unit tests

```
# tests/vpc.tftest.hcl
run "valid_cidr" {
  command = plan
  variables {
    cidr_block = "10.0.0.0/16"
  }
  assert {
    condition      = aws_vpc.this.cidr_block == "10.0.0.0/16"
    error_message = "VPC CIDR mismatch"
  }
}
```

Run `terraform test` in CI before `plan`.

23.8 22.8 Chapter summary

- Use **remote backends** with **locking**; never rely on local state in teams.
- **Workspaces** isolate state within one config; **separate state keys** per environment are often safer for production.
- CI should run **fmt**, **validate**, **test**, **scan**, and **plan** on every PR; **apply** only from protected branches with human approval.
- Prefer **OIDC** credentials and least-privilege IAM roles for pipeline identities.

23.9 Lab 22.1

1. Bootstrap an S3 bucket and DynamoDB table for Terraform state.
2. Migrate a Chapter 21 project from local to remote backend.
3. Create `dev` and `prod` workspaces (or separate backend keys) with different instance types.
4. Add a GitHub Actions workflow that runs `terraform plan` on PRs.
5. Import an existing S3 bucket into Terraform state.

23.10 Lab 22.2

1. Configure Atlantis (or TFC free tier) for automated PR plans.
2. Add `tflint` and `checkov` steps that fail the pipeline on high-severity findings.
3. Write a `terraform test` file validating a module output.

23.11 Review questions

1. What happens if two engineers run `terraform apply` simultaneously without state locking?
2. When would you choose separate directories over Terraform workspaces?
3. Why should the plan role in CI have fewer permissions than the apply role?
4. What is the difference between `terraform state rm` and `terraform destroy`?
5. How does OIDC improve on static AWS access keys in GitHub Actions?

Next: [*Chapter 23 — Ansible Playbooks, Roles, Inventories*](#)

24 Chapter 23: Ansible — Playbooks, Roles, and Inventories

DevOps Handbook — Part VI, Pages 431–450

24.1 23.1 Ansible in the IaC landscape

Ansible is an agentless configuration management and automation tool. It connects over SSH (Linux) or WinRM (Windows), pushes **modules** that declare desired state, and exits. Unlike Terraform, Ansible does not maintain a state file—it interrogates each host every run.

Tool	Primary use	State
Terraform	Provision cloud resources	Remote state file
Ansible	Configure OS, deploy apps, orchestrate	None (facts cache optional)
Packer	Build machine images	Ephemeral
Kubernetes	Run container workloads	etcd

Ansible complements Terraform: Terraform creates the VPC and EC2 instances; Ansible installs nginx, agents, and application releases.

24.2 23.2 Inventory — defining targets

An **inventory** lists hosts and groups. Static INI format:

```
# inventories/production/hosts.ini
[web]
web01.example.com ansible_host=10.0.1.10
web02.example.com ansible_host=10.0.1.11

[db]
db01.example.com ansible_host=10.0.2.10

[web:vars]
ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/prod.pem

[production:children]
web
db
```

YAML inventory:

```
all:
  children:
    web:
      hosts:
        web01.example.com:
          ansible_host: 10.0.1.10
      vars:
        http_port: 80
    db:
      hosts:
        db01.example.com:
          ansible_host: 10.0.2.10
```

24.2.1 Dynamic inventory

Query cloud APIs at runtime:

```
# AWS EC2 dynamic inventory (ansible-core 2.12+)
ansible-inventory -i aws_ec2.yml --graph
```

aws_ec2.yml :

```
plugin: amazon.aws.aws_ec2
regions:
  - us-east-1
filters:
  tag:Environment: production
  instance-state-name: running
keyed_groups:
  - key: tags.Role
    prefix: role
hostnames:
  - private-ip-address
```

Run against a group: `ansible-playbook site.yml -i aws_ec2.yml --limit role_web`

24.3 23.3 Playbooks — ordered automation

A **playbook** is YAML listing **plays** (mapped to host groups) and **tasks** (module calls).

site.yml :

```
---
- name: Configure web tier
  hosts: web
  become: true
  vars:
    app_version: "2.4.1"

  pre_tasks:
    - name: Wait for SSH
      wait_for_connection:
        timeout: 300

  roles:
    - common
    - nginx
    - app_deploy

  post_tasks:
    - name: Verify health endpoint
      uri:
        url: "http://localhost/health"
        status_code: 200
      register: health
      retries: 5
      delay: 10
      until: health.status == 200
```

playbook.yml for a single concern:

```
---
- name: Patch all production servers
  hosts: production
  become: true
  serial: "25%"           # Rolling update – 25% at a time
  max_fail_percentage: 0

  tasks:
    - name: Update packages (Debian)
      apt:
        update_cache: true
        upgrade: dist
      when: ansible_os_family == "Debian"
```

```

- name: Reboot if required
  reboot:
    msg: "Rebooting after patching"
  when: reboot_required_file.stat.exists

```

24.4 23.4 Modules, idempotency, and handlers

```

- name: Install and configure nginx
  hosts: web
  become: true

  tasks:
    - name: Install nginx package
      apt:
        name: nginx
        state: present
        update_cache: true

    - name: Deploy site config
      template:
        src: templates/nginx.conf.j2
        dest: /etc/nginx/sites-available/default
        validate: nginx -t -c %s
        notify: Reload nginx

    - name: Ensure nginx is running
      service:
        name: nginx
        state: started
        enabled: true

  handlers:
    - name: Reload nginx
      service:
        name: nginx
        state: reloaded

```

Handlers run once at the end of the play, only if notified. **Templates** use Jinja2 (`{{ ansible_hostname }}`). The `validate` parameter prevents deploying broken configs.

24.4.1 Check mode and diff

```

ansible-playbook site.yml --check --diff  # Dry run with file diffs
ansible-playbook site.yml --tags deploy  # Run tagged tasks only

```

24.5 23.5 Roles — reusable bundles

Role layout:

```

roles/nginx/
├── defaults/main.yml      # Low precedence variables
├── vars/main.yml          # Role-internal vars
├── tasks/main.yml
├── handlers/main.yml
├── templates/nginx.conf.j2
├── files/ssl.crt
├── meta/main.yml          # Dependencies
└── README.md

```

roles/nginx/tasks/main.yml :

```

---
- name: Install nginx

```

```

package:
  name: "{{ nginx_package }}"
  state: present

- name: Configure worker processes
  lineinfile:
    path: /etc/nginx/nginx.conf
    regexp: '^worker_processes'
    line: "worker_processes {{ nginx_worker_processes }}"
    notify: Reload nginx

```

roles/nginx/defaults/main.yml :

```

nginx_package: nginx
nginx_worker_processes: auto

```

meta/main.yml declares dependencies:

```

dependencies:
  - role: common

```

Install from Ansible Galaxy: `ansible-galaxy install geerlingguy.docker`

24.6 23.6 Variables and facts precedence

Ansible merges variables from many sources. Highest wins (simplified):

Priority (high → low)	Source
Extra vars (<code>-e</code>)	CLI
Task vars	vars: on task
Block/play vars	vars: in play
Host facts / host_vars	host_vars/web01.yml
Role defaults	defaults/main.yml

```
ansible-playbook site.yml -e "app_version=2.5.0"
```

Gather facts: `setup` module runs automatically. Disable with `gather_facts: false` for speed on large fleets.

Custom facts in `/etc/ansible/facts.d/*.fact` (JSON or executable).

24.7 23.7 Vault, collections, and project structure

Encrypt secrets with **Ansible Vault**:

```

ansible-vault create group_vars/production/vault.yml
ansible-vault edit group_vars/production/vault.yml
ansible-playbook site.yml --ask-vault-pass
# Or --vault-password-file for CI

```

```

# group_vars/production/vault.yml (encrypted)
db_password: "s3cr3t"

```

Reference in templates: `{{ db_password }}` — never commit plaintext secrets.

Collections package modules and roles (`ansible-galaxy collection install amazon.aws`).

Recommended layout:

```
ansible/
├─ ansible.cfg
├─ inventories/
│  └─ staging/
│  └─ production/
├─ group_vars/
├─ host_vars/
├─ roles/
├─ playbooks/
│  └─ site.yml
└─ requirements.yml
```

ansible.cfg:

```
[defaults]
inventory = inventories/production
roles_path = roles
host_key_checking = True
retry_files_enabled = False
interpreter_python = auto_silent

[privilege_escalation]
become = True
become_method = sudo
```

24.8 23.8 Ansible in CI/CD

```
# GitHub Actions excerpt
- name: Run Ansible lint
  run: pip install ansible-lint && ansible-lint playbooks/

- name: Molecule test
  run: cd roles/nginx && molecule test

- name: Deploy to staging
  run: |
    ansible-playbook playbooks/site.yml \
      -i inventories/staging \
      --private-key "${{ secrets.SSH_KEY }}" \
      --vault-password-file vault-pass.sh
```

Use **Molecule** for role testing with Docker or cloud drivers. Limit production deploys to tagged releases and `--check` on canary hosts first.

24.9 23.9 Chapter summary

- **Inventory** defines where Ansible runs; use **dynamic inventory** for cloud fleets.
- **Playbooks** orchestrate tasks; **roles** encapsulate reusable configuration.
- Ansible is **idempotent**—second runs should change nothing if state is correct.
- Protect secrets with **Ansible Vault**; test roles with **Molecule** and lint with **ansible-lint**.

24.10 Lab 23.1

1. Create a static inventory with `web` and `db` groups (use Vagrant or multipass VMs).
2. Write a playbook that installs nginx and deploys a custom `index.html` via template.
3. Extract nginx configuration into a role with defaults and handlers.
4. Encrypt a database password with Ansible Vault and use it in a template.

24.11 Lab 23.2

1. Configure AWS EC2 dynamic inventory and run `ansible all -m ping`.
 2. Add a `serial: 1` rolling restart play for a fake application service.
 3. Set up Molecule with the Docker driver to test your nginx role.
-

24.12 Review questions

1. Why is Ansible described as “agentless,” and what are the trade-offs?
 2. What triggers a handler to run, and when do handlers execute?
 3. How does `--check` mode differ from running tasks with `state: absent`?
 4. Where should role defaults go versus play-level `vars`?
 5. How would you safely store and use production secrets in Ansible?
-

Next: [Chapter 24 — Packer & Golden Images](#)

25 Chapter 24: Packer & Golden Images

DevOps Handbook — Part VI, Pages 451–465

25.1 24.1 Golden images and the immutable infrastructure pattern

A **golden image** (or **machine image**) is a pre-baked AMI, Azure image, or GCP image with the OS hardened, agents installed, and dependencies cached. New instances launch from the image instead of running lengthy bootstrap scripts on every boot.

Approach	Boot time	Drift risk	Rollback
Configure at boot (cloud-init, Ansible)	Slower	Higher (scripts change)	Harder
Golden image (Packer)	Fast	Lower (versioned AMIs)	Revert launch template
Containers	Fastest	Lowest for app layer	Image tag/digest

Packer (HashiCorp) builds images across providers from a single template. Pair Packer with Terraform (provision infrastructure) and Ansible (configure during build) for a complete pipeline.

25.2 24.2 Packer workflow

```
packer init . # Install plugins
packer validate amazon-linux.pkr.hcl
packer build amazon-linux.pkr.hcl
packer build -var "region=us-west-2" amazon-linux.pkr.hcl
```

Packer launches a temporary builder instance, runs **provisioners**, creates an image snapshot, then terminates the builder.

25.3 24.3 AWS EBS builder example

amazon-linux.pkr.hcl:

```
packer {
  required_version = ">= 1.10.0"
  required_plugins {
    amazon = {
      version = ">= 1.2.8"
      source  = "github.com/hashicorp/amazon"
    }
    ansible = {
      version = ">= 1.1.0"
      source  = "github.com/hashicorp/ansible"
    }
  }
}

variable "region" {
  type    = string
  default = "us-east-1"
}

variable "ami_prefix" {
  type    = string
  default = "acme-base"
```

```

}

locals {
  timestamp = regex_replace(timestamp(), "[- TZ:]", "")
}

source "amazon-ebs" "base" {
  ami_name      = "${var.ami_prefix}-${local.timestamp}"
  instance_type = "t3.small"
  region        = var.region

  source_ami_filter {
    filters = {
      name      = "al2023-ami-*-x86_64"
      root-device-type = "ebs"
      virtualization-type = "hvm"
    }
    owners      = ["amazon"]
    most_recent = true
  }

  ssh_username = "ec2-user"

  tags = {
    Name        = "${var.ami_prefix}-${local.timestamp}"
    BuiltBy     = "packer"
    Environment = "shared"
  }
}

build {
  sources = ["source.amazon-ebs.base"]

  provisioner "shell" {
    inline = [
      "sudo dnf update -y",
      "sudo dnf install -y amazon-cloudwatch-agent jq"
    ]
  }

  provisioner "ansible" {
    playbook_file = "./ansible/base.yml"
    extra_arguments = [
      "--become",
      "-e", "ansible_python_interpreter=/usr/bin/python3"
    ]
  }

  provisioner "shell" {
    inline = [
      "sudo cloud-init clean --logs",
      "sudo rm -f /root/.ssh/authorized_keys /home/ec2-user/.ssh/authorized_keys",
      "sudo shred -u /etc/ssh/ssh_host_* || true"
    ]
  }
}

```

The final **cleanup provisioner** removes SSH host keys and authorized keys so clones do not share credentials.

25.4 24.4 Provisioners

Provisioner	Use case
shell	Inline scripts, package installs
ansible / ansible-local	Configuration management during build
file	Upload static files
windows-shell / powershell	Windows images
docker	Build Docker images (different builder)

Order matters: install packages before Ansible, clean secrets last.

ansible/base.yml :

```
---
- hosts: default
  become: true
  tasks:
    - name: Harden sshd
      lineinfile:
        path: /etc/ssh/sshd_config
        regexp: '^PermitRootLogin'
        line: 'PermitRootLogin no'
        notify: restart sshd

    - name: Install node exporter
      unarchive:
        src: https://github.com/prometheus/node_exporter/releases/download/v1.8.2/node_exporter-1.8.2.linux-amd64.tar.gz
        dest: /usr/local/bin
        remote_src: true
        extra_opts: [--strip-components=1]
        include: ['node_exporter']

    - name: Enable node exporter systemd unit
      copy:
        dest: /etc/systemd/system/node_exporter.service
        content: |
          [Unit]
          Description=Node Exporter
          [Service]
          ExecStart=/usr/local/bin/node_exporter
          [Install]
          WantedBy=multi-user.target
        notify: daemon reload

  handlers:
    - name: restart sshd
      service: { name: sshd, state: restarted }
    - name: daemon reload
      systemd: { daemon_reload: true }
```

25.5 24.5 Multi-region and shared images

```
build {
  sources = [
    "source.amazon-ebs.us_east",
    "source.amazon-ebs.eu_west"
  ]
  # Same provisioners run in each region
}
```

Use **AMI sharing** across accounts via `ami_users` or AWS Organizations. In Terraform, data source the latest Packer AMI:

```
data "aws_ami" "app_base" {
  most_recent = true
  owners      = ["self"]

  filter {
    name   = "name"
    values = ["acme-base-*"]
  }

  filter {
    name   = "tag:BuiltBy"
    values = ["packer"]
  }
}

resource "aws_launch_template" "app" {
  name = "app-lt"
```



```
image_id      = data.aws_ami.app_base.id
instance_type = "t3.medium"
}
```

25.6 24.6 Versioning, promotion, and CI

Image naming strategy:

```
acme-base-20250901120000 # Timestamp build
acme-base-v1.4.2         # Semantic version tag
```

Pipeline stages:

1. **Build** on merge to `main` (Packer)
2. **Scan** AMI or exported container with **Inspector**, **Trivy**, or **ECR scan**
3. **Promote** to staging launch template
4. **Bake period** — run integration tests
5. **Promote** to production; keep last N AMIs for rollback

```
# CI excerpt
- run: packer build -machine-readable . | tee build.log
- run: |
    AMI_ID=$(grep 'artifact,0,id' build.log | cut -d, -f6 | cut -d: -f2)
    aws ec2 create-tags --resources "$AMI_ID" --tags Key=Version,Value=${{ github.sha }}
```

Store Packer variables and plugin versions in Git; pin plugin versions in `required_plugins`.

25.7 24.7 Security and compliance

- **No secrets in images** — use instance roles and secrets managers at runtime
- **CIS benchmarks** — apply hardening roles (DevSec hardening collection)
- **SBOM** — document packages; scan for CVEs before promotion
- **Immutable updates** — new AMI + rolling instance refresh, not SSH patching in prod
- **Signed AMIs** — AWS EC2 Image Builder supports compliance pipelines

Anti-pattern	Better approach
Embedding API keys in image	IAM instance profile
<code>:latest</code> untagged AMIs	Explicit version tags + lifecycle policy
Manual AMI copies	Automated pipeline with scan gates

25.8 24.8 Chapter summary

- **Golden images** reduce boot time and configuration drift; Packer automates their creation.
- Use **provisioners** in order: update OS, configure, then **sanitize** SSH keys and logs.
- Integrate Packer with **CI scan gates** and Terraform **launch templates** referencing latest approved AMI.
- Treat images as **versioned artifacts** with promotion and rollback paths.

25.9 Lab 24.1

1. Install Packer and initialize the Amazon plugin.

2. Build an Amazon Linux 2023 AMI with nginx and the CloudWatch agent.
 3. Launch an EC2 instance from the AMI and verify services without extra user-data.
 4. Add a cleanup provisioner that removes build-time SSH artifacts.
-

25.10 Lab 24.2

1. Wire Packer to Ansible for CIS-style sshd hardening.
 2. Output the AMI ID from CI and pass it to a Terraform `aws_launch_template`.
 3. Implement AMI retention: keep last 5 images, deregister older AMIs.
-

25.11 Review questions

1. Why should SSH host keys be regenerated (or removed) during the Packer build?
 2. When would you choose cloud-init bootstrap over golden images?
 3. How does Terraform discover the latest Packer-built AMI safely?
 4. What security risks exist if secrets are baked into an AMI?
 5. Describe a promotion pipeline from build to production launch template.
-

Next: [Chapter 25 — Policy as Code](#)

26 Chapter 25: Policy as Code — OPA, Checkov, and Sentinel

DevOps Handbook — Part VI, Pages 466–480

26.1 25.1 Why policy as code?

Infrastructure and deployment mistakes scale with automation. **Policy as Code (PaC)** encodes guardrails—encryption required, no public S3 buckets, approved regions—as executable rules evaluated in CI, admission controllers, or Terraform runs **before** changes reach production.

Without PaC	With PaC
Manual checklist in PR template	Automated deny/warn on violation
Post-incident “what broke the rule?”	Audit log of policy decisions
Inconsistent interpretation	Same rule in dev and prod

PaC complements but does not replace peer review and threat modeling.

26.2 25.2 Policy evaluation points

Developer → PR → [Lint/Scan: Checkov] → Plan → [OPA/Sentinel] → Apply → Runtime [Gatekeeper/Kyverno]

Stage	Tools	Input format
Static IaC scan	Checkov, tfsec, KICS	.tf, .yaml, .dockerfile
Plan-time	OPA (conftest), Sentinel	terraform plan JSON
K8s admission	OPA Gatekeeper, Kyverno	Kubernetes API requests
General purpose	Open Policy Agent (OPA)	JSON

26.3 25.3 Checkov — static IaC scanning

Checkov (Bridgecrew/Prisma) scans Terraform, CloudFormation, Kubernetes, Dockerfiles, and more.

```
pip install checkov
checkov -d infra/                                # Directory scan
checkov -f main.tf --framework terraform
checkov -d . --skip-check CKV_AWS_20             # Skip specific check
checkov -d . --soft-fail                         # Warn only (not for prod)
```

Example finding: S3 bucket without encryption or public access block.

Suppress false positives inline:

```
resource "aws_s3_bucket" "logs" {
  #checkov:skip=CKV_AWS_18:Access logging handled centrally
  bucket = "acme-logs"
}
```

`.checkov.yml` for CI:

```
quiet: false
compact: true
framework:
  - terraform
  - dockerfile
soft-fail: false
skip-check:
  - CKV_AWS_144 # Document why in SECURITY.md
```

GitHub Actions:

```
- name: Checkov scan
  uses: bridgecrewio/checkov-action@master
  with:
    directory: infra/
    framework: terraform
    soft_fail: false
```

26.4 25.4 Open Policy Agent (OPA) and Rego

OPA is a general-purpose policy engine. Policies are written in **Rego**. **conftest** runs Rego against JSON/YAML files.

Install: `brew install conftest` or download from openpolicyagent.org.

Policy: `policy/s3.rego`

```
package terraform.s3

deny[msg] {
  resource := input.resource_changes[_]
  resource.type == "aws_s3_bucket"
  not resource.change.after.server_side_encryption_configuration
  msg := sprintf("S3 bucket %s must enable encryption", [resource.address])
}

deny[msg] {
  resource := input.resource_changes[_]
  resource.type == "aws_s3_bucket"
  resource.change.after.acl == "public-read"
  msg := sprintf("S3 bucket %s must not be public", [resource.address])
}
```

Generate plan JSON and test:

```
terraform plan -out=tfplan
terraform show -json tfplan > plan.json
conftest test plan.json -p policy/
```

OPA on Kubernetes (**Gatekeeper**):

```
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sRequiredLabels
metadata:
  name: require-team-label
spec:
  match:
    kinds:
      - apiGroups: [""]
        kinds: ["Pod"]
  parameters:
    labels: ["team", "cost-center"]
```

Violating pods are rejected at admission time.

26.5 25.5 HashiCorp Sentinel (Terraform Cloud/Enterprise)

Sentinel is HashiCorp’s policy language for Terraform Cloud/Enterprise. It integrates directly with the run pipeline.

Example: restrict instance types

```
# policy/limit-instance-type.sentinel
import "tfplan/v2" as tfplan

allowed = ["t3.micro", "t3.small", "t3.medium"]

main = rule {
  all tfplan.resource_changes as _, rc {
    rc.type is not "aws_instance" or
    rc.change.after.instance_type in allowed
  }
}
```

Enforcement levels:

Level	Behavior
advisory	Log only
soft-mandatory	Overridable with permission
hard-mandatory	Cannot apply

Sentinel has first-class Terraform plan imports (`tfplan` , `tfconfig` , `tfstate`). OPA is more portable across tools; Sentinel is tightly coupled to HashiCorp ecosystem.

26.6 25.6 Comparison matrix

Feature	Checkov	OPA/conftest	Sentinel
Languages	Python checks	Rego	Sentinel DSL
Terraform plan	Indirect (HCL)	Plan JSON	Native
K8s admission	Limited	Gatekeeper	N/A
License	Apache 2.0	Apache 2.0	BSL / TFC
Best for	Quick security baseline	Multi-tool policy	TFC enterprise governance

Use **Checkov** for broad static scans, **OPA** for custom plan logic and Kubernetes, **Sentinel** if you are standardized on Terraform Cloud with hierarchical policies.

26.7 25.7 Building a policy program

1. **Start with deny-list basics** — public exposure, unencrypted storage, open security groups
2. **Map to compliance** — tag each rule to CIS, SOC2, or internal standard
3. **Warn then enforce** — `soft-fail` for 2 weeks, then hard fail
4. **Exception process** — ticket ID in `#checkov:skip` or OPA `data.exceptions`
5. **Measure** — track violations per team; fix generators not just resources

`policy/exceptions.json` for OPA:

```
{
  "exceptions": [
    {
      "resource": "aws_s3_bucket.legacy_logs",
      "policy": "terraform.s3.deny",
      "reason": "TICKET-4521 migration Q4",
      "expires": "2026-12-31"
    }
  ]
}
```

```
exception[resource] {
  ex := data.exceptions[_]
  resource := ex.resource
  time.parse_rfc3339_ns(ex.expires) > time.now_ns()
}
```

26.8 25.8 CI pipeline integration

```
jobs:
  policy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
      - run: terraform init && terraform plan -out=tfplan
      - run: terraform show -json tfplan > plan.json
      - run: checkov -d infra/ --quiet
      - run: conftest test plan.json -p policy/ --all-namespaces
```

Fail the build on any `deny` rule. Store policy repos separately for platform teams to version independently.

26.9 25.9 Chapter summary

- **Policy as Code** automates compliance and prevents recurring misconfigurations.
- **Checkov** provides out-of-the-box IaC security checks with minimal setup.
- **OPA/Rego** is flexible for Terraform plans, Kubernetes admission, and custom JSON APIs.
- **Sentinel** integrates with Terraform Cloud for plan-time enforcement in enterprise workflows.
- Combine static scan + plan-time policy + runtime admission for defense in depth.

26.10 Lab 25.1

1. Run Checkov on a Terraform project; fix three high-severity findings.
2. Write a Rego policy denying `0.0.0.0/0` ingress on security groups.
3. Integrate Checkov and conftest into a GitHub Actions workflow.

26.11 Lab 25.2

1. Install OPA Gatekeeper on a test cluster and deploy a `require-labels` constraint.
2. Create an exception file with expiring waivers for one resource.
3. Document a team policy: which checks are hard-fail vs advisory.

26.12 Review questions

1. What is the difference between static IaC scanning and plan-time policy evaluation?
 2. When would you use Gatekeeper instead of only Checkov?
 3. Why should policy exceptions have expiration dates?
 4. How does Sentinel's `soft-mandatory` differ from `hard-mandatory`?
 5. Name three resource types you would prioritize in a new PaC rollout.
-

Next: [*Chapter 26 — Docker Fundamentals*](#)

27 Chapter 26: Docker — Images, Dockerfile, and Multi-stage Builds

DevOps Handbook — Part VII, Pages 481–505

27.1 26.1 Containers in one paragraph

A **container** packages an application with its dependencies into an isolated process on a shared Linux kernel. Unlike VMs, containers share the host OS kernel—lightweight and fast to start.

Docker is the dominant tooling for building, shipping, and running containers locally and in CI.

27.2 26.2 Core concepts

Concept	Description
Image	Read-only template (layers)
Container	Running instance of an image
Dockerfile	Recipe to build an image
Registry	Storage for images (Docker Hub, ECR)
Volume	Persistent data outside container lifecycle
Network	Connect containers to each other and the host

27.3 26.3 Essential commands

```
docker pull nginx:1.25-alpine
docker images
docker run -d -p 8080:80 --name web nginx:1.25-alpine
docker ps
docker logs web
docker exec -it web sh
docker stop web && docker rm web
docker build -t myapp:1.0 .
docker tag myapp:1.0 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:1.0
docker push 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:1.0
```

27.4 26.4 Dockerfile best practices

```
# Use specific version tags, not :latest in production
FROM node:20-alpine AS builder

WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production=false

COPY . .
RUN npm run build

# Multi-stage: smaller final image
```



```

FROM node:20-alpine AS runtime

RUN addgroup -g 1001 -S app && adduser -u 1001 -S app -G app
WORKDIR /app
USER app

COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
COPY package.json .

EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=3s CMD wget -q0- http://localhost:3000/health || exit 1

CMD ["node", "dist/server.js"]

```

27.4.1 Best practices checklist

- Use **multi-stage builds** to shrink images
- Run as **non-root** user
- Minimize layers; combine RUN commands where sensible
- Use `.dockerignore` (exclude `node_modules`, `.git`, etc.)
- Pin base image digests for supply chain security
- Add **HEALTHCHECK** for orchestrators

27.5 26.5 Docker Compose for local development

`docker-compose.yml`:

```

services:
  api:
    build: .
    ports:
      - "3000:3000"
    environment:
      DATABASE_URL: postgres://user:pass@db:5432/app
    depends_on:
      db:
        condition: service_healthy

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: app
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U user -d app"]
      interval: 5s
      timeout: 5s
      retries: 5

volumes:
  pgdata:

```

```

docker compose up -d
docker compose logs -f api
docker compose down -v # Remove volumes too

```

27.6 26.6 Security basics

- Scan images: `docker scout cves` or **Trivy**
- Do not store secrets in images or Dockerfile `ENV`

- Use read-only root filesystem where possible (`--read-only`)
 - Drop capabilities: `--cap-drop=ALL`
-

27.7 26.7 Chapter summary

- Images are built from Dockerfiles; containers are ephemeral runtime instances.
 - **Multi-stage builds** and non-root users are production essentials.
 - **Docker Compose** orchestrates multi-container local stacks.
-

27.8 Lab 26.1

1. Containerize a simple Node or Python web app.
 2. Add multi-stage build and health check.
 3. Run with Compose alongside PostgreSQL.
 4. Scan the image with Trivy.
-

Next: [Chapter 37 — CI/CD Pipeline Design](#)

28 Chapter 27: Container Networking, Volumes, and Security

DevOps Handbook — Part VII, Pages 506–525

28.1 27.1 Container networking model

Containers share the host Linux kernel but need isolated network namespaces. **Docker** (and containerd/CRI-O underneath) creates virtual interfaces, bridges, and iptables/nftables rules so containers get IPs and can reach each other and the outside world.

Namespace	Isolates
Network	Interfaces, routes, ports
PID	Process tree
Mount	Filesystem view
UTS	Hostname
IPC	Inter-process communication
User	UID/GID mapping (rootless)

Understanding namespaces explains why a process in a container sees its own `eth0` and port 80, while the host may map that to port 8080.

28.2 27.2 Docker network drivers

```
docker network ls
docker network inspect bridge
docker network create --driver bridge app-net
docker run -d --name api --network app-net nginx:alpine
docker run -d --name cache --network app-net redis:7-alpine
```

Driver	Scope	Use case
bridge	Single host	Default; container-to-container on one machine
host	Single host	Container uses host network stack (no NAT)
none	Isolated	No networking
overlay	Multi-host (Swarm)	Legacy Swarm clustering
macvlan	L2 segment	Containers appear as physical NICs on LAN

28.2.1 DNS on user-defined bridges

On the default `bridge`, containers resolve each other by IP only. On **user-defined bridge networks**, Docker’s embedded DNS resolves **container names**:

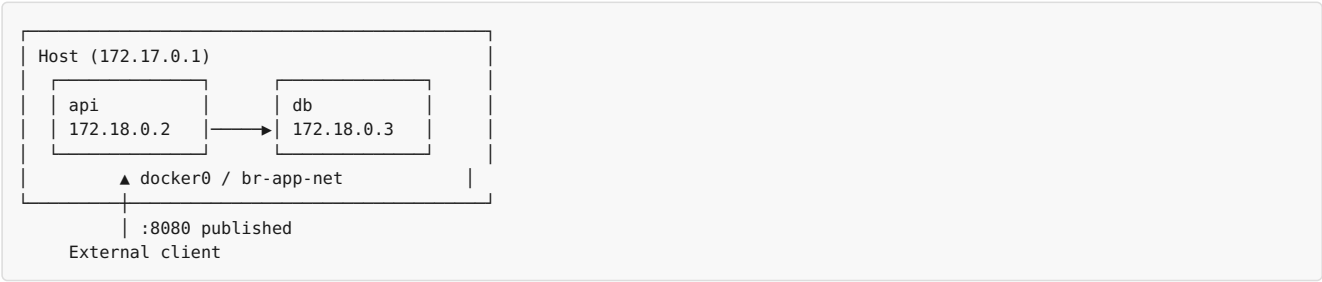
```
# From api container:
docker exec api ping -c1 cache # Resolves to cache container IP
```

28.2.2 Published ports

```
docker run -p 127.0.0.1:8080:80 nginx # Bind host localhost only
docker run -p 8080:80 nginx # All interfaces (careful in prod)
```

-p sets DNAT rules forwarding host port → container port.

28.3 27.3 Container-to-container and host communication



Host network mode (--network host): container shares host ports directly—useful for high-performance proxies; breaks port isolation.

Extra hosts (--add-host): inject static /etc/hosts entries for legacy apps.

28.4 27.4 Volumes and persistence

Container filesystems are ephemeral. **Volumes** persist data outside the container lifecycle.

Mechanism	Location	Best for
Volume	/var/lib/docker/volumes/	Databases, shared data
Bind mount	Host path	Dev hot-reload, config files
tmpfs	Memory	Secrets, temp cache

```
docker volume create pgdata
docker run -d \
  --name postgres \
  -v pgdata:/var/lib/postgresql/data \
  -e POSTGRES_PASSWORD=secret \
  postgres:16-alpine

# Bind mount for development
docker run -v $(pwd)/src:/app/src node:20-alpine npm run dev
```

Volume commands:

```
docker volume ls
docker volume inspect pgdata
docker run --rm -v pgdata:/data alpine ls /data
```

Named volumes are portable across hosts when using volume plugins or orchestrators (Kubernetes PVCs). **Bind mounts** tie data to a specific host path—avoid for production databases on single Docker hosts.

28.4.1 Read-only root filesystem

```
docker run --read-only \
  --tmpfs /tmp \
  -v app-cache:/var/cache \
```

```
myapp:1.0
```

28.5 27.5 Linux capabilities and seccomp

Containers are not VMs. Root in a container may still be dangerous on the host without proper isolation. Docker drops most **Linux capabilities** by default.

```
# Drop all, add only what's needed
docker run --cap-drop=ALL --cap-add=NET_BIND_SERVICE nginx:alpine

# Never do this in production
docker run --privileged myapp # Near-root on host
```

Capability	Risk if granted
SYS_ADMIN	Mount, cgroup manipulation
NET_RAW	Packet sniffing, ARP spoofing
DAC_OVERRIDE	Bypass file permissions

seccomp profiles filter syscalls. Docker uses a default profile; Kubernetes can set `seccompProfile: RuntimeDefault`.

AppArmor / **SELinux** add MAC labels:

```
docker run --security-opt apparmor=docker-default ...
```

28.6 27.6 Image and runtime security

28.6.1 Non-root user

```
RUN adduser -D -u 10001 appuser
USER appuser
```

28.6.2 Scanning

```
trivy image myapp:1.0
docker scout cves myapp:1.0
grype myapp:1.0
```

Fail CI if CRITICAL CVEs exist without accepted risk ticket.

28.6.3 Secrets management

Anti-pattern	Fix
ENV DB_PASSWORD=... in Dockerfile	Runtime env from vault/SSM
Secrets in image layers	BuildKit secrets, multi-stage discard
World-readable bind mounts	tmpfs or secret driver

BuildKit secret mount:

```
# syntax=docker/dockerfile:1
RUN --mount=type=secret,id=npnrc,target=/root/.npmrc npm ci
```

```
docker build --secret id=npmc,src=$HOME/.npmrc .
```

28.7 27.7 Resource limits and health

```
docker run -d \
  --memory=512m \
  --cpus=1.5 \
  --pids-limit=100 \
  --restart=unless-stopped \
  myapp:1.0
```

Without limits, one container can exhaust host memory (**OOM killer** may kill random processes).

Health checks (Dockerfile or run flag):

```
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
  CMD wget -q0- http://localhost:8080/health || exit 1
```

Orchestrators use health status for rolling updates and traffic routing.

28.8 27.8 Debugging networking issues

```
docker exec api ip addr
docker exec api cat /etc/resolv.conf
docker exec api wget -q0- http://cache:6379
iptables -t nat -L -n | grep 8080    # Host: inspect port publishing
docker logs api --tail 50
```

Common problems:

- Wrong network → container not on same user-defined bridge
- Published port conflict → another process bound to host port
- DNS failure → using default bridge instead of named network
- Firewall on host blocking forwarded traffic

28.9 27.9 Chapter summary

- Containers use **network namespaces**; user-defined **bridge networks** provide DNS-based service discovery.
- **Volumes** persist data; prefer named volumes over bind mounts for production state.
- Harden with **non-root users**, **cap-drop**, **read-only rootfs**, and **image scanning**.
- Never run `--privileged` in production; inject secrets at runtime, not in images.

28.10 Lab 27.1

1. Create a user-defined bridge network with `api` and `redis` containers; verify name resolution.
 2. Attach a named volume to PostgreSQL; delete and recreate the container without data loss.
 3. Run nginx with `--read-only`, `--cap-drop=ALL`, and `--cap-add=NET_BIND_SERVICE`.
-

28.11 Lab 27.2

1. Publish a port bound to `127.0.0.1` only; confirm remote access is blocked.
 2. Scan an image with Trivy; remediate one CVE by upgrading a base package.
 3. Debug a deliberate misconfiguration (wrong network) using `docker inspect` and `exec`.
-

28.12 Review questions

1. Why do containers on the default bridge network not resolve each other by name?
 2. What is the difference between a bind mount and a named volume?
 3. Which capability allows binding to ports below 1024 without running as root?
 4. Why is `--privileged` dangerous compared to adding specific capabilities?
 5. How does `-p 8080:80` affect host firewall and NAT rules?
-

Next: [*Chapter 28 — Docker Compose and Local Dev*](#)

29 Chapter 28: Docker Compose and Local Dev Environments

DevOps Handbook — Part VII, Pages 526–540

29.1 28.1 What Docker Compose solves

Docker Compose (v2, integrated as `docker compose`) defines multi-container applications in a single YAML file. Developers run `docker compose up` to start databases, caches, APIs, and frontends with consistent networking and volumes—eliminating “works on my machine” setup drift.

Compose is ideal for **local development**, **integration tests**, and **demo stacks**. Production orchestration typically moves to Kubernetes, ECS, or Nomad, but Compose files often inform those definitions.

29.2 28.2 Compose file structure

`compose.yaml`:

```
name: bookstore

services:
  api:
    build:
      context: ./api
      dockerfile: Dockerfile
      target: dev
    ports:
      - "3000:3000"
    environment:
      NODE_ENV: development
      DATABASE_URL: postgres://app:app@db:5432/bookstore
      REDIS_URL: redis://cache:6379/0
    volumes:
      - ./api/src:/app/src:ro
    depends_on:
      db:
        condition: service_healthy
      cache:
        condition: service_started
    develop:
      watch:
        - action: sync
          path: ./api/src
          target: /app/src
        - action: rebuild
          path: ./api/package.json

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: app
      POSTGRES_DB: bookstore
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app -d bookstore"]
      interval: 5s
      timeout: 5s
      retries: 5

  cache:
    image: redis:7-alpine
    command: redis-server --save "" --appendonly no
```



```

adminer:
  image: adminer:4
  ports:
    - "8080:8080"
  profiles:
    - tools

volumes:
  pgdata:

```

Key directives:

Key	Purpose
build / image	Build locally or pull image
ports	Publish container ports
environment / env_file	Configuration
volumes	Persistence and dev bind mounts
depends_on	Startup ordering (with health conditions)
profiles	Optional services (--profile tools)
develop.watch	File sync / rebuild (Compose 2.22+)

29.3 28.3 Essential commands

```

docker compose up -d           # Detached start
docker compose up --build      # Rebuild images
docker compose ps
docker compose logs -f api
docker compose exec api sh
docker compose run --rm api npm test
docker compose stop
docker compose down            # Stop and remove containers
docker compose down -v         # Also remove named volumes
docker compose config          # Validate and render merged YAML

```

Run one-off commands in the service context:

```
docker compose run --rm api npx prisma migrate deploy
```

29.4 28.4 Environment-specific overrides

compose.yaml — base stack

compose.override.yaml — auto-loaded local overrides (gitignored secrets)

compose.prod.yaml — production-like settings

```
docker compose -f compose.yaml -f compose.prod.yaml up -d
```

compose.override.yaml example:

```

services:
  api:
    ports:
      - "3000:3000"
      - "9229:9229" # Node debugger
    environment:
      LOG_LEVEL: debug

```

`.env` file (loaded automatically):

```
POSTGRES_USER=app
POSTGRES_PASSWORD=localdev
API_PORT=3000
```

Reference in compose: `${API_PORT}:3000`

29.5 28.5 Networking in Compose

Compose creates a **default network** named `{project}_default`. All services join it and resolve each other by **service name** (`db`, `cache`, `api`).

Custom network:

```
networks:
  frontend:
  backend:

services:
  web:
    networks: [frontend]
  api:
    networks: [frontend, backend]
  db:
    networks: [backend]
```

`db` is unreachable from `web`—only `api` bridges both networks. Useful for mimicking DMZ patterns locally.

29.6 28.6 Development workflows

29.6.1 Hot reload with bind mounts

Mount source code read-write; run dev server with file watcher inside container:

```
services:
  api:
    build: .
    command: npm run dev
    volumes:
      - ./api:/app
      - /app/node_modules # Anonymous volume preserves container node_modules
```

The anonymous volume prevents host `node_modules` from overwriting Linux binaries built in the image.

29.6.2 Compose Watch

```
develop:
  watch:
    - action: sync+restart
      path: ./config
      target: /app/config
```

`docker compose watch` syncs files and restarts affected services.

29.6.3 Seed data

```
services:
  db:
    volumes:
```

```
- pgdata:/var/lib/postgresql/data
- ./scripts/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
```

PostgreSQL runs scripts in `docker-entrypoint-initdb.d` on first initialization only.

29.7 28.7 Testing with Compose

`compose.test.yaml` :

```
services:
  api:
    build: .
    environment:
      DATABASE_URL: postgres://test:test@db:5432/test
    command: npm test
    depends_on:
      db:
        condition: service_healthy

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: test
      POSTGRES_PASSWORD: test
      POSTGRES_DB: test
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U test"]
      interval: 2s
      retries: 10
```

CI:

```
- run: docker compose -f compose.test.yaml up --build --abort-on-container-exit --exit-code-from api
- run: docker compose -f compose.test.yaml down -v
```

`--exit-code-from api` fails the job if tests fail.

29.8 28.8 Compose vs production

Concern	Local Compose	Production K8s/ECS
Scaling	<code>deploy.replicas</code> (Swarm only); manual	HPA, replica sets
Secrets	<code>.env</code> , Docker secrets	Vault, K8s Secrets, SSM
Health routing	None	Load balancers, probes
Rolling updates	Recreate containers	Controlled rollouts

Translate patterns, not files literally—tools like **Kompose** exist but output usually needs manual refinement.

29.9 28.9 Chapter summary

- **Compose** defines multi-service stacks with networking, volumes, and dependencies in YAML.
- Use **overrides**, `.env` , and **profiles** for flexible local vs CI environments.
- **Healthchecks** and `depends_on` conditions prevent race conditions on startup.
- **Compose Watch** and bind mounts enable fast inner-loop development.
- Run integration tests in CI with `docker compose up --abort-on-container-exit` .

29.10 Lab 28.1

1. Build a three-service stack: API, PostgreSQL, Redis.
 2. Add healthchecks and verify API waits for database readiness.
 3. Create a `tools` profile with Adminer or pgAdmin.
-

29.11 Lab 28.2

1. Split `compose.yaml` and `compose.override.yaml` for debug ports and log level.
 2. Add Compose Watch or a bind mount for live code reload.
 3. Write a CI job that runs tests via `compose.test.yaml`.
-

29.12 Review questions

1. How does Compose service discovery differ from linking containers manually?
 2. Why use an anonymous volume for `node_modules` in a bind-mounted Node app?
 3. What does `depends_on: condition: service_healthy` solve?
 4. When should you use `compose down -v` versus `compose down`?
 5. How do Compose profiles help keep optional services out of the default stack?
-

Next: [Chapter 29 — Container Registries and Image Signing](#)

30 Chapter 29: Container Registries and Image Signing

DevOps Handbook — Part VII, Pages 541–560

30.1 29.1 Container registries overview

A **container registry** stores and distributes OCI-compatible **images** (manifests + layers). Registries are the artifact hub of the container supply chain—CI pushes images; clusters pull them to run workloads.

Registry	Provider	Notes
Docker Hub	Docker	Public default; rate limits on anonymous pulls
Amazon ECR	AWS	IAM auth, image scanning, lifecycle policies
Google Artifact Registry	GCP	Multi-format (Docker, npm, maven)
Azure ACR	Azure	Geo-replication, content trust
GitHub Container Registry (GHCR)	GitHub	<code>ghcr.io/org/image</code>
Harbor	CNCF / self-hosted	RBAC, replication, signing

Images are referenced by **tag** (`myapp:1.2.3`) or immutable **digest** (`myapp@sha256:abc123...`). Prefer digests in production manifests for immutability.

30.2 29.2 Image naming and tagging strategy

```
<registry>/<namespace>/<repository>:<tag>
ghcr.io/acme-corp/payments-api:v1.4.2
123456789.dkr.ecr.us-east-1.amazonaws.com/web:main-abc1234
```

Tagging conventions:

Tag pattern	Use
<code>v1.2.3</code>	Release semver
<code>main-abc1234</code>	Git SHA from branch
<code>1.2.3-alpine</code>	Base OS variant
<code>latest</code>	Dev only—never prod deploy target

Never reuse tags for different content. CI should push unique tags per build; promotion means updating deployment to point at a tested tag/digest.

30.3 29.3 Authenticating to registries

30.3.1 Amazon ECR

```
aws ecr get-login-password --region us-east-1 | \
  docker login --username AWS --password-stdin \
    123456789.dkr.ecr.us-east-1.amazonaws.com

docker tag myapp:1.0 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:1.0
docker push 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:1.0
```

Create repository with Terraform:

```
resource "aws_ecr_repository" "myapp" {
  name                = "myapp"
  image_tag_mutability = "IMMUTABLE"

  image_scanning_configuration {
    scan_on_push = true
  }
}

resource "aws_ecr_lifecycle_policy" "myapp" {
  repository = aws_ecr_repository.myapp.name
  policy = jsonencode({
    rules = [{
      rulePriority = 1
      description = "Keep last 30 images"
      selection = {
        tagStatus    = "any"
        countType    = "imageCountMoreThan"
        countNumber = 30
      }
      action = { type = "expire" }
    }]
  })
}
```

30.3.2 GHCR

```
echo $GITHUB_TOKEN | docker login ghcr.io -u USERNAME --password-stdin
docker tag myapp:1.0 ghcr.io/acme-corp/myapp:1.0
docker push ghcr.io/acme-corp/myapp:1.0
```

Use fine-grained PAT or `GITHUB_TOKEN` in Actions with `packages: write` permission.

30.4 29.4 Pull policies in Kubernetes and beyond

```
spec:
  containers:
    - name: api
      image: 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:v1.4.2@sha256:deadbeef...
      imagePullPolicy: IfNotPresent
```

imagePullPolicy	Behavior
Always	Pull every pod start (required for <code>:latest</code>)
IfNotPresent	Pull if not cached locally
Never	Local only (kind/minikube dev)

Private registries need **imagePullSecrets** or cloud-specific workload identity (EKS, GKE).

30.5 29.5 Vulnerability scanning

Scan on push and on schedule:

```
trivy image ghcr.io/acme-corp/myapp:1.0
aws ecr describe-image-scan-findings \
  --repository-name myapp \
  --image-id imageTag=1.0
```

CI gate example:

```
- name: Build and push
  run: |
    docker build -t ghcr.io/acme-corp/myapp:${{ github.sha }} .
    docker push ghcr.io/acme-corp/myapp:${{ github.sha }}

- name: Scan image
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: ghcr.io/acme-corp/myapp:${{ github.sha }}
    severity: CRITICAL,HIGH
    exit-code: 1
```

Maintain **SBOMs** (Software Bill of Materials) with `syft` or `trivy sbom` for compliance and faster CVE matching.

30.6 29.6 Image signing and verification

Signing proves an image was built by a trusted pipeline and was not tampered with in transit or at rest in the registry.

30.6.1 Docker Content Trust (Notary v1)

Legacy; being superseded by **Sigstore/cosign** in most ecosystems.

30.6.2 Cosign (Sigstore)

```
# Generate key pair (prefer keyless in CI with OIDC)
cosign generate-key-pair

# Sign after push
cosign sign --key cosign.key ghcr.io/acme-corp/myapp:1.0

# Verify before deploy
cosign verify --key cosign.pub ghcr.io/acme-corp/myapp:1.0
```

Keyless signing in GitHub Actions:

```
permissions:
  id-token: write
  contents: read
  packages: write

- uses: sigstore/cosign-installer@main
- run: cosign sign ghcr.io/acme-corp/myapp:${{ steps.build.outputs.digest }}
  env:
    COSIGN_EXPERIMENTAL: "true"
```

Attach **SBOM** as attestation:

```
syft ghcr.io/acme-corp/myapp:1.0 -o spdx-json > sbom.spdx.json
cosign attest --predicate sbom.spdx.json --type spdx ghcr.io/acme-corp/myapp:1.0
```

30.6.3 Policy enforcement

Kyverno verifyImages policy:

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
```

```

metadata:
  name: verify-signed-images
spec:
  validationFailureAction: Enforce
  rules:
    - name: check-signature
      match:
        any:
          - resources:
              kinds: [Pod]
      verifyImages:
        - imageReferences: ["ghcr.io/acme-corp/*"]
          attestors:
            - entries:
                - keyless:
                    subject: "https://github.com/acme-corp/myapp/.github/workflows/release.yml@refs/heads/main"
                    issuer: "https://token.actions.githubusercontent.com"

```

Unsigned images are rejected at admission.

30.7 29.7 Registry operations

30.7.1 Mirroring and air-gapped pulls

Harbor and pull-through caches proxy Docker Hub to avoid rate limits and enable offline clusters.

30.7.2 Replication

Harbor and cloud registries replicate images across regions for disaster recovery and faster pulls.

30.7.3 Retention and garbage collection

Untagged manifests accumulate storage costs. Lifecycle policies expire old SHAs; run registry GC during maintenance windows.

30.7.4 RBAC

Role	Typical permissions
CI robot	Push to specific repos
Cluster nodes	Pull only
Developers	Pull dev; no prod push
Security	Scan, quarantine, delete

30.8 29.8 Supply chain best practices

1. **Pin base images** by digest in Dockerfile
2. **Build in CI** — developers do not push local images to prod registry
3. **Sign every release** artifact
4. **Verify signatures** in deploy pipeline and admission controller
5. **Immutable tags** on production repositories
6. **Separate registries** or repos per environment (dev vs prod)
7. **Audit logs** — who pushed/pulled what and when

30.9 29.9 Chapter summary

- Registries are the **distribution hub** for container images; use structured tagging and immutability.
 - Authenticate with **short-lived tokens** (ECR, OIDC) rather than long-lived passwords.
 - **Scan** images in CI and block critical CVEs; generate **SBOMs** for traceability.
 - **Sign** images with Cosign; **verify** at deploy time and with admission policies.
 - Apply **lifecycle policies** to control storage and reduce attack surface from stale images.
-

30.10 Lab 29.1

1. Create an ECR or GHCR repository and push an image from CI.
 2. Enable scan-on-push and review findings for one image.
 3. Tag images with Git SHA and semver; deploy using digest in a K8s manifest.
-

30.11 Lab 29.2

1. Sign an image with Cosign (keyless or key pair).
 2. Verify the signature in a deploy script before `kubectl apply`.
 3. Write a lifecycle policy retaining the last 20 images.
-

30.12 Review questions

1. Why should production deployments reference digests instead of mutable tags?
 2. What is the difference between image signing and vulnerability scanning?
 3. How does ECR authentication differ from Docker Hub username/password?
 4. What does an immutable tag policy prevent?
 5. Where in the pipeline should unsigned images be rejected?
-

Next: [Chapter 30 — Rootless Containers and Podman](#)

31 Chapter 30: Rootless Containers and Podman

DevOps Handbook — Part VII, Pages 561–580

31.1 30.1 Why rootless containers matter

Traditionally, Docker required a **root-owned daemon** (`dockerd`). Any user in the `docker` group effectively had root on the host—container escape or `-v /:/host` mounts could compromise the entire machine.

Rootless containers run the container runtime and workloads as an unprivileged user, mapping container UIDs to subordinate UID ranges on the host. Compromise of a container or runtime bug has **reduced blast radius**.

Mode	Daemon user	Container root maps to
Rootful Docker	root	host root (dangerous)
Rootless Podman/Docker	user namespace	unprivileged host UID
Kubernetes	varies	depends on PSA/ SCC

Production clusters still use rootful runtimes on nodes, but developers and CI increasingly adopt rootless tooling.

31.2 30.2 Podman overview

Podman (Red Hat / CNCF) is a daemonless OCI container engine. It pulls images, creates containers, and manages pods—compatible with Docker CLI for most workflows.

```
podman --version
podman run -d -p 8080:80 --name web docker.io/library/nginx:alpine
podman ps
podman logs web
podman exec -it web sh
podman stop web && podman rm web
```

Docker-compatible aliases:

```
alias docker=podman
alias docker-compose=podman-compose
```

Podman stores data in `~/.local/share/containers/` for rootless users—no central daemon socket owned by root.

31.3 30.3 Rootless setup

On Fedora/RHEL/Ubuntu with upstream packages:

```
# Install
sudo apt install podman slirp4netns fuse-overlayfs

# Enable lingering for systemd user services (servers)
loginctl enable-linger $USER

# Verify rootless
podman info --format '{{.Host.Security.Rootless}}'
```

`/etc/subuid` and `/etc/subgid` must allocate ranges:

```
testuser:100000:65536
```

Podman maps container UID 0 → host UID 100000, etc.

31.3.1 Limitations of rootless

Feature	Rootless constraint
Binding ports < 1024	Needs <code>sysctl</code> or rootful helper
Some volume mounts	Permission mapping complexity
<code>host</code> network	Limited compared to rootful
cgroup devices	May need delegated cgroup v2

Bind privileged ports rootlessly (Linux 5.7+):

```
echo "net.ipv4.ip_unprivileged_port_start=80" | \
sudo tee /etc/sysctl.d/99-unprivileged-ports.conf
sudo sysctl --system
```

31.4 30.4 Pods and systemd integration

Podman groups containers into **Pods** (shared network namespace, like a Kubernetes pod):

```
podman pod create --name app -p 8080:80
podman run -d --pod app --name web nginx:alpine
podman run -d --pod app --name metrics prom/prometheus
podman pod ps
```

Generate **systemd unit files** for production-like single-host deploys:

```
podman generate systemd --new --name web > container-web.service
systemctl --user enable --now container-web.service
journalctl --user -u container-web.service -f
```

`--new` recreates the container on each start from the image definition—closer to immutable infrastructure.

31.5 30.5 Podman Compose and Kubernetes YAML

podman-compose:

```
podman-compose -f compose.yaml up -d
```

Play kube — run Kubernetes manifests locally:

```
podman play kube deployment.yaml
podman play kube --down deployment.yaml
```

Useful for validating K8s YAML without a full cluster.

31.6 30.6 Rootless Docker (Moby)

Docker Engine 20.10+ supports **rootless mode** via `dockerd-rootless-setup tool.sh install`.

```
dockerd-rootless-setuptool.sh install
export DOCKER_HOST=unix:///${XDG_RUNTIME_DIR}/docker.sock
docker run hello-world
```

Rootless Docker still uses `containerd` and user namespaces but maintains the Docker CLI/API model. Podman is daemonless; choose based on team familiarity and RHEL vs Debian ecosystems.

31.7 30.7 Security comparison

Rootful Docker:
Client → `/var/run/docker.sock` (group=docker) → root dockerd → container (root in cgroup)

Rootless Podman:
Client → podman (same UID) → conmon + crun/runc → user namespace → mapped UIDs

Hardening tips for rootless dev:

- Do not `sudo podman` routinely—it defeats the model
- Use **SELinux** labels (`:Z` on volume mounts on RHEL)
- Scan images same as production (**Trivy**)
- Prefer **read-only** rootfs and **cap-drop**

```
podman run --read-only --tmpfs /tmp --cap-drop=ALL myapp:1.0
```

31.8 30.8 Buildah and Skopeo

The **containers** ecosystem includes:

Tool	Role
Buildah	Build OCI images without Dockerfile (or with)
Skopeo	Copy/sign/inspect images between registries
Podman	Run containers and pods
crun	OCI runtime (lightweight)

Build without daemon:

```
buildah bud -t myapp:dev .
buildah push myapp:dev docker://ghcr.io/acme/myapp:dev
```

Skopeo copy (air-gapped mirror):

```
skopeo copy docker://docker.io/library/alpine:3.20 \
  docker://registry.internal/alpine:3.20
```

31.9 30.9 When to use Podman vs Docker vs Kubernetes

Scenario	Recommendation
Developer laptop (security-conscious)	Rootless Podman
Existing Docker Desktop team	Docker with hardened settings

CI build agents	<code>buildah / kaniko / docker buildx</code>
Production orchestration	Kubernetes (containerd/CRI-O runtime)
Single-node edge	Podman + systemd units

Kubernetes nodes typically run **CRI-O** or **containerd**—not Podman—though Red Hat OpenShift uses CRI-O with similar roots.

31.10 Chapter summary

- **Rootless** containers reduce host compromise risk by eliminating a root-owned central daemon.
 - **Podman** is daemonless, Docker CLI-compatible, and integrates with **systemd** and **pods**.
 - Configure **subuid/subgid** ranges and understand rootless **port and volume** limitations.
 - **Buildah** and **Skopeo** complete the toolchain for build and registry operations without Docker.
 - Use rootless on workstations; enforce signing, scanning, and orchestrator policies in production.
-

31.11 Lab 30.1

1. Install Podman rootless; run nginx publishing port 8080.
 2. Create a pod with two containers sharing a network namespace.
 3. Generate a systemd user unit and manage the container with `systemctl --user`.
-

31.12 Lab 30.2

1. Convert a Docker Compose file to run under `podman-compose`.
 2. Build an image with Buildah and push to GHCR with Skopeo or `podman push`.
 3. Compare `podman info` security section between rootful and rootless modes.
-

31.13 Review questions

1. Why is membership in the `docker` group considered equivalent to root access?
 2. How do user namespaces map container root to the host?
 3. What problem do Podman pods solve compared to standalone containers?
 4. Name two limitations of rootless mode when binding network ports.
 5. When would you choose Buildah over `podman build`?
-

Next: [Chapter 31 — K8s Architecture](#)

32 Chapter 31: Kubernetes Architecture — Control Plane and Node Components

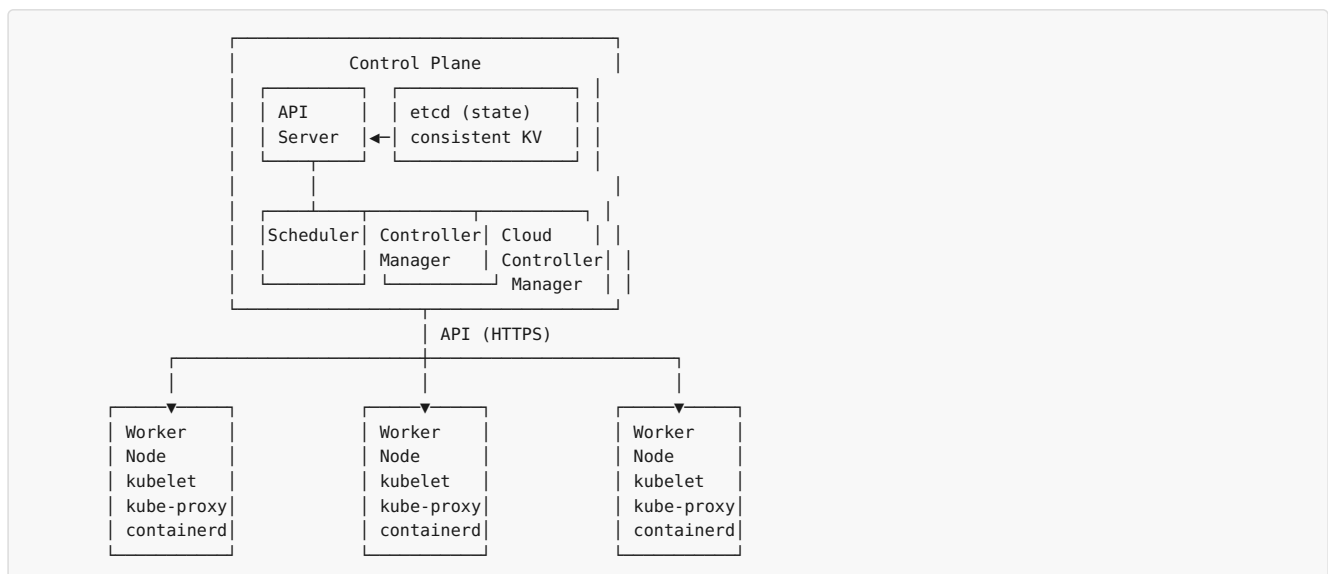
DevOps Handbook — Part VIII, Pages 581–605

32.1 31.1 What Kubernetes provides

Kubernetes (K8s) is a container orchestration platform that schedules workloads across a cluster, heals failed containers, scales replicas, rolls out updates, and exposes services. You declare **desired state** in YAML; controllers reconcile reality toward that state.

Kubernetes does not replace containers—it orchestrates **OCI images** running in **pods** on **nodes**, managed by a **control plane**.

32.2 31.2 Cluster architecture overview



32.3 31.3 Control plane components

32.3.1 API Server (kube-apiserver)

The **front door** to the cluster. All tools (`kubectl` , controllers, kubelet) talk to the API server over HTTPS. It validates requests, persists objects to etcd, and enforces authentication/authorization.

```
kubectl cluster-info
kubectl get --raw /healthz
kubectl api-resources
```

32.3.2 etcd

Distributed **key-value store** holding all cluster state: pods, services, secrets metadata, RBAC, etc. Production clusters run etcd with TLS, regular backups, and odd-numbered quorum (3 or 5 members).

```
# Backup (on etcd member - simplified)
ETCDCTL_API=3 etcdctl snapshot save backup.db \
--endpoints=https://127.0.0.1:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key
```

Never expose etcd to the internet.

32.3.3 Scheduler (kube-scheduler)

Watches unscheduled pods and assigns them to nodes based on resources, affinity, taints/tolerations, and topology spread.

32.3.4 Controller Manager (kube-controller-manager)

Runs control loops: ReplicaSet, Deployment, Node, Job, EndpointSlice controllers, etc. Example: Deployment controller creates ReplicaSets; ReplicaSet controller creates pods.

32.3.5 Cloud Controller Manager (CCM)

Integrates with cloud APIs: node lifecycle, load balancers, routes, volumes. On EKS/GKE/AKS, the cloud provider manages much of this.

32.4 31.4 Node components

32.4.1 kubelet

Agent on each node. Registers the node, watches pod specs assigned to it, pulls images, starts containers via the **CRI** (containerd, CRI-O), reports status, and runs probes.

32.4.2 kube-proxy

Maintains network rules (iptables, ipvs, or eBPF) so **Services** get stable virtual IPs and load-balance traffic to pod endpoints.

32.4.3 Container runtime

Runs containers. **containerd** is the default in modern distros; Docker was removed as a direct runtime after K8s 1.24 (use cri-dockerd shim only if needed).

```
kubectctl get nodes -o wide
crictl ps # On node - CRI debugging
systemctl status kubelet
```

32.5 31.5 Kubernetes objects and namespaces

Everything is an API **resource** with `apiVersion`, `kind`, `metadata`, `spec`, `status`.

```
apiVersion: v1
kind: Pod
metadata:
  name: demo
  namespace: dev
  labels:
    app: demo
spec:
  containers:
    - name: nginx
```

```
image: nginx:1.25-alpine
ports:
  - containerPort: 80
```

```
kubectl apply -f pod.yaml
kubectl get pods -n dev -o wide
kubectl describe pod demo -n dev
kubectl logs demo -n dev
kubectl delete pod demo -n dev
```

Namespaces isolate resources logically (not physically):

Namespace	Typical use
kube-system	Control plane addons
kube-public	Public cluster info
default	Avoid in prod
dev / staging / prod	Environment separation

32.6 31.6 kubectl and kubeconfig

~/.kube/config defines **clusters**, **users**, and **contexts**:

```
contexts:
- name: prod-eks
  context:
    cluster: prod-eks
    user: prod-eks-admin
    namespace: production
current-context: prod-eks
```

```
kubectl config get-contexts
kubectl config use-context staging-eks
kubectl config set-context --current --namespace=payments
```

Imperative vs declarative:

```
kubectl run nginx --image=nginx --port=80      # Imperative (avoid in prod)
kubectl apply -f deployment.yaml               # Declarative (preferred)
kubectl diff -f deployment.yaml
```

32.7 31.7 Managed vs self-managed clusters

Type	Examples	You manage
Managed control plane	EKS, GKE, AKS	Nodes, workloads, addons
Self-managed	kubeadm, kops	Control plane + nodes
Local dev	kind, minikube, k3d	Learning only

Managed services patch control plane versions; you still own upgrades, node AMIs, CNI, and workload security.

32.8 31.8 Addons and the CNCF landscape

Common cluster addons:

Addon	Purpose
CNI (Calico, Cilium, AWS VPC CNI)	Pod networking
CoreDNS	Service discovery DNS
Metrics Server	<code>kubectl top</code> , HPA
Ingress controller	HTTP routing
CSI driver	Persistent volumes
cert-manager	TLS certificates

Install with Helm or cloud marketplace; pin versions and test upgrades in staging.

32.9 31.9 High availability and etcd sizing

Production control plane:

- **3+ API server** instances behind load balancer
- **3 or 5 etcd** members across failure domains
- Regular **etcd backup** and restore drills
- **Separate** etcd from worker nodes

Worker nodes:

- Spread across **availability zones**
 - Use **PodDisruptionBudgets** during node drains
 - **Cluster Autoscaler** adds/removes nodes based on pending pods
-

32.10 31.10 Chapter summary

- The **control plane** (API server, etcd, scheduler, controllers) manages desired state; **nodes** run workloads via kubelet and runtime.
 - All interaction flows through the **API server**; protect it with RBAC, audit logs, and network policies.
 - **etcd** is the source of truth—backup and secure it aggressively.
 - Use **declarative YAML** and **kubectl apply**; understand namespaces and contexts for multi-environment work.
-

32.11 Lab 31.1

1. Create a local cluster with `kind` or `minikube`.
 2. Deploy a pod, inspect it with `describe`, and view logs.
 3. List control plane pods in `kube-system` and identify CoreDNS, kube-proxy.
-

32.12 Lab 31.2

1. Configure two contexts (e.g., `kind` + `cloud sandbox`) and switch between them.
2. Take an etcd snapshot on a kubeadm lab cluster (or document EKS backup approach).
3. Draw your org's cluster diagram: control plane ownership, node pools, addons.

32.13 Review questions

1. Which component is the only one that writes to etcd for API requests?
2. What is the kubelet's relationship to the container runtime?
3. Why should etcd never be exposed publicly?
4. What is the difference between the scheduler and a controller?
5. Name three addons required for a minimally functional cluster network and DNS.

Next: [Chapter 32 — K8s Workloads](#)

33 Chapter 32: Kubernetes Workloads — Pod, Deployment, StatefulSet, DaemonSet

DevOps Handbook — Part VIII, Pages 606–630

33.1 32.1 Pods — the atomic unit

A **Pod** is the smallest deployable unit in Kubernetes: one or more containers sharing network and storage namespaces, scheduled together on one node.

```
apiVersion: v1
kind: Pod
metadata:
  name: web
  labels:
    app: web
spec:
  containers:
    - name: nginx
      image: nginx:1.25-alpine
      ports:
        - containerPort: 80
      resources:
        requests:
          cpu: 100m
          memory: 128Mi
        limits:
          cpu: 500m
          memory: 256Mi
      livenessProbe:
        httpGet:
          path: /
          port: 80
        initialDelaySeconds: 10
        periodSeconds: 10
      readinessProbe:
        httpGet:
          path: /
          port: 80
        periodSeconds: 5
```

Probe	Purpose
liveness	Restart if deadlocked
readiness	Include in Service endpoints
startup	Slow-start apps before liveness kicks in

Sidecar pattern — helper container in same pod:

```
spec:
  containers:
    - name: app
      image: myapp:1.0
      volumeMounts:
        - name: logs
          mountPath: /var/log/app
    - name: log-shipper
      image: fluent/fluent-bit:2
      volumeMounts:
        - name: logs
          mountPath: /var/log/app
  volumes:
    - name: logs
      emptyDir: {}
```

Pods are ephemeral—do not manage them directly in production. Use controllers.

33.2 32.2 Deployments — stateless apps

A **Deployment** manages ReplicaSets to run N identical pod replicas with rolling updates and rollbacks.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
  labels:
    app: api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: api
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: api
    spec:
      containers:
        - name: api
          image: ghcr.io/acme/api:v1.2.0
          ports:
            - containerPort: 8080
          env:
            - name: LOG_LEVEL
              value: info
```

```
kubectl apply -f deployment.yaml
kubectl rollout status deployment/api
kubectl set image deployment/api api=ghcr.io/acme/api:v1.3.0
kubectl rollout history deployment/api
kubectl rollout undo deployment/api
kubectl scale deployment/api --replicas=5
```

33.2.1 Rolling update flow

```
v1 v1 v1 → v1 v1 v2 → v1 v2 v2 → v2 v2 v2
```

`maxUnavailable: 0` ensures capacity during rollout; `maxSurge` allows extra pods temporarily.

33.3 32.3 ReplicaSets and labels

Deployments own **ReplicaSets**; ReplicaSets own **Pods**. Labels in `selector.matchLabels` must match pod template labels.

```
kubectl get rs
kubectl get pods --show-labels
```

Never change immutable selector fields on existing Deployments—create a new Deployment instead.

33.4 32.4 StatefulSets — stable identity and storage

StatefulSets assign stable network IDs (`web-0` , `web-1`) and persistent storage per replica. Use for databases, Kafka, ZooKeeper.

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: postgres-headless
  replicas: 3
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:16
          ports:
            - containerPort: 5432
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        storageClassName: gp3
        resources:
          requests:
            storage: 50Gi
```

Headless Service for stable DNS:

```
apiVersion: v1
kind: Service
metadata:
  name: postgres-headless
spec:
  clusterIP: None
  selector:
    app: postgres
  ports:
    - port: 5432
```

Pod DNS: `postgres-0.postgres-headless.default.svc.cluster.local`

StatefulSet pods are created/deleted in order (0, 1, 2...). Scaling and updates are slower but predictable.

33.5 32.5 DaemonSets — one pod per node

DemonSets run a pod on every (or selected) node—agents, log collectors, CNI plugins, node exporters.

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-exporter
spec:
  selector:
    matchLabels:
      app: node-exporter
  template:
    metadata:
      labels:
        app: node-exporter
    spec:
      hostNetwork: true
```

```

tolerations:
  - operator: Exists
containers:
  - name: exporter
    image: prom/node-exporter:v1.8.2
    ports:
      - containerPort: 9100

```

`tolerations: operator: Exists` schedules on tainted control plane nodes if needed.

33.6 32.6 Jobs and CronJobs

Job — run to completion:

```

apiVersion: batch/v1
kind: Job
metadata:
  name: db-migrate
spec:
  backoffLimit: 3
  template:
    spec:
      restartPolicy: OnFailure
      containers:
        - name: migrate
          image: ghcr.io/acme/api:v1.2.0
          command: ["npm", "run", "migrate"]

```

CronJob — scheduled jobs:

```

apiVersion: batch/v1
kind: CronJob
metadata:
  name: nightly-report
spec:
  schedule: "0 2 * * *"
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: report
              image: ghcr.io/acme/reporter:latest

```

33.7 32.7 Choosing a workload type

Workload	Controller	When
HTTP API, workers	Deployment	Stateless, horizontal scale
Postgres cluster	StatefulSet	Stable ID + PVC per pod
Log agent	DaemonSet	Per-node background service
Migration, batch	Job	Run once to completion
Reports, backups	CronJob	Time-based

33.8 32.8 Pod scheduling controls

```
spec:
  affinity:
    podAntiAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
            matchLabels:
              app: api
          topologyKey: kubernetes.io/hostname
    topologySpreadConstraints:
      - maxSkew: 1
        topologyKey: topology.kubernetes.io/zone
        whenUnsatisfiable: DoNotSchedule
        labelSelector:
          matchLabels:
            app: api
```

Taints and tolerations repel pods unless tolerated:

```
# Node taint: dedicated=gpu:NoSchedule
spec:
  tolerations:
    - key: dedicated
      operator: Equal
      value: gpu
      effect: NoSchedule
```

33.9 32.9 Chapter summary

- **Pods** run containers; use probes and resource requests/limits.
- **Deployments** manage stateless apps with rolling updates and rollbacks.
- **StatefulSets** provide stable identity and storage for stateful systems.
- **DaemonSets** run per-node agents; **Jobs/CronJobs** handle batch work.
- Match workload type to data and scaling requirements—do not run databases as bare Deployments without understanding data loss risk.

33.10 Lab 32.1

1. Deploy a 3-replica nginx Deployment; practice rollout, rollback, and scale.
2. Add liveness and readiness probes; break the app and observe restarts vs traffic removal.
3. Deploy a StatefulSet with `volumeClaimTemplates` and verify ordered pod names.

33.11 Lab 32.2

1. Create a DaemonSet node-exporter and confirm one pod per node.
2. Run a Job migration container and inspect completion status.
3. Add pod anti-affinity so replicas spread across nodes/zones.

33.12 Review questions

1. What happens if readiness probe fails but liveness passes?
2. Why do StatefulSets need a headless Service?
3. When should you use a Job instead of a Deployment with `restartPolicy Never`?
4. What do `maxSurge` and `maxUnavailable` control during rollouts?
5. Why is running PostgreSQL as a single-replica Deployment risky?

Next: [*Chapter 33 — K8s Services and Ingress*](#)

34 Chapter 33: Kubernetes Services, Ingress, and Network Policies

DevOps Handbook — Part VIII, Pages 631–655

34.1 33.1 Kubernetes networking model

Every pod gets a cluster-routable **IP address**. Pods can talk to any other pod without NAT (CNI permitting). **Services** provide stable virtual IPs and DNS names in front of ephemeral pod IPs. **Ingress** (or Gateway API) routes HTTP/S from outside the cluster to Services.

Internet → Load Balancer → Ingress Controller → Service → Pod endpoints

34.2 33.2 Service types

```
apiVersion: v1
kind: Service
metadata:
  name: api
spec:
  type: ClusterIP
  selector:
    app: api
  ports:
    - port: 80
      targetPort: 8080
      protocol: TCP
```

Type	Reachability	Use case
ClusterIP	Internal only	Default; microservice-to-microservice
NodePort	Host port 30000–32767	Dev, legacy exposure
LoadBalancer	Cloud LB → Service	Public APIs on managed clouds
ExternalName	DNS CNAME	External SaaS alias

DNS inside cluster: `api.default.svc.cluster.local` (short: `api` in same namespace).

```
kubectl get svc
kubectl get endpoints api
kubectl run -it --rm debug --image=busybox -- wget -qO- http://api
```

Headless Service (`clusterIP: None`) returns pod A records directly—used by StatefulSets.

34.3 33.3 Endpoints and EndpointSlices

The **Endpoints** (or **EndpointSlice**) controller populates backend pod IPs for Services matching `selector`. Readiness probe failure removes a pod from endpoints—traffic stops without deleting the pod.

```
kubectl get endpointslices -l kubernetes.io/service-name=api
```

Services without selectors can manually point to external IPs:

```
spec:
  ports:
    - port: 443
      type: ClusterIP
  ---
  apiVersion: v1
  kind: Endpoints
  metadata:
    name: external-db
  subsets:
    - addresses:
        - ip: 10.0.50.10
      ports:
        - port: 5432
```

34.4 33.4 Ingress — HTTP routing

Ingress requires an **Ingress Controller** (nginx, traefik, AWS ALB, etc.).

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web-ingress
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - app.example.com
      secretName: app-tls
  rules:
    - host: app.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend
                port:
                  number: 80
    - host: api.example.com
      http:
        paths:
          - path: /v1
            pathType: Prefix
            backend:
              service:
                name: api
                port:
                  number: 80
```

pathType	Match
Prefix	/api matches /api/users
Exact	Exact path only
ImplementationSpecific	Controller-defined

Annotations configure TLS, redirects, rate limits, body size—varies by controller.

34.5 33.5 LoadBalancer and NodePort in cloud

EKS example — Service type LoadBalancer provisions an ELB:

```

apiVersion: v1
kind: Service
metadata:
  name: api-public
  annotations:
    service.beta.kubernetes.io/aws-load-balancer-type: "nlb"
    service.beta.kubernetes.io/aws-load-balancer-scheme: "internet-facing"
spec:
  type: LoadBalancer
  selector:
    app: api
  ports:
    - port: 443
      targetPort: 8080

```

Prefer **Ingress** or **Gateway API** over many LoadBalancers—cost and IP management.

34.6 33.6 Gateway API (modern alternative)

Gateway API separates **Gateway** (infrastructure) from **HTTPRoute** (application routing):

```

apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: api-route
spec:
  parentRefs:
    - name: public-gateway
  hostnames:
    - api.example.com
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /v1
      backendRefs:
        - name: api
          port: 80

```

More expressive than Ingress; adoption growing on cloud providers.

34.7 33.7 Network Policies — zero trust inside the cluster

By default, all pods can reach all pods (**allow-all**). **NetworkPolicy** restricts traffic at L3/L4.

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api-allow
  namespace: production
spec:
  podSelector:
    matchLabels:
      app: api
  policyTypes:
    - Ingress
    - Egress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: frontend
        - namespaceSelector:
            matchLabels:
              name: ingress-nginx
      ports:
        - protocol: TCP

```

```

      port: 8080
    egress:
      - to:
          - podSelector:
              matchLabels:
                app: postgres
            ports:
              - protocol: TCP
                port: 5432
        - to:                                # DNS
            - namespaceSelector: {}
              podSelector:
                matchLabels:
                  k8s-app: kube-dns
            ports:
              - protocol: UDP
                port: 53

```

Requires a CNI that enforces NetworkPolicy (**Calico**, **Cilium**, **Weave**). AWS VPC CNI needs policy enforcement addon.

Default deny pattern:

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]

```

Then add explicit allow policies per app.

34.8 33.8 Debugging connectivity

```

kubectl run netshoot --rm -it --image=nicolaka/netshoot -- bash
curl -v http://api.production.svc.cluster.local
nslookup api.production.svc.cluster.local
kubectl describe networkpolicy -n production
kubectl get ingress
kubectl logs -n ingress-nginx deploy/ingress-nginx-controller

```

Symptom	Check
503 from Ingress	Endpoints empty? Readiness failing?
Timeout pod-to-pod	NetworkPolicy deny? Wrong namespace DNS?
Works by IP not name	CoreDNS issue
External no route	LB security groups, Ingress class

34.9 33.9 Chapter summary

- **Services** abstract pod IPs with ClusterIP, NodePort, and LoadBalancer types.
- **Ingress** (or **Gateway API**) routes HTTP host/path to Services; requires a controller.
- **NetworkPolicies** implement least-privilege pod communication—start with default deny in production namespaces.
- Always verify **endpoints** and **readiness** when traffic fails mysteriously.

34.10 Lab 33.1

1. Expose a Deployment with ClusterIP; reach it from a debug pod via DNS.
 2. Install nginx Ingress Controller; route two hostnames to two Services.
 3. Add TLS with cert-manager (or self-signed for lab).
-

34.11 Lab 33.2

1. Apply default-deny NetworkPolicy in a namespace.
 2. Add allow rules so frontend → api → postgres works; verify blocked paths fail.
 3. Document required egress to kube-dns for DNS resolution.
-

34.12 Review questions

1. How does a Service know which pods to send traffic to?
 2. What is the difference between ClusterIP and Headless Services?
 3. Why does Ingress require a separate controller?
 4. What happens to Service traffic when all pods fail readiness?
 5. Which CNIs enforce NetworkPolicy, and what breaks if yours does not?
-

Next: [Chapter 34 — K8s Config and Storage](#)

35 Chapter 34: Kubernetes ConfigMaps, Secrets, and Storage (PV/PVC)

DevOps Handbook — Part VIII, Pages 656–675

35.1 34.1 Separating config from images

Twelve-factor apps store config in the environment, not in images. Kubernetes provides **ConfigMaps** for non-sensitive config and **Secrets** for sensitive data—mounted as files or injected as environment variables.

Object	Data	Base64	Encryption at rest
ConfigMap	Plain text	No	Optional
Secret	Sensitive	Yes (not encryption)	Enable KMS provider

Never commit plaintext Secrets to Git. Use **Sealed Secrets**, **External Secrets Operator**, or **SOPS**.

35.2 34.2 ConfigMaps

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: api-config
data:
  LOG_LEVEL: debug
  APP_ENV: staging
  nginx.conf: |
    server {
      listen 80;
      location / { proxy_pass http://127.0.0.1:8080; }
    }
```

Reference in a Deployment:

```
spec:
  containers:
    - name: api
      image: ghcr.io/acme/api:1.0
      envFrom:
        - configMapRef:
            name: api-config
      env:
        - name: LOG_LEVEL
          valueFrom:
            configMapKeyRef:
              name: api-config
              key: LOG_LEVEL
      volumeMounts:
        - name: config-vol
          mountPath: /etc/nginx
          readOnly: true
  volumes:
    - name: config-vol
      configMap:
        name: api-config
        items:
          - key: nginx.conf
            path: nginx.conf
```

```
kubectl create configmap api-config --from-literal=LOG_LEVEL=info
```

```
kubectl create configmap app-config --from-file=./config/
kubectl get configmap api-config -o yaml
```

Immutable ConfigMaps (recommended): set `immutable: true` to prevent accidental updates causing rolling reload storms.

35.3 34.3 Secrets

```
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
type: Opaque
stringData:      # Plain text in manifest – encode on apply
  username: appuser
  password: s3cr3t-db-pwd
```

```
kubectl create secret generic db-credentials \
  --from-literal=username=appuser \
  --from-literal=password='s3cr3t'

kubectl create secret docker-registry ghcr-creds \
  --docker-server=ghcr.io \
  --docker-username=user \
  --docker-password=$TOKEN
```

Mount as files (preferred for apps that read credential files):

```
volumeMounts:
- name: db-secret
  mountPath: /etc/secrets/db
  readOnly: true
volumes:
- name: db-secret
  secret:
    secretName: db-credentials
    defaultMode: 0400
```

RBAC restricts who reads Secrets. Enable **encryption at rest** with KMS (AWS, GCP, Azure integrations).

35.3.1 External Secrets Operator

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: db-credentials
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secrets-manager
    kind: ClusterSecretStore
  target:
    name: db-credentials
  data:
  - secretKey: password
    remoteRef:
      key: prod/bookstore/db
      property: password
```

Secrets sync from AWS Secrets Manager, Vault, or Parameter Store into native K8s Secrets.

35.4 34.4 Storage basics — PV, PVC, StorageClass

Pod filesystems are ephemeral. **PersistentVolume (PV)** is cluster storage; **PersistentVolumeClaim (PVC)** is a pod's request for storage.

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gp3
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
  encrypted: "true"
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data-pvc
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: gp3
resources:
  requests:
    storage: 20Gi
```

```
spec:
  containers:
    - name: postgres
      image: postgres:16
      volumeMounts:
        - name: data
          mountPath: /var/lib/postgresql/data
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: data-pvc
```

Access mode	Meaning
ReadWriteOnce (RWO)	One node read/write
ReadOnlyMany (ROX)	Many nodes read
ReadWriteMany (RWX)	Many nodes read/write (EFS, NFS)

WaitForFirstConsumer delays volume provisioning until pod is scheduled—important for zonal disks.

35.5 34.5 StatefulSet storage pattern

StatefulSets use **volumeClaimTemplates** for per-replica PVCs (`data-postgres-0` , `data-postgres-1`).

Scaling down does **not** delete PVCs by default—data survives for rescheduling. Orphan PVC cleanup must be deliberate.

```
kubectl get pvc
kubectl describe pvc data-pvc
kubectl exec postgres-0 -- df -h /var/lib/postgresql/data
```

35.6 34.6 Config reload and updates

Changing a ConfigMap mounted as env var **does not** update running containers—restart required. Mounted

volumes can refresh (with delay; subPath does not auto-update).

Patterns:

- **Reloader** operator watches ConfigMaps/Secrets and rolls Deployments
- Sidecar **inotify** watcher triggers app reload
- Immutable versions: `api-config-v3` → update deployment reference

For Secrets, prefer short-lived tokens and automatic rotation via External Secrets.

35.7 34.7 Backup and snapshots

CSI **VolumeSnapshot**:

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata:
  name: data-snap-20250901
spec:
  volumeSnapshotClassName: csi-aws-vsc
  source:
    persistentVolumeClaimName: data-pvc
```

Backup tools: **Velero** (cluster resources + PV snapshots), cloud-native backup services.

Test restores regularly—an untested backup is wishful thinking.

35.8 34.8 Security checklist

- No Secrets in container images or public Git
 - Enable **encryption at rest** for etcd Secrets
 - Restrict Secret get/list with RBAC
 - Use **readOnly** volume mounts where possible
 - Prefer **projected volumes** combining multiple sources
 - Scan manifests in CI for hardcoded credentials (**gitleaks**, **checkov**)
-

35.9 34.9 Chapter summary

- **ConfigMaps** hold non-sensitive config; **Secrets** hold sensitive data—neither replaces a secrets manager at scale.
 - Mount config as **volumes** for file-based apps; use **env** for simple key-value.
 - **PVCs** request storage; **StorageClasses** enable dynamic provisioning.
 - Match **access modes** and **StorageClass** topology to workload (RWO for EBS, RWX for shared files).
 - Integrate **External Secrets** and **volume snapshots** for production operations.
-

35.10 Lab 34.1

1. Create ConfigMap and Secret; deploy an app consuming both via env and volume mounts.
 2. Provision a PVC with dynamic StorageClass; verify data survives pod deletion.
 3. Expand a PVC if `allowVolumeExpansion` is enabled.
-

35.11 Lab 34.2

1. Deploy External Secrets Operator (or Sealed Secrets) syncing one secret from a vault/cloud store.
 2. Take a VolumeSnapshot and restore to a new PVC.
 3. Add Reloader or manual rollout workflow when ConfigMap changes.
-

35.12 Review questions

1. Why is base64 encoding in Secrets not considered encryption?
 2. What happens to a PVC when its pod is deleted?
 3. When should you use ReadWriteMany instead of ReadWriteOnce?
 4. Why do env-var ConfigMap changes not appear in running pods?
 5. What does `volumeBindingMode: WaitForFirstConsumer` prevent?
-

Next: [Chapter 35 — Helm and GitOps](#)

36 Chapter 35: Helm, Kustomize, and GitOps (Argo CD)

DevOps Handbook — Part VIII, Pages 676–700

36.1 35.1 The packaging problem

Raw YAML does not scale: duplicated labels, repeated image tags across environments, and manual `kubectl` apply drift from Git. **Helm** packages charts; **Kustomize** overlays bases; **GitOps** (Argo CD, Flux) continuously reconciles cluster state from Git.

```
Git (desired state) → CI builds image → GitOps controller → Kubernetes (actual state)
```

36.2 35.2 Helm charts

A **chart** is a directory of templated Kubernetes manifests plus metadata.

```
mychart/
├── Chart.yaml
├── values.yaml
├── templates/
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── ingress.yaml
│   └── _helpers.tpl
└── charts/          # Subchart dependencies
```

Chart.yaml :

```
apiVersion: v2
name: api
description: Bookstore API
type: application
version: 0.1.0
appVersion: "1.2.0"
```

values.yaml :

```
replicaCount: 2
image:
  repository: ghcr.io/acme/api
  tag: "1.2.0"
  pullPolicy: IfNotPresent
service:
  type: ClusterIP
  port: 80
ingress:
  enabled: true
  host: api.example.com
resources:
  requests:
    cpu: 100m
    memory: 128Mi
```

templates/deployment.yaml excerpt:

```
apiVersion: apps/v1
kind: Deployment
metadata:
```

```

name: {{ include "api.fullname" . }}
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    spec:
      containers:
        - name: api
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          ports:
            - containerPort: 8080

```

Commands:

```

helm repo add bitnami https://charts.bitnami.com/bitnami
helm search repo nginx
helm install my-nginx bitnami/nginx --namespace web --create-namespace
helm upgrade my-nginx bitnami/nginx -f values-prod.yaml
helm list
helm uninstall my-nginx
helm template api ./mychart -f values-staging.yaml # Render locally

```

Override per environment:

```

# values-prod.yaml
replicaCount: 5
image:
  tag: "1.2.0"
ingress:
  host: api.example.com

```

36.3 35.3 Helm hooks and releases

Hooks run jobs at pre/post install/upgrade (`helm.sh/hook: pre-upgrade`). Use for DB migrations—carefully, with rollback strategy.

Release = installed chart instance stored as Secrets in `sh.helm.release.v1.*` (history for rollback).

```

helm history api
helm rollback api 3

```

Helm 3 is **client-side only**—no Tiller. RBAC applies to the user or CI service account running Helm.

36.4 35.4 Kustomize — patch without templates

Kustomize is built into `kubectl apply -k`. No templating language—bases + overlays with patches and image transforms.

```

deploy/
├─ base/
│  ├─ kustomization.yaml
│  ├─ deployment.yaml
│  └─ service.yaml
└─ overlays/
   ├─ staging/
   │  ├─ kustomization.yaml
   │  └─ patch-replicas.yaml
   └─ production/
      ├─ kustomization.yaml
      └─ patch-resources.yaml

```

base/kustomization.yaml :

```

apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - deployment.yaml
  - service.yaml
commonLabels:
  app: api

```

overlays/production/kustomization.yaml :

```

apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - ../../base
namespace: production
replicas:
  - name: api
    count: 5
images:
  - name: ghcr.io/acme/api
    newTag: v1.2.0
patches:
  - path: patch-resources.yaml
configMapGenerator:
  - name: api-config
    literals:
      - LOG_LEVEL=warn

```

```

kubectl apply -k deploy/overlays/production
kubectl diff -k deploy/overlays/staging

```

Tool	Strength	Weakness
Helm	Packaging, versioning, ecosystem charts	Go templates complexity
Kustomize	Simple overlays, native kubectl	Less packaging metadata

Many teams use **Helm charts rendered by Kustomize** (`helm template | kustomize`) or Helm with post-render Kustomize.

36.5 35.5 GitOps principles

1. **Git is the source of truth** for desired cluster state
2. **Declarative** manifests (YAML, Helm, Kustomize)
3. **Automated reconciliation** — controller applies diffs
4. **Observable** — drift visible in UI/CLI

Anti-pattern: `kubectl apply` from laptops to production. GitOps makes changes via PR merge.

36.6 35.6 Argo CD

Argo CD watches Git repos and syncs Applications to clusters.

```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: bookstore-api
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/acme-corp/gitops.git
    targetRevision: main

```

```

    path: apps/bookstore/overlays/production
  destination:
    server: https://kubernetes.default.svc
    namespace: production
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true

```

```

argocd app list
argocd app diff bookstore-api
argocd app sync bookstore-api
argocd app rollback bookstore-api

```

App of Apps pattern bootstraps many Applications from one root manifest.

36.6.1 Sync strategies

Mode	Behavior
Manual	Human clicks Sync
Automated	Sync on Git change
selfHeal	Revert manual kubectl edits
prune	Delete resources removed from Git

Use **prune** cautiously with shared resources; test in staging first.

36.7 35.7 Promotion pipeline

Typical flow:

1. CI builds image → pushes `ghcr.io/acme/api:abc123`
2. CI or bot opens PR updating `newTag` in `overlays/staging`
3. Argo CD syncs staging; run smoke tests
4. PR promotes same tag to `overlays/production`
5. Argo CD syncs prod (manual approval optional via Argo CD projects)

Image updater tools (Argo CD Image Updater, Flux image automation) can write back to Git when new tags appear—pin policies to semver or regex.

36.8 35.8 Flux (brief comparison)

Flux CD uses GitRepository + Kustomization/HelmRelease CRDs:

```

apiVersion: source.toolkit.fluxcd.io/v1
kind: GitRepository
metadata:
  name: gitops
spec:
  url: https://github.com/acme-corp/gitops
  ref:
    branch: main
---
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
  name: production
spec:

```

```

sourceRef:
  kind: GitRepository
  name: gitops
  path: ./apps/bookstore/overlays/production
  interval: 5m
  prune: true

```

	Argo CD	Flux
UI	Rich web UI	CLI / Weave GitOps UI
Model	Application CRD	Modular controllers
Multi-cluster	ApplicationSet	Cluster API integration

Both are CNCF graduated/incubating; choice often organizational.

36.9 35.9 Secrets in GitOps

Never plain Secrets in Git. Options:

- **Sealed Secrets** — encrypt for cluster-only decryption
- **SOPS** + KMS — Mozilla SOPS with age/PGP/cloud KMS
- **External Secrets** — reference cloud vault; not stored in Git

Argo CD supports helm-secrets, ksops plugins for encrypted manifests.

36.10 35.10 Chapter summary

- **Helm** packages and parameterizes Kubernetes apps; **Kustomize** patches bases per environment.
- **GitOps** reconciles cluster state from Git—eliminating ad-hoc kubectl changes.
- **Argo CD** and **Flux** automate deploys with visibility into drift and sync status.
- Protect secrets with encryption or external stores; promote images via Git PRs, not manual image bumps on nodes.

36.11 Lab 35.1

1. Package a Deployment + Service as a Helm chart; install with staging and prod values files.
2. Replicate the same app with Kustomize base + two overlays.
3. Compare rendered YAML from `helm template` vs `kubectl kustomize`.

36.12 Lab 35.2

1. Install Argo CD on a test cluster; deploy an Application from a Git repo.
2. Change a replica count in Git; observe auto-sync (if enabled).
3. Make a manual `kubectl scale` change; observe selfHeal revert it.

36.13 Review questions

1. What is the difference between a Helm chart and a Helm release?
2. When would Kustomize be preferable to Helm?

3. What risks does `prune: true` introduce?
4. How does GitOps change the role of `kubectl apply` in production?
5. Name two approaches for storing secrets in a GitOps repository.

Next: [*Chapter 36 — K8s RBAC and Security*](#)

37 Chapter 36: Kubernetes RBAC, Pod Security, and Hardening

DevOps Handbook — Part VIII, Pages 701–720

37.1 36.1 Defense in depth for Kubernetes

Cluster security spans authentication (who), authorization (what they can do), admission control (what can be created), network policy (who can talk to whom), and workload hardening (how pods run). No single control is sufficient—layer them.

```
Request → AuthN (cert/OIDC) → AuthZ (RBAC) → Admission (PSA/OPA) → Pod sandbox → NetworkPolicy
```

37.2 36.2 Authentication overview

Human users should not use long-lived client certificates in production. Prefer **OIDC** integration (Google, Azure AD, Okta) via `kubect1` and cloud IAM for CI.

Service accounts authenticate **in-cluster** workloads:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: api
  namespace: production
```

```
spec:
  serviceAccountName: api
  containers:
  - name: api
    image: ghcr.io/acme/api:1.0
```

Each ServiceAccount gets a mounted token (bound to audience in K8s 1.24+). Use **IRSA** (EKS), **Workload Identity** (GKE), or **Azure Workload Identity** to map SA → cloud IAM role—never static cloud keys in pods.

```
kubectl auth can-i create deployments --namespace production
kubectl auth can-i --list --as=system:serviceaccount:production:api
```

37.3 36.3 RBAC — Roles and Bindings

RBAC grants permissions via **Role** (namespace) or **ClusterRole** (cluster-wide), bound with **RoleBinding** or **ClusterRoleBinding**.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: deployer
  namespace: staging
rules:
  - apiGroups: ["apps"]
    resources: ["deployments"]
    verbs: ["get", "list", "watch", "create", "update", "patch"]
  - apiGroups: [""]
    resources: ["pods", "pods/log"]
```

```

    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: ci-deployer
  namespace: staging
subjects:
- kind: User
  name: github-actions-ci
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: deployer
  apiGroup: rbac.authorization.k8s.io

```

Least privilege verbs:

Verb	Risk
get, list, watch	Read
create, update, patch	Modify
delete	Destructive
*	Avoid

ClusterRole for read-only auditors:

```

apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: view-all
rules:
- apiGroups: ["*"]
  resources: ["*"]
  verbs: ["get", "list", "watch"]

```

Bind to group `viewers` from OIDC. Never grant `cluster-admin` casually—use break-glass accounts with MFA and audit logging.

37.4 36.4 Aggregated roles and default bindings

Built-in ClusterRoles: `view`, `edit`, `admin`, `cluster-admin`. Namespace `RoleBinding` to `cluster-admin` is still namespace-scoped for namespaced resources but powerful.

Audit bindings:

```

kubectl get rolebindings,clusterrolebindings --all-namespaces
kubectl describe clusterrolebinding cluster-admin

```

Remove default permissions for unauthenticated or overly broad groups in managed clusters.

37.5 36.5 Pod Security Admission (PSA)

Pod Security Admission replaces deprecated PodSecurityPolicy. Namespace labels enforce **privileged**, **baseline**, or **restricted** profiles.

```

apiVersion: v1
kind: Namespace
metadata:
  name: production

```

```
labels:
  pod-security.kubernetes.io/enforce: restricted
  pod-security.kubernetes.io/enforce-version: latest
  pod-security.kubernetes.io/audit: restricted
  pod-security.kubernetes.io/warn: restricted
```

Profile	Constraints
privileged	Unrestricted
baseline	No hostNetwork, hostPID, privileged containers
restricted	Non-root, drop caps, seccomp, no privilege escalation

Restricted-compliant pod excerpt:

```
spec:
  securityContext:
    runAsNonRoot: true
    seccompProfile:
      type: RuntimeDefault
  containers:
    - name: api
      image: ghcr.io/acme/api:1.0
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities:
          drop: ["ALL"]
      volumeMounts:
        - name: tmp
          mountPath: /tmp
  volumes:
    - name: tmp
      emptyDir: {}
```

Test violations:

```
kubectl label namespace production pod-security.kubernetes.io/enforce=restricted --overwrite
kubectl apply -f pod.yaml # Should fail if non-compliant
```

37.6 36.6 OPA Gatekeeper and Kyverno

Admission controllers enforce custom policies beyond PSA.

Kyverno — validate/mutate/generate:

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: require-non-root
spec:
  validationFailureAction: Enforce
  rules:
    - name: check-runasnonroot
      match:
        any:
          - resources:
              kinds: [Pod]
      validate:
        message: "Containers must run as non-root"
        pattern:
          spec:
            containers:
              - securityContext:
                  runAsNonRoot: true
```

Gatekeeper uses Rego (see Chapter 25). Start with PSA labels, add policy engines for org-specific rules (required

labels, approved registries, resource limits).

37.7 36.7 Workload hardening checklist

Control	Implementation
Non-root user	<code>runAsUser: 10001, runAsNonRoot: true</code>
Read-only rootfs	<code>readOnlyRootFilesystem: true</code> + tmpfs volumes
Drop capabilities	<code>capabilities.drop: [ALL]</code>
No privileged	<code>privileged: false</code>
Resource limits	Prevent noisy neighbor / DoS
Seccomp/AppArmor	<code>seccompProfile: RuntimeDefault</code>
Image pull policy	<code>IfNotPresent</code> + pinned tags/digests
NetworkPolicy	Default deny + explicit allow

```
resources:
  requests:
    cpu: 100m
    memory: 128Mi
  limits:
    cpu: "1"
    memory: 512Mi
```

Pods without limits may be BestEffort QoS and evicted first under pressure.

37.8 36.8 Secrets, audit logs, and API security

Enable **audit logging** on API server (managed clouds: control plane logs). Ship to SIEM; alert on `secrets` access and `clusterrolebinding` changes.

Restrict **etcd** and **API server** network access. Disable anonymous auth. Rotate credentials.

RBAC for Secrets:

```
rules:
- apiGroups: [""]
  resources: ["secrets"]
  verbs: ["get", "list"]
```

Limit to operators and controllers—not all developers.

37.9 36.9 Node and supply chain security

- **Minimal node OS** — Bottlerocket, COS, hardened AMIs
- **Auto-upgrade** node pools on CVE patches
- **Admission** — only signed images from approved registries (Chapter 29)
- **Falco** — runtime threat detection (syscall anomalies)
- **Vulnerability scanning** — workload images in CI and admission

Disable mounting Docker socket into pods (`/var/run/docker.sock`)—instant cluster compromise vector.

37.10 36.10 Incident response primitives

```
# Contain compromised pod
kubectl cordon node-name
kubectl delete pod compromised-pod --grace-period=0
kubectl scale deployment/suspicious --replicas=0

# Investigate
kubectl auth can-i --list --as=system:serviceaccount:ns:sa
kubectl get events -A --sort-by='.lastTimestamp'
kubectl logs -n kube-system -l component=kube-apiserver
```

Maintain runbooks for credential rotation, etcd restore, and compromised service account token revocation.

37.11 36.11 Chapter summary

- **RBAC** enforces least privilege—prefer namespace Roles over cluster-admin.
 - **Service accounts** should map to cloud IAM via workload identity, not static keys.
 - **Pod Security Admission** enforces baseline/restricted profiles at namespace level.
 - Layer **admission policies**, **NetworkPolicies**, and **runtime monitoring** for production hardening.
 - Enable **audit logs** and test restricted pod specs before enforcing in production namespaces.
-

37.12 Lab 36.1

1. Create a Role allowing deploy updates only in `staging`; bind to a test user or SA.
 2. Label a namespace `restricted` and fix a Deployment until it passes PSA.
 3. Run `kubectl auth can-i` checks validating your RBAC design.
-

37.13 Lab 36.2

1. Install Kyverno or Gatekeeper; deny containers running as root cluster-wide.
 2. Apply default-deny NetworkPolicy plus explicit allows for one app stack.
 3. Review API audit logs (or simulate with `kubectl` commands) for sensitive resource access.
-

37.14 Review questions

1. What is the difference between a Role and a ClusterRole?
 2. Which PSA profile should production app namespaces use?
 3. Why should pods not use cluster-admin ServiceAccounts?
 4. Name three fields required for a restricted Pod Security profile.
 5. How do admission controllers differ from RBAC?
-

Next: [Chapter 37 — CI/CD Pipeline Design](#)

38 Chapter 37: CI/CD Concepts and Pipeline Design

DevOps Handbook — Part IX, Pages 721–745

38.1 37.1 Continuous Integration (CI)

CI means developers merge code to a shared branch **frequently** (at least daily), and each merge triggers an **automated build and test** pipeline.

Goals:

- Detect integration bugs early
- Keep `main` always deployable
- Reduce merge hell from long-lived branches

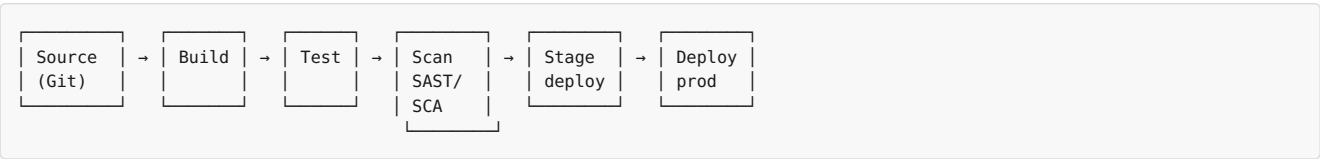
38.2 37.2 Continuous Delivery vs Continuous Deployment

Practice	Definition
Continuous Delivery	Every change is releasable; deploy to prod is a manual (or one-click) decision
Continuous Deployment	Every change that passes tests is automatically deployed to production

Most enterprises use **Continuous Delivery** with approval gates for production.

38.3 37.3 Pipeline stages

A typical pipeline:



Stage	Activities
Source	Webhook on push/PR
Build	Compile, docker build, artifact upload
Test	Unit, integration, contract tests
Scan	SAST, dependency scan, container scan, IaC scan
Stage deploy	Deploy to staging; smoke tests
Prod deploy	Blue/green, canary, or rolling; approval gate

38.4 37.4 GitHub Actions example

.github/workflows/ci.yml:

```
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Node
        uses: actions/setup-node@v4
        with:
          node-version: "20"
          cache: npm

      - run: npm ci
      - run: npm run lint
      - run: npm test -- --coverage

      - name: Build Docker image
        run: docker build -t myapp:${{ github.sha }} .

      - name: Scan image
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: myapp:${{ github.sha }}
          severity: CRITICAL,HIGH
          exit-code: 1

  deploy-staging:
    needs: build-and-test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    environment: staging
    steps:
      - uses: actions/checkout@v4
      - name: Deploy to staging
        run: ./scripts/deploy.sh staging ${github.sha}
```

38.5 37.5 Branching and trunk-based development

Trunk-based development (recommended for CI/CD):

- Short-lived feature branches (< 2 days)
- Frequent merges to `main`
- Feature flags hide incomplete work

Avoid long-running `develop` branches that diverge from production reality.

38.6 37.6 Deployment strategies

Strategy	How it works	Risk
Rolling	Replace instances gradually	Medium
Blue/green	Switch traffic between two identical envs	Low downtime
Canary	Route small % traffic to new version	Lowest blast radius
Recreate	Tear down old, start new	Downtime

38.7 37.7 Pipeline secrets and artifacts

- Store secrets in **GitHub Secrets**, **Vault**, or cloud secret managers—never in YAML.
 - Pin action versions (@v4 , commit SHA) for supply chain security.
 - Upload build artifacts (JAR, Docker image digest) for downstream deploy jobs.
 - Sign artifacts (Sigstore/cosign) for provenance.
-

38.8 37.8 DORA metrics in your pipeline

Track:

1. **Deployment frequency** — How often you deploy to prod
2. **Lead time for changes** — Commit to prod duration
3. **Change failure rate** — % of deploys causing incidents
4. **MTTR** — Time to restore after failure

Instrument pipelines to emit these metrics to your observability platform.

38.9 37.9 Chapter summary

- CI validates every merge; CD/CD automates release to staging/prod.
 - Design pipelines with **build → test → scan → deploy** stages.
 - Prefer trunk-based dev, pinned dependencies, and safe deployment strategies.
-

38.10 Lab 37.1

Build a full GitHub Actions pipeline for a sample app: lint, test, Docker build, Trivy scan, and deploy to staging on merge to `main`.

Continue: Chapter 38 — GitHub Actions Deep Dive (outline in TOC)

39 Chapter 38: GitHub Actions — Workflows, Secrets, Runners

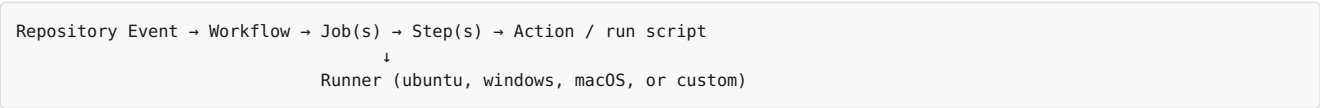
DevOps Handbook — Part IX, Pages 746–770

39.1 38.1 GitHub Actions architecture

GitHub Actions is GitHub’s native CI/CD platform. Workflows are triggered by repository events (push, pull request, schedule, manual dispatch) and execute as one or more **jobs** on **runners**—virtual machines or containers that run your steps.

Core components:

Component	Description
Workflow	YAML file in <code>.github/workflows/</code> defining automation
Event	Trigger (push , pull_request , workflow_dispatch , etc.)
Job	A set of steps that run on the same runner
Step	Individual task: run a script or use an action
Action	Reusable unit (checkout, setup-node, deploy)
Runner	Execution environment (GitHub-hosted or self-hosted)
Artifact	Files persisted between jobs or after workflow completion
Environment	Named deployment target with optional protection rules



GitHub Actions integrates tightly with GitHub: PR checks, branch protection, environments, and OIDC federation for cloud deployments without long-lived credentials.

39.2 38.2 Workflow syntax fundamentals

Every workflow begins with a name, trigger, and jobs block:

```
name: CI Pipeline

on:
  push:
    branches: [main, release/*]
  pull_request:
    branches: [main]
  workflow_dispatch:
    inputs:
      environment:
        description: Target environment
        required: true
        type: choice
        options: [staging, production]

concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
```

```

permissions:
  contents: read
  id-token: write # Required for OIDC to AWS/GCP/Azure

jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 15
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-go@v5
        with:
          go-version: "1.22"
          cache: true
      - run: go test ./... -race -coverprofile=coverage.out
      - uses: actions/upload-artifact@v4
        with:
          name: coverage
          path: coverage.out

```

Key syntax elements:

Key	Purpose
<code>on</code>	Events and filters (paths, branches, tags)
<code>jobs.<id>.needs</code>	Job dependency graph
<code>jobs.<id>.if</code>	Conditional execution
<code>jobs.<id>.strategy.matrix</code>	Parallel builds across OS/versions
<code>\${{ }}</code>	Expression syntax for contexts and functions
<code>env</code>	Environment variables at workflow, job, or step level

Contexts provide runtime data: `github`, `env`, `job`, `steps`, `secrets`, `vars`. Use `github.sha`, `github.ref_name`, and `github.event_name` for dynamic behavior.

39.3 38.3 Reusable workflows and composite actions

Avoid duplicating pipeline logic across repositories with **reusable workflows** and **composite actions**.

Reusable workflow (called from another repo's workflow):

```

# .github/workflows/reusable-test.yml
on:
  workflow_call:
    inputs:
      node-version:
        required: true
        type: string
    secrets:
      NPM_TOKEN:
        required: true
    outputs:
      image-tag:
        description: Built image tag
        value: ${ jobs.build.outputs.tag }

jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
      tag: ${ steps.meta.outputs.tag }
    steps:
      - uses: actions/checkout@v4
      - id: meta
        run: echo "tag=${{ github.sha }}" >> $GITHUB_OUTPUT

```

Caller workflow:

```
jobs:
  call-reusable:
    uses: ../github/workflows/reusable-test.yml
    with:
      node-version: "20"
    secrets:
      NPM_TOKEN: ${ secrets.NPM_TOKEN }
```

Composite actions bundle multiple steps into one `uses:` reference—ideal for org-wide lint or deploy patterns stored in a dedicated `actions` repository.

39.4 38.4 Secrets, variables, and security

Secrets must never appear in logs. GitHub automatically masks values registered as secrets.

Secret scope	Visibility
Repository secrets	Single repo workflows
Environment secrets	Jobs targeting named environment
Organization secrets	Selected repos in org
Dependabot secrets	Dependabot workflows only

```
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - name: Deploy with API key
        env:
          DEPLOY_KEY: ${ secrets.DEPLOY_API_KEY }
        run: ./scripts/deploy.sh
```

Security best practices:

- 1. **Least privilege** — Set `permissions:` explicitly; default was historically overly broad.
- 2. **Pin actions** — Use commit SHA or version tag (`@v4`), not `@main` .
- 3. **OIDC over static keys** — Federate to AWS IAM, Azure, GCP without storing cloud credentials in GitHub.
- 4. **Environment protection** — Require reviewers for production; limit branches that can deploy.
- 5. **Fork PR safety** — Workflows from forks do not receive secrets unless explicitly configured (and carefully).

OIDC example for AWS:

```
permissions:
  id-token: write
  contents: read

steps:
  - uses: aws-actions/configure-aws-credentials@v4
    with:
      role-to-assume: arn:aws:iam::123456789012:role/GitHubActionsDeploy
      aws-region: us-east-1
```

Repository **variables** (`vars`) store non-sensitive configuration (region names, feature flags) visible to collaborators with appropriate access.

39.5 38.5 Runners: hosted vs self-hosted

GitHub-hosted runners are managed by GitHub (Ubuntu, Windows, macOS). They are ephemeral, pre-installed with common tools, and billed by minute (included minutes on free/team plans, then usage-based).

Self-hosted runners run on your infrastructure—on-premises VMs, Kubernetes, or cloud instances you control.

Aspect	GitHub-hosted	Self-hosted
Maintenance	GitHub	You
Custom software	Limited image	Full control
Network access	Public internet	Private VPC, internal APIs
Cost model	Per-minute	Your infra + runner software
Isolation	Fresh VM per job	Shared or dedicated per label
Scale	Automatic	You add runners

Self-hosted runner registration:

```
# On your runner machine
mkdir actions-runner && cd actions-runner
curl -o actions-runner-linux-x64-2.311.0.tar.gz -L \
  https://github.com/actions/runner/releases/download/v2.311.0/actions-runner-linux-x64-2.311.0.tar.gz
tar xzf actions-runner-linux-x64-2.311.0.tar.gz
./config.sh --url https://github.com/myorg/myrepo --token RUNNER_TOKEN
sudo ./svc.sh install
sudo ./svc.sh start
```

Use **runner labels** (`runs-on: [self-hosted, linux, gpu]`) to route jobs. For Kubernetes, projects like **Actions Runner Controller (ARC)** scale ephemeral runners as pods.

Runner groups (organization level) restrict which repos can use which runners—critical for security when runners access internal networks.

39.6 38.6 Matrix builds, caching, and artifacts

Matrix strategies run the same job across multiple configurations:

```
jobs:
  test-matrix:
    strategy:
      fail-fast: false
      matrix:
        os: [ubuntu-latest, macos-latest]
        python: ["3.11", "3.12"]
    runs-on: ${ matrix.os }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${ matrix.python }
      - run: pip install -r requirements.txt && pytest
```

Caching speeds dependency installation:

```
- uses: actions/cache@v4
  with:
    path: ~/.npm
    key: npm-${ runner.os }-${ hashFiles('**/package-lock.json') }
    restore-keys: npm-${ runner.os }-
```

Artifacts pass build outputs between jobs or retain them for download:

```
- uses: actions/upload-artifact@v4
```

```
with:
  name: dist
  path: dist/
  retention-days: 7
```

Use artifacts for test reports, built binaries, or Terraform plans—not for large Docker images (push to a registry instead).

39.7 38.7 Environments, deployments, and approvals

Environments model deployment targets (staging, production) with optional rules:

- Required reviewers before job runs
- Wait timer (e.g., 5 minutes before prod deploy)
- Branch restrictions (only `main` deploys to prod)
- Environment-specific secrets

```
jobs:
  deploy-prod:
    needs: test
    runs-on: ubuntu-latest
    environment:
      name: production
      url: https://app.example.com
    steps:
      - uses: actions/github-script@v7
        with:
          script: |
            github.rest.repos.createDeployment({
              owner: context.repo.owner,
              repo: context.repo.repo,
              ref: context.sha,
              environment: 'production',
              auto_merge: false,
              required_contexts: []
            })
      - run: ./deploy.sh production
```

The **deployments API** surfaces deployment history in the GitHub UI, linking commits to running releases.

39.8 38.8 Debugging and troubleshooting

Problem	Diagnostic approach
Workflow not triggering	Check <code>on:</code> filters, branch names, path filters
Secret empty in fork PR	Expected behavior; use <code>pull_request_target</code> only with extreme care
Action version break	Pin SHA; read action changelog
Runner offline	Check self-hosted runner service, token expiry
Job queued forever	Org concurrency limits; no available runners
Expression errors	Enable debug logging: <code>ACTIONS_STEP_DEBUG: true</code>

Enable workflow run logs retention in repo settings. Use **workflow visualization** in the Actions tab to inspect job graphs.

39.9 38.9 Chapter summary

- GitHub Actions workflows are event-driven YAML pipelines composed of jobs and steps on runners.
 - Secure pipelines with scoped permissions, pinned actions, OIDC, and environment protection rules.
 - Choose hosted runners for simplicity; self-hosted for private network access and custom tooling.
 - Reusable workflows, matrix builds, caching, and artifacts scale CI across teams and tech stacks.
-

39.10 Lab 38.1 — Multi-environment deploy pipeline

1. Create `.github/workflows/deploy.yml` with jobs: `lint`, `test`, `build`, `deploy-staging`, `deploy-production`.
 2. Configure a `staging` environment with no approval and `production` with required reviewer.
 3. Add OIDC or repository secrets for your target cloud.
 4. Use a matrix to test on Node 18 and 20.
 5. Upload test coverage as an artifact; verify download after run.
 6. Trigger via `workflow_dispatch` and confirm environment gates block production until approval.
-

39.11 Lab 38.2 — Self-hosted runner in Docker

1. Register a self-hosted runner in a test organization (or use a disposable repo).
 2. Run the runner in a Docker container with label `docker-runner`.
 3. Create a workflow targeting `runs-on: [self-hosted, docker-runner]` that accesses an internal mock service on `host.docker.internal`.
 4. Document cleanup: remove runner registration and revoke token.
-

39.12 Review questions

1. What is the difference between a **job**, a **step**, and an **action**?
 2. Why should you prefer OIDC over storing AWS access keys in GitHub Secrets?
 3. When would you choose a self-hosted runner over a GitHub-hosted runner?
 4. Explain how `concurrency` groups prevent duplicate deploys.
 5. What security risk does `pull_request_target` introduce, and when is it justified?
 6. How do environment protection rules support Continuous Delivery without Continuous Deployment?
 7. Describe two ways to share pipeline logic across multiple repositories.
-

Continue: Chapter 39 — Jenkins Pipelines as Code

40 Chapter 39: Jenkins Pipelines as Code

DevOps Handbook — Part IX, Pages 771–790

40.1 39.1 Jenkins in the modern CI/CD landscape

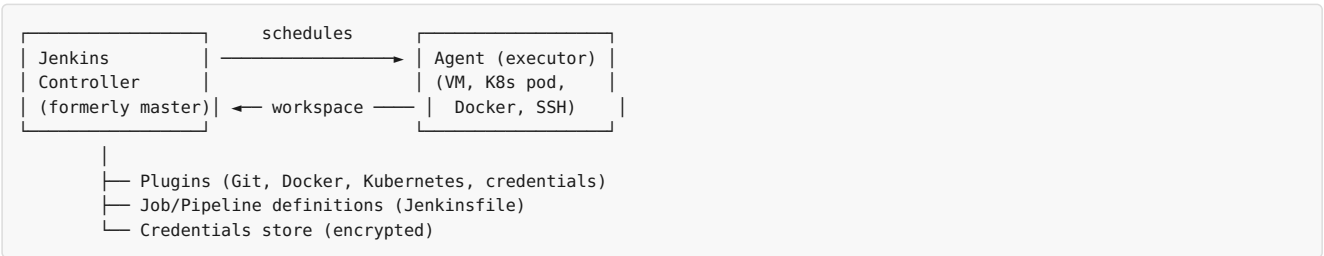
Jenkins is the longest-running open-source automation server in DevOps. While cloud-native platforms (GitHub Actions, GitLab CI) dominate greenfield projects, Jenkins remains entrenched in enterprises with thousands of plugins, existing investment, and air-gapped environments.

Modern Jenkins centers on **Pipeline as Code**: pipelines defined in `Jenkinsfile` files stored in Git, versioned alongside application code, and reviewed through pull requests—eliminating click-configured jobs that drift from documentation.

Strength	Limitation
2,000+ plugins	Plugin maintenance burden
Self-hosted control	You operate HA, upgrades, security
Flexible agents	Complex agent provisioning
Declarative & Scripted DSL	Groovy learning curve

This chapter focuses on **Declarative Pipeline** syntax—the recommended approach for readability and Jenkins validation.

40.2 39.2 Architecture: controller, agents, and plugins



- **Controller** — Orchestrates builds, serves UI, stores configuration. Never run heavy builds on the controller in production.
- **Agent** — Executes pipeline steps. Labels route jobs (`agent { label 'linux && docker' }`).
- **Plugins** — Extend functionality; pin versions and audit for CVEs.
- **Credentials** — Bound to folders or jobs; referenced by ID in pipelines.

For Kubernetes, the **Kubernetes plugin** creates ephemeral agent pods per build—isolating workloads and scaling horizontally.

40.3 39.3 Declarative Pipeline structure

A minimal `Jenkinsfile` in the repository root:

```
pipeline {
  agent any

  options {
```

```

    timestamps()
    timeout(time: 30, unit: 'MINUTES')
    buildDiscarder(logRotator(numToKeepStr: '20'))
  }

  environment {
    APP_NAME = 'order-service'
    REGISTRY = 'registry.example.com'
  }

  stages {
    stage('Checkout') {
      steps {
        checkout scm
      }
    }

    stage('Build') {
      steps {
        sh 'make build'
      }
    }

    stage('Test') {
      steps {
        sh 'make test'
      }
      post {
        always {
          junit '**/target/surefire-reports/*.xml'
        }
      }
    }

    stage('Docker Build & Push') {
      when {
        branch 'main'
      }
      steps {
        script {
          docker.withRegistry("https://${REGISTRY}", 'registry-credentials-id') {
            def image = docker.build("${APP_NAME}:${env.BUILD_NUMBER}")
            image.push('latest')
            image.push("${env.GIT_COMMIT.take(7)}")
          }
        }
      }
    }
  }

  post {
    success {
      slackSend channel: '#ci', message: "Build ${env.BUILD_URL} succeeded"
    }
    failure {
      slackSend channel: '#ci', message: "Build ${env.BUILD_URL} FAILED"
    }
  }
}

```

Declarative blocks:

Block	Role
agent	Where pipeline runs
environment	Variables and credentials binding
stages / stage	Logical pipeline phases
steps	Commands and plugin calls
post	Actions on success/failure/always
when	Conditional stage execution
options	Timeouts, concurrency, log retention

40.4 39.4 Agents and parallelism

Agent directives control execution placement:

```
pipeline {
  agent none

  stages {
    stage('Parallel Tests') {
      parallel {
        stage('Unit') {
          agent { label 'linux' }
          steps { sh 'npm run test:unit' }
        }
        stage('Integration') {
          agent { label 'linux && docker' }
          steps { sh 'npm run test:integration' }
        }
        stage('Lint') {
          agent { label 'linux' }
          steps { sh 'npm run lint' }
        }
      }
    }
  }
}
```

Docker agent runs the entire pipeline inside a container:

```
pipeline {
  agent {
    docker {
      image 'node:20-alpine'
      args '-v /var/run/docker.sock:/var/run/docker.sock'
    }
  }
  stages {
    stage('Build') {
      steps { sh 'npm ci && npm run build' }
    }
  }
}
```

Kubernetes agent pod template (requires Kubernetes plugin):

```
pipeline {
  agent {
    kubernetes {
      yaml '''
apiVersion: v1
kind: Pod
spec:
  containers:
  - name: maven
    image: maven:3.9-eclipse-temurin-17
    command: ["sleep"]
    args: ["99d"]
    resources:
      limits:
        memory: "1Gi"
        cpu: "1000m"
...
      }
    }
  }
  stages {
    stage('Maven Build') {
      steps {
        container('maven') {
          sh 'mvn -B clean verify'
        }
      }
    }
  }
}
```

```
}  
}
```

40.5 39.5 Credentials, parameters, and shared libraries

Never hardcode secrets in Jenkinsfiles. Use the **Credentials Binding** plugin:

```
stage('Deploy') {  
    steps {  
        withCredentials([  
            string(credentialsId: 'deploy-api-token', variable: 'TOKEN'),  
            usernamePassword(credentialsId: 'git-deploy', usernameVariable: 'GIT_USER', passwordVariable: 'GIT_PASS')  
        ]) {  
            sh '''  
                curl -H "Authorization: Bearer $TOKEN" \  
                    -X POST https://api.example.com/deploy  
            ...  
            }  
        }  
    }  
}
```

Parameters enable manual runs with choices:

```
parameters {  
    choice(name: 'ENV', choices: ['staging', 'production'], description: 'Target')  
    booleanParam(name: 'RUN_INTEGRATION', defaultValue: true)  
}  
  
stage('Deploy') {  
    when {  
        expression { params.ENV == 'production' }  
    }  
    steps {  
        input message: 'Deploy to production?', ok: 'Deploy'  
        sh "./deploy.sh ${params.ENV}"  
    }  
}
```

Shared Libraries centralize Groovy functions across pipelines. Define in **Manage Jenkins → Configure System → Global Pipeline Libraries**, then:

```
@Library('my-devops-lib@v2') _  
  
pipeline {  
    agent any  
    stages {  
        stage('Scan') {  
            steps {  
                devopsSecurity.scanImage('myapp:latest')  
            }  
        }  
    }  
}
```

Libraries live in a Git repo with `vars/` (global steps) and `src/` (classes).

40.6 39.6 Multibranch and organization folders

Multibranch Pipeline jobs automatically discover branches and PRs with a `Jenkinsfile`, creating child jobs per branch:

1. Connect GitHub/GitLab/Bitbucket via webhook or polling.
2. Jenkins scans repository for branches containing `Jenkinsfile`.
3. Each branch gets an isolated pipeline history.

Organization Folder scans an entire GitHub organization—ideal for microservice estates with one repo per service.

`Jenkinsfile` location can be customized (`Script Path`) for monorepos: `services/payments/Jenkinsfile` .

40.7 39.7 Pipeline durability and best practices

Practice	Rationale
Declarative over Scripted	Easier review, built-in validation
Keep controller lightweight	Agents run builds
Pin plugin versions	Reproducible builds, fewer surprises
Use <code>options { disableConcurrentBuilds() }</code> on deploy pipelines	Prevent overlapping deploys
Archive artifacts selectively	Disk fills quickly on busy controllers
Blue Ocean or Stage View	Visualize parallel stages
Backup <code>\$JEN_HOME</code>	Jobs, credentials, config
RBAC via Matrix Authorization	Least privilege for Jenkins UI

Scripted Pipeline (full Groovy) remains for edge cases but should be wrapped in `script { }` blocks inside Declarative when needed.

40.8 39.8 Jenkins vs cloud-native CI (when to stay)

Stay on Jenkins when:

- Regulatory requirements mandate on-premises CI
- Heavy customization via niche plugins is required
- Existing pipeline investment exceeds migration cost

Migrate when:

- Plugin debt and controller HA consume disproportionate ops time
- Teams want CI tightly integrated with Git hosting
- Ephemeral cloud runners reduce agent maintenance

Hybrid patterns exist: Jenkins orchestrates legacy jobs while new services use GitHub Actions, unified by deployment tooling (Argo CD, Spinnaker).

40.9 39.9 Chapter summary

- Jenkins Pipeline as Code (`Jenkinsfile`) brings version control and review to CI/CD definitions.
- Declarative Pipeline provides structured stages, agents, credentials, and post-build actions.
- Agents on VMs, Docker, or Kubernetes execute work; the controller orchestrates only.
- Multibranch pipelines and shared libraries scale Jenkins across many services and teams.

40.10 Lab 39.1 — Declarative pipeline from scratch

1. Install Jenkins (Docker or local) with Git, Pipeline, Docker Pipeline, and JUnit plugins.
 2. Create a Multibranch Pipeline job connected to a sample repo.
 3. Add a `Jenkinsfile` with checkout, build, test, and Docker publish stages.
 4. Store registry credentials in Jenkins Credentials; reference via `withCredentials`.
 5. Add a `when { branch 'main' }` gate on publish; verify feature branches skip deploy.
 6. Configure Slack or email notification in `post { failure }`.
-

40.11 Lab 39.2 — Kubernetes ephemeral agents

1. Deploy Jenkins with Kubernetes plugin on a test cluster.
 2. Define a pod template with `jnlp` and `maven` containers.
 3. Run a pipeline that builds a Java app inside the `maven` container.
 4. Confirm agent pod is deleted after build completes.
-

40.12 Review questions

1. What problem does Pipeline as Code solve compared to UI-configured jobs?
 2. Why should production builds not run on the Jenkins controller?
 3. Explain the difference between Declarative and Scripted Pipeline.
 4. How do Multibranch Pipelines discover which branches to build?
 5. What is a Jenkins Shared Library, and when should you use one?
 6. Describe two ways to inject secrets safely into a pipeline step.
 7. What are the main operational costs of running Jenkins at enterprise scale?
-

Continue: Chapter 40 — GitLab CI and Multi-platform Comparison

41 Chapter 40: GitLab CI and Multi-platform Comparison

DevOps Handbook — Part IX, Pages 791–810

41.1 40.1 GitLab CI/CD overview

GitLab CI/CD is built into GitLab—same platform for source control, merge requests, registry, and pipelines. Configuration lives in `.gitlab-ci.yml` at the repository root (or custom path). GitLab Runner executes jobs; GitLab Server (gitlab.com or self-managed) schedules pipelines.

Architecture:

```
.gitlab-ci.yml → GitLab CI/CD → Runner(s) → Job containers/shell
|
├─ Variables & CI/CD settings
├─ Environments & deployments
└─ Container Registry / Package Registry
```

GitLab emphasizes **DevOps platform** consolidation: one vendor for SCM, CI, security scanning (Ultimate tier), and deployment tracking.

41.2 40.2 `.gitlab-ci.yml` fundamentals

```
stages:
  - build
  - test
  - security
  - deploy

default:
  image: node:20-alpine
  cache:
    key: ${CI_COMMIT_REF_SLUG}
    paths:
      - node_modules/

variables:
  NODE_ENV: test

build:
  stage: build
  script:
    - npm ci
    - npm run build
  artifacts:
    paths:
      - dist/
    expire_in: 1 week

unit-test:
  stage: test
  script:
    - npm ci
    - npm run test:unit
  coverage: '/Lines\s*:\s*(\d+\.\d+)%/'

integration-test:
  stage: test
  services:
    - name: postgres:15
      alias: db
  variables:
```

```

POSTGRES_DB: testdb
POSTGRES_USER: test
POSTGRES_PASSWORD: test
DATABASE_URL: postgres://test:test@db:5432/testdb
script:
  - npm run test:integration

deploy-staging:
  stage: deploy
  environment:
    name: staging
    url: https://staging.example.com
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
  script:
    - ./deploy.sh staging
  needs:
    - build
    - unit-test

```

Core concepts:

Concept	Description
Stage	Ordered phase; jobs in same stage run in parallel
Job	Unit of work with <code>script</code>
Runner	Executes jobs (shared, group, project-specific)
Artifact	Files passed between jobs
Cache	Speed up dependency installs across pipelines
Service	Sidecar containers (DB, Redis) for tests
Rules	Modern conditional syntax replacing <code>only/except</code>

41.3 40.3 Runners, executors, and isolation

GitLab Runner is a separate agent process registering to GitLab.

Executor	Use case
Docker	Most common; job runs in container
Kubernetes	Ephemeral pods per job
Shell	Direct on VM; least isolation
SSH	Remote machine execution
Instance	Auto-scaled cloud VMs (Premium+)

Register a runner:

```

gitlab-runner register \
  --url https://gitlab.com/ \
  --registration-token PROJECT_TOKEN \
  --executor docker \
  --docker-image alpine:latest \
  --description "docker-runner-01" \
  --tag-list "docker,linux"

```

Use **tags** in jobs: `tags: [docker, linux]`. **Protected runners** run only on protected branches—critical for production deploy jobs accessing sensitive credentials.

41.4 40.4 Variables, environments, and child pipelines

CI/CD variables scope: project, group, environment. Mark sensitive values as **masked** and **protected**.

```
deploy-production:
  stage: deploy
  environment:
    name: production
    url: https://app.example.com
    deployment_tier: production
  rules:
    - if: $CI_COMMIT_TAG =~ /^v\d+\.\d+\.\d+$/
  script:
    - echo "Deploying $CI_COMMIT_TAG"
  variables:
    KUBE_CONTEXT: prod-cluster
```

Manual jobs (when: manual) implement approval gates. **Resource groups** serialize deploys to the same environment.

Child pipelines split monorepo CI:

```
trigger-backend:
  trigger:
    include: backend/.gitlab-ci.yml
    strategy: depend
  rules:
    - changes:
      - backend/**/*
```

Merge request pipelines run on MR events; **merged results pipelines** test the post-merge state—reducing “green MR, red main” surprises.

41.5 40.5 GitLab-specific features

Feature	Benefit
Auto DevOps	Opinionated default pipeline for build/test/deploy
Review Apps	Dynamic environments per MR
Container Registry	Built-in image storage tied to project
Dependency Proxy	Cache upstream Docker images
Compliance pipelines	Enforce stages across org (Ultimate)
DAST/SAST templates	Security jobs from template catalog

Review App example:

```
review:
  stage: deploy
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    url: https://$CI_ENVIRONMENT_SLUG.example.com
    on_stop: stop_review
  script:
    - deploy-review.sh
  rules:
    - if: $CI_MERGE_REQUEST_IID

stop_review:
  stage: deploy
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    action: stop
  when: manual
```

```
script:
  - teardown-review.sh
```

41.6 40.6 Multi-platform CI/CD comparison

Dimension	GitHub Actions	GitLab CI	Jenkins
Config file	<code>.github/workflows/*.yml</code>	<code>.gitlab-ci.yml</code>	<code>Jenkinsfile</code>
Hosting	GitHub cloud/Enterprise	GitLab.com/self-managed	Self-hosted (typical)
Runners	Hosted + self-hosted	Runner required (shared or own)	Agents
Marketplace	Actions marketplace	CI templates, components	Plugins
MR/PR integration	Native checks	Native pipelines	Plugin-dependent
OIDC to cloud	Excellent	Supported	Plugin-dependent
Learning curve	Moderate YAML	Moderate YAML	Groovy + ops
Cost	Minutes-based	Per-user + compute	Infra + maintenance

Azure Pipelines (YAML, Microsoft-hosted agents, Azure DevOps integration) fits Microsoft-centric orgs. **CircleCI**, **Buildkite**, and **Tekton** (Kubernetes-native) appear in cloud-native and high-scale scenarios.

Decision framework:

1. **Where is code hosted?** Align CI with SCM when possible.
2. **Compliance** — Data residency, air-gap, audit requirements.
3. **Runner strategy** — Shared vs dedicated, spot instances, K8s.
4. **Total cost** — License + compute + engineer maintenance.
5. **Ecosystem** — Required integrations (Jira, SonarQube, Vault).

41.7 40.7 Portable pipeline patterns

Regardless of platform, mature pipelines share structure:

```
lint → unit test → build artifact → security scan → integration test → deploy staging → deploy prod
```

Portable techniques:

- **Containerize builds** — Same Dockerfile locally and in CI
- **Make/task runners** — `make test` invoked identically everywhere
- **Composite/reusable units** — GitHub reusable workflows, GitLab `include:`, Jenkins shared libraries
- **Policy gates** — Fail on critical CVEs, coverage thresholds
- **Immutable artifacts** — Promote the same image digest staging → prod

Example GitLab `include` for shared org template:

```
include:
  - project: 'platform/ci-templates'
    ref: main
    file: '/nodejs/build-test-deploy.yml'
```


41.8 40.8 Migration considerations

Migrating Jenkins → GitLab or GitHub:

1. Inventory jobs, plugins, credentials, and agent labels.
2. Rewrite pipelines incrementally (strangler pattern)—one service at a time.
3. Map credentials to new secret stores; rotate during migration.
4. Parallel-run old and new pipelines until confidence is high.
5. Decommission Jenkins agents only after traffic cutover.

Tooling like **GitHub Actions Importer** and community converters assist but require manual validation for complex Groovy logic.

41.9 40.9 Chapter summary

- GitLab CI/CD uses `.gitlab-ci.yml` with stages, jobs, runners, artifacts, and environments in one platform.
 - Runners with Docker or Kubernetes executors provide flexible, isolated build environments.
 - Compare CI platforms on hosting, runner model, integration depth, security, and operational cost—not syntax alone.
 - Portable pipeline design (containers, shared templates, immutable artifacts) reduces vendor lock-in.
-

41.10 Lab 40.1 — GitLab pipeline with services

1. Create a GitLab project (gitlab.com free tier or self-managed).
 2. Add `.gitlab-ci.yml` with Postgres `services` for integration tests.
 3. Configure CI/CD variables for a mock deploy token (masked).
 4. Add `deploy-staging` with `environment` and `rules` for `main` only.
 5. Create an MR and verify MR pipeline runs; merge and confirm staging deploy.
-

41.11 Lab 40.2 — Platform comparison exercise

1. Implement the same three-stage pipeline (lint, test, docker build) in GitHub Actions and GitLab CI for one sample repo.
 2. Document: lines of config, runner setup time, secret handling, and MR/PR integration UX.
 3. Present a one-page recommendation for a hypothetical 50-developer org.
-

41.12 Review questions

1. What is the role of GitLab Runner, and how does it differ from GitLab Server?
 2. Explain `rules` vs deprecated `only/except` in GitLab CI.
 3. How do **Review Apps** improve merge request workflows?
 4. Name three criteria for choosing between GitHub Actions and GitLab CI.
 5. What is a child pipeline, and when is it useful in monorepos?
 6. Why should production deploy runners be **protected**?
 7. What pipeline patterns remain portable across CI platforms?
-

42 Chapter 41: Deployment Strategies — Blue/Green, Canary, Feature Flags

DevOps Handbook — Part IX, Pages 811–825

42.1 41.1 Why deployment strategy matters

Releasing software is not binary—“deploy or don’t deploy.” How you roll out changes determines **downtime**, **blast radius**, **rollback speed**, and **confidence**. A pipeline that builds and tests flawlessly still causes outages if production traffic shifts incorrectly.

Deployment strategies sit at the intersection of **CI/CD**, **infrastructure**, and **application design**. This chapter covers rolling, recreate, blue/green, canary, and feature flags—with Kubernetes and load balancer examples.

Strategy	Downtime	Rollback speed	Infra cost	Complexity
Recreate	Yes	Slow (redeploy)	Low	Low
Rolling	Minimal	Moderate	Low	Low
Blue/green	Near zero	Fast (switch)	2× during cutover	Medium
Canary	None	Fast (traffic shift)	Partial duplicate	High
Feature flags	None	Instant (toggle)	Low	App changes

42.2 41.2 Rolling deployment

Rolling replaces instances incrementally: old and new versions run simultaneously until all instances update.

Kubernetes default `Deployment` strategy:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
spec:
  replicas: 5
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    spec:
      containers:
        - name: api
          image: registry.example.com/api:v2.3.0
          readinessProbe:
            httpGet:
              path: /health
              port: 8080
```

Parameter	Effect
maxSurge	Extra pods above desired count during rollout
maxUnavailable	Pods that can be down during rollout

Pros: No duplicate full environment; built into K8s. **Cons:** Mixed versions during rollout—API compatibility

required; slow rollback if new image already propagated.

42.3 41.3 Recreate deployment

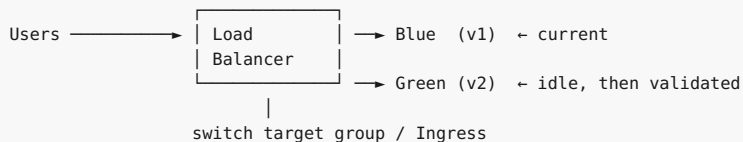
Recreate terminates all old instances before starting new ones—simple but causes downtime.

```
strategy:
  type: Recreate
```

Use for: stateful batch jobs, maintenance windows, or when only one version can run (license constraints). Avoid for user-facing services with SLA requirements.

42.4 41.4 Blue/green deployment

Blue/green maintains two identical environments—**blue** (current production) and **green** (new version). After validating green, traffic switches atomically.



AWS ALB example (conceptual):

1. Green ASG or target group runs v2 behind same ALB with zero weight.
2. Run smoke tests against green via internal DNS or test header.
3. Swap listener rules: green 100%, blue 0%.
4. Keep blue warm for quick rollback.

Kubernetes with two Deployments and Service selector swap—or use **Argo Rollouts** blue/green:

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: api-rollout
spec:
  replicas: 5
  strategy:
    blueGreen:
      activeService: api-active
      previewService: api-preview
      autoPromotionEnabled: false
  selector:
    matchLabels:
      app: api
  template:
    metadata:
      labels:
        app: api
    spec:
      containers:
        - name: api
          image: registry.example.com/api:v2.3.0
```

Pros: Instant rollback (switch back to blue); clear validation window on preview URL. **Cons:** Double resource cost during transition; database schema migrations need careful handling (expand/contract pattern).

42.5 41.5 Canary deployment

Canary routes a small percentage of traffic to the new version, monitors metrics, then progressively increases traffic.

```
Traffic: 95% → stable (v1)
         5% → canary (v2) → 50/50 → 100% v2
```

Istio VirtualService example:

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: api
spec:
  hosts:
  - api.example.com
  http:
  - route:
    - destination:
        host: api-stable
        weight: 95
    - destination:
        host: api-canary
        weight: 5
```

Flagger (Kubernetes) automates canary analysis using Prometheus metrics:

```
apiVersion: flagger.app/v1beta1
kind: Canary
metadata:
  name: api
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api
  service:
    port: 8080
  analysis:
    interval: 1m
    threshold: 5
    maxWeight: 50
    stepWeight: 10
    metrics:
    - name: request-success-rate
      thresholdRange:
        min: 99
      interval: 1m
    - name: request-duration
      thresholdRange:
        max: 500
      interval: 1m
```

Canary success criteria typically include: error rate, latency p99, business KPIs (conversion rate). Automated rollback triggers when thresholds breach.

Pros: Smallest blast radius; data-driven promotion. **Cons:** Requires observability, traffic management, and often service mesh or advanced ingress.

42.6 41.6 Feature flags (dark launches)

Feature flags decouple **deployment** (code in production) from **release** (feature visible to users). Code ships disabled; operators enable for internal testers, then percentage of users, then everyone.

```
from flagsmith import Flagsmith
```

```

flagsmith = Flagsmith(environment_key=os.environ["FLAGSMITH_KEY"])

def checkout_handler(request):
    flags = flagsmith.get_identity_flags(str(request.user.id))
    if flags.is_feature_enabled("new_checkout_flow"):
        return new_checkout(request)
    return legacy_checkout(request)

```

Flag management platforms: **LaunchDarkly**, **Flagsmith**, **Unleash**, **Split.io**, or open-source **GO Feature Flag**.

Flag type	Use case
Release	Gradual rollout of new UI/API
Ops/kill switch	Disable expensive endpoint under load
Experiment	A/B test variants
Permission	Entitlements, beta access

Best practices:

- Flags are **temporary**—remove after full rollout (flag debt causes untestable combinatorial states).
- Default to **off** in production; explicit enablement.
- Audit flag changes; tie to incident runbooks for kill switches.
- Test both flag states in CI where feasible.

Feature flags complement canary: canary validates infrastructure and baseline metrics; flags control user-visible behavior within the same binary.

42.7 41.7 Database migrations and deployment coupling

Schema changes break naive blue/green:

Phase	Pattern
Expand	Add new column/table; old code ignores it
Migrate data	Backfill asynchronously
Contract	Remove old column after all code uses new

Never deploy code that requires a schema change **before** the schema is backward-compatible. Tools: **Flyway**, **Liquibase**, **Alembic**—run migrations as pipeline stage with rollback plans.

42.8 41.8 Choosing a strategy

Decision tree:

1. **Zero downtime required?** → Eliminate recreate.
2. **Strong observability + traffic control?** → Canary.
3. **Need instant rollback without redeploy?** → Blue/green or feature flags.
4. **Simple stateless app, K8s?** → Rolling often sufficient.
5. **Hide incomplete features on main?** → Feature flags + trunk-based development.

Many organizations combine: **rolling** for frequent small patches, **canary** for high-risk services, **feature flags** for product experimentation.

42.9 41.9 Chapter summary

- Deployment strategy determines outage risk and rollback speed—not just pipeline success.
 - Rolling and recreate suit simpler cases; blue/green enables instant traffic reversal.
 - Canary reduces blast radius when metrics-driven automation validates new versions.
 - Feature flags separate code deployment from feature release; manage flag lifecycle diligently.
 - Coordinate database migrations with expand/contract to support parallel versions.
-

42.10 Lab 41.1 — Kubernetes rolling vs recreate

1. Deploy a sample app with 3 replicas and `RollingUpdate (maxUnavailable: 0)`.
 2. Update image tag; watch `kubectl rollout status` and observe zero downtime with readiness probes.
 3. Change strategy to `Recreate`; repeat update and measure downtime with `curl` in a loop.
-

42.11 Lab 41.2 — Feature flag rollout

1. Deploy open-source Unleash or Flagsmith locally.
 2. Wrap a sample endpoint with a feature flag defaulting to off.
 3. Enable for 10% of user IDs; verify split behavior.
 4. Document flag removal checklist for post-rollout cleanup.
-

42.12 Review questions

1. Compare rolling and blue/green in terms of rollback speed and resource cost.
 2. What metrics would you use to automate canary promotion?
 3. Why must feature flags be removed after full rollout?
 4. Explain the expand/contract pattern for database migrations.
 5. When is recreate deployment acceptable?
 6. How do feature flags differ from canary deployments?
 7. What role does a service mesh play in canary releases?
-

Continue: Chapter 42 — DORA Metrics and Continuous Improvement

43 Chapter 42: DORA Metrics and Continuous Improvement

DevOps Handbook — Part IX, Pages 826–840

43.1 42.1 What DORA measures

The **DevOps Research and Assessment (DORA)** program, led by Nicole Forsgren, Jez Humble, and Gene Kim, identified four metrics that distinguish high-performing software delivery organizations from low performers. Published in *Accelerate* (2018) and updated in annual **State of DevOps** reports, these metrics provide a **evidence-based** framework—not vanity metrics like lines of code or story points.

The four **DORA metrics**:

Metric	Definition	Elite benchmark (indicative)
Deployment frequency	How often code deploys to production	Multiple per day
Lead time for changes	Commit to production duration	Less than one hour
Change failure rate	% of changes causing failure in prod	0–15%
Time to restore service (MTTR)	Recovery time after incident	Less than one hour

A fifth metric—**reliability** (availability, latency SLOs)—was added in later research as a companion to delivery speed: speed without stability is unsustainable.

43.2 42.2 Why these four metrics matter

Traditional KPIs often optimize local maxima: development velocity without operational stability, or uptime achieved by never deploying. DORA metrics capture **both** speed and stability—the CALMS balance from Part I.

High performers demonstrate:

- Small, frequent changes (lower risk per change)
- Fast feedback when something breaks
- Culture of learning, not blame

Low performers often show:

- Infrequent, large releases (“release trains” without automation)
- Long manual testing phases
- Heroics during outages instead of systematic recovery

Important: Metrics drive behavior. Gaming DORA (deploying trivial config changes to inflate frequency) destroys trust. Use metrics for **improvement**, not individual performance review.

43.3 42.3 Measuring deployment frequency

Deployment frequency counts production deployments over time—typically per day or per week per service or team.

What counts as a deployment:

Counts	Does not count
New app version to prod	Deploy to staging only
Config/IaC change affecting prod	Failed deploy that never served traffic
Hotfix rollout	Restart without version change (debatable—document policy)

Instrumentation:

```
# GitHub Actions – emit deployment event
- name: Record deployment
  if: success()
  run: |
    curl -X POST "${METRICS_API}/deployments" \
      -H "Authorization: Bearer ${ secrets.METRICS_TOKEN }" \
      -d '{
        "service": "order-api",
        "environment": "production",
        "commit_sha": "${ github.sha }",
        "timestamp": "$(date -u +%Y-%m-%dT%H:%M:%SZ)"
      }'
```

Sources: CI/CD deployment logs, GitHub Deployments API, GitLab environments, Argo CD sync events, Spinnaker pipeline history.

Aggregate by **team** or **service**—not entire company—so teams see actionable data.

43.4 42.4 Measuring lead time for changes

Lead time for changes is the elapsed time from **code committed** to **running successfully in production**.

Lead Time = prod_deploy_timestamp - first_commit_timestamp

For a feature spanning multiple commits, use the merge commit to `main` or the tag triggering release—**define consistently**.

Break down lead time into segments (value stream map):

Segment	Typical bottlenecks
Code review wait	Large PRs, reviewer availability
CI queue + duration	Slow tests, insufficient runners
Approval gates	Manual change advisory boards
Deploy execution	Manual steps, change windows

Example calculation from Git + CI:

```
# Pseudocode: compute lead time for a deployment
deploy = get_production_deployment(service="api", sha="abc123")
commit_time = git.commit(deploy.sha).committed_at
lead_time_hours = (deploy.finished_at - commit_time).total_seconds() / 3600
record_metric("lead_time_hours", lead_time_hours, labels={"service": "api"})
```

Reducing lead time usually means: trunk-based development, automated tests, parallel CI, automated promotion, and eliminating manual handoffs.

43.5 42.5 Change failure rate

Change failure rate = (deployments causing prod failure) / (total deployments) in a period.

Define **failure** explicitly:

- Rollback required
- Hotfix within 24 hours of deploy
- SEV-1/SEV-2 incident linked to a change
- SLO breach attributed to deployment (via incident tagging)

```
-- Example: weekly change failure rate per service
SELECT
  service,
  COUNT(*) FILTER (WHERE caused_incident) * 100.0 / COUNT(*) AS failure_rate_pct
FROM deployments
WHERE environment = 'production'
  AND deployed_at >= NOW() - INTERVAL '7 days'
GROUP BY service;
```

Link deployments to incidents in your incident tool (PagerDuty, Jira, incident.io) via **deployment ID** or **commit SHA** in incident metadata.

Improve change failure rate with: automated testing, canary analysis, feature flags, and post-deploy smoke tests —not by deploying less frequently (that hides problems in bigger batches).

43.6 42.6 Time to restore service (MTTR)

MTTR (Mean Time to Restore) measures how quickly service returns to normal after an incident—distinct from **MTTD** (detect) and **MTTR** repair vs **MTBF** (between failures).

```
MTTR = incident_resolved_at - incident_started_at
```

Use **customer-impacting** start time (SLO breach or alert), not internal discovery time, for user-centric measurement.

Improve MTTR with:

- Runbooks and automated remediation
- Observability (Chapter 43–47)
- Blameless postmortems with action items
- Chaos engineering validating recovery paths
- On-call rotation and escalation policies

Track MTTR separately for **deployment-related** vs **infrastructure** incidents to target improvements.

43.7 42.7 Performance clusters and benchmarking

DORA research clusters organizations into **elite**, **high**, **medium**, **low** performers based on metric combinations —not single metrics in isolation.

Cluster	Typical profile
Elite	On-demand deploys, sub-hour lead time, low CFR, fast restore
High	Daily-weekly deploys, day-scale lead time
Medium	Monthly deploys, weeks lead time

Low	Quarterly+ deploys, months lead time
-----	--------------------------------------

Compare teams to **past self**, not blindly to FAANG elite—context (regulated industry, legacy monolith) matters. Use benchmarks to set **directional goals**.

43.8 42.8 Building a metrics program

Implementation roadmap:

1. **Define** — Document what counts as deploy, failure, production for each service.
2. **Instrument** — CI/CD hooks, deployment APIs, incident linkage.
3. **Visualize** — Dashboards (Grafana, Datadog, custom) per team.
4. **Review** — Weekly team reviews; monthly cross-team patterns.
5. **Improve** — Pick one bottleneck from lead time breakdown per quarter.
6. **Protect culture** — No individual DORA targets; blameless analysis.

Sample dashboard panels:

- Deploy frequency (7-day rolling)
- Lead time p50 / p95
- Change failure rate (30-day)
- MTTR (30-day)
- Deployments vs incidents overlay chart

43.9 42.9 Continuous improvement loops

DORA metrics feed **Plan-Do-Check-Act (PDCA)** and **Three Ways** (Part I):

Three Way	DORA connection
Flow	Lead time, deployment frequency
Feedback	Change failure rate, MTTR
Continual learning	Postmortems, experiment with pipeline changes

Combine with **SPACE** framework (Developer productivity—Satisfaction, Performance, Activity, Communication, Efficiency) to avoid over-optimizing deploy count at developer experience expense.

Space for improvement without DORA data: value stream mapping workshops, identifying wait states, automating top three manual steps.

43.10 42.10 Chapter summary

- DORA’s four metrics—deployment frequency, lead time, change failure rate, MTTR—measure delivery speed and stability together.
- Define measurement policies consistently; instrument CI/CD and incident systems.
- Improve metrics through systemic changes (automation, testing, observability)—not by gaming numbers.
- Benchmark against past performance; use metrics in blameless continuous improvement cycles.

43.11 Lab 42.1 — Deployment event pipeline

1. Add a post-deploy step to your CI pipeline that POSTs deployment metadata to a spreadsheet, database, or metrics endpoint.
 2. Tag each deployment with service name, environment, commit SHA, and duration.
 3. Build a simple dashboard (Grafana or spreadsheet chart) showing deployment frequency over 2 weeks of test data.
-

43.12 Lab 42.2 — Lead time value stream map

1. Pick one recent production release.
 2. Record timestamps: first commit, PR opened, PR merged, CI complete, staging deploy, prod deploy.
 3. Calculate segment durations; identify the largest wait state.
 4. Propose one automation or process change targeting that bottleneck.
-

43.13 Review questions

1. Name the four core DORA metrics and what each measures.
 2. Why should DORA metrics not be used for individual performance evaluation?
 3. How do you define whether a deployment “failed” for change failure rate?
 4. What is the difference between lead time for changes and CI pipeline duration?
 5. How does MTTR relate to observability and incident response practices?
 6. What lead time segments would you investigate first in a team with 5-day lead time?
 7. How do elite performers differ from low performers in deployment frequency patterns?
-

Continue: Chapter 43 — Monitoring Fundamentals

44 Chapter 43: Monitoring Fundamentals — Metrics, Logs, Traces

DevOps Handbook — Part X, Pages 841–865

44.1 43.1 From monitoring to observability

Monitoring asks: “Is the system up? Are known thresholds breached?” Traditional monitoring relies on predefined dashboards and alerts—CPU > 80%, error rate > 1%.

Observability asks: “Why is the system behaving this way?” A system is observable when you can infer internal state from **external outputs**—especially for novel failures nobody anticipated.

The three pillars of observability:

Pillar	Question it answers	Typical tools
Metrics	How much? How fast? Trend over time?	Prometheus, Datadog, CloudWatch
Logs	What happened? Context and narrative?	ELK, Loki, CloudWatch Logs
Traces	Where did time go? Cross-service path?	Jaeger, Tempo, Zipkin, X-Ray

Metrics are cheap at scale; logs are verbose; traces connect distributed requests. Mature platforms **correlate** all three—click from alert → logs → trace span.

44.2 43.2 Metrics: types and best practices

Metrics are numeric measurements aggregated over time.

Type	Description	Example
Counter	Monotonically increasing	http_requests_total
Gauge	Value that goes up/down	memory_usage_bytes , queue depth
Histogram	Distribution in buckets	Request latency buckets
Summary	Quantiles (client-side)	http_request_duration_seconds{quantile="0.99"}

Prometheus exposition format:

```
# HELP http_requests_total Total HTTP requests
# TYPE http_requests_total counter
http_requests_total{method="GET",status="200",handler="/api/orders"} 15432
http_requests_total{method="POST",status="500",handler="/api/orders"} 12
```

RED method for services:

- **Rate** — Requests per second
- **Errors** — Failed requests per second
- **Duration** — Latency distribution

USE method for resources:

- **Utilization** — % busy
- **Saturation** — Queue length / wait

- **Errors** — Error count

Naming conventions: include unit suffix (`_seconds` , `_bytes` , `_total` for counters). Avoid high-cardinality labels (user IDs, unbounded paths)—they explode storage and query cost.

```
from prometheus_client import Counter, Histogram, start_http_server

REQUEST_COUNT = Counter(
    "http_requests_total", "Total requests",
    ["method", "endpoint", "status"]
)
REQUEST_LATENCY = Histogram(
    "http_request_duration_seconds", "Latency",
    ["method", "endpoint"],
    buckets=[0.01, 0.05, 0.1, 0.5, 1.0, 2.5, 5.0]
)

def handle_request(method, endpoint, handler):
    with REQUEST_LATENCY.labels(method, endpoint).time():
        status = handler()
        REQUEST_COUNT.labels(method, endpoint, str(status)).inc()
```

44.3 43.3 Logs: structure and aggregation

Logs are discrete events with timestamps—historically unstructured text, now increasingly **structured** (JSON).

Unstructured (hard to query):

```
2026-09-01T14:32:01Z ERROR payment failed user=12345 order=987 reason=timeout
```

Structured (machine-parseable):

```
{
  "timestamp": "2026-09-01T14:32:01.123Z",
  "level": "ERROR",
  "service": "payment-api",
  "trace_id": "4bf92f3577b329da7c93630a8a323129",
  "span_id": "00f067aa0ba902b7",
  "message": "payment failed",
  "user_id": "12345",
  "order_id": "987",
  "error": "gateway timeout"
}
```

Best practices:

Practice	Rationale
Structured JSON	Filter/aggregate in log platforms
Consistent field names	Cross-service queries
Include trace/span IDs	Jump to distributed trace
Log levels (DEBUG–ERROR)	Control volume in prod
Never log secrets/PII	Compliance and security
Centralize	SSHing to grep individual servers does not scale

Log levels guide:

- **DEBUG** — Development troubleshooting (usually off in prod)
- **INFO** — Normal lifecycle events (startup, deploy, key business events)
- **WARN** — Recoverable anomalies
- **ERROR** — Failures requiring attention

44.4 43.4 Traces: distributed request flow

In microservices, a single user request traverses many services. **Distributed tracing** assigns a **trace ID** to the request; each service creates **spans** (timed operations) linked by parent-child relationships.

```
Trace ID: abc123
├─ Span: API Gateway (50ms)
│   └─ Span: Auth Service (10ms)
│       └─ Span: Order Service (35ms)
│           └─ Span: Postgres query (5ms)
│               └─ Span: Inventory RPC (20ms)
```

OpenTelemetry span attributes:

```
from opentelemetry import trace

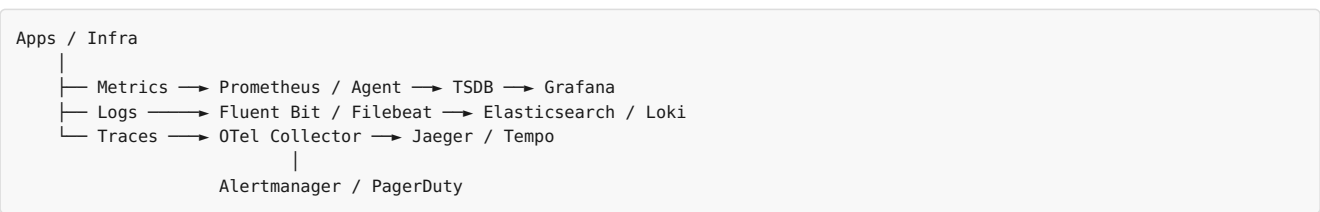
tracer = trace.get_tracer(__name__)

with tracer.start_as_current_span("process_order") as span:
    span.set_attribute("order.id", order_id)
    span.set_attribute("customer.tier", "premium")
    result = charge_payment(order_id)
    span.add_event("payment_completed", {"amount": result.amount})
```

Traces answer: Which downstream call caused latency? Did error originate in service A or B? What was the critical path?

Sample **tail-based sampling**: keep all error traces, sample 1% of success traces—balancing cost and debuggability.

44.5 43.5 The observability data pipeline



OpenTelemetry (OTel) is the vendor-neutral standard for instrumenting metrics, logs, and traces with one SDK and exporting to many backends.

Design principles:

- 1. **Instrument at boundaries** — HTTP handlers, RPC, DB calls, queue publish/consume.
- 2. **SLO-driven** — Alert on user-impacting symptoms, not every metric (Chapter 47).
- 3. **Cardinality control** — Label discipline prevents metric explosion.
- 4. **Retention tiers** — Hot (days), warm (weeks), cold (compliance archive).

44.6 43.6 Alerting philosophy

Alerts should be **actionable**, **urgent**, and **symptom-based**.

Bad alert	Good alert
CPU > 70% on one pod	Error budget burn rate > 2x for 15m
Disk 90% full	Disk will fill in 4h at current rate

DISK 60% FULL	DISK WILL FULL IN 4h at current rate
Any ERROR log	Payment success rate < 99.5% for 5m

Alert routing:

- **Page** (wake someone) — Customer-impacting SLO breach
- **Ticket** — Degraded but not urgent
- **Dashboard** — Informational trend

Reduce **alert fatigue**—on-call engineers ignoring alerts is worse than no alerts.

44.7 43.7 Golden signals and SLIs

Google SRE **four golden signals**:

1. **Latency** — Time to serve request (distinguish success vs error latency)
2. **Traffic** — Demand on system
3. **Errors** — Rate of failed requests
4. **Saturation** — How “full” the system is

Map golden signals to **Service Level Indicators (SLIs)**—quantifiable measure of service aspect (availability, latency, throughput). Chapter 47 expands SLOs and error budgets.

Example SLI:

```
Availability SLI = successful_requests / total_valid_requests
Latency SLI      = proportion of requests < 300ms
```

44.8 43.8 Instrumentation ownership

Role	Responsibility
Application team	Business metrics, RED metrics, trace propagation
Platform team	Collectors, TSDB, dashboards, alert routing
SRE	SLO definition, on-call, incident process

“You build it, you run it” includes **you instrument it**. Platform provides plumbing; teams own service-level observability.

44.9 43.9 Chapter summary

- Observability combines metrics (aggregates), logs (events), and traces (request paths) to explain system behavior.
- Use RED/USE methods, structured logging, and trace propagation with correlation IDs.
- Control cardinality, retention, and alert quality to keep systems operable at scale.
- OpenTelemetry unifies instrumentation; backend choice can evolve without re-instrumenting apps.

44.10 Lab 43.1 — RED metrics for a sample API

1. Add Prometheus client to a sample HTTP service exposing `/metrics`.

2. Instrument request count, error count, and latency histogram per route.
 3. Scrape with Prometheus; graph rate and p99 latency in Grafana.
 4. Generate load with `hey` or `k6`; observe metrics change.
-

44.11 Lab 43.2 — Correlate logs and traces

1. Instrument a two-service demo with OpenTelemetry.
 2. Emit JSON logs including `trace_id`.
 3. Trigger a request; find log line in Loki/Elasticsearch and open linked trace in Jaeger.
-

44.12 Review questions

1. Distinguish monitoring from observability with an example novel failure.
 2. What are the four metric types in Prometheus, and when use each?
 3. Explain RED and USE methods and their target systems.
 4. Why is structured logging preferred over plain text in production?
 5. What is a trace span, and how do spans form a trace tree?
 6. Why is high cardinality dangerous in metric labels?
 7. What makes an alert “actionable” vs noise?
-

Continue: Chapter 44 — Prometheus & Grafana Stack

45 Chapter 44: Prometheus & Grafana Stack

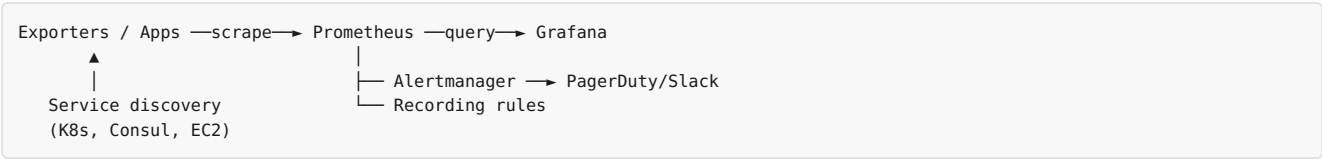
DevOps Handbook — Part X, Pages 866–890

45.1 44.1 The Prometheus ecosystem

Prometheus is an open-source monitoring system and time-series database (TSDB) designed for cloud-native environments. Created at SoundCloud, now a CNCF graduated project, it powers metrics collection for Kubernetes and microservices worldwide.

Grafana visualizes metrics from Prometheus (and 100+ other sources), providing dashboards, alerting UI, and exploration.

Typical stack:



Prometheus uses a **pull model**: it scrapes HTTP `/metrics` endpoints on intervals. Pushgateway exists for short-lived batch jobs but is not the default pattern.

45.2 44.2 Prometheus architecture

Component	Role
Prometheus server	Scrapes, stores TSDB, evaluates rules
Exporters	Expose third-party metrics (node_exporter, mysqld_exporter)
Alertmanager	Dedupes, routes, silences alerts
Pushgateway	Accept push for batch jobs
Service discovery	Dynamic scrape targets

Data model:

- **Metric name + labels** identify a time series
- **Samples**: float value + millisecond timestamp
- Retention default ~15 days (configurable); long-term storage via Thanos, Cortex, Mimir

45.3 44.3 Prometheus configuration

prometheus.yml :

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s
  external_labels:
    cluster: production
    region: us-east-1

alerting:
```

```

alertmanagers:
  - static_configs:
    - targets: ['alertmanager:9093']

rule_files:
  - /etc/prometheus/rules/*.yaml

scrape_configs:
  - job_name: prometheus
    static_configs:
      - targets: ['localhost:9090']

  - job_name: node
    static_configs:
      - targets: ['node-exporter:9100']

  - job_name: kubernetes-pods
    kubernetes_sd_configs:
      - role: pod
    relabel_configs:
      - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
        action: keep
        regex: true
      - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_port]
        action: replace
        target_label: __address__
        regex: (.+)
        replacement: $1

```

Relabeling filters and transforms targets before scrape—essential for Kubernetes pod discovery.

Pod annotation example:

```

metadata:
  annotations:
    prometheus.io/scrape: "true"
    prometheus.io/port: "8080"
    prometheus.io/path: "/metrics"

```

45.4 44.4 PromQL essentials

PromQL (Prometheus Query Language) queries time series.

Basic selectors:

```

# Instant vector: current value
http_requests_total

# Filter by label
http_requests_total{status="500", service="api"}

# Rate over 5 minutes (counter → per-second rate)
rate(http_requests_total[5m])

# Error ratio
sum(rate(http_requests_total{status=~"5.."}[5m]))
/
sum(rate(http_requests_total[5m]))

# p99 latency from histogram
histogram_quantile(0.99,
  sum(rate(http_request_duration_seconds_bucket[5m])) by (le, service)
)

# CPU usage (node exporter)
100 - (avg by(instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)

```

Common functions:

Function	Use
----------	-----

<code>rate()</code>	Per-second rate for counters
<code>increase()</code>	Total increase over range
<code>histogram_quantile()</code>	Percentiles from histogram buckets
<code>avg_over_time()</code>	Gauge averages
<code>predict_linear()</code>	Disk fill prediction
<code>absent()</code>	Detect missing metrics

45.5 44.5 Alerting rules and Alertmanager

Recording rules pre-compute expensive queries; alerting rules fire when conditions hold.

rules/api.yml :

```
groups:
- name: api_alerts
  interval: 30s
  rules:
    - record: job:http_requests:rate5m
      expr: sum(rate(http_requests_total[5m])) by (job, service)

    - alert: HighErrorRate
      expr: |
        (
          sum(rate(http_requests_total{status=~"5.."}[5m])) by (service)
          /
          sum(rate(http_requests_total[5m])) by (service)
        ) > 0.05
      for: 5m
      labels:
        severity: critical
      annotations:
        summary: "High 5xx rate on {{ $labels.service }}"
        description: "Error rate {{ $value | humanizePercentage }} for 5m"

    - alert: HighLatencyP99
      expr: |
        histogram_quantile(0.99,
          sum(rate(http_request_duration_seconds_bucket[5m])) by (le, service)
        ) > 1
      for: 10m
      labels:
        severity: warning
```

Alertmanager routing (alertmanager.yml):

```
route:
  receiver: default
  group_by: ['alertname', 'service']
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
  routes:
    - match:
        severity: critical
      receiver: pagerduty
    - match:
        severity: warning
      receiver: slack

receivers:
- name: pagerduty
  pagerduty_configs:
    - service_key: SECRET
- name: slack
  slack_configs:
    - api_url: SECRET
      channel: '#alerts'
```

```

- name: default
  slack_configs:
    - api_url: SECRET
      channel: '#alerts-low'

```

Use **inhibition rules** to suppress noisy dependent alerts (node down inhibits all pod alerts on that node).

45.6 44.6 Grafana dashboards

Install Grafana, add Prometheus data source (`http://prometheus:9090`), build dashboards.

Example panel queries:

Panel	PromQL
Request rate	<code>sum(rate(http_requests_total[5m])) by (service)</code>
Error rate	ratio query above
p50/p95/p99	<code>histogram_quantile(0.95, ...)</code>
Saturation	<code>process_resident_memory_bytes</code> , queue depth gauges

Dashboard best practices:

- **Row per service** or golden signals template
- **Variables** — `$service` , `$environment` dropdowns via label values
- **Annotations** — Deploy markers from CI webhook
- **Version control** — Export JSON to Git (Grafana provisioning or Grafana-as-code tools like **Grafonnet**, **Terraform grafana provider**)

Provisioning example `datasources.yml` :

```

apiVersion: 1
datasources:
- name: Prometheus
  type: prometheus
  access: proxy
  url: http://prometheus:9090
  isDefault: true

```

45.7 44.7 Kubernetes deployment patterns

kube-prometheus-stack Helm chart bundles Prometheus, Alertmanager, Grafana, and default Kubernetes dashboards:

```

helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm install kube-prometheus prometheus-community/kube-prometheus-stack \
  --namespace monitoring --create-namespace

```

Prometheus Operator manages Prometheus, Alertmanager, ServiceMonitor, PodMonitor CRDs:

```

apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: api-metrics
  namespace: monitoring
spec:
  selector:
    matchLabels:
      app: api
  namespaceSelector:

```

```
matchNames:
  - production
endpoints:
  - port: metrics
    interval: 30s
    path: /metrics
```

Thanos or **Grafana Mimir** add long-term storage and global query across clusters.

45.8 44.8 Exporters and instrumentation

Common exporters:

Exporter	Metrics source
node_exporter	Linux host CPU, memory, disk, network
blackbox_exporter	Probe HTTP/TCP/ICMP uptime
kube-state-metrics	Kubernetes object state
postgres_exporter	PostgreSQL stats
redis_exporter	Redis stats

Application instrumentation via Prometheus client libraries (Go, Python, Java, etc.) or **OpenTelemetry** exporting to Prometheus format.

Blackbox probe config:

```
modules:
  http_2xx:
    prober: http
    timeout: 5s
    http:
      valid_status_codes: [200, 201, 204]
      preferred_ip_protocol: ip4
```

45.9 44.9 Operations and troubleshooting

Issue	Investigation
Target down	Prometheus → Status → Targets; check network, TLS, path
Missing metrics	Verify exposition format, label typos
High memory	Reduce retention, cardinality, scrape frequency
Alert storm	Alertmanager grouping, inhibition, for: duration
Query timeout	Recording rules, optimize PromQL, scale Prometheus

Capacity planning: ~1-2 bytes per sample; estimate $(\text{metrics} \times \text{labels} \times \text{scrape_freq} \times \text{retention})$.

Security: authenticate Grafana, restrict Prometheus admin API, TLS for scrape in zero-trust networks.

45.10 44.10 Chapter summary

- Prometheus scrapes and stores metrics; PromQL analyzes time series; Alertmanager routes alerts.
- Grafana visualizes Prometheus data with dashboards, variables, and provisioning-as-code.

- Kubernetes deployments use ServiceMonitors, kube-prometheus-stack, and optional long-term storage (Thanos/Mimir).
 - Control cardinality, use recording rules, and design actionable alerts tied to symptoms.
-

45.11 Lab 44.1 — Local Prometheus + Grafana

1. Run Prometheus, node_exporter, and Grafana via Docker Compose.
 2. Scrape node metrics; build a dashboard with CPU, memory, disk panels.
 3. Add an alerting rule for disk > 85%; route to a webhook receiver (use webhook.site for testing).
-

45.12 Lab 44.2 — ServiceMonitor on Kubernetes

1. Deploy kube-prometheus-stack on a test cluster.
 2. Expose a sample app with /metrics ; create ServiceMonitor.
 3. Confirm targets appear in Prometheus UI; import a RED metrics dashboard.
-

45.13 Review questions

1. Why does Prometheus use pull-based scraping instead of push by default?
 2. Explain the difference between recording rules and alerting rules.
 3. Write PromQL for 5xx error rate as a percentage over 5 minutes.
 4. What problem does Alertmanager solve that Prometheus alerts alone do not?
 5. How do ServiceMonitors differ from static scrape configs?
 6. Why is high label cardinality a problem for Prometheus?
 7. When would you add Thanos or Mimir to the stack?
-

Continue: Chapter 45 — ELK/EFK Log Aggregation

46 Chapter 45: ELK/EFK Log Aggregation

DevOps Handbook — Part X, Pages 891–910

46.1 45.1 Centralized logging rationale

Production systems generate millions of log lines across containers, VMs, and serverless functions. SSHing to individual hosts and running `grep` does not scale and fails when pods are ephemeral.

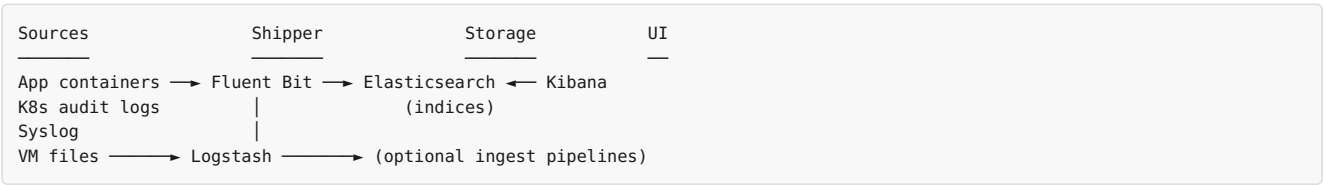
Centralized log aggregation collects, indexes, and searches logs from all sources in one platform—enabling correlation, compliance retention, and security analytics.

The dominant open-source stack:

Stack	Components
ELK	Elasticsearch, Logstash, Kibana
EFK	Elasticsearch, Fluent Bit/Fluentd, Kibana
ECK	Elastic Cloud on Kubernetes (operator)

Alternatives: **Grafana Loki** (label-based, pairs with Prometheus), **OpenSearch** (AWS fork of Elasticsearch), cloud-native **CloudWatch Logs**, **Datadog**, **Splunk**.

46.2 45.2 Architecture overview



Data flow:

1. **Generate** — Apps write stdout/stderr (JSON preferred) or log files
2. **Ship** — Agents (Fluent Bit, Filebeat) forward logs
3. **Process** — Parse, enrich, filter (Logstash or ingest pipelines)
4. **Store** — Elasticsearch indexes documents for search
5. **Visualize** — Kibana Discover, Dashboards, alerting

46.3 45.3 Elasticsearch fundamentals

Elasticsearch is a distributed search and analytics engine storing logs as JSON documents in **indices**.

Concepts:

Term	Meaning
Index	Collection of documents (like a database)
Document	Single log entry (JSON)
Shard	Horizontal partition of index

Replica	Copy for HA and read scaling
Mapping	Schema (field types)

Index naming patterns:

```
logs-{service}-{yyyy.MM.dd}
filebeat-8.11.0-2026.09.01
```

Index Lifecycle Management (ILM) automates rollover, warm, cold, delete phases—controlling cost:

Phase	Action
Hot	Active writes, fast storage
Warm	Shrink shards, reduce replicas
Cold	Searchable snapshots, cheap storage
Delete	Remove after retention period

Example ILM policy (conceptual): hot 7 days → warm 30 days → delete at 90 days.

46.4 45.4 Fluent Bit vs Logstash vs Filebeat

Tool	Role	Resource use
Fluent Bit	Lightweight forwarder; K8s DaemonSet standard	Very low
Fluentd	Richer routing; heavier	Medium
Filebeat	Elastic Beats shipper; simple file/container logs	Low
Logstash	Heavy transformation pipeline	High (JVM)

Fluent Bit Kubernetes config snippet:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: fluent-bit-config
data:
  fluent-bit.conf: |
    [SERVICE]
      Flush          5
      Log_Level      info
      Daemon         off

    [INPUT]
      Name           tail
      Tag            kube.*
      Path           /var/log/containers/*.log
      Parser         docker
      DB             /var/log/flb_kube.db
      Mem_Buf_Limit  50MB

    [FILTER]
      Name           kubernetes
      Match          kube.*
      Kube_URL       https://kubernetes.default.svc:443
      Merge_Log      0n
      Keep_Log       0ff
      K8S-Logging.Parser 0n

    [OUTPUT]
```


Name	es
Match	*
Host	elasticsearch.logging.svc
Port	9200
Index	k8s-logs
Suppress_Type_Name	On
tls	On
tls.verify	On

Deploy Fluent Bit as **DaemonSet**—one pod per node tailing `/var/log/containers/*.log`.

46.5 45.5 Logstash pipelines

Logstash receives, transforms, and outputs events:

```
input {
  beats {
    port => 5044
  }
}

filter {
  if [service] == "payment-api" {
    json {
      source => "message"
    }
    mutate {
      remove_field => ["message"]
    }
  }

  if [level] == "ERROR" {
    mutate {
      add_tag => ["needs_review"]
    }
  }

  geoip {
    source => "client_ip"
  }
}

output {
  elasticsearch {
    hosts => ["https://elasticsearch:9200"]
    user => "logstash_internal"
    password => "${LOGSTASH_PASSWORD}"
    index => "logs-%[service]}-%{+YYYY.MM.dd}"
  }
}
```

Ingest pipelines (Elasticsearch-native) reduce Logstash dependency for simpler transforms.

Prefer parsing at **application** (structured JSON on stdout)—shipper stays dumb, fast, reliable.

46.6 45.6 Kibana for search and dashboards

Kibana provides:

- **Discover** — Ad-hoc search with KQL (Kibana Query Language)
- **Visualize** — Charts, tables, maps
- **Dashboard** — Combined operational views
- **Alerting** — Log threshold rules (Elastic Stack alerting)

KQL examples:

```
service: "payment-api" and level: "ERROR"
trace_id: "4bf92f3577b329da7c93630a8a323129"
@timestamp >= "2026-09-01T00:00:00" and kubernetes.namespace: "production"
response.status >= 500
```

Link logs to traces: include `trace_id` and `span_id` fields matching OpenTelemetry/Jaeger.

Saved searches for on-call:

- Payment errors last hour
- OOMKilled container events
- Authentication failures spike

46.7 45.7 EFK on Kubernetes (ECK)

Elastic **ECK operator** manages Elasticsearch, Kibana, Beats CRDs:

```
apiVersion: elasticsearch.k8s.elastic.co/v1
kind: Elasticsearch
metadata:
  name: logging
spec:
  version: 8.11.0
  nodeSets:
    - name: default
      count: 3
      config:
        node.store.allow_mmap: false
      volumeClaimTemplates:
        - metadata:
            name: elasticsearch-data
          spec:
            accessModes: ["ReadWriteOnce"]
            resources:
              requests:
                storage: 100Gi
```

Sizing guidance:

Daily log volume	Starting cluster
< 50 GB/day	3 nodes, 8 GB heap each
50–500 GB/day	Dedicated masters, hot/warm tiers
> 500 GB/day	Index lifecycle, dedicated ingest, consult Elastic sizing

Heap: stay below 50% RAM, max ~31 GB per node (compressed pointers).

46.8 45.8 Security and compliance

- **TLS everywhere** — Shipper to Elasticsearch, Kibana access
- **Authentication** — Native, SAML, OIDC integration
- **RBAC** — Index-level, document-level security (paid features in Elastic)
- **Audit logs** — Who searched what (compliance)
- **PII scrubbing** — Filter at ingest; never index credit cards, passwords

```
# Logstash mutate – drop sensitive fields
filter {
  mutate {
    remove_field => ["password", "credit_card", "ssn"]
  }
}
```

Regulatory retention: ILM delete phase after required period; legal hold overrides.

46.9 45.9 Loki as an alternative

Grafana Loki indexes labels (like Prometheus), not full text—lower cost, less flexible search.

```
{namespace="production", app="api"} |= "error" | json | level="ERROR"
```

Choose Loki when already on Grafana/Prometheus and logs are structured. Choose ELK when full-text search, complex analytics, and Elastic ecosystem (SIEM) are required.

46.10 45.10 Operational best practices

Practice	Benefit
JSON structured logs	Faster queries, smaller pipelines
Consistent @timestamp UTC	Cross-region correlation
Sample DEBUG in prod	Cost control
Dedicated logging cluster	Blast radius isolation from app cluster
Monitor the monitors	Alert on ingest lag, cluster health yellow/red
Test disaster recovery	Snapshot/restore to S3 repository

Common failures:

- **Mapping explosions** — Dynamic fields from poorly structured logs
- **Hot shards** — Uneven index sizing; use rollover
- **Ingest backpressure** — Scale Fluent Bit buffers or Elasticsearch ingest nodes

46.11 45.11 Chapter summary

- ELK/EFK centralizes logs: shippers (Fluent Bit/Filebeat) → Elasticsearch → Kibana.
- Structure logs at the source; use ILM for retention and cost control.
- Fluent Bit DaemonSets are the Kubernetes standard for container log collection.
- Secure the stack with TLS, RBAC, and PII filtering; consider Loki for label-centric workloads.

46.12 Lab 45.1 — Docker Compose ELK

1. Deploy Elasticsearch, Logstash, Kibana, and Filebeat via Docker Compose.
2. Configure a sample app to log JSON to stdout; Filebeat ships to Logstash → ES.
3. In Kibana Discover, search by `level:ERROR` and create a visualization of errors over time.

46.13 Lab 45.2 — Fluent Bit on Kubernetes

1. Install Fluent Bit DaemonSet pointing to a managed or local Elasticsearch.

2. Deploy a pod that logs JSON with `trace_id` field.
 3. Query logs in Kibana by `trace_id`; verify kubernetes metadata enrichment (pod name, namespace).
-

46.14 Review questions

1. Compare ELK and EFK—what component differs and why choose Fluent Bit?
 2. What is an Elasticsearch index lifecycle policy, and why use it?
 3. Why should applications emit structured JSON instead of plain text?
 4. Explain the role of a DaemonSet log shipper in Kubernetes.
 5. What causes mapping explosions, and how do you prevent them?
 6. When would you choose Grafana Loki over Elasticsearch?
 7. How do you correlate logs with distributed traces in Kibana?
-

Continue: Chapter 46 — OpenTelemetry Distributed Tracing

47 Chapter 46: OpenTelemetry Distributed Tracing

DevOps Handbook — Part X, Pages 911–930

47.1 46.1 The tracing problem

Microservices decompose monoliths into independently deployable units—each request may traverse API gateways, auth services, business logic, databases, caches, and message queues. When latency spikes or errors appear, **which hop failed?**

Distributed tracing records the path of a request across process boundaries. **OpenTelemetry (OTel)** is the CNCF standard unifying tracing, metrics, and logs instrumentation under one vendor-neutral API—replacing OpenTracing and OpenCensus.

Benefits:

- End-to-end latency breakdown
- Dependency mapping (service graph)
- Root cause analysis for errors
- Performance regression detection per release

47.2 46.2 Traces, spans, and context propagation

Concept	Definition
Trace	End-to-end journey of a request (shared trace ID)
Span	Single timed operation within a trace
Parent span	Upstream operation that triggered child work
Context propagation	Passing trace ID/span ID across HTTP/gRPC/message headers

W3C **Trace Context** standard headers:

```
traceparent: 00-4bf92f3577b329da7c93630a8a323129-00f067aa0ba902b7-01
tracestate: congo=t61rcWkgMzE
```

Format: `version-trace_id-parent_span_id-flags`

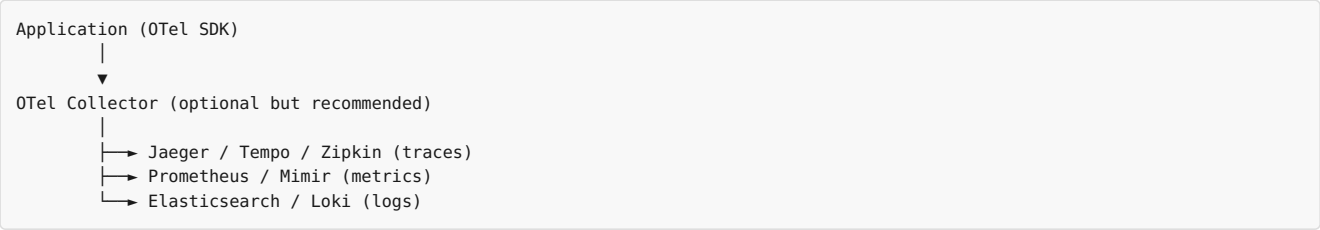
```
Trace 4bf92f3577b329da...
├─ [span A] API Gateway      120ms
│   └─ [span B] Auth validate 15ms
│       └─ [span C] Order service 95ms
│           └─ [span D] DB query 8ms
│               └─ [span E] Inventory gRPC 70ms ← bottleneck
```

Span attributes (metadata):

- `http.method`, `http.status_code`
- `db.system`, `db.statement` (sanitized)
- `service.name`, `deployment.environment`

Span events: point-in-time annotations within a span (e.g., “cache miss”).

47.3 46.3 OpenTelemetry architecture



Components:

Component	Role
OTel API	Instrumentation interfaces in app code
OTel SDK	Sampling, batching, export
Instrumentation libraries	Auto-instrument HTTP, gRPC, DB drivers
OTel Collector	Receive, process, export; vendor-agnostic hub
Backend	Storage and UI (Jaeger, Grafana Tempo)

47.4 46.4 Instrumentation: auto vs manual

Auto-instrumentation (agents) attaches without code changes:

```
# Python example
opentelemetry-instrument \
  --traces_exporter otlp \
  --metrics_exporter otlp \
  --service_name order-api \
  --exporter_otlp_endpoint http://otel-collector:4317 \
  python app.py
```

Java: `-javaagent:opentelemetry-javaagent.jar`

Manual instrumentation for business logic:

```
from opentelemetry import trace
from opentelemetry.trace import Status, StatusCode

tracer = trace.get_tracer("order-service", "1.0.0")

def create_order(order_data):
    with tracer.start_as_current_span("create_order") as span:
        span.set_attribute("order.customer_id", order_data["customer_id"])
        try:
            validate(order_data)
            order_id = persist(order_data)
            span.set_attribute("order.id", order_id)
            return order_id
        except ValidationError as e:
            span.record_exception(e)
            span.set_status(Status(StatusCode.ERROR, str(e)))
            raise
```

Ensure **context propagation** across async boundaries (thread pools, asyncio, message queues)—dropped context breaks trace chains.

47.5 46.5 OTel Collector configuration

Collector pipelines: **receivers** → **processors** → **exporters**

```
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318

processors:
  batch:
    timeout: 5s
    send_batch_size: 1024
  memory_limiter:
    check_interval: 1s
    limit_mib: 512
  resource:
    attributes:
      - key: deployment.environment
        value: production
        action: upsert
  tail_sampling:
    decision_wait: 10s
    policies:
      - name: errors
        type: status_code
        status_code:
          status_codes: [ERROR]
      - name: latency
        type: latency
        latency:
          threshold_ms: 1000
      - name: probabilistic
        type: probabilistic
        probabilistic:
          sampling_percentage: 5

exporters:
  otlp/jaeger:
    endpoint: jaeger-collector:4317
    tls:
      insecure: true
  prometheus:
    endpoint: 0.0.0.0:8889

service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, tail_sampling, batch, resource]
      exporters: [otlp/jaeger]
    metrics:
      receivers: [otlp]
      processors: [batch, resource]
      exporters: [prometheus]
```

Tail sampling at collector evaluates complete traces—keep all errors and slow traces, sample routine success paths.

47.6 46.6 Sampling strategies

Tracing every request at high QPS is expensive. Sampling reduces volume while preserving debuggability.

Strategy	Behavior
Always on	Dev/test only
Probabilistic head	Random % at trace start; simple but may miss errors

Rate limiting	Max traces per second
Tail sampling	Decide after trace completes; keep errors/slow

Head sampling in SDK:

```
from opentelemetry.sdk.trace.sampling import TraceIdRatioBased

sampler = TraceIdRatioBased(0.1) # 10% of traces
```

Production recommendation: **tail sampling in Collector** with error and latency policies plus low baseline probabilistic sample.

47.7 46.7 Backends: Jaeger and Grafana Tempo

Jaeger (CNCF graduated): UI for trace search, dependency graphs, compare traces.

Deploy Jaeger on Kubernetes:

```
# Simplified – use Jaeger Operator in production
apiVersion: apps/v1
kind: Deployment
metadata:
  name: jaeger
spec:
  template:
    spec:
      containers:
        - name: jaeger
          image: jaegertracing/all-in-one:1.52
          ports:
            - containerPort: 16686 # UI
            - containerPort: 4317 # OTLP gRPC
```

Grafana Tempo: object storage backend (S3/GCS), integrates natively with Grafana—trace-to-log correlation via `trace_id`.

Trace search:

- By trace ID (from logs or error message)
- By service + operation + duration
- By tags (`http.status_code=500`)

47.8 46.8 Correlating traces, logs, and metrics

Exemplars link metrics to traces—a histogram bucket points to example trace ID.

Logs include trace context:

```
{
  "message": "order created",
  "trace_id": "4bf92f3577b329da7c93630a8a323129",
  "span_id": "00f067aa0ba902b7"
}
```

Grafana **Trace to logs** jump: from span → Loki query `{trace_id="..."}`.

Metrics derived from traces (span metrics connector in Collector): RED metrics per service from trace data without duplicate instrumentation.

47.9 46.9 Service mesh tracing

Istio and **Linkerd** generate spans automatically for mesh traffic—application instrumentation still needed for in-process detail.

Istio enables tracing via Telemetry API; spans exported to OTel Collector.

Mesh traces show network latency (TLS, retries); app traces show business logic—combine both.

47.10 46.10 Adoption roadmap

1. **Deploy Collector** — Central export point before picking backend
2. **Auto-instrument ingress** — API gateway, primary services
3. **Propagate context** — Verify cross-service trace continuity
4. **Add manual spans** — Critical business operations
5. **Tail sampling** — Cost control in production
6. **Integrate Grafana/Jaeger** — On-call workflows, dashboards
7. **SLO dashboards** — Latency from trace metrics

Anti-patterns:

- Instrumentation without sampling plan (cost surprise)
 - Broken context in async/message paths
 - Storing full SQL with PII in span attributes
-

47.11 46.11 Chapter summary

- OpenTelemetry standardizes traces, metrics, and logs with vendor-neutral SDKs and Collector.
 - Traces consist of spans linked by context propagation (W3C Trace Context headers).
 - Use auto-instrumentation plus manual spans; tail sampling preserves errors while controlling cost.
 - Correlate traces with logs (`trace_id`) and metrics (exemplars) for full observability.
-

47.12 Lab 46.1 — OTel + Jaeger locally

1. Run OTel Collector, Jaeger, and a two-service demo (e.g., frontend → backend).
 2. Auto-instrument both services with OTel agents.
 3. Generate traffic; verify end-to-end trace in Jaeger UI.
 4. Introduce artificial 2s delay in backend; identify span in critical path.
-

47.13 Lab 46.2 — Tail sampling

1. Configure Collector tail sampling: 100% errors, 5% success.
 2. Generate 1000 successful requests and 50 failing requests.
 3. Compare trace counts in Jaeger; confirm error traces retained.
-

47.14 Review questions

1. What is the difference between a trace and a span?

2. Explain W3C Trace Context propagation across HTTP services.
 3. Why is tail sampling preferred over head sampling in production?
 4. What role does the OpenTelemetry Collector play?
 5. How do you correlate logs with traces in Grafana?
 6. When is manual instrumentation necessary beyond auto-instrumentation?
 7. What are exemplars, and how do they connect metrics to traces?
-

Continue: Chapter 47 — SLOs, Error Budgets, Incident Response

48 Chapter 47: SLOs, Error Budgets, and Incident Response

DevOps Handbook — Part X, Pages 931–945

48.1 47.1 Reliability as a product decision

Uptime is not maximized at 100%—infinite reliability costs infinite money and slows feature delivery. **Site Reliability Engineering (SRE)** treats reliability as a **product feature** with an explicit target negotiated between engineering and business.

Service Level Objective (SLO) — Target reliability (e.g., 99.9% availability over 30 days).

Service Level Indicator (SLI) — Measured metric (e.g., successful requests / total requests).

Service Level Agreement (SLA) — Contract with customers (often looser than internal SLO).

Error budget — Allowed unreliability: $1 - \text{SLO}$. If SLO is 99.9%, budget is 0.1% (~43 minutes downtime/month).

When error budget is ** exhausted, **freeze risky releases and invest in reliability**. When budget is healthy**, ship features aggressively.

48.2 47.2 Choosing SLIs and SLOs

SLIs must reflect **user experience**, not internal convenience.

Service type	Good SLI	Poor SLI
Request/response API	Success rate + latency	CPU utilization
Data pipeline	Freshness, completeness	Worker pod count
Storage	Durability, read success	Disk free space alone

Example API SLIs:

```
# Availability SLI (30-day window)
sum(rate(http_requests_total{status!~"5.."}[5m]))
/
sum(rate(http_requests_total[5m]))

# Latency SLI: proportion under 300ms
sum(rate(http_request_duration_seconds_bucket{le="0.3"}[5m]))
/
sum(rate(http_request_duration_seconds_count[5m]))
```

SLO targets (illustrative):

Tier	Availability	Latency (p99)
Critical payment	99.99%	< 200ms
Core API	99.9%	< 500ms
Internal tooling	99.5%	< 2s

Document SLOs in a **SLO document** per service: SLI definition, query, target, window, owners, escalation.

48.3 47.3 Error budgets in practice

Error budget calculation (30-day rolling):

```
Budget = (1 - SL0) × total_events
Consumed = failed_events or slow_events
Remaining = Budget - Consumed
```

Burn rate alerts predict budget exhaustion before window ends:

Burn rate	Meaning	Action
1×	On track to exhaust exactly at window end	Monitor
2×	Exhaust in half the window	Investigate
14.4×	Exhaust in ~2 days at 99.9% monthly	Page on-call

Multi-window burn alert (Google SRE):

```
# Conceptual – implement via Prometheus recording rules
# Fast burn: 2% budget in 1 hour → page
# Slow burn: 5% budget in 6 hours → ticket
```

Policy when budget depleted:

- 1. Stop non-critical releases
- 2. Focus engineering on reliability work
- 3. Executive visibility—product tradeoff discussion
- 4. Resume feature velocity when budget recovers

Error budgets **align** dev and ops: shared metric replaces “ops blocks deploys” vs “dev ignores stability.”

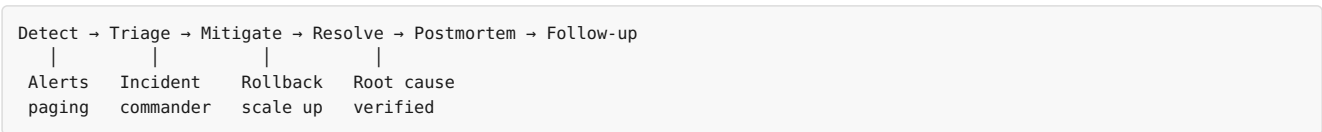
48.4 47.4 Incident severity and lifecycle

Incident — Unplanned interruption or degradation requiring coordinated response.

Severity levels (example):

Severity	Impact	Response
SEV-1	Customer-facing outage, revenue impact	Immediate all-hands, exec comms
SEV-2	Major degradation, workaround exists	On-call + team lead within 15m
SEV-3	Minor impact, limited users	Business hours response
SEV-4	Cosmetic, no user impact	Backlog

Incident lifecycle:



Roles:

Role	Responsibility
Incident Commander (IC)	Coordinates response, decisions, comms

Technical Lead	Drives debugging and mitigation
Communications Lead	Status page, stakeholder updates
Scribe	Timeline, actions log

48.5 47.5 On-call and escalation

Effective on-call requires:

- **Runbooks** — Step-by-step for known alerts
- **Dashboards** — Golden signals per service linked from alerts
- **Escalation policy** — Primary → secondary → manager
- **Handoff** — Document active issues between shifts
- **Toil limits** — On-call should not mean endless manual work

PagerDuty/Opsgenie escalation example:

1. Primary on-call (5 min)
2. Secondary on-call (10 min)
3. Engineering manager (15 min)
4. Incident commander pool (SEV-1)

Alert quality directly affects on-call burnout—every page must require human action (Chapter 43).

48.6 47.6 Incident response procedures

Detection — Automated alerts on SLO burn, synthetic probes, customer reports.

Triage checklist:

1. Confirm user impact (not false positive)
2. Assign severity and IC
3. Open incident channel (`#inc-2026-09-01-api-outage`)
4. Start timeline document
5. Communicate status page update within 15 min (SEV-1)

Mitigation over root cause — Restore service first (rollback, failover, scale, feature flag off), investigate after.

Common mitigations:

Symptom	Fast mitigation
Bad deploy	Rollback to previous version
Traffic spike	Scale horizontally, rate limit
Dependency down	Circuit breaker, degrade gracefully
Data corruption	Stop writes, restore from backup

Resolution — Verify SLI recovery, close incident, schedule postmortem within 5 business days.

48.7 47.7 Blameless postmortems

Postmortem documents **what happened**, **why** (contributing factors, not single root cause), and **action items**—

without blaming individuals.

Template sections:

1. Summary (one paragraph)
2. Impact (duration, users affected, SLO breach)
3. Timeline (UTC timestamps)
4. Root cause analysis (5 whys, fishbone)
5. What went well / what went poorly
6. Action items (owner, due date, priority)
7. Lessons learned

Blameless culture encourages reporting near-misses—hidden failures repeat.

Action item tracking: treat reliability fixes like product backlog items with SLAs.

48.8 47.8 Status communication

Status page (Statuspage, Instatus, custom):

- Investigating → Identified → Monitoring → Resolved
- Update every 30–60 min during SEV-1
- Honest scope—avoid “all systems operational” when degraded

Internal vs external messaging:

- Internal: technical detail, hypothesis, commands run
 - External: user impact, workaround, ETA if confident
-

48.9 47.9 Tooling integration

Integrate observability → incident → postmortem:

```
Prometheus alert → Alertmanager → PagerDuty → Slack #incidents
Deploy webhook → Timeline auto-entry
Grafana snapshot → Postmortem attachment
Jira/Linear → Action item tickets from postmortem template
```

Tag incidents with **deployment ID** and **trace IDs** for DORA change failure analysis.

48.10 47.10 Chapter summary

- SLOs define target reliability; SLIs measure it; error budgets balance feature velocity and stability.
 - Burn rate alerts provide early warning before budget exhaustion.
 - Incident response prioritizes mitigation, clear roles, and stakeholder communication.
 - Blameless postmortems drive systemic improvements with tracked action items.
-

48.11 Lab 47.1 — Define SLOs for a sample service

1. Pick a sample HTTP API with Prometheus metrics.
2. Write SLI queries for availability and latency.
3. Set SLO target 99.9% / 30 days; calculate monthly error budget in minutes.
4. Create Grafana dashboard showing budget remaining.

48.12 Lab 47.2 — Tabletop incident exercise

1. Scenario: deploy causes 10% 5xx rate spike.
 2. Assign roles (IC, tech lead, comms, scribe).
 3. Run 30-minute simulated response: triage, mitigation decision, status update draft.
 4. Write blameless postmortem with 3 action items.
-

48.13 Review questions

1. Distinguish SLI, SLO, and SLA with an example.
 2. How is error budget calculated for 99.95% over 30 days?
 3. What should happen when error budget is exhausted mid-quarter?
 4. Why prioritize mitigation over root cause during an active incident?
 5. What makes a postmortem blameless, and why does it matter?
 6. Explain multi-window burn rate alerting.
 7. What SLIs would you choose for a batch data pipeline?
-

Continue: Chapter 48 — Chaos Engineering and Game Days

49 Chapter 48: Chaos Engineering and Game Days

DevOps Handbook — Part X, Pages 946–960

49.1 48.1 Proactive resilience testing

Production fails in unexpected ways—despite staging environments, tests, and code review. **Chaos engineering** deliberately injects failures into systems to validate resilience **before** real outages expose gaps.

“Chaos engineering is the discipline of experimenting on a system in order to build confidence in the system’s capability to withstand turbulent conditions in production.” — Principles of Chaos Engineering

Not reckless destruction: chaos experiments are **hypothesis-driven, controlled, measured, and bounded** in blast radius.

Relationship to other practices:

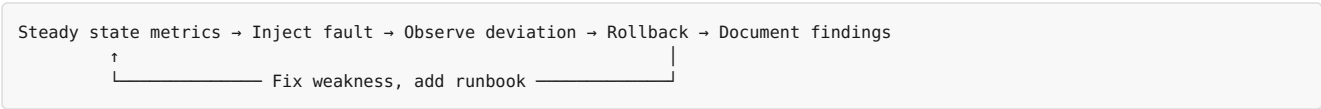
Practice	Focus
Testing	Known expected behavior
Chaos engineering	Unknown failure modes, emergent behavior
Game days	Scheduled team exercises simulating incidents
DR drills	Region/account failover

Netflix **Chaos Monkey** (2011) randomly terminated production instances—proving redundancy worked. The practice evolved into comprehensive fault injection platforms.

49.2 48.2 Principles of chaos engineering

1. **Build a hypothesis around steady state** — Define normal (SLO metrics, error rate, latency).
2. **Vary real-world events** — Server failure, network partition, dependency timeout, disk full.
3. **Run experiments in production** — Staging lacks scale and traffic patterns (start in non-prod, graduate to prod with safeguards).
4. **Automate experiments** — Manual one-offs do not scale; continuous verification.
5. **Minimize blast radius** — Start small (one pod, one AZ, 1% traffic).

Experiment workflow:



49.3 48.3 Failure modes to inject

Category	Examples
Infrastructure	Kill pod/VM, drain node, AZ failure
Network	Latency, packet loss, DNS failure, partition
Dependency	Upstream 503, slow responses, certificate expiry
	CPU/memory pressure, disk fill, connection pool

Resource	exhaustion
Application	Exception injection, feature flag misconfiguration
Data	Replica lag, split-brain simulation
Human	On-call unreachable (game day scenario)

Prioritize failures matching **real incident history**—postmortem action items often become chaos experiments.

49.4 48.4 Chaos tools landscape

Tool	Environment	Capabilities
Litmus Chaos	Kubernetes	CRD-based experiments, hub of scenarios
Chaos Mesh	Kubernetes	Pod kill, network chaos, IO chaos, stress
Gremlin	K8s, VM, bare metal	SaaS/enterprise fault injection
AWS FIS	AWS	EC2 stop, AZ impairment, RDS failover
Azure Chaos Studio	Azure	Managed chaos experiments
Toxiproxy	Dev/test	Network condition proxy
Pumba	Docker	Container chaos

Litmus experiment example:

```
apiVersion: litmuschaos.io/v1alpha1
kind: ChaosEngine
metadata:
  name: api-pod-delete
  namespace: production
spec:
  appinfo:
    appns: production
    applabel: "app=api"
    appkind: deployment
  chaosServiceAccount: litmus-admin
  experiments:
    - name: pod-delete
      spec:
        components:
          env:
            - name: TOTAL_CHAOS_DURATION
              value: "60"
            - name: CHAOS_INTERVAL
              value: "10"
            - name: FORCE
              value: "false"
```

Chaos Mesh NetworkChaos (latency):

```
apiVersion: chaos-mesh.org/v1alpha1
kind: NetworkChaos
metadata:
  name: delay-inventory
spec:
  action: delay
  mode: one
  selector:
    namespaces:
      - production
    labelSelectors:
      app: order-api
  delay:
```

```
latency: "500ms"
correlation: "100"
jitter: "50ms"
direction: to
target:
  selector:
    namespaces:
      - production
    labelSelectors:
      app: inventory-service
```

49.5 48.5 Safe experimentation guardrails

Before running in production:

Guardrail	Purpose
Abort conditions	Auto-stop if error rate > threshold
Business hours vs off-peak	Limit user impact window
One variable at a time	Isolate cause and effect
Rollback plan	Immediate experiment halt mechanism
Stakeholder notification	Support, leadership aware
Exclude critical paths initially	Payment during peak—defer
Observability first	Cannot validate hypothesis without metrics

Integrate with **OpenTelemetry/Prometheus** dashboards dedicated to experiment runs.

Steady state hypothesis example:

"When one API pod is terminated, p99 latency remains < 500ms and error rate < 0.1% because Kubernetes reschedules within 30s and HPA maintains capacity."

If hypothesis fails: fix (PDB, probes, HPA tuning), do not dismiss the experiment.

49.6 48.6 Game days

Game days are scheduled, cross-functional exercises—often simulating major incidents without continuous automated chaos.

Structure (4-hour example):

Time	Activity
0:00	Briefing, roles assigned, rules of engagement
0:15	Inject scenario (facilitator triggers faults)
0:15–2:00	Team responds as real incident
2:00	End injection, debrief hot wash
2:30	Document gaps, update runbooks

Scenarios:

- Primary database failover
- Region loss (traffic shift drill)
- Certificate expiry on ingress

- DDoS simulation (with provider coordination)
- Supply chain compromise response

Facilitator **does not** fix—observes communication, decision quality, tooling gaps.

Include: engineering, SRE, support, comms, product—for SEV-1 realism.

Frequency: quarterly per critical service; annual org-wide DR game day.

49.7 48.7 From experiment to improvement

Chaos findings feed backlog:

Finding	Remediation
No PDB on critical Deployment	Add PodDisruptionBudget
HPA too slow	Tune metrics, reduce cooldown
Retry storm on dependency failure	Circuit breaker + bulkhead
Alert did not fire	Fix SLO burn alert
Runbook outdated	Update with verified steps

Track **resilience score**: % of critical services with passing monthly chaos experiments.

Automate regression: experiment suite runs in staging on every release candidate.

49.8 48.8 Organizational maturity

Maturity	Behavior
Level 0	No chaos; surprises in prod
Level 1	Ad-hoc manual tests in staging
Level 2	Scheduled game days, documented scenarios
Level 3	Automated chaos in staging CI
Level 4	Continuous limited chaos in production with guardrails
Level 5	Chaos-driven architecture; resilience SLOs

Start at Level 1–2; production chaos requires strong observability and org trust.

49.9 48.9 Chapter summary

- Chaos engineering validates resilience through controlled, hypothesis-driven failure injection.
- Tools like Litmus and Chaos Mesh inject infrastructure and network faults in Kubernetes.
- Guardrails (abort conditions, blast radius limits) make production experiments safe.
- Game days train teams on incident response; findings improve architecture, alerts, and runbooks.

49.10 Lab 48.1 — Pod delete experiment

1. Deploy sample app with 3 replicas, PDB, and HPA in test cluster.
 2. Run Litmus or kubectl delete pod experiment during load test.
 3. Measure recovery time and error rate; document hypothesis pass/fail.
-

49.11 Lab 48.2 — Mini game day

1. Design a 1-hour scenario: dependency returns 503 for 10 minutes.
 2. Facilitate team response with IC and scribe roles.
 3. Debrief: what broke, what worked, 3 action items.
-

49.12 Review questions

1. State the five principles of chaos engineering.
 2. Why run experiments in production rather than only in staging?
 3. What abort conditions would you set for a pod-delete experiment?
 4. Compare chaos engineering and game days.
 5. Name three failure modes worth injecting for a microservices API.
 6. How should a failed chaos hypothesis be handled?
 7. What organizational prerequisites are needed before production chaos?
-

Continue: Chapter 49 — Shift-Left Security

50 Chapter 49: Shift-Left Security in the Pipeline

DevOps Handbook — Part XI, Pages 961–980

50.1 49.1 DevSecOps and shift-left

DevSecOps integrates security into the software delivery lifecycle—not as a final gate before release, but as continuous practice from design through operation. **Shift-left** means moving security activities **earlier**—cheaper to fix vulnerabilities in IDE or PR than in production breach.

Traditional model:

```
Dev → Test → Security audit (weeks) → Deploy → Pen test → Incident
```

Shift-left model:

```
Threat model → Secure coding → Pre-commit → PR scan → CI scan → Deploy → Runtime monitor
```

Security becomes **everyone’s responsibility** with platform-enabling tooling—not solely a separate security team blocking releases.

50.2 49.2 Security activities across the SDLC

Phase	Activities
Design	Threat modeling (STRIDE), security requirements, data classification
Develop	Secure coding standards, IDE plugins, pre-commit hooks
Build	SAST, SCA, secret scanning, IaC scan
Test	DAST, fuzzing, penetration test automation
Deploy	Image signing, admission control, least-privilege IAM
Operate	Runtime detection, WAF, audit logs, incident response
Respond	Vulnerability disclosure, patch SLAs, forensics

The **software supply chain** spans all phases—dependencies, build systems, registries, and deployment pipelines are attack surfaces (SolarWinds, Log4Shell lessons).

50.3 49.3 Threat modeling fundamentals

STRIDE categorizes threats:

Letter	Threat	Example
S	Spoofing	Fake JWT
T	Tampering	Modify API payload
R	Repudiation	Deny transaction without audit log
I	Information disclosure	Log leaking PII

D	Denial of service	Unbounded query
E	Elevation of privilege	IDOR to admin

Process:

1. Draw data flow diagram (users, services, data stores)
2. Identify trust boundaries
3. Apply STRIDE per component
4. Prioritize mitigations (risk = likelihood × impact)
5. Track in backlog

Tools: **Microsoft Threat Modeling Tool**, **OWASP Threat Dragon**, Miro templates.

Output feeds security test cases and pipeline scan configuration.

50.4 49.4 Pipeline security gates

Integrate checks in CI without destroying developer velocity:

```
Commit → Secret scan → Lint/IaC policy → SAST → Unit tests → Build →
Container scan → SCA → Sign artifact → Deploy (policy) → DAST (staging)
```

Gate philosophy:

Severity	Policy example
Critical	Block merge/deploy
High	Block deploy to prod; warn on PR
Medium	Ticket required; SLA 30 days
Low	Informational

Policy as Code (OPA, Conftest) enforces gates consistently:

```
# deny deployment if critical CVEs in scan result
package deploy

deny[msg] {
  input.scan.critical_count > 0
  msg := sprintf("Critical vulnerabilities: %d", [input.scan.critical_count])
}
```

Balance **speed vs safety**: fast feedback on PR (< 10 min for secret/SAST); heavier scans async or nightly.

50.5 49.5 Secret scanning

Secrets in Git history are permanent until rotated. Scan **commits**, **PRs**, and **history**.

Tools:

Tool	Integration
GitLeaks	Pre-commit, CI
TruffleHog	CI, deep history
GitHub secret scanning	Native on GitHub

GitLab secret detection	Native on GitLab
-------------------------	------------------

Pre-commit hook:

```
# .pre-commit-config.yaml
repos:
- repo: https://github.com/gitleaks/gitleaks
  rev: v8.18.0
  hooks:
  - id: gitleaks
```

CI step:

```
- name: Secret scan
  uses: gitleaks/gitleaks-action@v2
  env:
    GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
```

On leak: **rotate immediately**, purge from history (`git filter-repo`) if exposed publicly—scanning alone is insufficient.

50.6 49.6 Secure defaults and platform guardrails

Platform engineering enables secure defaults:

- **Base images** — Hardened, scanned, regularly rebuilt
- **Template repos** — CI security jobs pre-configured
- **Namespace/network policies** — Deny-by-default in Kubernetes
- **IAM roles** — Short-lived credentials, no long-lived keys in repos
- **Dependency update bots** — Renovate, Dependabot with auto-merge for patches

Golden paths make the secure path the easy path—developers opt out explicitly, not opt in.

50.7 49.7 Security champions program

Scale security expertise without bottleneck:

Role	Responsibility
Central security	Policy, tooling, compliance, escalation
Security champion (per team)	Triage findings, threat model reviews, training
Developers	Fix findings, secure coding

Champions attend monthly syncs, get early tool access, relay team friction to security org.

Metrics: mean time to remediate (MTTR) vulnerabilities by severity, not raw finding count (discourages hiding issues).

50.8 49.8 Compliance integration

Map shift-left controls to frameworks:

Control area	Pipeline automation
Access control	IAM review in IaC scans

Access control	IAM review in IaC scan
Change management	Git audit trail, approved PRs
Vulnerability mgmt	SCA/SAST gates, ticket integration
Logging/monitoring	Mandatory observability checks
Encryption	TLS policy, secret management validation

Evidence collection: export CI scan reports, deployment logs, and policy decisions for SOC 2, ISO 27001, PCI audits—**continuous compliance** vs annual scramble.

50.9 49.9 Culture and psychological safety

Developers must report security concerns without fear. Blameless response to leaked secrets encourages fast rotation.

Training:

- OWASP Top 10 awareness
- Secure coding labs (SQLi, XSS in sandbox)
- Annual phishing simulations

Security team as **enabler** (“here’s how to fix”) not **gatekeeper** (“no”).

50.10 49.10 Chapter summary

- Shift-left embeds security early: threat modeling, scanning, and policy gates in CI/CD.
 - Secret scanning, SAST/SCA/DAST (Chapters 50–52), and policy-as-code form layered defense.
 - Platform golden paths and security champions scale practices across teams.
 - Balance blocking gates with remediation SLAs and developer experience.
-

50.11 Lab 49.1 — Pre-commit security stack

1. Add pre-commit with gitleaks and detect-private-key hooks to a sample repo.
 2. Attempt to commit a fake AWS key; confirm hook blocks.
 3. Add CI job duplicating secret scan for defense in depth.
-

50.12 Lab 49.2 — Threat model exercise

1. Draw data flow for a three-tier web app.
 2. Apply STRIDE to API gateway and database boundary.
 3. Produce 5 mitigations mapped to pipeline controls (scan type or policy).
-

50.13 Review questions

1. What does “shift-left” mean in DevSecOps?
2. List security activities appropriate at design vs deploy phases.
3. Explain STRIDE with one example threat per category.
4. How should CI gates differ for critical vs medium findings?

5. Why rotate secrets immediately when leaked to Git?
 6. What is a security champion, and why use this model?
 7. How does shift-left support compliance audits?
-

Continue: Chapter 50 — SAST, DAST, SCA, Container Scanning

51 Chapter 50: SAST, DAST, SCA, and Container Scanning

DevOps Handbook — Part XI, Pages 981–1000

51.1 50.1 Application security testing taxonomy

Modern applications combine custom code, open-source dependencies, container images, and infrastructure configuration—each layer requires specialized scanning.

Acronym	Full name	What it analyzes	When
SAST	Static Application Security Testing	Source code without running	PR, CI build
DAST	Dynamic Application Security Testing	Running application (black-box)	Staging, pre-prod
SCA	Software Composition Analysis	Dependencies, licenses, CVEs	CI, daily scheduled
IAST	Interactive AST	Instrumented runtime + code	Optional hybrid
Container scan	Image/filesystem CVE scan	OS packages, layers	After docker build
IaC scan	Infrastructure as Code	Terraform, K8s manifests	PR, CI

No single tool covers all—**defense in depth** layers findings with deduplication and prioritization.

51.2 50.2 SAST — static analysis

SAST parses source code and bytecode for vulnerability patterns: SQL injection, XSS, path traversal, hardcoded secrets, insecure crypto.

Popular tools:

Tool	Languages	Notes
Semgrep	Multi-language, custom rules	Fast, CI-friendly
SonarQube	30+ languages	Quality + security
CodeQL	Multi (GitHub)	Semantic analysis
Bandit	Python	Python-specific
gosec	Go	Go-specific
Checkmarx / Veracode	Enterprise	Broad coverage

Semgrep CI example:

```
- name: Semgrep SAST
  uses: returntocorp/semgrep-action@v1
  with:
    config: >-
```

```
p/security-audit
p/owasp-top-ten
p/ci
```

Custom Semgrep rule:

```
rules:
- id: sql-injection-concat
  patterns:
    - pattern: |
        $QUERY = "... " + $USER_INPUT + "... "
    - pattern: execute($QUERY)
  message: Possible SQL injection via string concatenation
  severity: ERROR
  languages: [python, javascript]
```

SAST limitations:

- **False positives** — Requires tuning and baselines
- **No runtime context** — Cannot see config-dependent bugs
- **Late custom rules** — Novel vulns need rule updates

Triage workflow: suppress with justification (`nosemgrep` comment + ticket), fix true positives within SLA.

51.3 50.3 DAST — dynamic testing

DAST probes **running** applications like an attacker—HTTP requests, form fuzzing, authentication bypass attempts.

Tools:

Tool	Type
OWASP ZAP	Open-source proxy + automation
Burp Suite	Manual + automated (pro)
Nuclei	Template-based scanner
StackHawk	CI-integrated DAST
Acunetix	Commercial

ZAP baseline scan in CI (against staging):

```
- name: OWASP ZAP Baseline
  uses: zaproxy/action-baseline@v0.12.0
  with:
    target: https://staging.example.com
    rules_file_name: .zap/rules.tsv
    cmd_options: -a
```

DAST requires:

- Deployed environment with realistic data (sanitized)
- Authentication handling (ZAP contexts, scripts)
- Scope limits—never scan production without approval

Finds: misconfigured headers, exposed admin panels, reflected XSS, CSRF gaps—issues SAST may miss.

51.4 50.4 SCA — dependency analysis

SCA inventories direct and transitive dependencies, matches against CVE databases (NVD, GitHub Advisory), flags license violations.

Tools:

Tool	Integration
Dependabot	GitHub native PRs
Renovate	Multi-platform, configurable
Snyk	SCA + container + IaC
Grype	Anchore ecosystem
OWASP Dependency-Check	Jenkins, CLI

GitHub Dependabot config:

```
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: npm
    directory: "/"
    schedule:
      interval: daily
    open-pull-requests-limit: 10
    groups:
      production-dependencies:
        patterns: ["*"]
        exclude-patterns: ["@types/*"]
```

Syft + Grype pipeline:

```
syft packages dir:. -o cyclonedx-json > sbom.json
grype sbom:sbom.json --fail-on critical
```

Prioritize by **exploitability** (EPSS score), **reachability** (is vulnerable code path used?), and **severity**—not all CVEs are exploitable in your context.

51.5 50.5 Container and image scanning

Container images bundle OS packages and application dependencies—both need scanning.

Scanner	Focus
Trivy	OS + language deps, misconfig, secrets
Grype	CVE matching via SBOM
Clair	Layer-based analysis
Snyk Container	Integrated platform
Amazon ECR scanning	Native AWS

Trivy in GitHub Actions:

```
- name: Build image
  run: docker build -t myapp:${{ github.sha }} .

- name: Trivy scan
  uses: aquasecurity/trivy-action@0.24.0
  with:
    image-ref: myapp:${{ github.sha }}
    format: sarif
    output: trivy-results.sarif
```

```

severity: CRITICAL,HIGH
exit-code: 1

- name: Upload SARIF
  uses: github/codeql-action/upload-sarif@v3
  with:
    sarif_file: trivy-results.sarif

```

Scan **base images** regularly—pin to digest, not `:latest`. Rebuild golden images on CVE patches.

Distroless/minimal bases reduce attack surface and scan noise.

51.6 50.6 IaC and Kubernetes scanning

Tool	Targets
Checkov	Terraform, CloudFormation, K8s, Dockerfile
tfsec / Trivy config	Terraform
kube-score	Kubernetes best practices
Polaris	K8s policy validation
Kubescape	NSA/CIS benchmarks

Checkov example:

```

checkov -d terraform/ --framework terraform \
--check CKV_AWS_79,CKV_AWS_80 \
--soft-fail-on LOW

```

Common findings: public S3 buckets, overly permissive security groups, containers running as root, missing resource limits.

51.7 50.7 Unified vulnerability management

Centralize findings from SAST, DAST, SCA, container scans:

```

CI tools → SARIF / API → DefectDojo / GitHub Security / Snyk dashboard
                        |
                        +-- Prioritization
                        +-- SLA tracking
                        +-- Jira tickets

```

SARIF (Static Analysis Results Interchange Format) standardizes results across tools for GitHub Advanced Security integration.

Remediation SLAs (example):

Severity	Fix SLA
Critical (exploitable)	7 days
High	30 days
Medium	90 days
Low	Next quarter

51.8 50.8 Reducing noise

Technique	Benefit
Baseline accepted risk	Stop re-alerting known issues
Reachability analysis	Focus on used vulnerable code
EPSS prioritization	Exploit probability scoring
Consolidate tools	One SCA source of truth
Fix upstream	Patch base image vs per-app

Developer experience: surface findings in PR comments with fix guidance—not email-only reports ignored for weeks.

51.9 50.9 Chapter summary

- SAST analyzes source; DAST probes running apps; SCA tracks dependency CVEs; container scans cover image layers.
 - Integrate scans in CI with severity gates; export SARIF for unified tracking.
 - Combine tools for defense in depth; prioritize by exploitability and reachability.
 - Maintain base images, dependency bots, and remediation SLAs for sustainable security.
-

51.10 Lab 50.1 — Full scan pipeline

1. Add Semgrep, Trivy (filesystem + image), and Dependabot to a sample Node.js + Docker project.
 2. Introduce deliberate vulnerable dependency (`npm install lodash@4.17.20`); confirm SCA flags CVE.
 3. Fix via upgrade; verify clean scan.
-

51.11 Lab 50.2 — DAST against staging

1. Deploy sample app to staging with intentional missing security header.
 2. Run OWASP ZAP baseline; document finding.
 3. Add header middleware; re-scan to confirm resolution.
-

51.12 Review questions

1. Compare SAST and DAST—what can each detect that the other cannot?
 2. Why scan container images after build, not only application dependencies?
 3. What is SARIF, and why use it?
 4. How do EPSS and reachability improve SCA prioritization?
 5. Name three common Checkov findings in Terraform.
 6. What are typical false positive sources in SAST, and how handle them?
 7. Why should DAST run against staging, not production?
-

52 Chapter 51: Secrets Management — Vault, SOPS, External Secrets

DevOps Handbook — Part XI, Pages 1001–1020

52.1 51.1 The secrets problem

Applications need credentials—database passwords, API keys, TLS certificates, cloud IAM tokens. Storing them in Git, environment files on laptops, or plaintext Kubernetes Secrets violates basic security hygiene and compliance.

Requirements for secrets management:

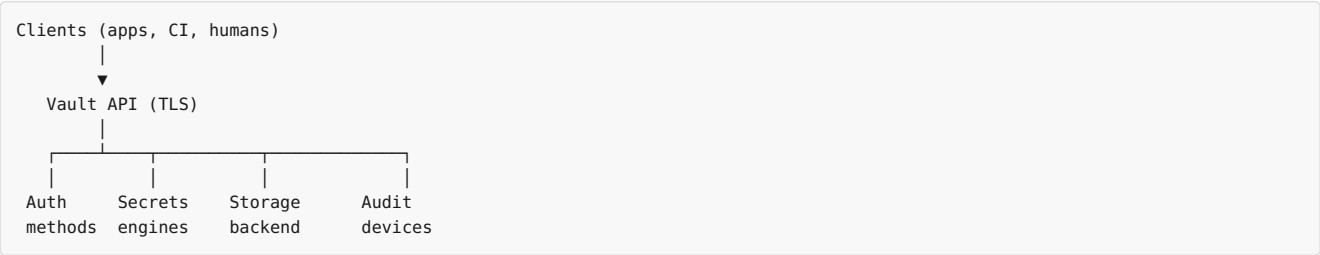
Requirement	Description
Encryption at rest	Secrets stored encrypted, not plaintext on disk
Encryption in transit	TLS for all secret API calls
Access control	Least privilege, audit who read what
Rotation	Periodic and emergency rotation without downtime
Dynamic secrets	Short-lived credentials generated on demand
Audit trail	Immutable log of access events

This chapter covers **HashiCorp Vault**, **Mozilla SOPS**, and **External Secrets Operator**—common patterns in cloud-native DevOps.

52.2 51.2 HashiCorp Vault architecture

Vault is a secrets engine and identity-aware broker.

Components:



Concept	Role
Seal/Unseal	Master key protects data; Shamir shards or auto-unseal (KMS)
Auth methods	Kubernetes, AWS IAM, OIDC, AppRole, userpass
Secrets engines	KV, database, PKI, AWS, transit encryption
Policies	HCL paths controlling read/write
Namespaces	Multi-tenancy (Enterprise)

Vault CLI basics:

```

export VAULT_ADDR=https://vault.example.com:8200
vault login -method=oidc

# Read static secret
vault kv get secret/data/myapp/database

# Write secret
vault kv put secret/myapp/api key=NEW_VALUE

# Generate dynamic DB creds (TTL 1h)
vault read database/creds/myapp-role

```

Dynamic database credentials: Vault creates temporary DB user, returns username/password, revokes on lease expiry—eliminating long-lived DB passwords in apps.

52.3 51.3 Vault on Kubernetes

Vault Agent Injector sidecar mounts secrets into pods:

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
  annotations:
    vault.hashicorp.com/agent-inject: "true"
    vault.hashicorp.com/role: "api"
    vault.hashicorp.com/agent-inject-secret-db: "secret/data/api/database"
    vault.hashicorp.com/agent-inject-template-db: |
      {{- with secret "secret/data/api/database" -}}
      DATABASE_URL=postgres://{{ .Data.data.username }}:{{ .Data.data.password }}@db:5432/app
      {{- end }}
spec:
  template:
    spec:
      serviceAccountName: api
      containers:
        - name: api
          image: myapp:latest

```

Kubernetes auth binds ServiceAccount → Vault role → policy.

Vault Secrets Operator (newer) syncs Vault secrets to Kubernetes Secret objects natively.

Operational concerns:

- **HA deployment** — Raft integrated storage or Consul backend
- **Auto-unseal** — AWS KMS, GCP CKMS, Azure Key Vault
- **Backup** — Snapshot Raft; test restore
- **Upgrade path** — Read release notes; staged rollout

52.4 51.4 Mozilla SOPS — Git-friendly encryption

SOPS (Secrets OperationS) encrypts YAML/JSON/env files **in Git**—only encrypted blobs committed; decryption keys from KMS/PGP/age.

Use case: team wants secrets versioned alongside config with PR review, without plaintext in repo.

Encrypted file example (secrets.enc.yaml):

```

db_password: ENC[AES256_GCM,data:xxx,iv:yyy,tag:zzz,type:str]
sops:
  kms:
    - arn: arn:aws:kms:us-east-1:123456789012:key/abc-123
      created_at: "2026-09-01T12:00:00Z"

```



```
enc: ...
lastmodified: "2026-09-01T12:00:00Z"
version: 3.8.1
```

Commands:

```
# Encrypt
sops --encrypt --kms arn:aws:kms:us-east-1:123:key/abc secrets.yaml > secrets.enc.yaml

# Edit in place (decrypt-edit-reencrypt)
sops secrets.enc.yaml

# Decrypt to stdout for deploy
sops --decrypt secrets.enc.yaml > /tmp/secrets.yaml
```

CI integration:

```
- name: Decrypt secrets
  env:
    AWS_REGION: us-east-1
  run: |
    sops --decrypt deploy/secrets.enc.yaml > deploy/secrets.yaml
    kubectl apply -f deploy/
```

age keys for teams without cloud KMS:

```
age-keygen -o key.txt
sops --encrypt --age age1xxx secrets.yaml > secrets.enc.yaml
```

SOPS tradeoffs: secrets in Git history remain encrypted but **key compromise** exposes all; rotate keys periodically.

52.5 51.5 External Secrets Operator (ESO)

External Secrets Operator syncs secrets from external providers into Kubernetes Secrets—bridging Vault, AWS Secrets Manager, GCP Secret Manager, Azure Key Vault, Parameter Store.

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: api-db-credentials
  namespace: production
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secrets-manager
    kind: ClusterSecretStore
  target:
    name: api-db-credentials
    creationPolicy: Owner
  data:
    - secretKey: DATABASE_URL
      remoteRef:
        key: production/api/database
        property: url
```

ClusterSecretStore for AWS:

```
apiVersion: external-secrets.io/v1beta1
kind: ClusterSecretStore
metadata:
  name: aws-secrets-manager
spec:
  provider:
    aws:
      service: SecretsManager
      region: us-east-1
```

```

auth:
  jwt:
    serviceAccountRef:
      name: external-secrets
      namespace: external-secrets

```

Benefits:

- No secrets in Git
- Central rotation in AWS/Vault propagates to K8s on refresh interval
- IRSA (IAM Roles for Service Accounts) for authentication—no static AWS keys

52.6 51.6 Cloud-native secret managers

Provider	Service	Integration
AWS	Secrets Manager, SSM Parameter Store	ESO, Lambda rotation
GCP	Secret Manager	ESO, GKE workload identity
Azure	Key Vault	ESO, managed identity

AWS Secrets Manager rotation Lambda template rotates RDS passwords automatically.

Compare Vault vs cloud-native:

Factor	Vault	Cloud SM
Multi-cloud	Yes	Per-cloud
Dynamic secrets	Rich engines	Limited
Operational load	Higher	Managed
Cost	Self-host + HCP option	Per-secret pricing

Many orgs: Vault for dynamic/complex; cloud SM for simple static secrets.

52.7 51.7 CI/CD secrets patterns

Anti-pattern	Better approach
Long-lived cloud keys in GitHub Secrets	OIDC federation to IAM role
Same secret all environments	Separate secrets per env
Manual rotation	Automated rotation + ESO refresh
Echo secrets in CI logs	Mask variables; use secret refs

GitHub OIDC → AWS (no static key):

```

permissions:
  id-token: write
steps:
  - uses: aws-actions/configure-aws-credentials@v4
    with:
      role-to-assume: arn:aws:iam::123456789012:role/GHA-Deploy
      aws-region: us-east-1
  - run: aws secretsmanager get-secret-value --secret-id prod/api

```

Short-lived tokens reduce blast radius of CI compromise.

52.8 51.8 Rotation and emergency response

Rotation checklist:

1. Generate new secret in manager
2. Dual-write period (app accepts old + new) if applicable
3. Update consumers via ESO refresh or redeploy
4. Revoke old secret after validation
5. Audit access logs for anomalies

Emergency: leaked secret → revoke immediately, rotate, scan Git history, review Vault/AWS audit logs for unauthorized access.

52.9 51.9 Chapter summary

- Vault provides dynamic secrets, fine-grained policies, and audit for enterprise secret management.
 - SOPS enables encrypted secrets in Git with KMS/age keys for reviewable configuration.
 - External Secrets Operator syncs cloud/Vault secrets into Kubernetes without plaintext in repos.
 - Prefer OIDC and short-lived credentials in CI; automate rotation and maintain audit trails.
-

52.10 Lab 51.1 — SOPS with age

1. Generate age key pair; encrypt a YAML file with DB credentials.
 2. Commit encrypted file to Git; verify plaintext never appears in history.
 3. Decrypt in CI script simulating deploy.
-

52.11 Lab 51.2 — External Secrets on Kubernetes

1. Store secret in AWS Secrets Manager (or LocalStack for local test).
 2. Install ESO; configure ClusterSecretStore and ExternalSecret.
 3. Mount synced K8s Secret in pod; verify app reads value.
-

52.12 Review questions

1. Compare static secrets in K8s Secrets vs Vault dynamic database credentials.
 2. When is SOPS appropriate vs External Secrets Operator?
 3. What is Vault seal/unseal, and why use auto-unseal with KMS?
 4. How does IRSA enable ESO to access AWS Secrets Manager without static keys?
 5. Describe a safe secret rotation procedure for a database password.
 6. Why use OIDC in GitHub Actions instead of storing AWS access keys?
 7. What audit capabilities should a secrets platform provide?
-

Continue: Chapter 52 — Supply Chain, SBOM, Sigstore, SLSA

53 Chapter 52: Supply Chain Security — SBOM, Sigstore, SLSA

DevOps Handbook — Part XI, Pages 1021–1040

53.1 52.1 Software supply chain threats

Modern software is assembled from thousands of components—open-source libraries, base images, CI plugins, Terraform modules. Attackers target the **supply chain** because compromising one dependency affects thousands of downstream users.

Notable incidents:

Incident	Mechanism
SolarWinds (2020)	Build system compromise, trojanized updates
Log4Shell (2021)	Ubiquitous vulnerable dependency
Codecov (2021)	CI script compromise exfiltrating secrets
npm package typosquatting	Malicious packages mimicking popular names
3CX (2023)	Signed malware in legitimate software update

Supply chain security verifies **provenance** (where artifacts came from), **integrity** (unchanged since build), and **trust** (who built and signed).

53.2 52.2 SBOM — Software Bill of Materials

An **SBOM** is a machine-readable inventory of components in a software artifact—like nutrition labels for code.

Format	Maintainer
CycloneDX	OWASP / ECMA standard
SPDX	Linux Foundation ISO standard

SBOM enables:

- Rapid CVE response (“are we affected by CVE-2024-XXXX?”)
- License compliance audits
- Vendor risk assessment
- Regulatory requirements (US EO 14028, EU CRA)

Generate SBOM with **Syft**:

```
# Filesystem
syft dir:. -o cyclonedx-json > sbom.json

# Container image
syft registry.example.com/myapp:v1.2.3 -o spdx-json > sbom.spdx.json

# Attach to image attestation
syft attest registry.example.com/myapp:v1.2.3 -o cyclonedx-json
```

CI integration:

```

- name: Generate SBOM
  uses: anchore/sbom-action@v0
  with:
    image: myapp:${{ github.sha }}
    format: cyclonedx-json
    output-file: sbom.cdx.json

- name: Upload SBOM artifact
  uses: actions/upload-artifact@v4
  with:
    name: sbom
    path: sbom.cdx.json

```

Store SBOMs alongside artifacts in registry (OCI referrer) or artifact repository—queryable when new CVEs emerge.

VEX (Vulnerability Exploitability eXchange) documents why listed CVEs are not exploitable in context—reduces false panic.

53.3 52.3 SLSA — supply chain levels

SLSA (Supply-chain Levels for Software Artifacts) defines progressive security levels for build pipelines.

Level	Requirements (summary)
SLSA 1	Provenance exists (build documented)
SLSA 2	Hosted build service, signed provenance
SLSA 3	Hardened build platform, non-falsifiable provenance
SLSA 4	Two-person review, reproducible builds

Provenance attestation (conceptual):

```

{
  "_type": "https://in-toto.io/Statement/v1",
  "subject": [{"name": "myapp:v1.2.3", "digest": {"sha256": "abc..."}},
  "predicateType": "https://slsa.dev/provenance/v1",
  "predicate": {
    "buildDefinition": {
      "buildType": "https://github.com/Attestations/GitHubActionsWorkflow@v1",
      "externalParameters": {
        "workflow": {"ref": "refs/heads/main", "repository": "https://github.com/org/myapp"}
      }
    },
    "runDetails": {
      "builder": {"id": "https://github.com/org/myapp/.github/workflows/build.yml@refs/heads/main"},
      "metadata": {"invocationId": "1234567890"}
    }
  }
}

```

GitHub Actions can generate SLSA provenance with **slsa-github-generator** workflow.

53.4 52.4 Sigstore and cosign

Sigstore provides free, automated code signing and verification—**cosign** signs container images and blobs; **Fulcio** issues short-lived certificates bound to OIDC identity; **Rekor** is transparency log.

Sign image after build:

```

# Keyless signing (OIDC identity from CI)
cosign sign registry.example.com/myapp:v1.2.3

```

```
# Verify
cosign verify registry.example.com/myapp:v1.2.3 \
  --certificate-identity-regexp="https://github.com/myorg/myapp/.github/workflows/.*" \
  --certificate-oidc-issuer="https://token.actions.githubusercontent.com"
```

GitHub Actions:

```
- name: Install cosign
  uses: sigstore/cosign-installer@v3

- name: Sign image
  run: cosign sign --yes registry.example.com/myapp:${{ github.sha }}
  env:
    COSIGN_EXPERIMENTAL: "true"
```

Kubernetes admission — Verify signatures before deploy:

```
# Policy Controller / Kyverno verifyImages
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: verify-signed-images
spec:
  validationFailureAction: Enforce
  rules:
    - name: verify-cosign-signature
      match:
        any:
          - resources:
              kinds: [Pod]
      verifyImages:
        - imageReferences:
            - "registry.example.com/myapp:*"
          attestors:
            - count: 1
              entries:
                - keyless:
                    subject: "https://github.com/myorg/myapp/.github/workflows/.*"
                    issuer: "https://token.actions.githubusercontent.com"
```

Only signed images from trusted CI workflows deploy to production.

53.5 52.5 Dependency and build hardening

Control	Implementation
Pin dependencies	Lock files, digest-pinned base images
Verify checksums	npm/yarn integrity, Go sum
Private registry proxy	Pull-through cache, vulnerability gate
Minimal CI permissions	permissions: contents: read default
Signed commits/tags	GPG or Sigstore gitsign
Branch protection	Required reviews, status checks
Hermetic builds	Reproducible, no network in compile (Bazel, Nix)

Dependabot/Renovate with grouped updates and CI validation before merge.

Block unpinned actions in CI:

```
# Policy: actions must use SHA pin
# Semgrep or custom linter on workflow files
uses: actions/checkout@b4ffde65f46336ab88eb53be808477a3936bae11 # v4.1.1
```

53.6 52.6 in-toto and attestations

in-toto framework chains attestations through supply chain steps: design → source → build → test → deploy.

Each step produces signed metadata; final deployment verifies entire chain.

GitHub Artifact Attestations (2024+) attach build provenance to artifacts in GitHub UI and API.

OCI **referrers** attach SBOM and signatures to container images in registry—Cosign, ORAS CLI.

```
oras attach registry.example.com/myapp:v1.2.3 \
  --artifact-type application/vnd.cyclonedx+json \
  --file sbom.json
```

53.7 52.7 Organizational supply chain program

1. **Inventory** — SBOM for every production artifact
2. **Sign** — cosign keyless in CI for all release images
3. **Verify** — Admission policy in K8s clusters
4. **Monitor** — Alert on new CVEs affecting SBOM components
5. **Respond** — Playbook linking CVE → affected services via SBOM query
6. **Vendor** — Require SBOM + SLSA level from third-party software

Metrics:

- % artifacts with SBOM + signature
- Mean time to identify affected services for new CVE
- % deployments blocked by unsigned image policy (should be 0 in prod bypass)

53.8 52.8 Chapter summary

- SBOMs inventory software components for CVE response and compliance.
- SLSA defines progressive build hardening levels with provenance attestations.
- Sigstore/cosign enables keyless signing and Kubernetes verification policies.
- Pin dependencies, harden CI, and maintain organizational SBOM + signing program.

53.9 Lab 52.1 — SBOM generation and CVE lookup

1. Build Docker image for sample app; generate CycloneDX SBOM with Syft.
2. Scan SBOM with Gype; document findings.
3. Upgrade vulnerable package; regenerate SBOM; confirm clean scan.

53.10 Lab 52.2 — Sign and verify with cosign

1. Build and push image to registry (GHCR or local).
 2. Sign with cosign keyless in GitHub Actions (or local key).
 3. Configure Kyverno or conftest policy rejecting unsigned images.
 4. Attempt deploy unsigned image; confirm rejection.
-

53.11 Review questions

1. What is an SBOM, and why generate it for every release?
2. Compare CycloneDX and SPDX formats.
3. Explain SLSA Level 2 vs Level 3 requirements at high level.
4. How does cosign keyless signing bind identity to CI workflow?
5. What is VEX, and when use it?
6. Name three supply chain attacks and their mechanisms.
7. How do Kubernetes admission policies enforce signed images?

Continue: Chapter 53 — Runtime Security

54 Chapter 53: Runtime Security — Falco and OPA Gatekeeper

DevOps Handbook — Part XI, Pages 1041–1060

54.1 53.1 Defense beyond the pipeline

Shift-left security catches vulnerabilities before deploy—but **runtime** threats remain: zero-days, insider abuse, container escapes, cryptomining, lateral movement after breach. **Runtime security** detects and responds to malicious behavior in executing systems.

Two complementary approaches:

Approach	Mechanism	Example tools
Threat detection	Monitor syscalls, K8s audit events	Falco, Sysdig, Aqua
Policy enforcement	Block non-compliant resources at admission	OPA Gatekeeper, Kyverno, Pod Security Admission

Detection alerts; admission prevents. Use both.

54.2 53.2 Falco — cloud-native runtime detection

Falco (CNCF) detects unexpected behavior via **kernel events** (syscalls via eBPF or kernel module) and **Kubernetes audit logs**.

Detection examples:

- Shell spawned in container
- Unexpected outbound connection
- Sensitive file read (`/etc/shadow`)
- Privilege escalation attempt
- Write below `/etc` or `/usr/bin`

Architecture:



Install on Kubernetes (Helm):

```
helm repo add falcosecurity https://falcosecurity.github.io/charts
helm install falco falcosecurity/falco \
  --namespace falco --create-namespace \
  --set driver.kind=ebpf
```

Custom rule:

```
# /etc/falco/rules.d/custom-rules.yaml
- rule: Terminal shell in container
  desc: Detect shell spawned in running container
  condition: >
    spawned_process and container and
    proc.name in (bash, sh, zsh) and
    not proc.pname in (bash, sh, docker-init, tini)
```

```

output: >
  Shell spawned in container
  (user=%user.name container=%container.name
  image=%container.image.repository cmdline=%proc.cmdline)
priority: WARNING
tags: [container, shell, mitre_execution]

- rule: Outbound connection to suspicious port
desc: Unexpected outbound connection on non-standard port
condition: >
  outbound and container and
  fd.sport != 443 and fd.sport != 80 and
  not fd.sip in (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16)
output: Suspicious outbound (container=%container.name connection=%fd.name)
priority: NOTICE

```

Tune rules to reduce false positives—baseline normal behavior in staging first.

Response integration via **Falcosidekick**:

```

config:
  slack:
    webhookurl: "https://hooks.slack.com/..."
    minimumpriority: warning
  pagerduty:
    routingkey: SECRET
    minimumpriority: error

```

54.3 53.3 Falco response actions

Detection alone is insufficient—define response playbooks:

Alert	Response
Shell in prod container	Kill pod, page on-call, capture forensics
Cryptomining outbound	NetworkPolicy block, isolate node
Sensitive file access	Audit user/service account, revoke credentials

Falco Response Engine (evolving) can trigger Kubernetes actions (delete pod, label node) via webhooks—use cautiously with human approval for destructive actions.

Combine with **NetworkPolicies** and **seccomp/AppArmor** profiles to limit blast radius when detection triggers.

54.4 53.4 OPA and Gatekeeper

Open Policy Agent (OPA) evaluates policy decisions via Rego language. **Gatekeeper** is OPA's Kubernetes admission controller—validates and mutates resources at create/update.

```

kubectrl apply → API Server → Gatekeeper (OPA) → Allow/Deny
                        |
                        ConstraintTemplate
                        Constraint

```

Install Gatekeeper:

```
kubectrl apply -f https://raw.githubusercontent.com/open-policy-agent/gatekeeper/v3.14.0/deploy/gatekeeper.yaml
```

ConstraintTemplate (require labels):

```
apiVersion: templates.gatekeeper.sh/v1
```

```

kind: ConstraintTemplate
metadata:
  name: k8srequiredlabels
spec:
  crd:
    spec:
      names:
        kind: K8sRequiredLabels
      validation:
        openAPIV3Schema:
          type: object
          properties:
            labels:
              type: array
              items:
                type: string
      targets:
        - target: admission.k8s.gatekeeper.sh
          rego: |
            package k8srequiredlabels
            violation[{"msg": msg}] {
              required := input.parameters.labels
              provided := input.review.object.metadata.labels
              missing := required[_]
              not provided[missing]
              msg := sprintf("Missing required label: %v", [missing])
            }

```

Constraint (enforce):

```

apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sRequiredLabels
metadata:
  name: require-team-label
spec:
  match:
    kinds:
      - apiGroups: [""]
        kinds: ["Pod"]
      namespaces: ["production"]
    parameters:
      labels: ["team", "app"]

```

Block privileged containers:

```

package k8spsprivileged

violation[{"msg": msg}] {
  input.review.object.spec.containers[_].securityContext.privileged
  msg := "Privileged containers are not allowed"
}

```

Gatekeeper **mutation** (assign labels, defaults) via AssignMetadata, Assign mutations.

54.5 53.5 Policy libraries and Pod Security

Use community policy libraries:

- **Gatekeeper library** — CIS benchmarks, PSA equivalents
- **Kyverno policies** — Alternative YAML-native policy engine
- **Pod Security Admission (PSA)** — Built-in K8s 1.25+ (`restricted` , `baseline` , `privileged`)

Namespace labels for PSA:

```

apiVersion: v1
kind: Namespace
metadata:
  name: production
  labels:

```

```
pod-security.kubernetes.io/enforce: restricted
pod-security.kubernetes.io/enforce-version: latest
```

Compare:

Engine	Language	Strength
Gatekeeper/OPA	Rego	Flexible, complex policies
Kyverno	YAML	K8s-native, easier onboarding
PSA	Built-in	Baseline pod hardening

Many orgs: PSA for baseline + Gatekeeper/Kyverno for org-specific rules.

54.6 53.6 Runtime security program

Layered Kubernetes security:

1. Pod Security Admission (restricted)
2. Gatekeeper/Kyverno admission policies
3. NetworkPolicies (default deny)
4. Falco runtime detection
5. Audit logging → SIEM
6. Regular policy review and game days

Metrics:

- Admission deny rate (misconfigurations vs attacks)
- Falco alert volume and true positive rate
- Time to remediate policy violations

54.7 53.7 Chapter summary

- Falco detects anomalous runtime behavior via kernel events and K8s audit with customizable rules.
- OPA Gatekeeper enforces Rego policies at Kubernetes admission—preventing non-compliant workloads.
- Combine admission control (prevent) with runtime detection (detect/respond).
- Use PSA, NetworkPolicies, and policy libraries for defense in depth.

54.8 Lab 53.1 — Falco shell detection

1. Install Falco on test cluster (eBPF driver).
2. Deploy nginx pod; exec bash inside container.
3. Confirm Falco alert in logs or Falcosidekick webhook.
4. Add exception rule for approved debug namespace.

54.9 Lab 53.2 — Gatekeeper privileged block

1. Install Gatekeeper; apply ConstraintTemplate blocking privileged pods.
2. Attempt `kubectl run` with `--privileged`; confirm denial.
3. Deploy compliant pod; confirm success.

54.10 Review questions

1. Distinguish runtime detection from admission control with examples.
2. How does Falco obtain visibility into container behavior?
3. What are ConstraintTemplate and Constraint in Gatekeeper?
4. Write Rego logic conceptually to deny containers running as root.
5. Compare OPA Gatekeeper and Kyverno for Kubernetes policy.
6. How do Pod Security Admission levels differ?
7. What tuning is needed before Falco alerts page on-call?

Continue: Chapter 54 — Compliance Automation

55 Chapter 54: Compliance Automation and Audit Trails

DevOps Handbook — Part XI, Pages 1061–1080

55.1 54.1 Compliance in continuous delivery

Regulated industries—finance, healthcare, government—must demonstrate **controls** are designed and operating effectively: SOC 2, ISO 27001, PCI DSS, HIPAA, FedRAMP, GDPR. Traditional compliance meant annual audits with manual evidence collection. **Compliance automation** embeds controls in pipelines and infrastructure, producing **continuous evidence**.

DevOps alignment:

Traditional compliance	Automated compliance
Annual spreadsheet audit	Continuous control monitoring
Manual change tickets	Git history + approved PRs
Screenshot of firewall	IaC + drift detection
Quarterly access review	IAM analytics + automated reports

Goal: **audit-ready by default**—evidence generated as a side effect of normal operations.

55.2 54.2 Control frameworks mapping

Map technical controls to framework requirements:

Framework domain	Technical implementation
Access control	IAM, RBAC, MFA, SSO
Change management	Git PR approval, CI gates, deployment logs
Encryption	TLS, KMS, encrypted volumes
Logging/monitoring	Centralized logs, immutable audit trails
Vulnerability management	SCA/SAST gates, patch SLAs
Backup/DR	Automated backups, tested restore
Incident response	On-call, postmortems, retention

SOC 2 Trust Service Criteria (Security, Availability, Processing Integrity, Confidentiality, Privacy) map to pipeline and platform controls. **PCI DSS** requires segmentation, logging, vulnerability scanning for cardholder data environments.

Maintain a **control matrix** spreadsheet/database linking control ID → implementation → evidence source → owner.

55.3 54.3 Audit trails — what to log

Immutable audit logs for:

Event type	Examples
Authentication	Login, MFA, failed attempts
Authorization	IAM policy change, role assignment
Data access	Admin read of customer data
Infrastructure change	Terraform apply, kubectl apply
Deployment	Who deployed what commit when
Secret access	Vault/AWS Secrets Manager read
Security findings	Scan results, waiver approvals

Kubernetes audit policy:

```
apiVersion: audit.k8s.io/v1
kind: Policy
rules:
- level: Metadata
  omitStages: [RequestReceived]
- level: RequestResponse
  verbs: ["create", "update", "patch", "delete"]
  resources:
    - group: ""
      resources: ["secrets", "configmaps"]
- level: RequestResponse
  verbs: ["create", "update", "patch", "delete"]
  resources:
    - group: "rbac.authorization.k8s.io"
      resources: ["roles", "rolebindings", "clusterroles", "clusterrolebindings"]
```

Ship audit logs to SIEM (Elastic Security, Splunk) with **WORM** storage (S3 Object Lock) for tamper evidence.

CloudTrail (AWS) / Cloud Audit Logs (GCP) / Activity Log (Azure)—enable organization trail, log integrity validation.

55.4 54.4 Policy as Code for compliance

Encode compliance rules as executable policy:

Checkov CIS benchmarks:

```
checkov -d terraform/ --framework terraform --check CKV_AWS_* \
  --download-external-modules true \
  --output sarif --output-file-path checkov.sarif
```

OPA/Conftest custom org policies:

```
# require S3 bucket encryption
package terraform.s3

deny[msg] {
  resource := input.resource.aws_s3_bucket[name]
  not input.resource.aws_s3_bucket_server_side_encryption_configuration[name]
  msg := sprintf("S3 bucket %s missing encryption config", [name])
}
```

Cloud Custodian — AWS/GCP/Azure compliance automation:

```
policies:
- name: s3-require-encryption
  resource: aws.s3
  filters:
    - type: bucket-encryption
      state: false
  actions:
```

```
- type: notify
- type: mark-for-op
  op: encrypt
```

Run in CI (prevent) and scheduled scans (detect drift).

55.5 54.5 Continuous control monitoring

CCM tools aggregate evidence:

Tool	Function
Vanta	Automated SOC 2 evidence
Drata	Continuous compliance monitoring
Secureframe	Control tracking
OpenSCAP	NIST/CIS scanning
InSpec	Compliance profiles as code

Custom dashboard example metrics:

- % production resources passing Checkov CIS
- Mean patch time for critical CVEs
- % employees with MFA enabled
- PR approval compliance rate (no self-merge to main)

Automated evidence collection:

```
# Weekly compliance report job
- name: Collect evidence
  run: |
    checkov -d infra/ -o json > evidence/checkov-$(date +%F).json
    aws iam generate-credential-report
    aws iam get-credential-report --query Content --output text | base64 -d > evidence/iam-$(date +%F).csv
    aws s3 cp evidence/ s3://compliance-evidence-bucket/$(date +%Y/%m/%d)/ --recursive
```

55.6 54.6 Change management automation

Auditors ask: “How do you control production changes?”

Automated answer:

1. All changes via **Git PR** with required reviewers
2. Branch protection: status checks (CI, security scans) must pass
3. Deployment linked to commit SHA in CI logs and GitHub Deployments API
4. **Immutable infrastructure** — no SSH manual edits
5. Emergency break-glass procedure documented with retroactive PR

Terraform Cloud / Atlantis audit trail:

```
PR #1234 → plan posted → approved → apply → state updated → audit log entry
```

Separation of duties: developer proposes (PR), different person approves, CI applies—SOD for SOC 2.

55.7 54.7 Data privacy and retention

GDPR/CCPA requirements:

- **Data inventory** — Map PII flows (SBOM-like for data)
- **Retention policies** — ILM on logs (Chapter 45)
- **Right to erasure** — Procedures in runbooks
- **DPIA** — Data Protection Impact Assessment for new features

Log redaction at ingest—never store full credit card numbers in application logs.

Retention schedule:

Data type	Retention	Rationale
Application logs	90 days	Operations
Audit logs	7 years	Regulatory
CI artifacts	1 year	Reproducibility
Backups	Per RPO/RTO policy	DR

55.8 54.8 Audit preparation workflow

Before external audit:

1. **Control matrix review** — Update implementations
2. **Evidence spot check** — Sample 30 days of deployment logs, access reviews
3. **Exception documentation** — Risk acceptance for waived findings
4. **Personnel interviews** — Engineers explain actual process matches documentation
5. **Pen test / vuln scan reports** — Current within policy window

During audit: provide read-only access to evidence repository—not production systems.

Post-audit: remediate findings, update automation to prevent recurrence.

55.9 54.9 Chapter summary

- Compliance automation maps framework controls to pipeline, IaC, and platform implementations.
- Immutable audit trails (K8s audit, CloudTrail, Git, Vault logs) provide continuous evidence.
- Policy-as-code (Checkov, OPA, Cloud Custodian) prevents and detects drift.
- CCM platforms and scheduled evidence jobs reduce audit scramble; maintain control matrix ownership.

55.10 Lab 54.1 — Kubernetes audit logging

1. Enable audit policy on test cluster logging RequestResponse for Secrets and RBAC.
2. Create and delete a Secret; query audit logs for event chain.
3. Forward logs to Elasticsearch; create Kibana dashboard of admin actions.

55.11 Lab 54.2 — Compliance evidence pipeline

1. Run Checkov against Terraform sample with intentional misconfigurations.

2. Export SARIF/JSON to `evidence/` directory with timestamp.
 3. Document control matrix entry linking CIS benchmark to Checkov check ID and evidence file.
-

55.12 Review questions

1. How does Git-based change management satisfy audit change control requirements?
 2. What events must Kubernetes audit logs capture for security compliance?
 3. Compare continuous compliance monitoring vs annual audit preparation.
 4. What is separation of duties in CI/CD, and why implement it?
 5. How do ILM policies support GDPR retention requirements?
 6. Name three tools for policy-as-code compliance scanning.
 7. What evidence would you provide for a “vulnerability management” control?
-

Continue: Chapter 55 — Platform Engineering

56 Chapter 55: Platform Engineering and Internal Developer Platforms

DevOps Handbook — Part XII, Pages 1081-1100

56.1 55.1 From DevOps to platform engineering

As organizations scale, each product team rebuilding CI templates, Kubernetes manifests, and observability stacks creates **cognitive load** and inconsistency. **Platform engineering** builds an **Internal Developer Platform (IDP)**—a curated set of tools, templates, and self-service capabilities that enable developers to ship software efficiently without becoming infrastructure experts.

Platform team mission (CNCF definition):

Treat platform as a product; optimize for developer experience and organizational outcomes.

Platform engineering vs related roles:

Role	Focus
DevOps	Culture + automation bridging dev and ops
SRE	Reliability, SLOs, toil reduction
Platform engineering	Self-service golden paths, IDP product
Infrastructure team	Raw compute, network, storage

Platform teams succeed when developers **choose** the platform because it accelerates work—not because mandates force compliance.

56.2 55.2 Internal Developer Platform components

Typical IDP layers:

Developer Portal (Backstage, Cortex, Port)			
Service catalog, docs, scaffolder, scorecards			
Golden paths: deploy web app, add cron job, expose API			
CI/CD templates	Runtime (K8s/EKS)	Observability stack	Security scanning
Infrastructure: Terraform modules, networking, IAM			

Key capabilities:

Capability	Purpose
Service catalog	Discoverable services, owners, dependencies
Software templates	Cookie-cutter repo + CI + infra
Environment provisioning	Dev/staging/prod namespaces or accounts
Deployment abstraction	GitOps, Helm charts, deploy buttons

Secrets integration	Vault/ESO wired by default
Observability bundle	Dashboards, alerts pre-configured
Documentation	TechDocs, runbooks linked from portal

56.3 55.3 Backstage — reference IDP portal

Backstage (CNCF, Spotify-origin) is the most adopted open-source developer portal.

Features:

- **Software Catalog** — YAML entity descriptors (`catalog-info.yaml`)
- **Software Templates (Scaffolder)** — Create repos from templates
- **TechDocs** — Docs-as-code in repo
- **Plugins** — Kubernetes, Argo CD, Grafana, CI visibility

Catalog entity:

```
# catalog-info.yaml
apiVersion: backstage.io/v1alpha1
kind: Component
metadata:
  name: order-api
  description: Order processing microservice
  annotations:
    github.com/project-slug: myorg/order-api
    backstage.io/techdocs-ref: dir:.
  tags:
    - python
    - kubernetes
spec:
  type: service
  lifecycle: production
  owner: team-checkout
  system: commerce
  dependsOn:
    - resource:default/postgres-orders
  providesApis:
    - order-api-rest
```

Scaffolder template (simplified):

```
apiVersion: scaffolder.backstage.io/v1beta3
kind: Template
metadata:
  name: python-microservice
  title: Python Microservice on EKS
spec:
  parameters:
    - title: Service info
      required: [name, owner]
      properties:
        name:
          type: string
        owner:
          type: string
          ui:field: OwnerPicker
  steps:
    - id: fetch
      action: fetch:template
      input:
        url: ./skeleton
        values:
          name: ${ parameters.name }
          owner: ${ parameters.owner }
    - id: publish
      action: publish:github
      input:
        repoUrl: github.com?repo=${ parameters.name }&owner=myorg
```

```
- id: register
  action: catalog:register
  input:
    repoContentsUrl: ${ steps.publish.output.repoContentsUrl }
```

56.4 55.4 Golden paths and paved roads

Golden path — Opinionated, supported workflow for common tasks:

“Create a REST API → get CI, EKS deployment, Prometheus metrics, and security scans automatically.”

Paved road vs unpaved road:

Paved (golden)	Unpaved (escape hatch)
Supported by platform team	Team owns fully
Fast onboarding	Flexibility for edge cases
Security/compliance built-in	Custom review required
SLAs on platform uptime	Best effort

Document escape hatches—forcing all workloads through inadequate abstractions drives shadow IT.

Measure golden path adoption: % new services created via scaffolder vs manual.

56.5 55.5 Platform as a product

Apply product management to platform:

Product practice	Platform application
User research	Developer interviews, surveys
Roadmap	Quarterly platform priorities
Metrics	DORA, time-to-first-deploy, CSAT
SLAs	IDP availability, template freshness
Support	#platform-support Slack, office hours
Deprecation policy	Notice before breaking template changes

Team Topologies (Part I): platform team is an **enabling team** or **platform team** interacting via **X-as-a-Service** with stream-aligned product teams.

Avoid **Ivory tower platform**—embed platform engineers in product teams periodically; consume your own golden paths (“eat your own dog food”).

56.6 55.6 Infrastructure abstractions

Platform hides complexity behind APIs:

Raw infrastructure	Platform abstraction
Terraform modules	“Create environment” button
Kubernetes YAML	Helm chart with values

IAM policies	Pre-approved roles per service type
Network policies	Automatic based on service catalog

Humanitec, Crossplane, Kratix — platform orchestration patterns composing infra resources from service definitions.

Crossplane Composite Resource:

```
apiVersion: platform.example.com/v1alpha1
kind: XServiceEnvironment
metadata:
  name: order-api-staging
spec:
  serviceName: order-api
  environment: staging
  region: us-east-1
  # Platform controller provisions RDS, EKS namespace, secrets, DNS
```

Developers declare intent; platform controllers reconcile infrastructure.

56.7 55.7 Developer experience metrics

Metric	Meaning
Time to first PR	New hire productivity
Time to production	Golden path efficiency
Template usage rate	Platform adoption
Developer satisfaction (CSAT)	Qualitative health
Toil tickets to platform	Gaps in self-service
Cognitive load survey	Team Topologies assessment

Improve metrics iteratively—platform backlog prioritized by developer pain, not loudest stakeholder.

56.8 55.8 Anti-patterns

Anti-pattern	Consequence
Platform without users	Wasted investment
Mandatory only, no DX	Shadow infrastructure
Endless customization	Unmaintainable templates
No versioning	Breaking all consumers at once
Ops team renamed “platform”	No product mindset change
Ignoring security/compliance	Golden path bypassed for speed

56.9 55.9 Chapter summary

- Platform engineering delivers an IDP with catalog, templates, and self-service golden paths.
- Backstage exemplifies developer portals integrating CI, runtime, docs, and ownership.

- Treat platform as a product with metrics, SLAs, and paved roads plus documented escape hatches.
 - Infrastructure abstractions (Crossplane, GitOps) hide complexity while enforcing standards.
-

56.10 Lab 55.1 — Backstage software catalog

1. Deploy Backstage locally or via demo instance.
 2. Register three sample services with `catalog-info.yaml`.
 3. Enable TechDocs plugin for one service; view rendered docs.
-

56.11 Lab 55.2 — Golden path template

1. Create scaffolder template: Node.js app + Dockerfile + GitHub Actions CI.
 2. Scaffold new service; verify repo contains working pipeline.
 3. Document golden path README with onboarding steps (< 30 min to first deploy).
-

56.12 Review questions

1. How does platform engineering differ from traditional DevOps team?
 2. What components belong in an Internal Developer Platform?
 3. Explain golden paths and why offer unpaved road escape hatches.
 4. What Backstage features support service discovery and documentation?
 5. How should platform teams measure success?
 6. What is Team Topologies' view of platform teams?
 7. Name two anti-patterns in platform engineering initiatives.
-

Continue: Chapter 56 — Multi-Cloud and Hybrid Patterns

57 Chapter 56: Multi-Cloud and Hybrid Patterns

DevOps Handbook — Part XII, Pages 1101–1115

57.1 56.1 Why multi-cloud and hybrid exist

Organizations rarely start with a single cloud strategy. **Multi-cloud** uses two or more public clouds (AWS + GCP + Azure). **Hybrid cloud** combines on-premises data centers with public cloud. Drivers include:

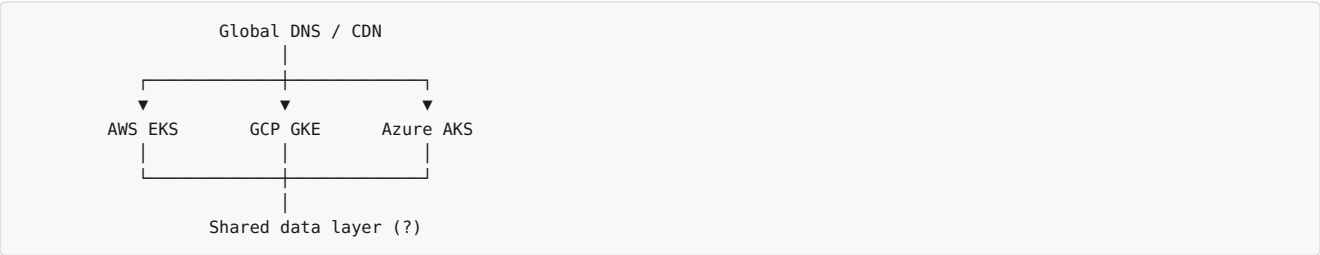
Driver	Example
Acquisition	Merged companies on different clouds
Regulatory	Data residency requires specific regions/providers
Vendor negotiation	Avoid lock-in for pricing leverage
Best-of-breed	GCP for ML, AWS for mature managed services
Legacy	Mainframe/on-prem cannot migrate quickly
Disaster recovery	Secondary region on different provider

Multi-cloud is **hard**—operational complexity often exceeds benefits without deliberate architecture. Prefer **single cloud + multi-region** unless a clear requirement forces multi-cloud.

57.2 56.2 Architecture patterns

57.2.1 Active-active multi-cloud

Traffic served from multiple clouds simultaneously—highest complexity:



Data consistency across clouds is the hard problem—avoid unless necessary.

57.2.2 Active-passive DR

Primary on AWS; GCP/Azure standby for failover:

- Replicate data asynchronously (RDS cross-region, object replication)
- DNS failover (Route 53, Cloud DNS health checks)
- Regular DR drills (Chapter 48)

57.2.3 Hybrid edge

On-prem core systems + cloud burst for scale:

On-prem:	ERP, legacy DB
Cloud:	Web tier, analytics, ML training
Link:	Direct Connect / ExpressRoute / VPN

57.2.4 Cloud-adjacent services

Keep sensitive data on-prem; process in cloud via encrypted links—**data gravity** pattern.

57.3 56.3 Portability strategies

Layer	Portability approach
Application	Containers, 12-factor apps, standard APIs
Orchestration	Kubernetes (EKS, GKE, AKS similar but not identical)
IaC	Terraform/Pulumi with provider modules
CI/CD	Platform-agnostic pipelines (GitHub Actions, GitLab)
Observability	OpenTelemetry, Prometheus, Grafana
Identity	OIDC/SAML federation; avoid cloud-specific auth in app
Data	Hardest—managed DB services differ significantly

Abstraction cost: Every portability layer adds indirection. Abstract **interfaces**, not every service feature.

Terraform multi-provider example:

```
# modules/kubernetes-cluster - cloud-agnostic interface
variable "cluster_name" {}
variable "node_count" {}
variable "region" {}

# AWS implementation
module "eks" {
  source = "../aws-eks"
  count  = var.cloud == "aws" ? 1 : 0
  # ...
}

# GCP implementation
module "gke" {
  source = "../gcp-gke"
  count  = var.cloud == "gcp" ? 1 : 0
  # ...
}
```

57.4 56.4 Networking across clouds and on-prem

Hybrid connectivity options:

Connection	Bandwidth	Latency	Use case
VPN	Moderate	Higher	Dev/test, small workloads
Direct Connect / ExpressRoute / Cloud Interconnect	High	Low	Production hybrid
SD-WAN	Optimized routing	Variable	Multi-site enterprises

Design considerations:

- **IP addressing** — Non-overlapping CIDR plans across clouds and on-prem
- **DNS** — Split-horizon or unified private zones
- **Firewall rules** — Centralized policy management

- **Latency** — Cross-cloud synchronous calls degrade UX

Service mesh (Istio multi-cluster) spans clusters across clouds—complex operational overhead.

57.5 56.5 GitOps and multi-cluster management

GitOps (Argo CD, Flux) scales to multi-cluster:

```
# ApplicationSet – deploy to all clusters
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: api-all-clusters
spec:
  generators:
    - clusters:
        selector:
          matchLabels:
            env: production
  template:
    metadata:
      name: 'api-{{name}}'
    spec:
      project: default
      source:
        repoURL: https://github.com/myorg/k8s-manifests
        path: apps/api/overlays/{{metadata.labels.cloud}}
      destination:
        server: '{{server}}'
        namespace: production
```

Cluster API provisions clusters consistently across providers.

Rancher, Anthos, Azure Arc — multi-cluster management planes with varying lock-in.

57.6 56.6 Data and state in multi-cloud

Pattern	Description
Single primary DB	One cloud owns data; others read via API
Async replication	Cross-cloud DB replication (lag, conflict resolution)
Per-cloud data	Regional data stays local; aggregate via analytics pipeline
Object storage replication	S3 CRR, GCS transfer, Azure Blob replication

Avoid dual-write across clouds without strong consistency model—leads to split-brain.

Event-driven sync:

Cloud A DB change → Kafka/EventBridge → Cloud B consumer updates read replica

57.7 56.7 Operational model

Challenge	Mitigation
Different IAM models	Central identity (Okta) + cloud role federation
Skill fragmentation	Platform golden paths per cloud + shared K8s layer

Cost visibility	FinOps tagging standards (Chapter 57)
Incident response	Unified on-call, cross-cloud runbooks
Compliance	Consistent policy-as-code adapted per provider

Cloud center of excellence (CCoE) — Governance without blocking teams: standards, modules, training—not approval for every resource.

57.8 56.8 When NOT to multi-cloud

Avoid multi-cloud when:

- Single region cloud meets all requirements
- Team lacks operational maturity for one cloud
- Motivation is vague “lock-in fear” without cost/benefit analysis
- Application is tightly coupled to one cloud’s proprietary services (DynamoDB, BigQuery ML)

Portable where it matters (compute, containers); **embrace managed services** where velocity wins.

57.9 56.9 Chapter summary

- Multi-cloud and hybrid architectures address DR, compliance, legacy, and vendor strategy—at operational cost.
 - Portability via containers, Kubernetes, Terraform, and OpenTelemetry reduces lock-in but adds abstraction.
 - GitOps and cluster management tools scale deployments across cloud boundaries.
 - Prefer single-cloud multi-region unless requirements clearly mandate multi-cloud; treat data as hardest portability layer.
-

57.10 Lab 56.1 — Terraform multi-provider module

1. Create Terraform module with `cloud` variable selecting AWS or GCP backend.
 2. Provision test cluster on one provider; switch variable; plan second (destroy first if budget limited).
 3. Document interface variables shared across implementations.
-

57.11 Lab 56.2 — Argo CD ApplicationSet

1. Register two clusters (or two namespaces simulating clusters) in Argo CD.
 2. Deploy ApplicationSet syncing same app to both.
 3. Update Git manifest; verify both destinations sync.
-

57.12 Review questions

1. Distinguish multi-cloud, hybrid cloud, and multi-region single cloud.
2. Why is data the hardest layer to make portable across clouds?
3. Name three legitimate business drivers for multi-cloud architecture.
4. How does GitOps help manage multi-cluster deployments?
5. What networking options connect on-premises to public cloud?
6. When should an organization avoid multi-cloud strategy?

7. What role does a Cloud Center of Excellence play?

Continue: Chapter 57 — FinOps

58 Chapter 57: FinOps and Cost-Aware Engineering

DevOps Handbook — Part XII, Pages 1116–1130

58.1 57.1 Cloud cost as engineering concern

Cloud’s pay-as-you-go model shifts infrastructure from CapEx (predictable depreciation) to OpEx (variable, opaque bills). Without discipline, costs scale faster than revenue—the **FinOps** discipline brings **financial accountability** to cloud spending through collaboration between engineering, finance, and business.

FinOps Foundation principles:

- 1. **Teams need to collaborate** — Finance, engineering, leadership share ownership
- 2. **Everyone takes ownership** — Engineers see and optimize their spend
- 3. **Reports should be accessible and timely** — Near real-time visibility
- 4. **Decisions are driven by business value** — Cost vs performance tradeoffs explicit
- 5. **Take advantage of the variable cost model** — Scale down, spot instances, right-size

FinOps is a **continuous cycle**: Inform → Optimize → Operate.

58.2 57.2 Cost visibility and allocation

You cannot optimize what you cannot attribute.

Tagging strategy (mandatory tags enforced by policy):

Tag	Purpose
team	Cost center owner
service	Application attribution
environment	dev/staging/prod
cost-center	Finance chargeback
project	Initiative tracking

AWS SCP / Tag Policies enforce tags at organization level. Kubernetes labels mirror tags for in-cluster cost tools.

Tools:

Tool	Scope
AWS Cost Explorer / CUR	AWS detailed billing
GCP Billing Export + BigQuery	GCP analysis
Azure Cost Management	Azure spend
Kubecost / OpenCost	Kubernetes cost by namespace/pod
CloudHealth / Cloudability	Multi-cloud FinOps platforms
Infracost	Terraform plan cost estimates in CI

OpenCost on Kubernetes:

```
apiVersion: apps/v1
```

```

kind: Deployment
metadata:
  name: opencost
  namespace: opencost
spec:
  template:
    spec:
      containers:
        - name: opencost
          image: opencost/opencost:latest
          env:
            - name: PROMETHEUS_SERVER_ENDPOINT
              value: "http://prometheus:9090"

```

Dashboard shows cost per namespace, deployment, label—allocating cluster overhead by CPU/RAM usage.

58.3 57.3 Right-sizing and waste elimination

Common waste sources:

Waste	Detection	Remediation
Idle resources	CloudWatch/GCP metrics low utilization	Stop dev instances nights/weekends
Oversized instances	CPU < 20% sustained	Downsize instance type
Unattached EBS volumes	Cost anomaly reports	Snapshot and delete
Old snapshots	Age-based report	Lifecycle policy
Over-provisioned K8s requests	Kubecost efficiency score	Reduce requests/limits
Forgotten environments	Tag + age audit	TTL automation
Data transfer	Network cost breakdown	CDN, VPC endpoints, region locality

Right-sizing workflow:

1. Identify top 20 cost drivers (Pareto: 80% spend in 20% resources)
2. Analyze 30-day utilization metrics
3. Test smaller size in staging
4. Apply change with rollback plan
5. Measure cost delta next billing cycle

AWS Compute Optimizer, GCP Recommender, Azure Advisor — automated right-sizing suggestions.

58.4 57.4 Pricing models and savings

Model	When to use
On-demand	Variable, unpredictable workloads
Reserved Instances / Savings Plans	Steady-state baseline (1–3 year commit)
Spot / Preemptible	Fault-tolerant batch, CI runners, stateless workers
Committed use discounts	GCP CUD, Azure RI

Spot instance pattern for CI:

```
# Kubernetes node pool – spot for CI workloads
nodeSelector:
  workload-type: ci-batch
tolerations:
  - key: "spot"
    operator: "Equal"
    value: "true"
    effect: "NoSchedule"
```

Mix: 70% reserved for baseline, 30% on-demand/spot for burst—model in spreadsheet before committing.

58.5 57.5 Cost in CI/CD and IaC

Shift cost awareness left:

Infracost in PR comments:

```
- name: Infracost
  uses: infracost/actions/setup@v3
- run: |
  infracost breakdown --path terraform/ --format json \
    --out-file infracost-base.json
  infracost diff --path terraform/ \
    --compare-to infracost-base.json
```

Developers see “+\$450/month” before merging Terraform expanding RDS instance.

Policies:

- Budget alerts at 80%, 100%, 120% of forecast
- Auto-shutdown Lambda for non-prod tagged `auto-shutdown=true`
- Maximum instance size guardrails in Terraform policy (OPA: deny `p4d.24xlarge` in dev)

58.6 57.6 Unit economics

Connect cloud spend to business metrics:

Metric	Formula
Cost per customer	Monthly infra / active customers
Cost per transaction	Infra / transactions
Cost per API request	Service cost / request volume
Margin per feature	Revenue - allocated infra - support

Engineering decisions framed in unit economics:

▮ “Caching reduces DB cost \$2K/month and improves latency—ROI positive in 2 weeks.”

Finance partnership: monthly **FinOps review** with engineering managers reviewing team dashboards—not surprise bills.

58.7 57.7 FinOps maturity stages

Stage	Characteristics
Crawl	Basic billing visibility, manual reports

Walk	Tagging enforced, team dashboards, anomaly alerts
Run	Unit economics, automated optimization, CI cost gates
Fly	Predictive forecasting, autoscaling cost-aware, culture embedded

58.8 57.8 Chapter summary

- FinOps aligns engineering, finance, and business on cloud cost accountability.
 - Tagging and tools (Cost Explorer, Kubecost, Infracost) enable visibility and allocation.
 - Right-sizing, reserved capacity, and spot instances optimize spend without sacrificing reliability.
 - Unit economics and CI cost feedback integrate financial awareness into daily engineering.
-

58.9 Lab 57.1 — Cost allocation tags

1. Define mandatory tag schema for Terraform modules.
 2. Apply OPA/Checkov policy denying untagged resources.
 3. Export cost report grouped by `team` tag (or simulate with labeled resources).
-

58.10 Lab 57.2 — Infracost in CI

1. Add Infracost to Terraform repo CI on pull requests.
 2. Open PR changing instance type; review cost diff comment.
 3. Document team policy for cost increase approval threshold.
-

58.11 Review questions

1. What are the three FinOps lifecycle phases?
 2. Why is tagging essential for cloud cost allocation?
 3. Compare reserved instances and spot instances use cases.
 4. How does Kubecost attribute Kubernetes cluster costs?
 5. What is unit economics in cloud engineering context?
 6. Name three common sources of cloud waste.
 7. How can IaC CI pipelines prevent costly infrastructure changes?
-

Continue: Chapter 58 — MLOps Overview

59 Chapter 58: MLOps Overview for DevOps Engineers

DevOps Handbook — Part XII, Pages 1131-1145

59.1 58.1 Why DevOps engineers need MLOps

Machine learning systems are **software systems**—they require versioning, testing, deployment, monitoring, and incident response. Unlike traditional apps, ML adds **data**, **models**, and **non-deterministic behavior**. **MLOps** (Machine Learning Operations) applies DevOps principles to the ML lifecycle.

DevOps engineers increasingly support data science teams—or operate ML platforms. Understanding MLOps prevents treating models as “black box scripts” outside standard engineering practices.

ML system components:

Data sources → Feature pipeline → Training → Model registry →
Deployment → Inference service → Monitoring (data + model drift)

59.2 58.2 ML lifecycle vs traditional SDLC

Traditional app	ML system
Code is primary artifact	Code + data + model weights
Tests verify logic	Tests verify data schema, model metrics
Deploy binary/container	Deploy model version + feature pipeline
Monitor errors/latency	Monitor + data drift + model performance
Rollback = previous release	Rollback = previous model + feature schema

Technical debt in ML accumulates silently—training-serving skew, stale features, label leakage—requiring explicit governance.

59.3 58.3 Key MLOps concepts

Concept	Definition
Feature store	Centralized, versioned features for training and serving
Model registry	Versioned model artifacts with metadata and stage (staging/prod)
Experiment tracking	Log params, metrics, artifacts per training run
Training pipeline	Automated, reproducible training workflow
Serving	Real-time (REST/gRPC) or batch inference
Data validation	Schema and statistical checks on input data
Model monitoring	Drift detection, performance decay alerts

Tools landscape:

Category	Tools
Experiment tracking	MLflow, Weights & Biases, Neptune
Orchestration	Kubeflow, Metaflow, Airflow, Argo Workflows
Feature store	Feast, Tecton, SageMaker Feature Store
Model registry	MLflow Model Registry, SageMaker, Vertex AI
Serving	KServe, Seldon Core, TorchServe, Triton
Monitoring	Evidently AI, WhyLabs, Arize

59.4 58.4 Reproducible training pipeline

Training must be **reproducible**: same data snapshot + code + hyperparameters = same model (within stochastic variance).

MLflow training example:

```
import mlflow
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

with mlflow.start_run():
    mlflow.log_param("n_estimators", 100)
    mlflow.log_param("max_depth", 10)

    model = RandomForestClassifier(n_estimators=100, max_depth=10)
    model.fit(X_train, y_train)

    preds = model.predict(X_test)
    accuracy = accuracy_score(y_test, preds)
    mlflow.log_metric("accuracy", accuracy)

    mlflow.sklearn.log_model(model, "model")
```

Pipeline orchestration (Kubeflow Pipelines):

```
@dsl.pipeline(name="Training pipeline")
def training_pipeline(data_path: str):
    validate_op = validate_data(data_path)
    train_op = train_model(validate_op.outputs["validated_data"])
    evaluate_op = evaluate_model(train_op.outputs["model"])
    with dsl.Condition(evaluate_op.outputs["accuracy"] > 0.85):
        deploy_op = register_model(train_op.outputs["model"])
```

CI for ML:

- Unit tests on feature engineering functions
- Data validation on sample datasets (Great Expectations)
- Model performance threshold gates before registry promotion

59.5 58.5 Model deployment patterns

Pattern	Description
Embedded model	Model loaded in app process (sklearn pickle, ONNX)
Model server	Separate inference service (Triton, TF Serving)
Sidecar	Model container alongside app pod in K8s

Serverless	Lambda/Cloud Functions for low-traffic inference
Batch	Scheduled jobs scoring datasets offline

KServe InferenceService:

```

apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: fraud-detector
spec:
  predictor:
    sklearn:
      storageUri: s3://models/fraud-detector/v3
      resources:
        requests:
          cpu: "500m"
          memory: "512Mi"
        limits:
          cpu: "1"
          memory: "1Gi"

```

Canary model rollout: route 5% traffic to new model version; compare prediction latency and business KPI before full promotion—same patterns as Chapter 41.

59.6 58.6 Feature stores and training-serving skew

Training-serving skew — Features computed differently in training vs production—silent accuracy degradation.

Feature store solves:

- **Single definition** — Feature logic defined once
- **Point-in-time correctness** — Training uses historical feature values as they existed
- **Online/offline serving** — Same features for batch training and real-time inference

Feast example:

```

from feast import FeatureStore

store = FeatureStore(repo_path=".")

# Training: historical features
training_df = store.get_historical_features(
    entity_df=entity_df,
    features=["customer_features:avg_order_value"],
).to_df()

# Serving: online features
features = store.get_online_features(
    features=["customer_features:avg_order_value"],
    entity_rows=[{"customer_id": 12345}],
).to_dict()

```

DevOps role: operate Feast infrastructure, monitor feature pipeline latency, integrate with data warehouse connectors.

59.7 58.7 Monitoring ML in production

Monitor four layers:

Layer	Signals
Infrastructure	CPU/GPU, memory, request rate (standard observability)

Service	Latency, errors, throughput
Data	Input distribution drift, missing features, schema violations
Model	Prediction distribution drift, accuracy on labeled feedback

Evidently drift report (conceptual):

```
from evidently.report import Report
from evidently.metric_preset import DataDriftPreset

report = Report(metrics=[DataDriftPreset()])
report.run(reference_data=training_sample, current_data=production_sample)
report.save_html("drift_report.html")
```

Alert when:

- Feature drift exceeds statistical threshold
- Model accuracy on holdout feedback drops
- Inference latency SLO breached

Retrain triggers: scheduled (weekly) + event-driven (drift alert).

59.8 58.8 Governance and responsible ML

- **Model cards** — Document intended use, limitations, bias evaluation
- **Lineage** — Track data sources, transformations, model dependencies
- **Access control** — Sensitive training data restricted
- **Audit** — Who promoted model to production
- **Rollback** — Previous model version in registry ready to deploy

Regulatory contexts (credit, healthcare) require explainability (SHAP, LIME) and audit trails—integrate with compliance automation (Chapter 54).

59.9 58.9 DevOps + MLOps collaboration

DevOps provides	Data science provides
K8s, CI/CD, observability	Models, features, metrics
GPU node pools, autoscaling	Training job requirements
Secrets, networking	Data access patterns
SLOs, on-call for serving	Model performance thresholds

Platform golden path for ML: “Deploy sklearn model from registry to KServe with monitoring”—Backstage template scaffolds InferenceService + Grafana dashboard.

59.10 58.10 Chapter summary

- MLOps extends DevOps to data, training, model registry, serving, and drift monitoring.
- Reproducible pipelines, experiment tracking, and feature stores reduce training-serving skew.
- Deploy models like services—with canaries, SLOs, and rollback via model registry.
- DevOps engineers operate ML infrastructure; data scientists own model quality—collaborate via platform golden paths.

59.11 Lab 58.1 — MLflow end-to-end

1. Train simple sklearn model; log to local MLflow.
 2. Register model; promote to Staging.
 3. Serve model via `mlflow models serve`; send test predictions.
-

59.12 Lab 58.2 — Model monitoring dashboard

1. Collect week of inference inputs (CSV).
 2. Run Evidently drift report against training baseline.
 3. Document retrain trigger policy based on drift score.
-

59.13 Review questions

1. How does an ML system differ from a traditional application in terms of artifacts?
 2. What is training-serving skew, and how do feature stores help?
 3. Name three components of an MLOps platform stack.
 4. Compare real-time and batch inference deployment patterns.
 5. What should ML monitoring cover beyond standard RED metrics?
 6. How do canary deployments apply to model rollouts?
 7. What is a model registry, and why version models?
-

Continue: Chapter 59 — Capstone Production System Design

60 Chapter 59: Capstone — End-to-End Production System Design

DevOps Handbook — Part XII, Pages 1146-1180

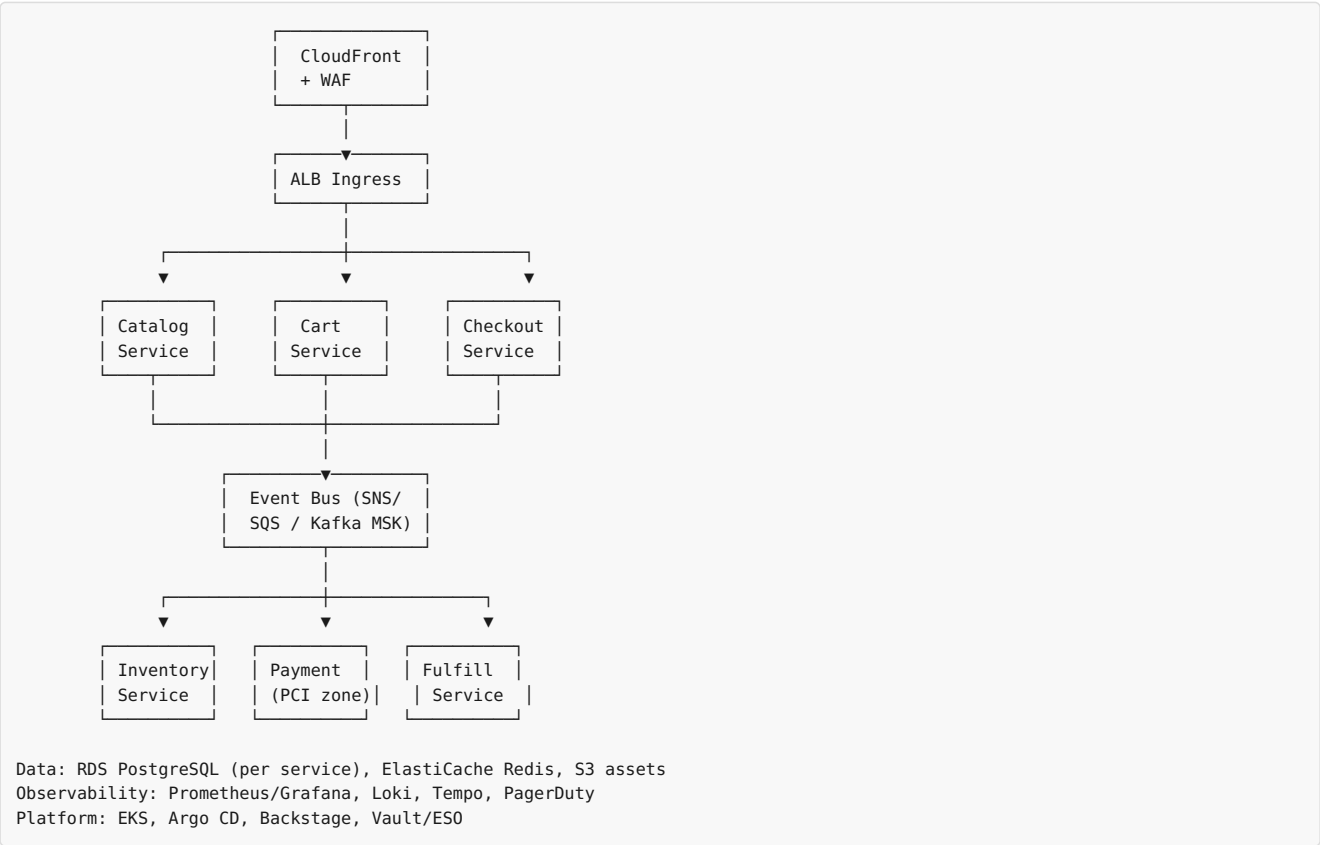
60.1 59.1 Capstone objectives

This chapter synthesizes the handbook into a **reference architecture** for a production-grade SaaS platform. You will design a system that embodies DevOps principles: automated delivery, observable operations, security by default, cost awareness, and reliable incident response.

Capstone scenario: **ShopStream** — a B2C e-commerce API handling product catalog, cart, checkout, and order fulfillment. Requirements:

Requirement	Target
Availability SLO	99.9% (43 min/month error budget)
Latency SLO	p99 < 500ms for checkout API
Scale	10K RPS peak, 50 microservices over 2 years
Compliance	PCI DSS for payment scope; SOC 2 Type II
Team	8 stream-aligned teams + 1 platform team
Cloud	AWS primary (reference AWS Handbook for service depth)

60.2 59.2 Architecture overview



Design principles applied:

- **Domain-driven service boundaries** — Checkout team owns checkout SLO
- **Async where possible** — Order events via queue, not synchronous chains
- **PCI segmentation** — Payment service in isolated subnet; no PAN in logs
- **Fail closed** — Auth failures reject; circuit breakers on dependencies

60.3 59.3 Repository and team structure

Team Topologies mapping:

Team	Services	Interaction
Catalog	product-api, search-indexer	Stream-aligned
Checkout	cart, checkout, payment-gateway	Stream-aligned
Fulfillment	inventory, shipping, notifications	Stream-aligned
Platform	IDP, clusters, shared modules	Platform X-as-a-Service
Security	Policies, scanning, audit	Enabling

Repository strategy: **polyrepo** with shared templates (one repo per service)—scaffolded via Backstage golden path.

Each service repo contains:

```
service-name/
├── src/
├── Dockerfile
├── helm/
├── terraform/           # Service-specific infra module
├── .github/workflows/
│   ├── ci.yml
│   └── deploy.yml
├── catalog-info.yaml
├── docs/
└── RUNBOOK.md
```

Monorepo optional for tightly coupled early-stage; migrate to polyrepo as teams scale.

60.4 59.4 CI/CD pipeline design

Per-service pipeline (GitHub Actions):

```
PR opened:
  lint → unit test → SAST (Semgrep) → SCA → IaC scan → build image →
  Trivy scan → push to ECR (SHA tag) → deploy preview (Review App)

Merge to main:
  integration tests → sign image (cosign) → generate SBOM →
  deploy staging (Argo CD sync) → DAST (ZAP baseline) →
  manual approval → canary 5% prod → metrics gate → full prod
```

Key configurations:

```
# Excerpt – production deploy with canary via Argo Rollouts
deploy-production:
  needs: [build, scan, staging-smoke]
  environment: production
  steps:
```

```

- uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: arn:aws:iam::ACCOUNT:role/GHA-Deploy-Prod
- run: |
    cosign verify ${ env.IMAGE } --certificate-oidc-issuer=...
    argocd app set checkout-api -p image.tag=${ github.sha }
    argocd app sync checkout-api --prune

```

DORA instrumentation: deployment webhook to metrics backend on every prod sync.

Deployment strategies by service criticality:

Service	Strategy
Catalog	Rolling
Checkout	Canary with automated analysis
Payment	Blue/green + manual approval

Feature flags (LaunchDarkly) for checkout UI experiments decoupled from deploy cadence.

60.5 59.5 Infrastructure as Code

Terraform layered modules:

```

terraform/
├── modules/
│   ├── vpc/
│   ├── eks/
│   ├── rds/
│   └── service-deployment/
├── environments/
│   ├── staging/
│   └── production/
└── global/
    └── iam/

```

State: S3 backend + DynamoDB lock per environment. CI runs `terraform plan` on PR, `apply` on merge to environment branch.

GitOps (Argo CD) reconciles Kubernetes from `k8s-manifests` repo:

```

apps/
├── checkout-api/
│   ├── base/
│   └── overlays/
│       ├── staging/
│       └── production/
└── catalog-api/
    └── ...

```

Platform provisions EKS, networking, shared RDS proxy; teams own service overlays.

Policy gates: Checkov in CI; Gatekeeper in cluster (no privileged pods, required labels, signed images).

60.6 59.6 Security architecture

Defense in depth layers:

Layer	Implementation
Edge	WAF, rate limiting, TLS 1.3

Identity	OAuth2/OIDC (Cognito/Auth0), mTLS service mesh (optional)
Network	VPC segmentation, NetworkPolicies, PCI isolated subnet
Secrets	Vault + ESO; dynamic DB creds for payment service
Supply chain	SBOM, cosign, Kyverno verifyImages
Runtime	Falco rules, PSA restricted
Compliance	Audit logs → SIEM; quarterly access reviews

PCI scope minimization: payment service tokenizes via Stripe—ShopStream never stores PAN; SAQ A scope where possible.

Security scanning integrated per Chapter 49–54; critical CVE blocks prod deploy.

60.7 59.7 Observability and SRE

Per-service SLOs documented in `slos/checkout-api.yaml` :

```
service: checkout-api
slos:
  - name: availability
    target: 99.9
    window: 30d
    sli:
      query: |
        sum(rate(http_requests_total{service="checkout",status!~"5.."}[5m]))
        / sum(rate(http_requests_total{service="checkout"}[5m]))
  - name: latency-p99
    target: 99.0 # 99% under 500ms
    window: 30d
    sli:
      query: |
        histogram_quantile(0.99, sum(rate(http_duration_bucket[5m])) by (le))
        < 0.5
```

Stack:

- **Metrics** — Prometheus + Grafana (kube-prometheus-stack)
- **Logs** — Fluent Bit → Loki (or OpenSearch for full-text SIEM)
- **Traces** — OpenTelemetry SDK → Collector → Tempo
- **Alerting** — Alertmanager → PagerDuty; multi-window burn alerts

Runbooks in repo `RUNBOOK.md` linked from Backstage. On-call rotation per team with escalation to platform for infra issues.

Incident process: SEV definitions, blameless postmortems within 5 days, action items in Jira.

Chaos: monthly Litmus pod-delete on staging; quarterly game day simulating payment dependency failure.

60.8 59.8 Data, DR, and scaling

Component	Pattern
PostgreSQL	RDS Multi-AZ; read replicas for catalog
Redis	ElastiCache cluster mode for session cache
Object storage	S3 + CloudFront for product images
Events	MSK Kafka for order stream; DLQ for failures

DR: RPO 1 hour, RTO 4 hours—warm standby in second region (us-west-2); Route 53 failover drill quarterly.

Autoscaling: HPA on CPU/RPS custom metrics; Karpenter for node provisioning; scale-to-zero for dev namespaces (FinOps).

60.9 59.9 FinOps and platform metrics

Mandatory tags: `team`, `service`, `environment`, `cost-center`.

Kubecost dashboards per team; monthly FinOps review with unit economics:

- Cost per order processed
- Cost per active user
- Idle resource report automated weekly

Infracost on Terraform PRs; budget alerts at org and team level.

Platform team metrics: golden path adoption %, developer CSAT, time-to-first-deploy for new services.

60.10 59.10 Capstone delivery roadmap

Phased implementation (technical sequencing, not calendar estimates):

Phase	Deliverables
1 — Foundation	EKS, VPC, Terraform modules, Argo CD, observability stack
2 — Platform	Backstage, golden path template, ESO, CI templates
3 — First service	Catalog API end-to-end: CI, deploy, SLO, runbook
4 — Critical path	Checkout + payment with PCI segmentation, canary
5 — Hardening	Security gates, chaos, DR drill, compliance evidence
6 — Scale	Event bus, additional services, FinOps optimization

Each phase ends with **definition of done**: deployed to staging, SLO dashboard live, runbook reviewed, security scan clean.

60.11 59.11 Design review checklist

Before production launch, verify:

- ☐ SLOs defined with error budget alerts
- ☐ CI: lint, test, SAST, SCA, container scan, IaC scan
- ☐ Signed images + admission policy enforced
- ☐ Secrets not in Git; ESO/Vault operational
- ☐ Centralized logs with trace correlation
- ☐ Runbooks and on-call rotation active
- ☐ DR failover tested within RTO target
- ☐ Tagging and cost dashboards per team
- ☐ Postmortem template and incident roles documented
- ☐ DORA metrics emitted from pipeline

60.12 59.12 Chapter summary

- ShopStream capstone integrates microservices, EKS, GitOps, CI/CD security, observability, and FinOps.
- Team Topologies and golden paths scale organizationally; polyrepo + templates balance autonomy and consistency.
- Layered security, SLO-driven operations, and phased delivery reduce risk while enabling continuous deployment.
- Use the design review checklist as a template for your own production systems.

60.13 Capstone Lab — Mini ShopStream

Build a reduced capstone (3 services: catalog, cart, checkout):

1. Scaffold services via template with shared CI pipeline.
2. Deploy to local k3d or cloud EKS with Argo CD.
3. Implement Prometheus metrics, one SLO dashboard, and alert.
4. Add Trivy scan gate and cosign sign (optional keyless).
5. Write RUNBOOK.md and conduct 30-minute game day (kill checkout pod).
6. Document architecture decision record (ADR) for one major choice.

60.14 Review questions

1. How do you decompose ShopStream into team-aligned services?
2. Why isolate payment in a separate network segment?
3. Describe the full CI/CD path from PR to production canary.
4. Which observability signals form checkout SLOs?
5. How does the platform team enable stream-aligned teams without bottleneck?
6. What DR strategy meets RPO 1h / RTO 4h?
7. Walk through the capstone design review checklist for catalog-api.

Continue: Chapter 60 — Appendix, Cheatsheets, Glossary

61 Chapter 60: Appendix — Cheatsheets, Glossary, and Index

DevOps Handbook — Part XII, Pages 1181-1200

61.1 60.1 kubectl quick reference

```
# Context and namespaces
kubectl config use-context prod
kubectl get pods -n production -o wide

# Deployments
kubectl rollout status deployment/api -n production
kubectl rollout undo deployment/api -n production
kubectl scale deployment/api --replicas=5

# Debugging
kubectl logs -f deployment/api -c api --tail=100
kubectl exec -it pod/api-xxx -- /bin/sh
kubectl describe pod api-xxx
kubectl get events --sort-by=.metadata.creationTimestamp

# Resources
kubectl top pods -n production
kubectl apply -f manifest.yaml --dry-run=server
kubectl diff -f manifest.yaml

# Port forward
kubectl port-forward svc/api 8080:80
```

61.2 60.2 Docker quick reference

```
# Build and run
docker build -t myapp:v1 .
docker run -d -p 8080:80 --name myapp myapp:v1
docker exec -it myapp sh

# Cleanup
docker system prune -a --volumes # caution: removes unused data
docker logs -f myapp

# Multi-stage build pattern
# FROM golang:1.22 AS builder
# FROM gcr.io/distroless/static-debian12
```

61.3 60.3 Terraform quick reference

```
terraform init
terraform plan -out=tfplan
terraform apply tfplan
terraform state list
terraform import aws_instance.web i-0abc123
terraform fmt -recursive
terraform validate

# Workspace
terraform workspace select staging
terraform workspace list
```

61.4 60.4 Git essentials

```
git status
git log --oneline --graph -20
git diff main...feature-branch
git rebase -i HEAD~3
git stash push -m "wip"
git cherry-pick abc1234

# Recover
git reflog
git reset --hard HEAD@{2}
```

61.5 60.5 Linux troubleshooting

```
# Processes
ps aux | grep nginx
systemctl status nginx
journalctl -u nginx -f --since "10 min ago"

# Network
ss -tlnp
curl -vI https://example.com
dig +short example.com
traceroute example.com

# Disk
df -h
du -sh /var/log/*
lsof +D /path/to/dir

# Resources
top -o %CPU
free -h
vmstat 1 5
iostat -x 1 3
```

61.6 60.6 PromQL cheatsheet

```
# Rate and errors
rate(http_requests_total[5m])
sum by (service) (rate(http_requests_total{status=~"5.."}[5m]))

# Latency percentiles
histogram_quantile(0.99, sum(rate(http_duration_bucket[5m])) by (le, service))

# Resource usage
100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)

# Predict disk full (4h)
predict_linear(node_filesystem_free_bytes[1h], 4*3600) < 0

# Absent metric alert helper
absent(up{job="api"})
```

61.7 60.7 CI/CD YAML snippets

GitHub Actions concurrency:

```
concurrency:
  group: deploy-${{ github.ref }}
  cancel-in-progress: true
```

GitLab rules:

```
rules:
  - if: $CI_PIPELINE_SOURCE == "merge_request_event"
  - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
```

Jenkins declarative post:

```
post {
  always { cleanWs() }
  failure { emailxnt subject: "Failed", body: "${env.BUILD_URL}" }
}
```

61.8 60.8 Security tool commands

```
# Secret scan
gitLeaks detect --source . -v

# Container scan
trivy image myapp:latest
trivy fs --severity CRITICAL,HIGH .

# SBOM
syft packages dir:. -o cyclonedx-json > sbom.json
grype sbom:sbom.json

# Sign and verify
cosign sign myregistry/myapp:v1
cosign verify myregistry/myapp:v1

# IaC scan
checkov -d terraform/ --framework terraform
```

61.9 60.9 Observability checklist

Signal	Minimum instrumentation
Metrics	RED per service; USE per node
Logs	JSON with timestamp, level, service, trace_id
Traces	W3C traceparent on all HTTP/gRPC exits
Alerts	SLO burn + symptom-based pages only
Dashboards	Golden signals per service + infra overview

61.10 60.10 SLO error budget reference

SLO	Monthly downtime budget	Quarterly budget
99%	~7.2 hours	~21.6 hours
99.9%	~43 minutes	~2.16 hours
99.95%	~22 minutes	~1.08 hours

99.95%	~22 minutes	~1.08 hours
99.99%	~4.3 minutes	~13 minutes

Formula: $\text{budget} = (1 - \text{SLO}) \times \text{period_seconds}$

61.11 60.11 Glossary

Term	Definition
Alert fatigue	Desensitization from excessive non-actionable alerts
Argo CD	GitOps continuous delivery tool for Kubernetes
Blue/green	Deployment with two environments; traffic switch for cutover
Canary	Gradual traffic shift to new version with monitoring
CI/CD	Continuous Integration / Continuous Delivery or Deployment
Container	Lightweight isolated process bundle with filesystem
DAST	Dynamic Application Security Testing on running apps
DevSecOps	Integrating security into DevOps lifecycle
DORA metrics	Deployment frequency, lead time, change failure rate, MTTR
Error budget	Allowed unreliability: $1 - \text{SLO}$
Feature flag	Runtime toggle decoupling deploy from release
FinOps	Cloud financial management discipline
GitOps	Declarative infra/apps with Git as source of truth
Golden path	Supported self-service workflow in platform engineering
Helm	Kubernetes package manager
IaC	Infrastructure as Code
IDP	Internal Developer Platform
Ingress	Kubernetes HTTP routing into cluster
Kubernetes	Container orchestration platform
MLOps	ML lifecycle automation and operations
MTTR	Mean Time to Restore service
Observability	Inferring internal state from external outputs
OIDC	OpenID Connect federated identity
OPA	Open Policy Agent for policy decisions
Pod	Smallest deployable unit in Kubernetes
Prometheus	Open-source metrics TSDB and scraper
Rego	OPA policy language
Runbook	Step-by-step operational procedure
SAML	Security Assertion Markup Language for SSO
SAST	Static Application Security Testing
SBOM	Software Bill of Materials

SCA	Software Composition Analysis for dependencies
Shift-left	Moving activities earlier in SDLC
SLO/SLI/SLA	Service Level Objective/Indicator/Agreement
SLSA	Supply-chain Levels for Software Artifacts
SRE	Site Reliability Engineering
Terraform	IaC tool by HashiCorp
Toil	Manual repetitive operational work
Trunk-based development	Short-lived branches merged frequently to main
Vault	HashiCorp secrets management
WAF	Web Application Firewall

61.12 60.12 Handbook chapter index

Part	Chapters	Topics
I	1-4	DevOps culture, CALMS, careers
II	5-8	Linux, systemd, networking, bash
III	9-11	Git, branching, collaboration
IV	12-15	Networking, HTTP/TLS, load balancing
V	16-19	Python, Go, templating, testing
VI	20-25	IaC, Terraform, Ansible, Packer, policy
VII	26-30	Docker, compose, registries, Podman
VIII	31-36	Kubernetes, Helm, GitOps, RBAC
IX	37-42	CI/CD, Actions, Jenkins, GitLab, deploy, DORA
X	43-48	Observability, Prometheus, ELK, OTel, SLO, chaos
XI	49-54	DevSecOps, scanning, secrets, supply chain, compliance
XII	55-60	Platform eng, multi-cloud, FinOps, MLOps, capstone

Cross-reference: [AWS Handbook](#) for cloud-specific service depth.

61.13 60.13 Recommended reading

Book	Author(s)	Topic
<i>The DevOps Handbook</i>	Kim, Humble, Debois, Willis	DevOps practices
<i>Accelerate</i>	Forsgren, Humble, Kim	DORA research
<i>Site Reliability Engineering</i>	Google	SRE principles
<i>The Phoenix Project</i>	Kim, Spafford, Behr	DevOps narrative
<i>Team Topologies</i>	Skelton, Pais	Organization design
<i>Kubernetes Up & Running</i>	Hightower, Burns, Beda	K8s operations
<i>Terraform: Up & Running</i>	Brikman	IaC patterns

Security DevOps	Bellomo et al.	DevSecOps
-----------------	----------------	-----------

Online: opentelemetry.io, prometheus.io, slsa.dev, finops.org

61.14 60.14 Certification paths (optional)

Certification	Focus
AWS Solutions Architect / DevOps Pro	AWS platform
CKA / CKAD / CKS	Kubernetes admin, app dev, security
Terraform Associate	IaC
LFCS / LFCE	Linux
GIAC / Security+	Security foundations

Certifications validate knowledge; hands-on labs in this handbook build practical skill.

61.15 60.15 Final review questions (handbook-wide)

1. Explain the Three Ways and how Parts IX-X implement the Second and Third Ways.
2. Design a CI pipeline listing stages from commit to production with security gates.
3. How do SLOs, error budgets, and DORA metrics interact in organizational decision-making?
4. Compare Terraform, Kubernetes, and GitOps responsibilities in platform architecture.
5. Describe a supply chain security program covering SBOM, signing, and admission control.
6. When would you choose Elasticsearch vs Loki for logging?
7. What capabilities belong in an Internal Developer Platform vs team-owned services?

61.16 60.16 Chapter summary

- This appendix provides command cheatsheets for daily DevOps operations.
- The glossary defines key terms used throughout the handbook.
- The chapter index and reading list support continued learning beyond these pages.

End of DevOps Handbook — Return to [Table of Contents](#)